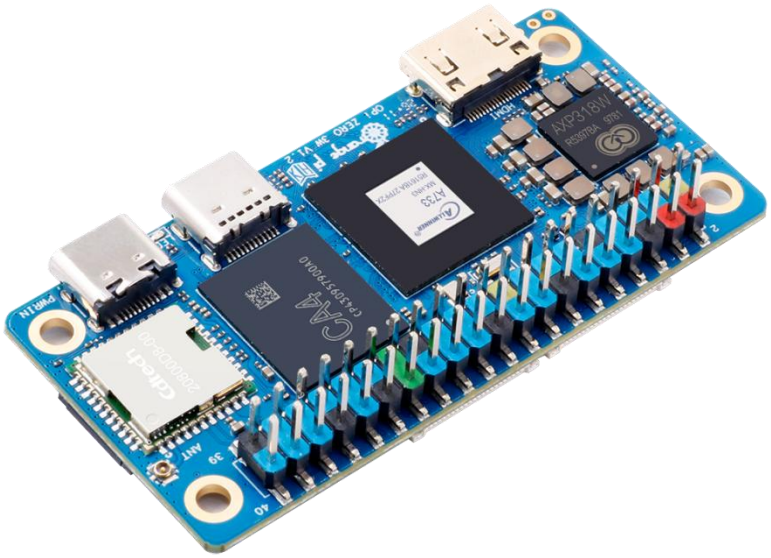
Orange Pi Zero 3w

用户手册



目录

1. Orange Pi Zero 3w 的基本特性 1
1.1. 什么是 Orange Pi Zero 3w 1
1.2. Orange Pi Zero 3w 的用途 1
1.3. Orange Pi Zero 3w 是为谁设计的 1
1.4. Orange Pi Zero 3w 的硬件特性 2
1.5. Orange Pi Zero 3w 的顶层视图和底层视图 4
1.6. Orange Pi Zero 3w 的接口详情图 5
2. 开发板使用介绍 7
2.1. 准备需要的配件 7
2.2. 下载开发板的镜像和相关的资料 10
2.3. 基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法 10
2.3.1. 使用 balenaEtcher 烧录 Linux 镜像的方法 10
2.3.2. 使用 Win32Diskimager 烧录 Linux 镜像的方法 14
2.4. 基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法 16
2.5. 烧写 Linux 镜像到 eMMC 中的方法 20
2.6. 烧写 Linux 镜像到 UFS 中的方法 20
2.7. 烧写 Android 镜像到 TF 卡的方法 20
2.8. 烧写 Android 镜像到 eMMC 的方法 26
2.9. 通过 TypeC 口烧录 Android 镜像到 eMMC 或 UFS 的方法 31
2.10. 启动香橙派开发板 35
2.11. 调试串口的使用方法 36
2.11.1. 调试串口的连接说明 36
2.11.2. Ubuntu 平台调试串口的使用方法 37
2.11.3. Windows 平台调试串口的使用方法 40

2.12. 使用开发板 40 pin 接口中的 5v 引脚供电说明 42
3. Debian/Ubuntu Server 和 Xfce 桌面系统使用说明 44
3.1. 已支持的 linux 镜像类型和内核版本 44
3.2. linux 内核驱动适配情况 45
3.3. 本手册 linux 命令格式说明 46
3.4. linux 系统登录说明 47
3.4.1. linux 系统默认登录账号和密码 47
3.4.2. 设置 linux 系统终端自动登录的方法 47
3.4.3. linux 桌面版系统自动登录说明 48
3.4.4. Linux 桌面版系统 root 用户自动登录的设置方法 49
3.5. 板载 LED 灯测试说明 49
3.6. TF 卡中 linux 系统 rootfs 分区容量操作说明 50
3.6.1. 第一次启动会自动扩容 TF 卡中 rootfs 分区的容量 50
3.6.2. 禁止自动扩容 TF 卡中 rootfs 分区容量的方法 51
3.6.3. 手动扩容 TF 卡中 rootfs 分区容量的方法 53
3.6.4. 缩小 TF 卡中 rootfs 分区容量的方法 58
3.7. 网络连接测试 62
3.7.1. WIFI 连接测试 62
3.7.2. 设置静态 IP 地址的方法 69
3.7.3. 设置 Linux 系统第一次启动自动连接网络的方法 70
3.8. SSH 远程登录开发板 73
3.8.1. Ubuntu 下 SSH 远程登录开发板 73
3.8.2. Windows 下 SSH 远程登录开发板 74
3.9. ADB 的使用方法 75
3.9.1. 网络 adb 的使用方法 75
3.9.2. 使用 type-c 数据线连接 adb 76
3.10. HDMI 测试 78
3.10.1. HDMI 显示测试 78
3.10.2. HDMI 转 VGA 显示测试 78
3.11. 蓝牙使用方法 79

3.11.1. 桌面版镜像的测试方法 79
3.11.2. 服务器版镜像的使用方法 82
3.12. USB 接口测试 85
3.12.1. USB 接口扩展说明 86
3.12.2. 连接 USB 鼠标或键盘测试 86
3.12.3. 连接 USB 存储设备测试 86
3.12.4. USB 以太网卡测试 87
3.12.5. USB 摄像头测试 88
3.13. 温度传感器 90
3.14. 40 pin 接口引脚说明 91
3.15. 安装 wiringOP 的方法 92
3.16. 40pin 接口 GPIO、I2C、UART、SPI 和 PWM 测试 93
3.16.1. 40pin GPIO 口测试 93
3.16.2. 40 pin GPIO 口上下拉电阻的设置方法 94
3.16.3. 40 pin SPI 测试 95
3.16.4. 40 pin I2C 测试 98
3.16.5. 40 pin 的 UART 测试 100
3.16.6. 使用/sys/class/pwm/测试 PWM 的方法 102
3.17. wiringOP-Python 的安装使用方法 105
3.17.1. wiringOP-Python 的安装方法 106
3.17.2. 40 pin GPIO 口测试 108
3.17.3. 40 pin SPI 测试 110
3.17.4. 40 pin I2C 测试 112
3.17.5. 40 pin 的 UART 测试 115
3.18. 硬件看门狗测试 118
3.19. 查看 A733 芯片的 chipid 118
3.20. Python 相关说明 119
3.20.1. Python 源码编译安装的方法 119
3.20.2. Python 更换 pip 源的方法 120
3.21. 安装 Docker 的方法 120
3.22. Home Assistant 的安装方法 121

3.22.1. 通过 docker 安装 121
3.22.2. 通过 python 安装 125
3.23. OpenCV 的安装方法 127
3.23.1. 使用 apt 来安装 OpenCV 127
3.24. 设置中文环境以及安装中文输入法 127
3.24.1. Debian 系统的安装方法 128
3.24.2. Ubuntu 22.04 系统的安装方法 134
3.25. 宝塔 Linux 面板的安装方法 139
3.26. QT 的安装方法 143
3.27. ROS 安装方法 149
3.27.1. Ubuntu22.04 安装 ROS 2 Humble 的方法 149
3.28. 安装内核头文件的方法 151
3.29. Linux 系统支持的部分编程语言测试 152
3.29.1. Debian Bullseye 系统 152
3.29.2. Debian Bookworm 系统 154
3.29.3. Ubuntu Jammy 系统 156
3.30. 上传文件到开发板 Linux 系统中的方法 158
3.30.1. 在 Ubuntu PC 中上传文件到开发板 Linux 系统中的方法 158
3.30.2. 在 Windows PC 中上传文件到开发板 Linux 系统中的方法 161
3.31. IMX219 MIPI 摄像头的使用方法 165
3.31.1. 摄像头的安装方法 165
3.31.2. 通过 Gstreamer 打开摄像头的方法 166
3.31.3. 通过 OpenCV 程序打开 MIPI 摄像头的方法 167
3.32. 10.1 寸 MIPI LCD 屏幕的使用方法 169
3.32.1. 10.1 寸 MIPI 屏幕的组装方法 169
3.32.2. 打开 10.1 寸 MIPI LCD 屏幕配置的方法 172
3.32.3. 服务器版镜像旋转显示方向的方法 174
3.33. Debian bullseye 系统硬解测试说明 175
3.34. Debian bullseye 系统 GPU 测试说明 176
3.35. NPU 的使用说明 178

3.35.1. PC 端开发环境配置 178
3.35.2. 板端模型部署 180
3.36. 烧写 linux 镜像到 eMMC 的方法 186
3.37. 烧写 linux 镜像到 UFS 的方法 188
3.38. 系统备份脚本 opi-bkimg 的使用方法 190
3.39. Linux Overlayroot 的使用方法 191
3.39.1. 配置 Overlayroot 为 tmpfs 模式 192
3.39.2. 配置 Overlayroot 为块设备存储模式 193
3.39.3. 禁用 Overlayroot 的方法 196
3.39.4. 通过 opi-bkimg 备份 overlayroot 镜像的方法 197
3.40. 关机和重启开发板的方法 198
4. Linux SDK——orangepi-build 使用说明 200
4.1. 编译系统需求 200
4.2. 获取 linux sdk 的源码 202
4.2.1. 下载 orangepi-build 202
4.2.2. 下载交叉编译工具链 203
4.2.3. orangepi-build 完整目录结构说明 205
4.3. 编译 u-boot 206
4.4. 编译 linux 内核 208
4.5. 编译 rootfs 212
4.6. 编译 linux 镜像 215
5. Android 13 系统使用说明 219
5.1. 已支持的 Android 版本 219
5.2. Android 13 功能适配情况 219
5.3. ADB 的使用方法 220
5.3.1. USB OTG 模式切换的方法 220
5.3.2. 使用数据线连接 adb 调试 223
5.3.3. 使用网络连接 adb 调试 224
5.3.4. HDMI 转 VGA 显示测试 225
5.4. WI-FI 的连接方法 225

5.5. 蓝牙的连接方法 228
5.6. MIPI 摄像头的使用方法 231
5.7. 10.1 寸 MIPI 屏幕的使用方法 232
5.8. USB 摄像头使用方法 233
5.9. Android 系统 ROOT 说明 235
5.10. 40 pin 接口 GPIO、UART、SPI 测试 237
5.10.1. 40 pin 的 GPIO 口测试方法 237
5.10.2. 40 pin 的 UART 测试方法 241
5.10.3. 40 pin 的 SPI 测试方法 243
5.10.4. 40 pin 的 I2C 测试方法 245
5.10.5. 40 pin 的 PWM 测试方法 249
6. Android 13 源码的编译方法 252
6.1. 下载 Android 13 的源码 252
6.2. 编译 Android 13 的源码 252
7. 附录 255
7.1. 用户手册更新历史 255
7.2. 镜像更新历史 255

1. Orange Pi Zero 3w 的基本特性

1.1. 什么是 Orange Pi Zero 3w

Orange Pi Zero 3w 采用了全志 A733 八核处理器，集成八核高性能 CPU，双核 Arm Cortex-A76 + 六核 Arm Cortex-A55 大小核架构，12nm 制程，最高主频 2.0GHz；支持 3Tops NPU，满足边缘智能 AI 加速应用；最高支持 16GB LPDDR5，支持最高 8K@24fps H.265/VP9/AVS2 视频格式解码，支持最高 4K@30fps H.265/H.264 视频格式编码；接口丰富，包括 PCIe3.0、TypeC USB3.0、UFS、MIPI-CSI、MIPI-DSI、40Pin 扩展接口等常用功能接口。支持 Ubuntu、Debian、Android 13 等操作系统。

Orange Pi Zero 3w 可为生成式 AI、人工智能 算法的场景化落地提供坚实的硬件基础，可广泛应用于智能工控、智能商显、零售支付、智慧教育、商用机器人、车载终端、视觉辅驾、边缘计算、智能配电终端等。

1.2. Orange Pi Zero 3w 的用途

我们可以用它实现：

- 一台小型的 Linux 桌面计算机
- 一台小型的 Linux 网络服务器
- Android 平板
- Android 游戏机等

当然还有其他更多的功能，依托强大的生态系统以及各式各样的扩展配件，Orange Pi 可以帮助用户轻松实现从创意到原型再到批量生产的交付，是创客、梦想家、业余爱好者的理想创意平台。

1.3. Orange Pi Zero 3w 是为谁设计的

Orange Pi 开发板不仅仅是一款消费品，同时也是给任何想用技术来进行创作创新的人设计的。它是一款简单、有趣、实用的工具，你可以用它去打造你身边的世界。

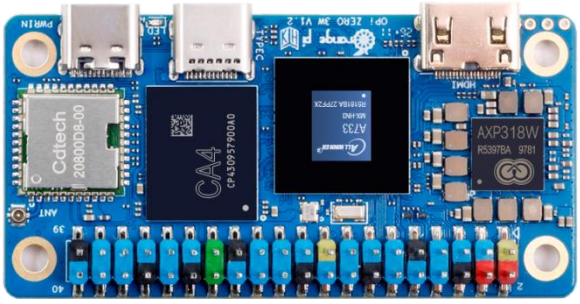
1.4. Orange Pi Zero 3w 的硬件特性

硬件特性介绍	
处理器	A733，2 个 Cortex-A76 核心 + 6 个 Cortex-A55 核心，最高主频 2.0GHz RISC-V E902 协处理器（200MHz） GPU：Imagination BXM-4-64 NPU：3TOPS
内存	最高支持 16GB LPDDR5
存储	板载 eMMC：8GB/16GB/32GB 可选（默认空贴） 板载 UFS：32GB/64GB/128GB 可选（默认空贴） TF 卡槽：最高支持 128GB
Wi-Fi+蓝牙	Wi-Fi+蓝牙二合一模块 Wi-Fi6 + BT 5.4，BLE
显示	1 x HDMI TX 2.0 interface up to 4K@60fps 1 x MIPI DSI 4 LANE 1 x DP1.4（DisplayPort）
摄像头	2x 4-lane MIPI-CSI camera interface
USB	1xType-C USB 3.0 HOST 1xType-C USB 2.0 支持 HOST 和 Device
FAN	1x 散热风扇接口
按键	1 * BOOT（测试点），1 * PWR ON（测试点）
40Pin	40Pin 功能扩展接口，支持以下接口类型： GPIO、UART、I2C、SPI、PWM
DEBUG	3Pin 调试串口
电源	Type-C 5V 3A DCIN
支持的 OS	Ubuntu、Debian、Android13 等
外观规格介绍	

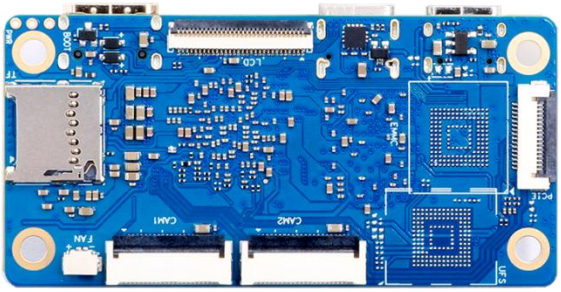
PCB	65mm*30mm
重量	14g

1.5. Orange Pi Zero 3w 的顶层视图和底层视图

顶层视图：

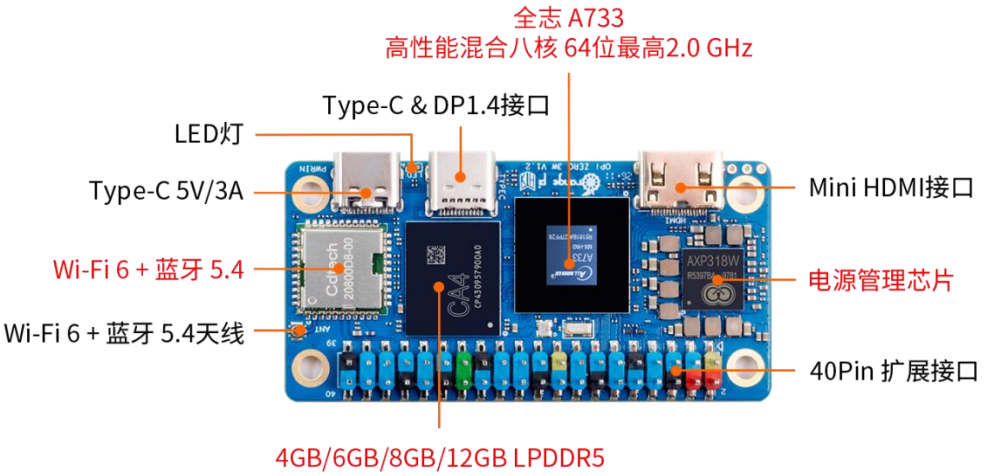


底层视图：

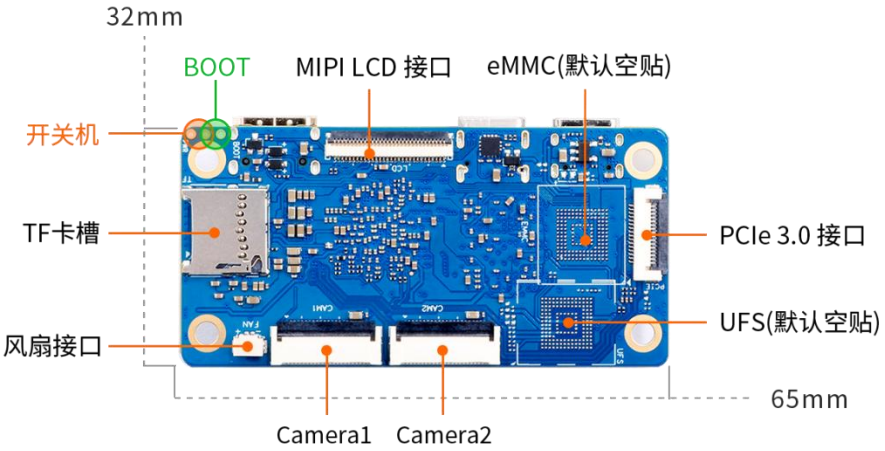


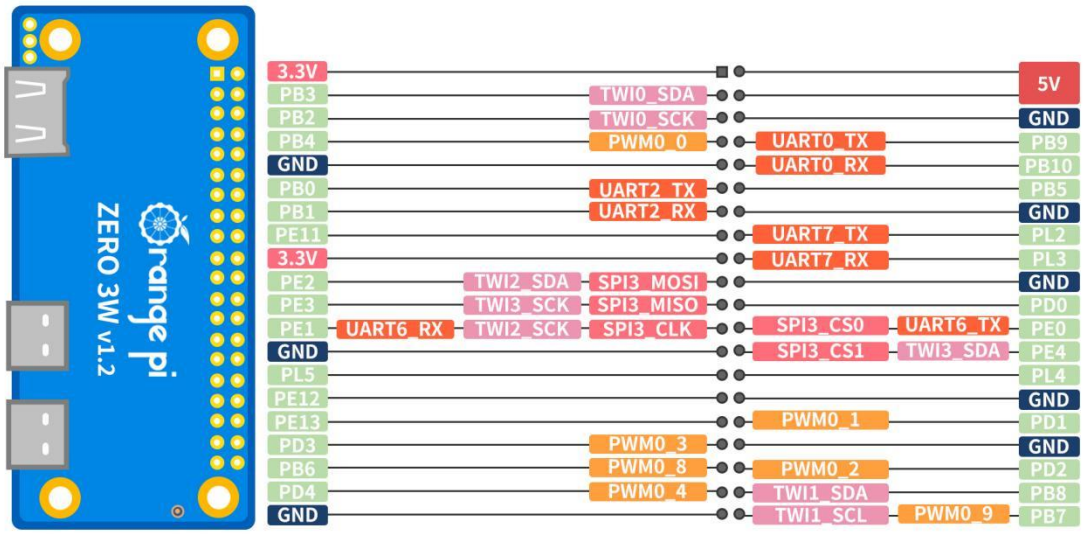
1.6. Orange Pi Zero 3w 的接口详情图

正面视图



背面视图





四个定位孔的直径都是 3.0mm。

2. 开发板使用介绍

2.1. 准备需要的配件

1) TF 卡，最小 8GB 容量的 **class10** 级或以上的高速闪迪卡



2) TF 卡读卡器，用于读写 TF 卡



3) HDMI 接口的显示器



4) Mini HDMI 转 HDMI 连接线，用于将开发板连接到 HDMI 显示器或者电视进行显示



5) Type-C转HDMI线，通过Type-C接口将开发板连接到HDMI显示器或者电视进行显示



6) 电源适配器，Orange Pi Zero 3w 建议使用 5V/3A 的 Type-C 电源供电



开发板的Type-C电源接口不支持PD协商功能，只支持固定的 5V电压输入。

7) USB接口的鼠标和键盘，只要是标准USB接口的鼠标和键盘都可以，鼠标和键盘可以用来控制Orange Pi开发板



8) Type-C 转 USB 线，用于接 USB 设备



9) USB 转 TTL 模块和杜邦线，使用串口调试功能时，需要 USB 转 TTL 模块和杜邦线来连接开发板和电脑





注意，开发板使用的TTL电平是 3.3v的，除了上图所示的USB转TTL模块外，其他类似的 3.3v的USB转TTL模块一般也都是可以的。

10) 安装有 Ubuntu 和 Windows 操作系统的 X64 电脑

1	Ubuntu22.04 PC	可选，用于编译 Android 和 Linux 源码
2	Windows PC	用于烧录 Android 和 Linux 镜像

2.2. 下载开发板的镜像和相关的资料

1) 中文版资料的下载网址为

http://www.orangepi.cn/html/hardWare/computerAndMicrocontrollers/service-and-support/Orange-Pi-Zero-3W.html

2) 英文版资料的下载网址为

http://www.orangepi.org/html/hardWare/computerAndMicrocontrollers/service-and-support/Orange-Pi-Zero-3W.html

3) 资料主要包含

a. **Linux 源码**：保存在 Github 上
b. **Android 镜像**：保存在百度云盘和谷歌网盘上
c. **Ubuntu 镜像**：保存在百度云盘和谷歌网盘上
d. **Debian 镜像**：保存在百度云盘和谷歌网盘上
e. **用户手册和原理图**：芯片相关的数据手册也会放在这里
f. **官方工具**：主要包括开发板使用过程中需要用到的软件

2.3. 基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法

注意，这里说的Linux镜像具体指的是从Orange Pi资料下载页面下载的Debian或者Ubuntu这样的Linux发行版镜像。

2.3.1. 使用 balenaEtcher 烧录 Linux 镜像的方法

1) 首先准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡

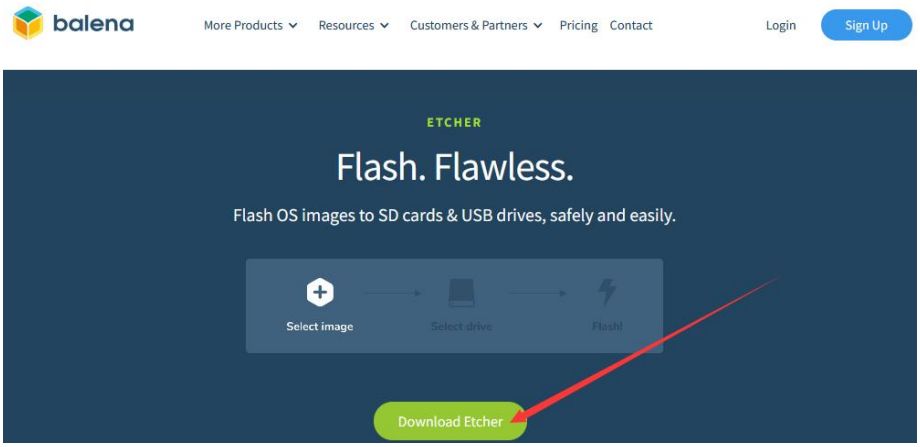
2) 然后使用读卡器把 TF 卡插入电脑

3) 从 Orange Pi 的资料下载页面下载想要烧录的 Linux 操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“.img”结尾的文件就是操作系统的镜像文件，大小一般都在 1GB 以上

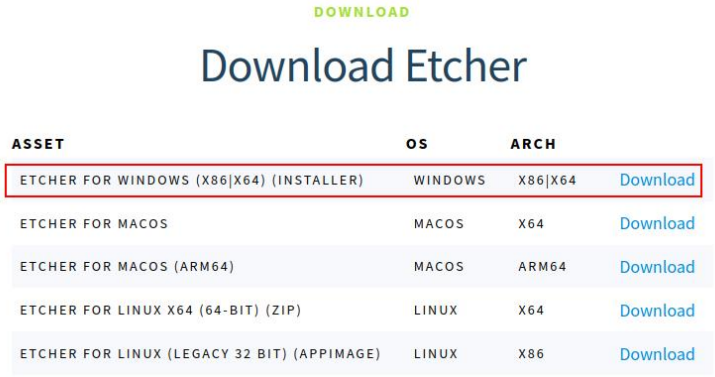
4) 然后下载 Linux 镜像的烧录软件——**balenaEtcher**，下载地址为

https://www.balena.io/etcher/

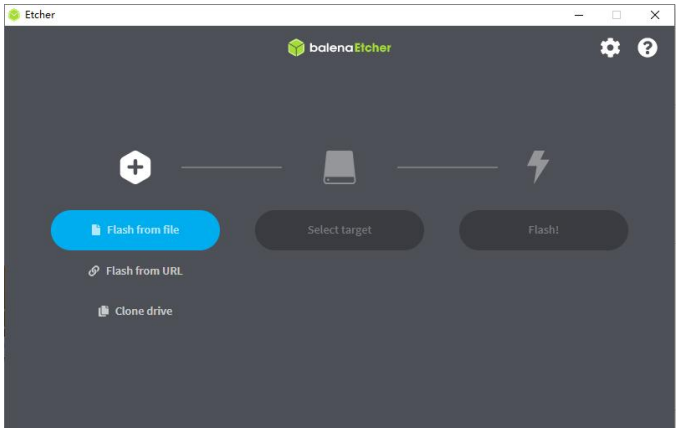
5) 进入 balenaEtcher 下载页面后，点击绿色的下载按钮会跳到软件下载选择界面



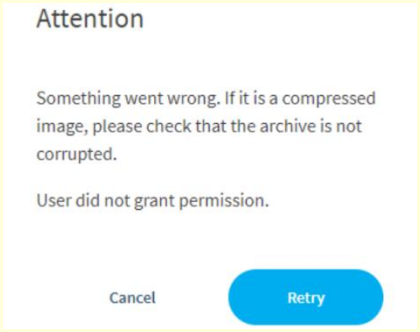
6) 然后下载 balenaEtcher Windows 版本的安装包。



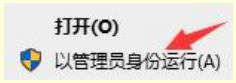
7) 然后安装下 balenaEtcher 再打开即可。打开后的 balenaEtcher 界面如下图所示



打开 balenaEtcher 时如果提示下面的错误：



请选择 balenaEtcher 后点击右键，然后选择以管理员身份运行。

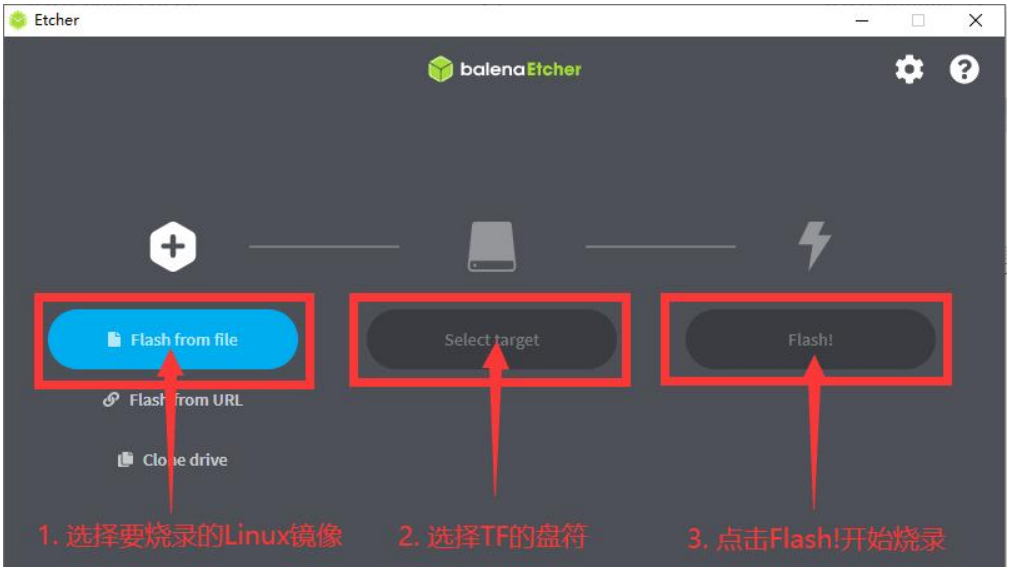


8) 使用 balenaEtcher 烧录 Linux 镜像的具体步骤如下所示

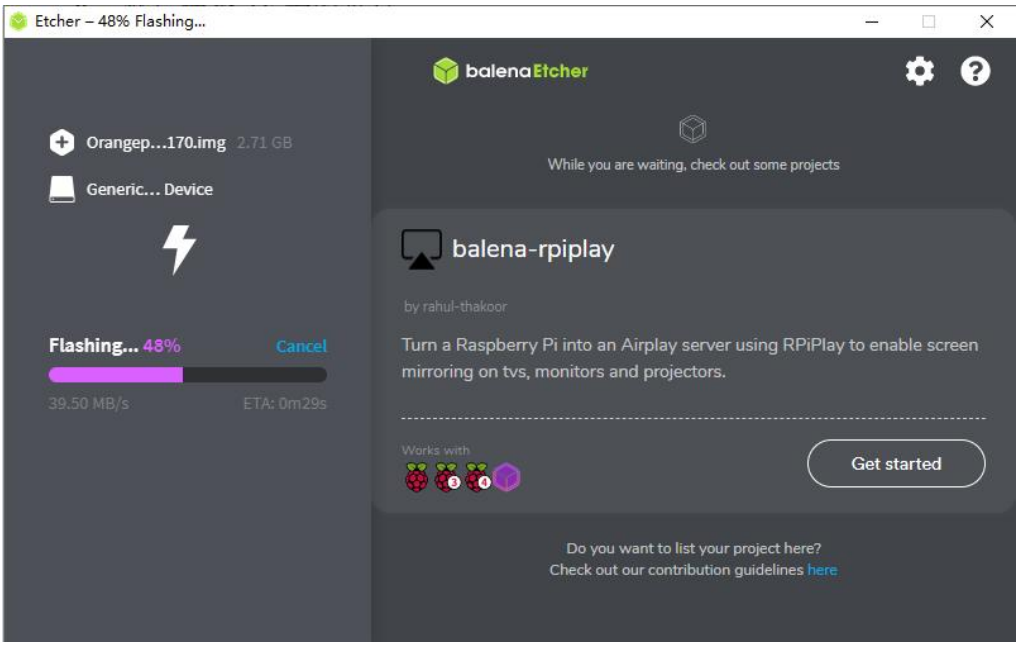
a. 首先选择要烧录的 Linux 镜像文件的路径

b. 然后选择 TF 卡的盘符

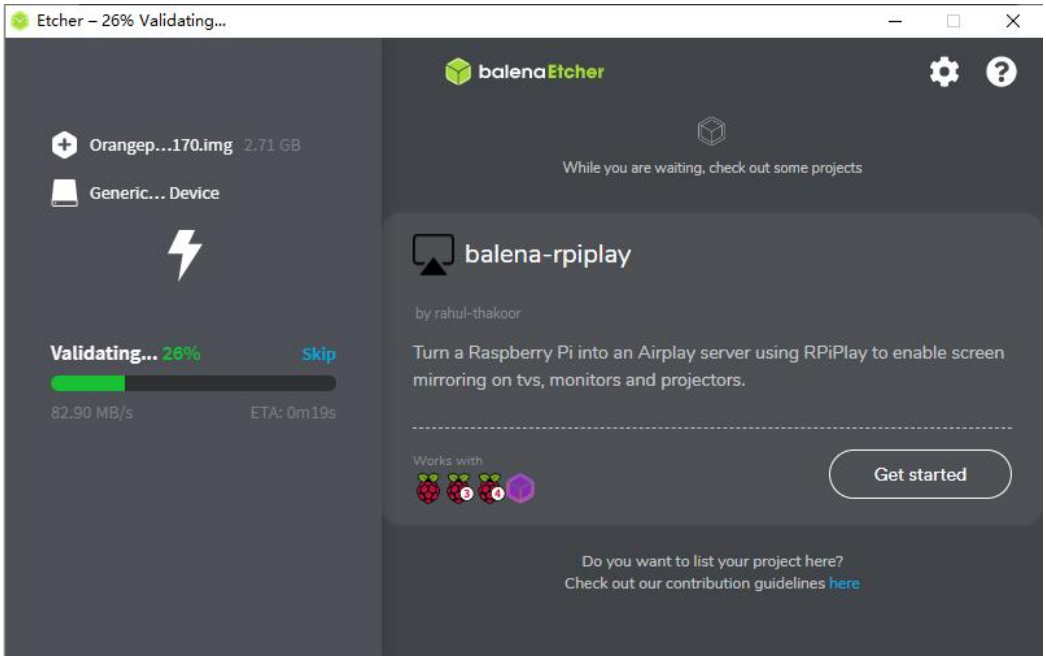
c. 最后点击 Flash 就会开始烧录 Linux 镜像到 TF 卡中



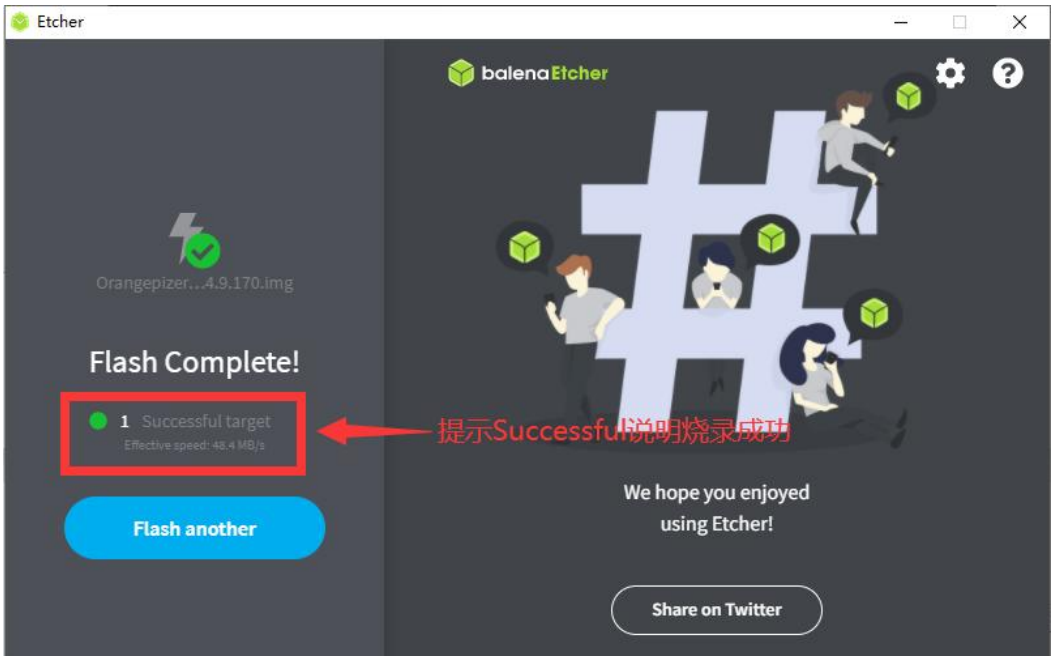
9) balenaEtcher 烧录 Linux 镜像的过程显示的界面如下图所示，另外进度条显示紫色表示正在烧录 Linux 镜像到 TF 卡中



10) Linux 镜像烧录完后，balenaEtcher 默认还会对烧录到 TF 卡中的镜像进行校验，确保烧录过程没有出问题。如下图所示，显示绿色的进度条就表示镜像已经烧录完成，balenaEtcher 正在对烧录完成的镜像进行校验



11) 成功烧录完成后 balenaEtcher 的显示界面如下图所示，如果显示绿色的指示图标说明镜像烧录成功，此时就可以退出 balenaEtcher，然后拔出 TF 卡插入到开发板的 TF 卡槽中使用了



2.3.2. 使用 Win32Diskimager 烧录 Linux 镜像的方法

1) 首先准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡

2) 然后使用读卡器把 TF 卡插入电脑

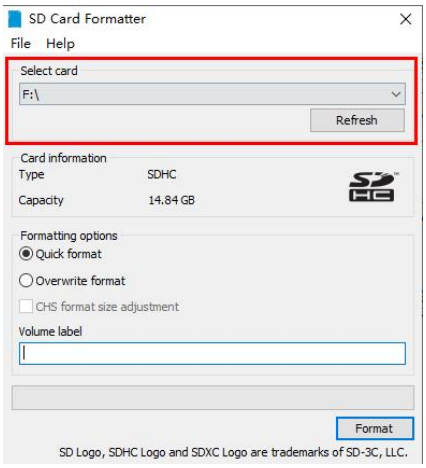
3) 接着格式化 TF 卡

a. 可以使用 **SD Card Formatter** 这个软件格式化 TF 卡，其下载地址为

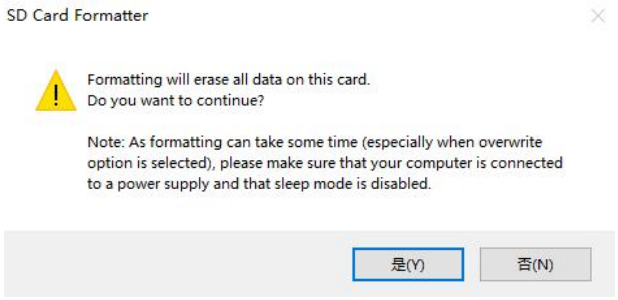
https://www.sdcard.org/downloads/formatter/eula_windows/SDCardFormatterv5_WinEN.zip

b. 下载完后直接解压安装即可，然后打开软件

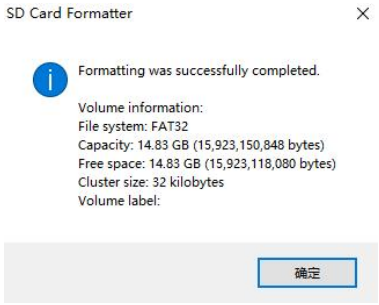
c. 如果电脑只插入了 TF 卡，则“**Select card**”一栏中会显示 TF 卡的盘符，如果电脑插入了多个 USB 存储设备，可以通过下拉框选择 TF 卡对应的盘符



d. 然后点击“**Format**”，格式化前会弹出一个警告框，选择“**是(Y)**”后就会开始格式化



e. 格式化完 TF 卡后会弹出下图所示的信息，点击确定即可



4) 从Orange Pi的资料下载页面下载想要烧录的Linux操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“**.img**”结尾的文件就是操作系统的镜像文件，大小一般都在 1GB以上

5) 使用 **Win32Diskimager** 烧录 Linux 镜像到 TF 卡

a. Win32Diskimager 的下载页面为

http://sourceforge.net/projects/win32diskimager/files/Archive/

b. 下载完后直接安装即可，Win32Diskimager 界面如下所示

a) 首先选择镜像文件的路径

b) 然后确认下 TF 卡的盘符和“**设备**”一栏中显示的一致

c) 最后点击“**写入**”即可开始烧录



c. 镜像写入完成后，点击“**退出**”按钮退出即可，然后就可以拔出 TF 卡插到开发板中启动

2.4. 基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法

注意，这里说的Linux镜像具体指的是从Orange Pi资料下载页面下载的Debian或者Ubuntu这样的Linux发行版镜像，Ubuntu PC指的是安装了Ubuntu系统的个人电脑。

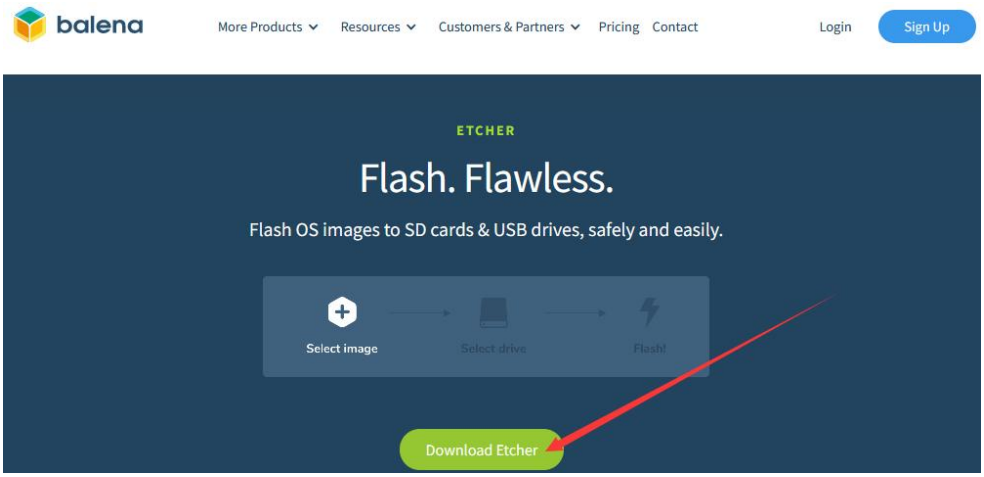
1) 首先准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡

2) 然后使用读卡器把 TF 卡插入电脑

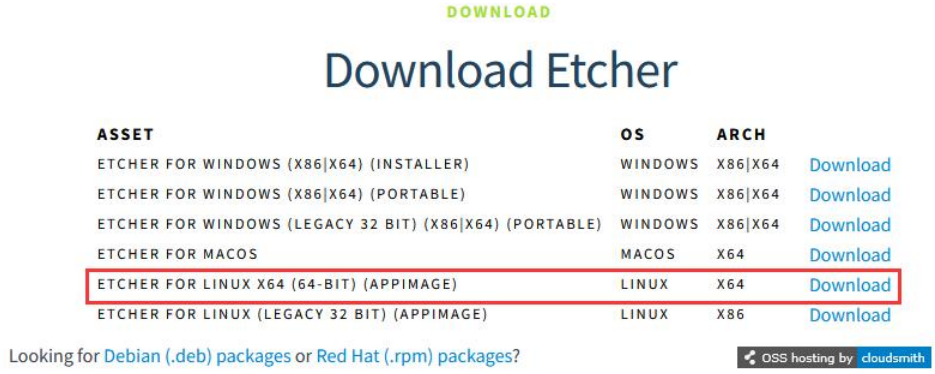
3) 下载 balenaEtcher 软件，下载地址为

https://www.balena.io/etcher/

4) 进入 balenaEtcher 下载页面后，点击绿色的下载按钮会跳到软件下载的地方



5) 然后选择下载 Linux 版本的软件即可



6) 从 Orange Pi 的资料下载页面下载想要烧录的 Linux 操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“**.img**”结尾的文件就是操作系统的镜像文件，大小一般都在 1GB 以上。7z 结尾的压缩包的解压命令如下所示：

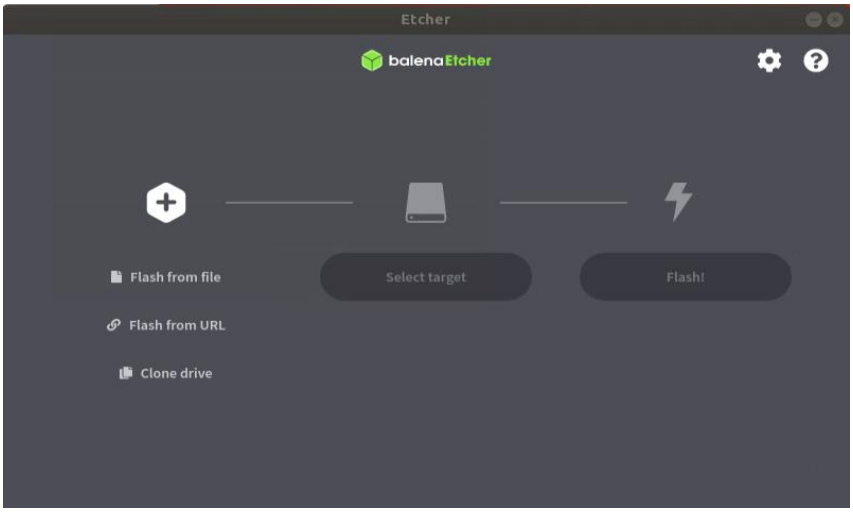
```
test@test:~$ 7z x Orangepizero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.7z
test@test:~$ ls Orangepizero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.*
Orangepizero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.7z
Orangepizero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.sha     #校验和文件
Orangepizero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.img     #镜像文件
```

7) 解压镜像后可以先用 **sha256sum -c *.sha** 命令计算下校验和是否正确，如果提示**成功**说明下载的镜像没有错，可以放心的烧录到 TF 卡，如果提示**校验和不匹配**说明下载的镜像有问题，请尝试重新下载

```
test@test:~$ sha256sum -c *.sha
```

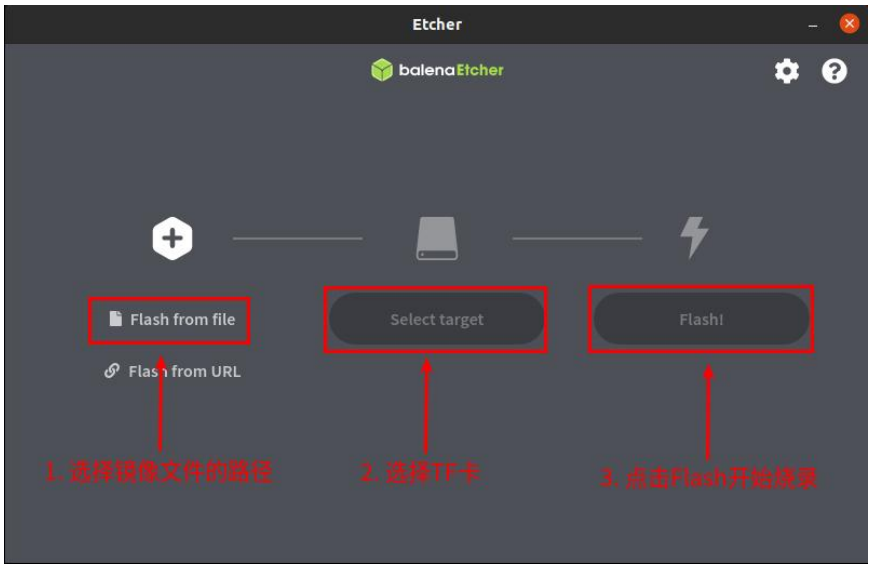
Orangepizero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.img: **成功**

8) 然后在 Ubuntu PC 的图形界面双击 **balenaEtcher-1.14.3-x64.AppImage** 即可打开 balenaEtcher（**无需安装**），balenaEtcher 打开后的界面显示如下图所示

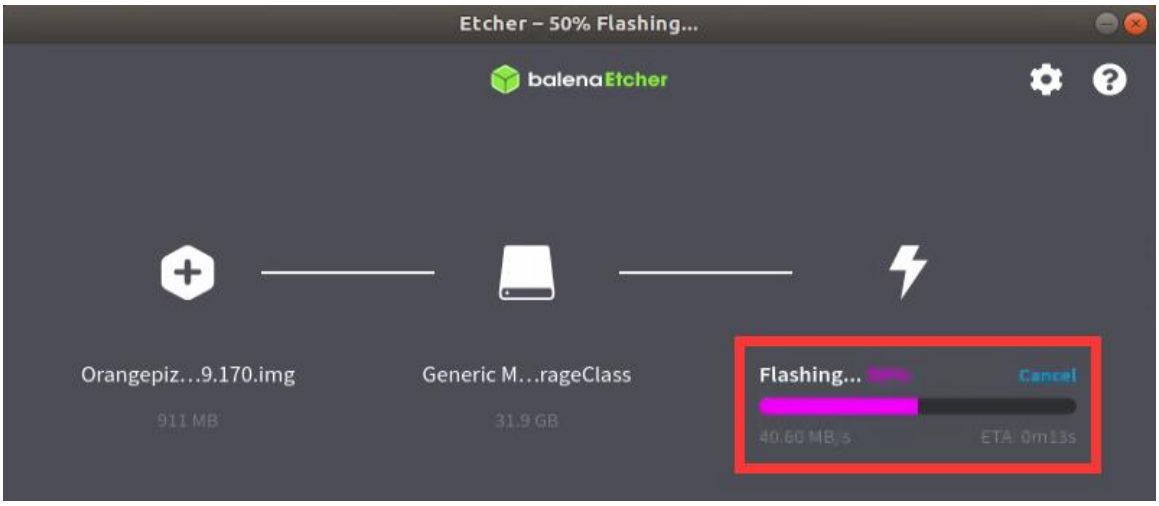


9) 使用 balenaEtcher 烧录 Linux 镜像的具体步骤如下所示

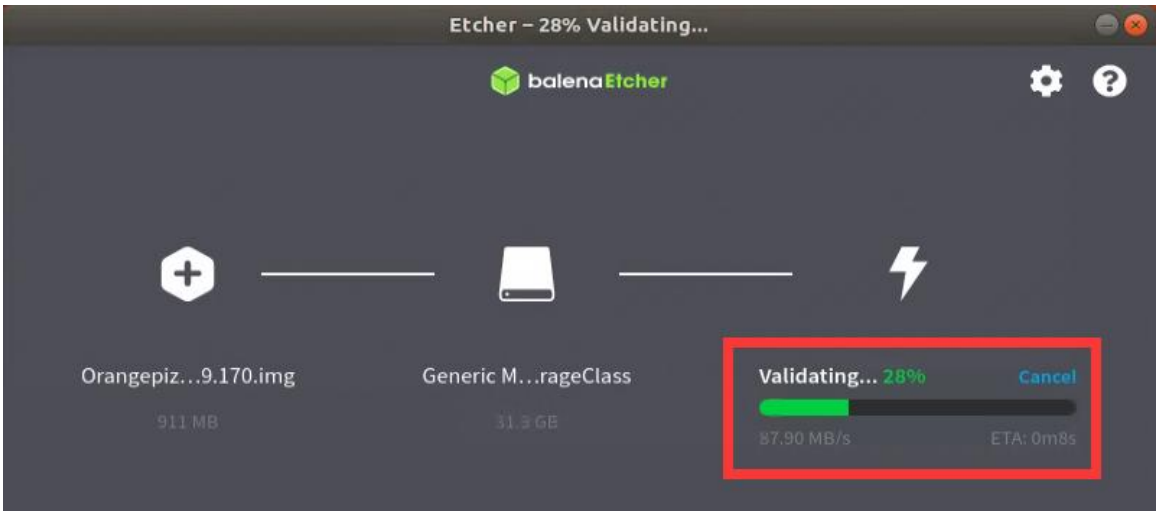
a. 首先选择要烧录的 Linux 镜像文件的路径
b. 然后选择 TF 卡的盘符
c. 最后点击 Flash 就会开始烧录 Linux 镜像到 TF 卡中



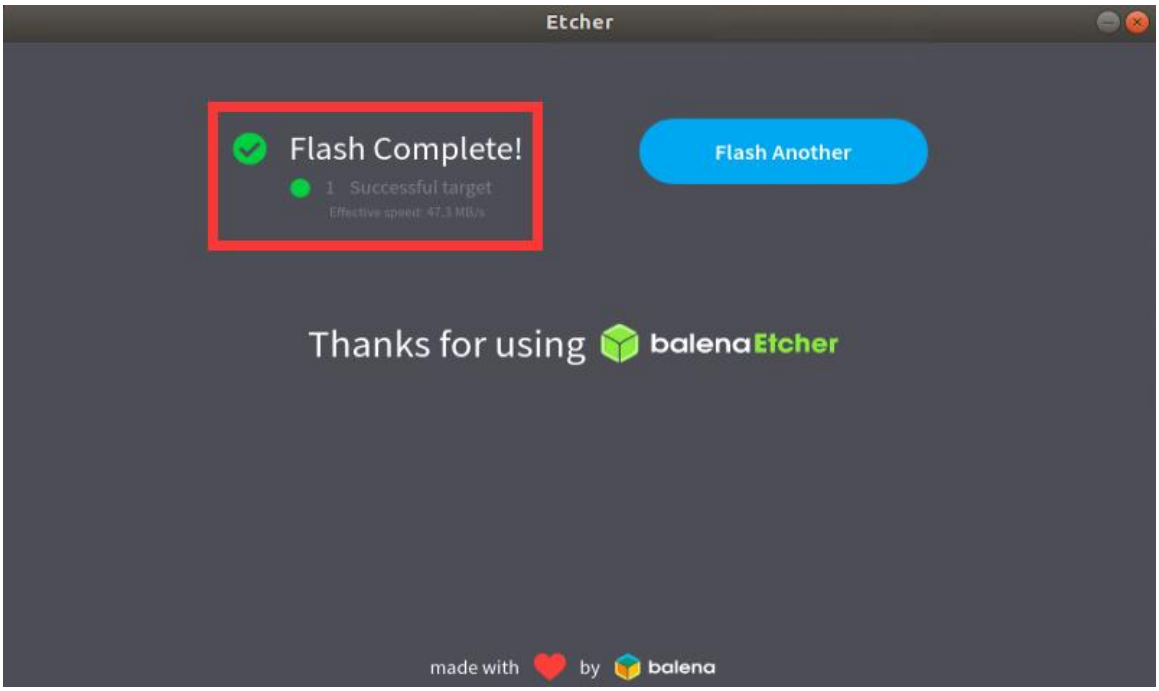
10) balenaEtcher 烧录 Linux 镜像的过程显示的界面如下图所示，另外进度条显示紫色表示正在烧录 Linux 镜像到 TF 卡中



11) Linux 镜像烧录完后，balenaEtcher 默认还会对烧录到 TF 卡中的镜像进行校验，确保烧录过程没有出问题。如下图所示，显示绿色的进度条就表示镜像已经烧录完成，balenaEtcher 正在对烧录完成的镜像进行校验



12) 成功烧录完成后 balenaEtcher 的显示界面如下图所示，如果显示绿色的指示图标说明镜像烧录成功，此时就可以退出 balenaEtcher，然后拔出 TF 卡插入到开发板的 TF 卡槽中使用了



2.5. 烧写 Linux 镜像到 eMMC 中的方法

见烧写 linux 镜像到 EMMC 的方法

2.6. 烧写 Linux 镜像到 UFS 中的方法

见烧写 linux 镜像到 UFS 的方法

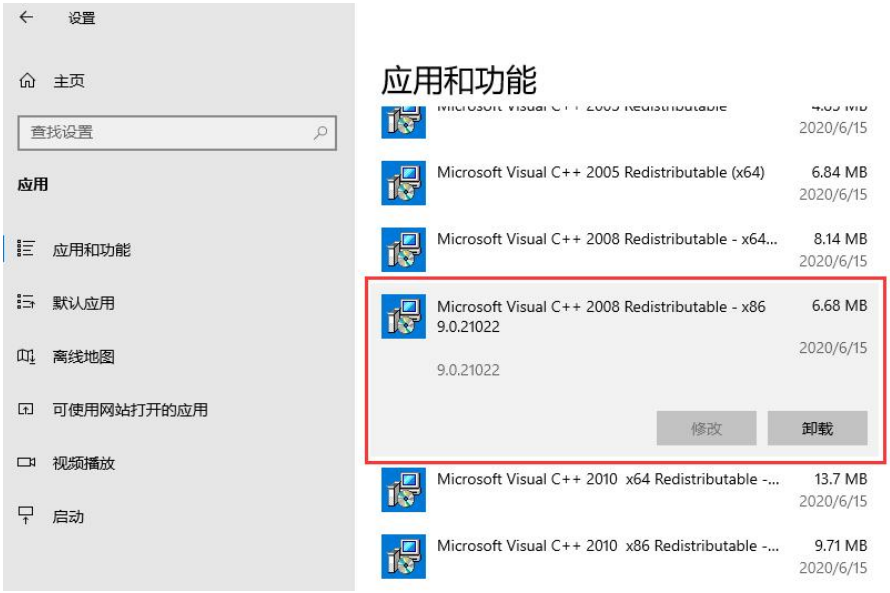
2.7. 烧写 Android 镜像到 TF 卡的方法

开发板的 Android 镜像只能在 Windows 平台下使用 PhoenixCard 软件烧录到 TF 卡中，PhoenixCard 软件的版本必须为 PhonixCard-4.2.8。

请不要用烧录 Linux 镜像的软件，如 Win32Diskimager 或者 balenaEtcher 来烧录安卓镜像。

另外 PhoenixCard 这款软件没有 Linux 和 Mac 平台的版本，所以在 Linux 和 Mac 平台下是无法烧录安卓镜像到 TF 卡中的。

1) 首先请确保 Windows 系统已经安装了 **Microsoft Visual C++ 2008 Redistrbutable - x86**

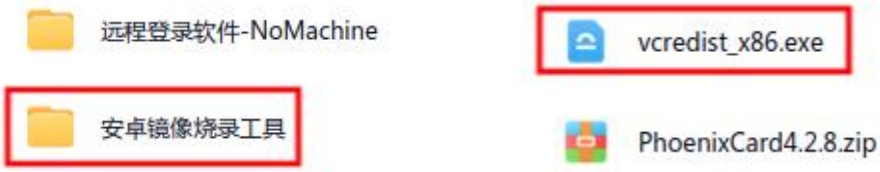


2) 如果没有安装 **Microsoft Visual C++ 2008 Redistrbutable - x86**，使用 PhoenixCard 格式化 TF 卡或者烧录 Android 镜像会提示下面的错误



3) **Microsoft Visual C++ 2008 Redistrbutable - x86** 的安装包可以从 Orange Pi Zero

3w 的官方工具中下载到，也可以去微软官网下载



4) 然后准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡

5) 然后使用读卡器把 TF 卡插入电脑

6) 从 Orange Pi 的资料下载页面下载 Android 镜像和 PhoenixCard 烧写工具，请确保 PhonenixCrad 工具的版本为 PhonixCard-4.2.8，请不要用低于 4.2.8 版本的 PhonixCard 软件来烧录 Android 镜像，低于这个版本的 PhonixCard 工具烧写的 Android 镜像可能会有问题

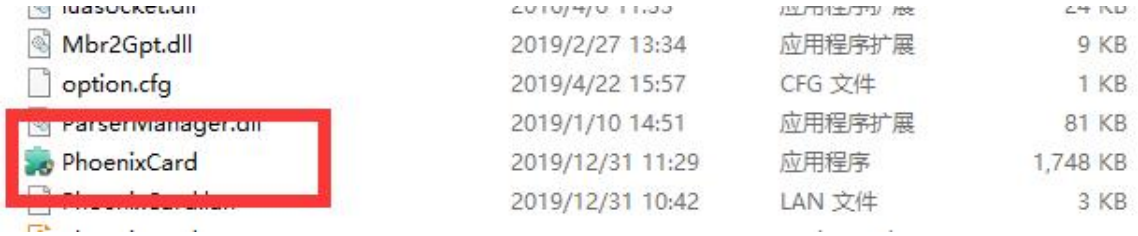


7) 然后使用解压软件解压下载的 Android 镜像的压缩包，解压后的文件中，以“**.img**”结尾的文件就是 Android 镜像文件，大小在 1GB 以上。如果不知道怎么解压 Android 镜像的压缩包，可以安装一个 360 压缩软件来解压镜像。



8) 然后使用解压软件解压 **PhonixCard4.2.8.zip**，此软件无需安装，在解压后的文

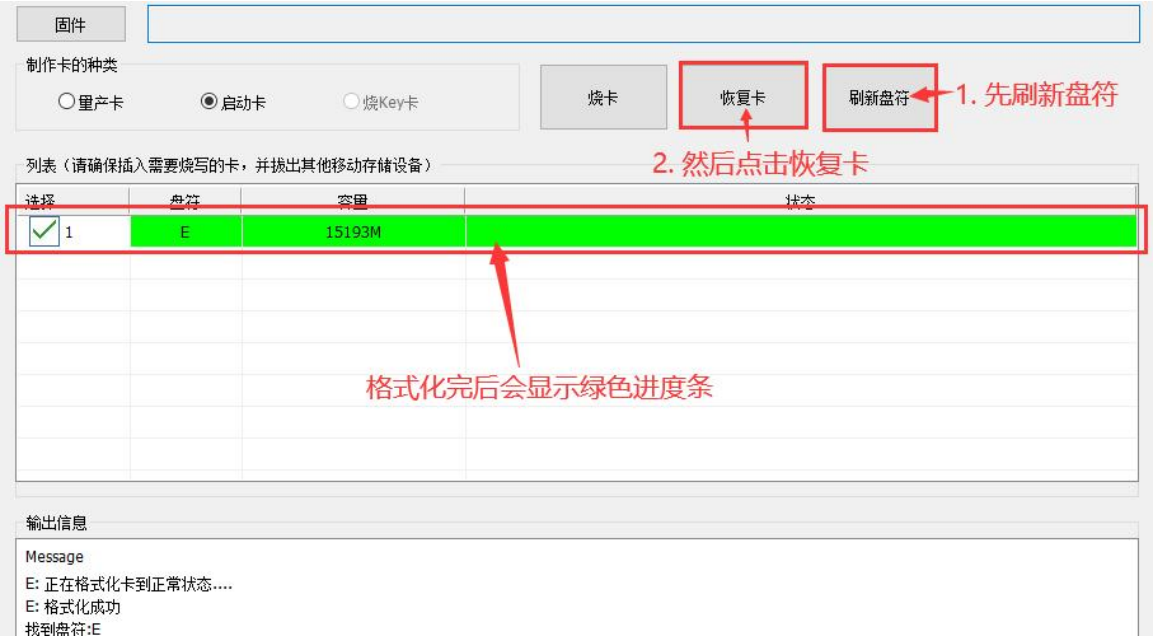
件夹中找到 PhoenixCard 打开即可



9) 打开 PhoenixCard 后，如果 TF 卡识别正常，会在中间的列表中显示 TF 卡的盘符和容量，**请务必确认显示的盘符和你想烧录的 TF 卡的盘符是一致的**，如果没有显示可以尝试拔插下 TF 卡，或者点击 PhoenixCard 中的“**刷新盘符**”按钮



10) 确认完盘符后，先格式化 TF 卡，点击 PhoenixCard 中“**恢复卡**”按钮即可（如果“**恢复卡**”按钮为灰色的无法按下，可以先点击下“**刷新盘符**”按钮）



如果格式化有问题，请尝试拔插下 TF 卡后再测试，如果重新拔插 TF 卡后还是

有问题，可以重启下 Window 电脑或者换一台电脑再试下。

11) 然后开始将 Android 镜像写入 TF 卡

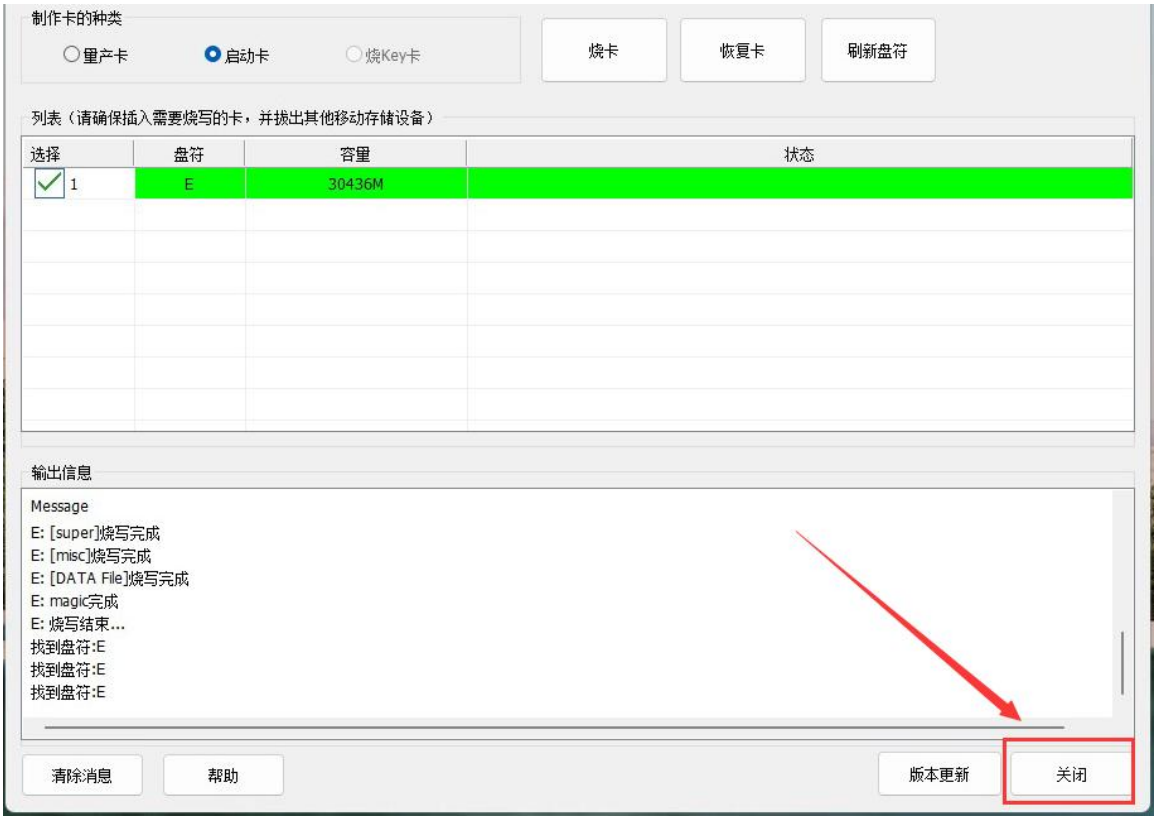
a. 首先在“**固件**”一栏中选择 Android 镜像的路径

b. 在“**制作卡的种类**”中选择“**启动卡**”

c. 然后点击“**烧卡**”按钮就会开始烧录



12) 烧录完后 PhoenixCard 的显示如下图所示，此时点击“**关闭**”按钮即可退出 PhoenixCard，然后就可以把 TF 卡从电脑中拔出来插到开发板中启动了



烧录完Android系统后在Windows中TF卡只能看到一个 128 MB的分区，显示的分区如下图所示（有些电脑可能会弹出二十几个磁盘分区，但也只能打开 128 MB的那个分区），请注意，这是正常的，请不要以为TF卡烧坏了。之所以这样，是因为安卓系统总共有二十几个分区，但大部分分区在Windows系统中是无法正常识别的。此时，请放心的拔下TF卡然后插入开发板中启动即可。



安卓系统启动后，使用下面的命令可以看到TF卡中的这二十几个分区：

```
console:/ # ls /dev/block/mmcblk0*
/dev/block/mmcblk0      /dev/block/mmcblk0p18  /dev/block/mmcblk0p27
/dev/block/mmcblk0p1    /dev/block/mmcblk0p19  /dev/block/mmcblk0p28
/dev/block/mmcblk0p10   /dev/block/mmcblk0p2   /dev/block/mmcblk0p3
/dev/block/mmcblk0p11   /dev/block/mmcblk0p20  /dev/block/mmcblk0p4
/dev/block/mmcblk0p12   /dev/block/mmcblk0p21  /dev/block/mmcblk0p5
/dev/block/mmcblk0p13   /dev/block/mmcblk0p22  /dev/block/mmcblk0p6
/dev/block/mmcblk0p14   /dev/block/mmcblk0p23  /dev/block/mmcblk0p7
/dev/block/mmcblk0p15   /dev/block/mmcblk0p24  /dev/block/mmcblk0p8
/dev/block/mmcblk0p16   /dev/block/mmcblk0p25  /dev/block/mmcblk0p9
/dev/block/mmcblk0p17   /dev/block/mmcblk0p26
```

使用df -h命令可以看到 32GB的TF卡烧录完安卓系统后大概还有 24 GB的空间可以用使用（二十几个分区并不会都挂载到安卓系统中，重点关注这些能看到的分

区即可）。

```
console:/ # df -h
Filesystem                     Size Used Avail Use% Mounted on
tmpfs                          963M 1.2M  961M   1% /dev
tmpfs                          963M    0  963M   0% /mnt
/dev/block/dm-0                803M 803M     0 100% /
/dev/block/dm-4                232K  36K  196K  16% /system_dlkm
/dev/block/dm-1                 88M  88M     0 100% /vendor
/dev/block/dm-3                 11M  11M     0 100% /vendor_dlkm
/dev/block/dm-2                106M 106M     0 100% /product
tmpfs                          963M 8.0K  963M   1% /apex
tmpfs                          963M 488K  962M   1% /linkerconfig
/dev/block/mmcblk0p21           10M 120K   10M   2% /metadata
/dev/block/mmcblk0p22           80M  54M   26M  67% /treadahead
/dev/block/mmcblk0p26           16M    0   16M   0% /oem
/dev/block/mmcblk0p27           64M 4.0K   64M   1% /Reserve0
/dev/block/dm-5                 26G 1.8G   24G   8% /data
tmpfs                          963M    0  963M   0% /data_mirror
/dev/fuse                       26G 1.8G   24G   8% /mnt/user/0/emulated
/dev/block/vold/public:179,1 128M  18M  109M  15% /mnt/media_rw/extsd
/dev/fuse                    128M  18M  109M  15% /mnt/user/0/extsd
```

2.8. 烧写 Android 镜像到 eMMC 的方法

开发板的 Android 镜像只能在 Windows 平台下使用 PhoenixCard 软件烧录到 eMMC 中，PhoenixCard 软件的版本必须为 PhonixCard-4.2.8。

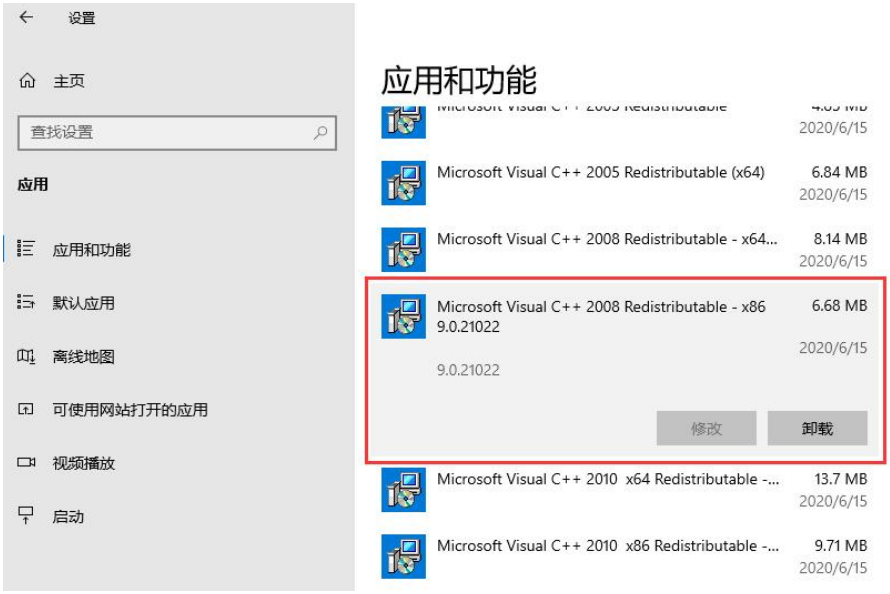
请不要用烧录 Linux 镜像的软件，如 Win32Diskimager 或者 balenaEtcher 来烧录安卓镜像。

另外 PhoenixCard 这款软件没有 Linux 和 Mac 平台的版本，所以在 Linux 和 Mac 平台下是无法烧录安卓镜像到 eMMC 中的。

在进行下面操作前，请确保开发板贴有 eMMC。

1) 首先请注意，此方法需要借助 TF 卡来完成，主要分为如下两步
   a. 先使用 PhoenixCard 将 Android 固件以量产卡的方式烧录到 TF 卡中
   b. 然后使用 TF 卡将 Android 固件烧录到 eMMC 中

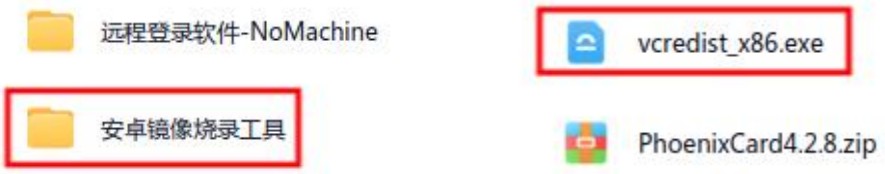
2) 请确保 Windows 系统已经安装了 **Microsoft Visual C++ 2008 Redistrbutable - x86**



3) 如果没有安装 **Microsoft Visual C++ 2008 Redistrbutable - x86**，使用 PhoenixCard 格式化 TF 卡或者烧录 Android 镜像会提示下面的错误



4) **Microsoft Visual C++ 2008 Redistrbutable - x86** 的安装包可以从 Orange Pi Zero 3w 的官方工具中下载到，也可以去微软官网下载



5) 然后准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡

6) 然后使用读卡器把 TF 卡插入电脑

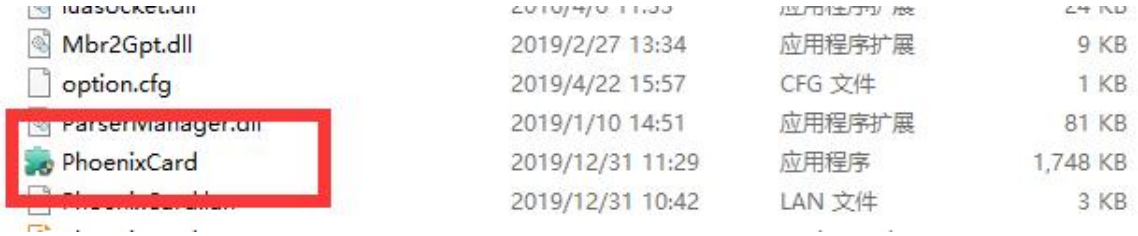
7) 从 Orange Pi 的资料下载页面下载 Android 镜像和 PhoenixCard 烧写工具，请确保 PhonenixCrad 工具的版本为 PhonixCard-4.2.8，请不要用低于 4.2.8 版本的 PhonixCard 软件来烧录 Android 镜像，低于这个版本的 PhonixCard 工具烧写的 Android 镜像可能会有问题



8) 然后使用解压软件解压下载的 Android 镜像的压缩包，解压后的文件中，以“**.img**”结尾的文件就是 Android 镜像文件，大小在 1GB 以上。如果不知道怎么解压 Android 镜像的压缩包，可以安装一个 360 压缩软件来解压镜像。



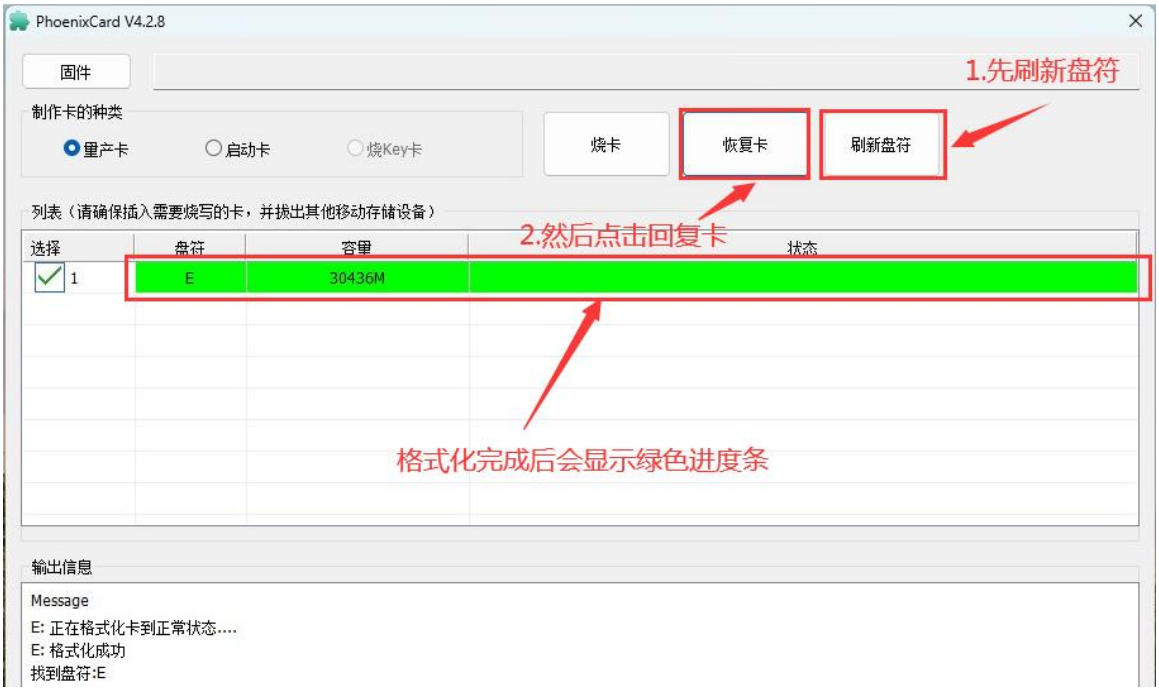
9) 然后使用解压软件解压 **PhonixCard4.2.8.zip**，此软件无需安装，在解压后的文件夹中找到 PhoenixCard 打开即可



10) 打开 PhoenixCard 后，如果 TF 卡识别正常，会在中间的列表中显示 TF 卡的盘符和容量，**请务必确认显示的盘符和你想烧录的 TF 卡的盘符是一致的**，如果没有显示可以尝试拔插下 TF 卡，或者点击 PhoenixCard 中的“**刷新盘符**”按钮



11) 确认完盘符后，先格式化 TF 卡，点击 PhoenixCard 中“**恢复卡**”按钮即可（如果“**恢复卡**”按钮为灰色的无法按下，可以先点击下“**刷新盘符**”按钮）



如果格式化有问题，请尝试拔插下 TF 卡后再测试，如果重新拔插 TF 卡后还是

有问题，可以重启下 Window 电脑或者换一台电脑再试下。

12) 然后开始将 Android 镜像写入 TF 卡

a. 首先在“**固件**”一栏中选择 Android 镜像的路径

b. 在“**制作卡的种类**”中选择“**量产卡**”

c. 然后点击“**烧卡**”按钮就会开始烧录



13) 烧录完后 PhoenixCard 的显示如下图所示，此时点击“**关闭**”按钮即可退出 PhoenixCard



14) 然后将 TF 卡插入开发板，上电启动开发板后会自动将 TF 卡中的 Android 固件烧录到开发板的 eMMC 中，烧录完成后会自动关机，开发板上的 LED 灯会熄灭

15) 此时就可以拔出 TF 卡，然后重新上电，就会启动 eMMC 中的 Android 系统了

2.9. 通过 TypeC 口烧录 Android 镜像到 eMMC 或 UFS 的方法

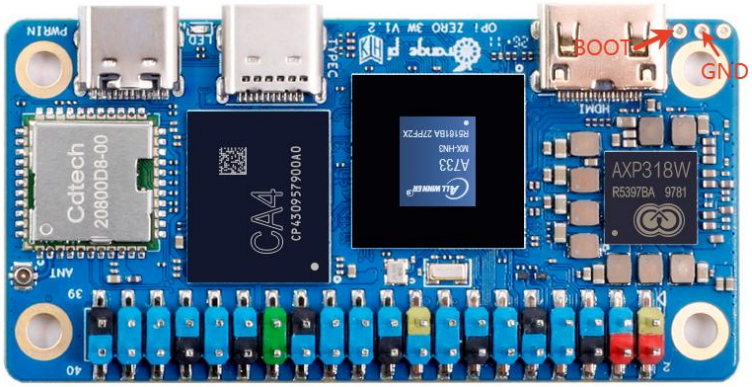
注意，下面所有的操作都是在 Windows 电脑中进行的，另外需要确保板子上已经贴有 eMMC 或者 UFS。

1) 首先需要准备一根品质良好的 Type-C 接口的数据线

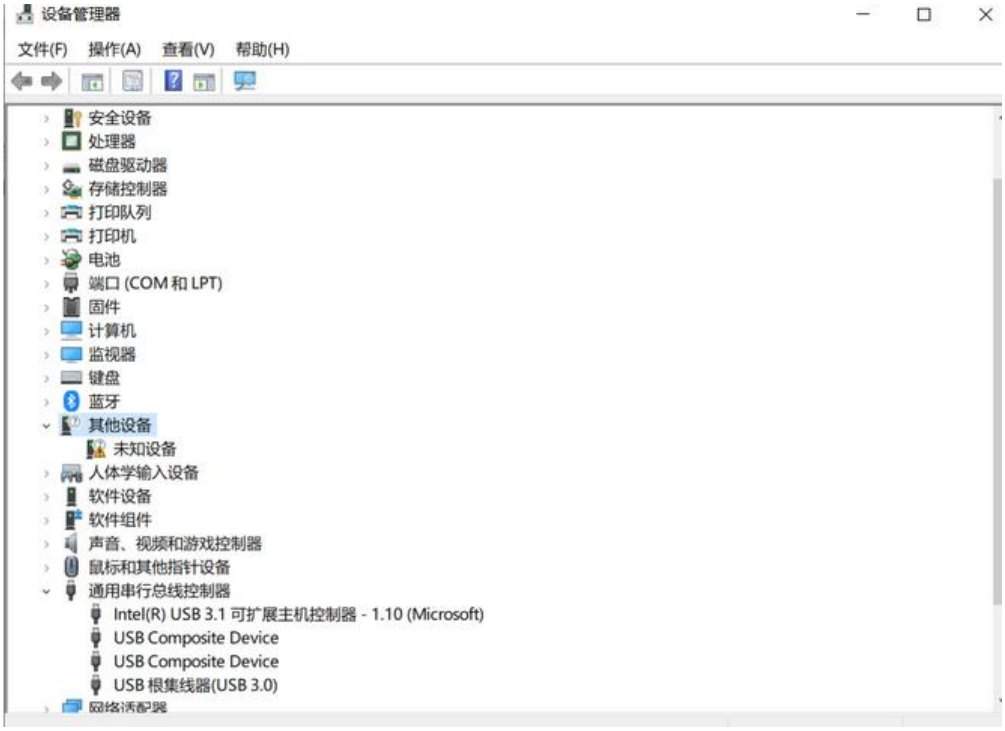


2）然后从 Orange Pi 的资料下载页面下载全志驱动 **AllwinnerUSBFlashDeviceDriver.zip** 和烧录工具 **AllwinnertechPhoeniSuit.zip**

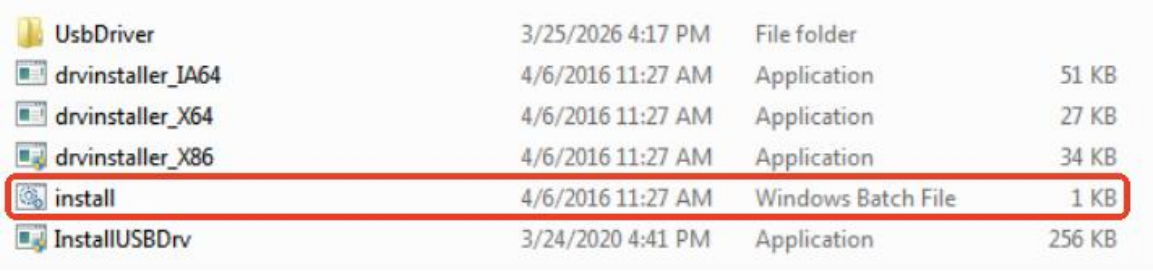
3）然后短接下图箭头指向的两个测试点，目的是将 BOOT 与 GND 短接进入 FEL 模式（**如果板子上没有系统不短接也可以**）



4）然后通过 TypeC 数据线连接开发板和电脑（**注意需要短接 BOOT 和 GND 再连接开发板和电脑上电**），此时电脑的设备管理器应该能够发现一个未知设备



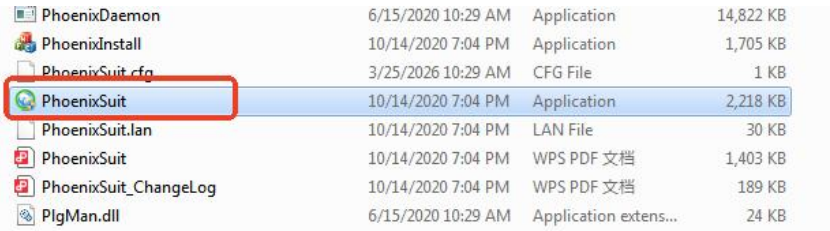
5）然后解压 **AllwinnerUSBFlashDeviceDriver.zip**，如果是 win7 系统，在解压后的文件夹中找到 **installe** 脚本可执行文件并打开即可开始安装

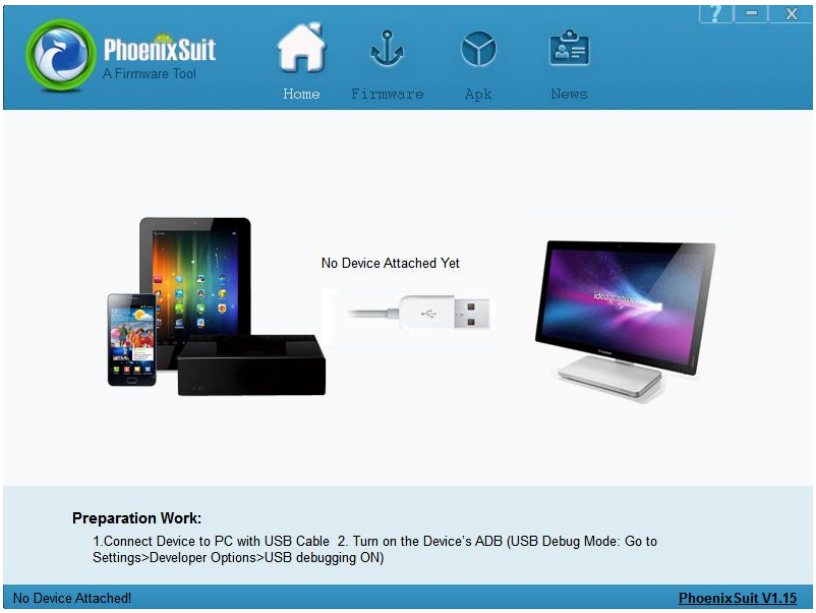


6) 如果是 win10 系统，需要鼠标右键点击未知设备，在弹出的对话框里点击浏**览我的计算机以查找驱动程序软件**，选择上面的驱动路径进行安装

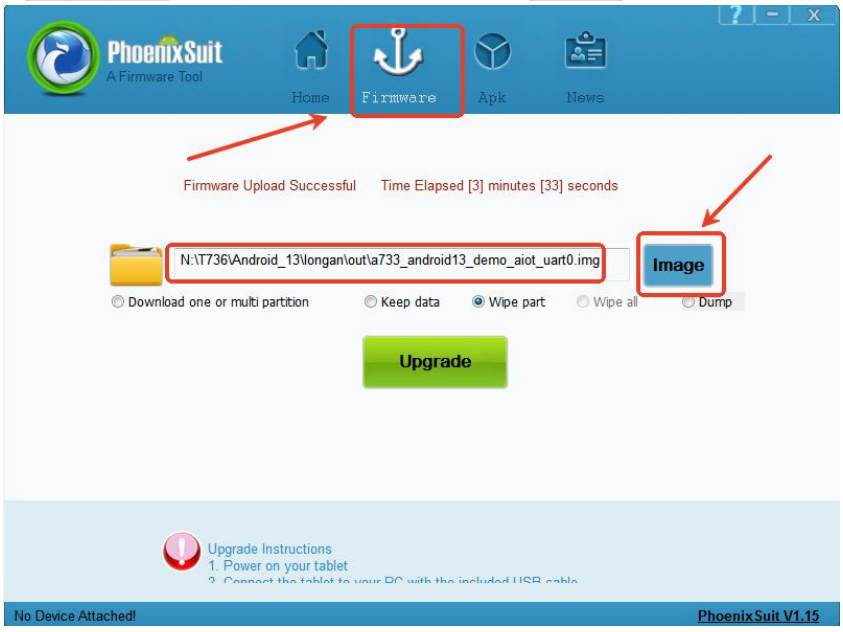


7) 全志 USB 驱动安装完成后，我们解压 **AllwinnertechPhoeniSuit.zip** 并打开 **PhoenixSuit** 烧录程序

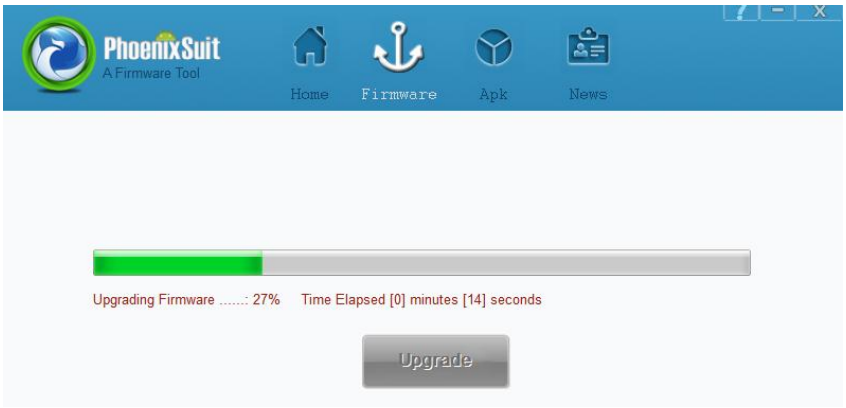




8) 然后点击 **Firmware** 切到刷机窗口，然后点击 **Image** 选择安卓镜像路径



9) 然后我们先断开 TypeC 数据线和电脑的连接，再短接 FEL 和 GND，再次连接开发板和电脑，此时工具会自动将镜像烧录到 eMMC 或者 UFS，烧录完成后板子会自动启动 Android 系统



2.10. 启动香橙派开发板

1) 将烧录好镜像的 TF 卡插入香橙派开发板的 TF 卡插槽中

2) 开发板有 Mini HDMI 接口，可以通过 Mini HDMI 转 HDMI 连接线把开发板连接到电视或者 HDMI 显示器。如果有 Type-C 转 HDMI 的线，也可以通过 Type-C 接口来显示开发板的系统界面。

3) 接上 USB 鼠标和键盘，用于控制香橙派开发板

4) 连接一个 5V/3A（5V/4A 的也可以）的 USB Type C 接口的**高品质**的电源适配器

切记不要插入电压输出大于 5V 的电源适配器，会烧坏开发板。

系统上电启动过程中很多不稳定的现象基本都是供电有问题导致的，所以一个靠谱的电源适配器很重要。如果启动过程中发现有不断重启的现象，请更换下电源或者 Type C 数据线再试下。

5) 然后打开电源适配器的开关，如果一切正常，此时 HDMI 显示器就能看到系统的启动画面了

6) 如果想通过调试串口查看系统的输出信息，请使用串口线将开发板连接到电脑，串口的连接方法请参看调试串口的使用方法一节

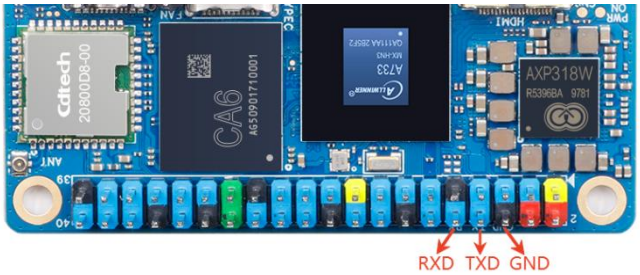
2.11. 调试串口的使用方法

2.11.1. 调试串口的连接说明

1) 首先需要准备一个 **3.3v** 的 USB 转 TTL 模块，然后将 USB 转 TTL 模块的 USB 接口一端插入到电脑的 USB 接口中



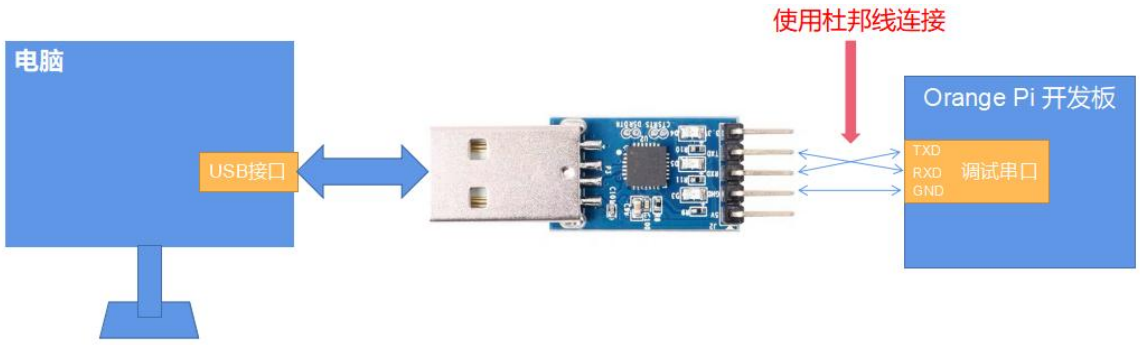
2) 开发板的调试串口 GND、TX 和 RX 引脚的对应关系如下图所示



3) USB 转 TTL 模块 GND、TX 和 RX 引脚需要通过杜邦线连接到开发板的调试串口上

a. USB 转 TTL 模块的 GND 接到开发板的 GND 上
b. USB 转 TTL 模块的 **RX 接到开发板的 TX 上**
c. USB 转 TTL 模块的 **TX 接到开发板的 RX 上**

4) USB 转 TTL 模块连接电脑和 Orange Pi 开发板的示意图如下所示



USB转TTL模块连接电脑和 Orange Pi 开发板的示意图

串口的 TX 和 RX 是需要交叉连接的，如果不想仔细区分 TX 和 RX 的顺序，可以把串口的 TX 和 RX 先随便接上，如果测试串口没有输出再交换下 TX 和 RX 的顺序，这样就总有一种顺序是对的。

2.11.2. Ubuntu 平台调试串口的使用方法

Linux 下可以使用的串口调试软件有很多，如 putty、minicom 等，下面演示下 putty 的使用方法。

1) 首先将 USB 转 TTL 模块插入 Ubuntu 电脑的 USB 接口，如果 USB 转 TTL 模块连接识别正常，在 Ubuntu PC 的**/dev** 下就可以看到对应的设备节点名，记住这个节点名，后面设置串口软件时会用到

```
test@test:~$ ls /dev/ttyUSB*
/dev/ttyUSB0
```

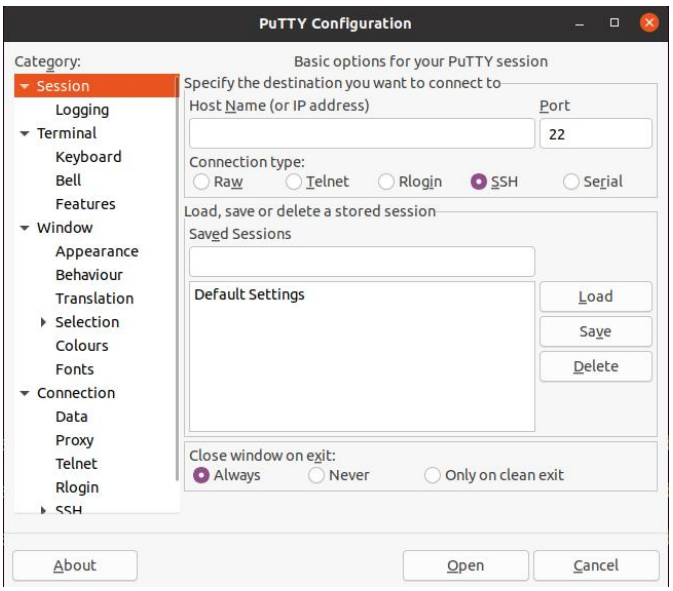
2) 然后使用下面的命令在 Ubuntu PC 上安装下 putty

```
test@test:~$ sudo apt update
test@test:~$ sudo apt install -y putty
```

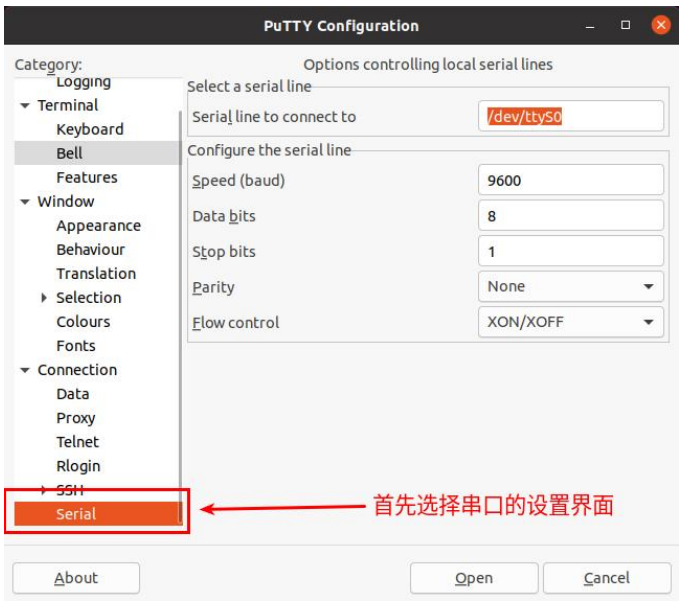
3) 然后运行 putty，**记得加 sudo 权限**

```
test@test:~$ sudo putty
```

4) 执行 putty 命令后会弹出下面的界面



5) 首先选择串口的设置界面

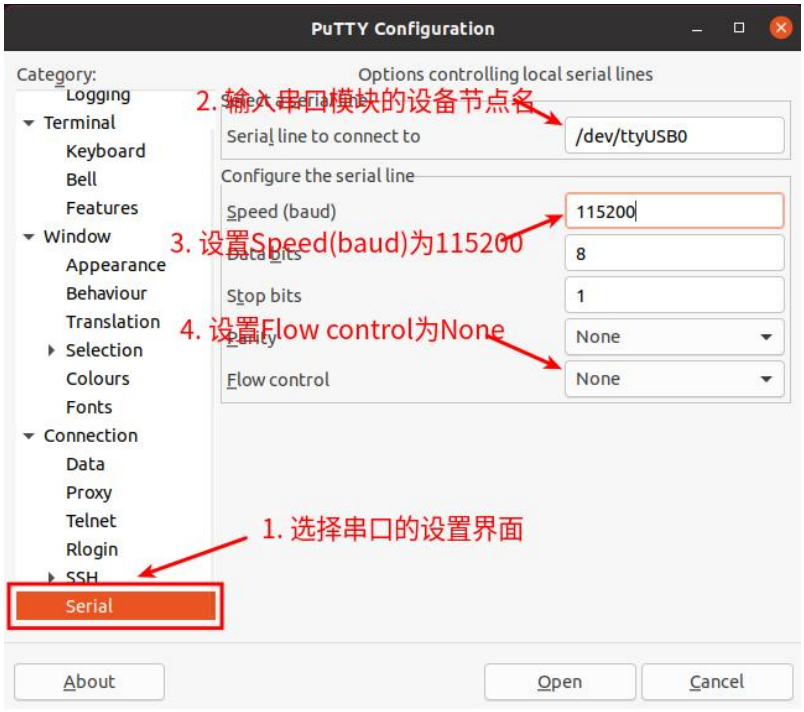


6) 然后设置串口的参数

a. 设置 **Serial line to connect to** 为**/dev/ttyUSB0**（修改为对应的节点名，一般为**/dev/ttyUSB0**）

b. 设置 **Speed(baud)**为 **115200**（串口的波特率）

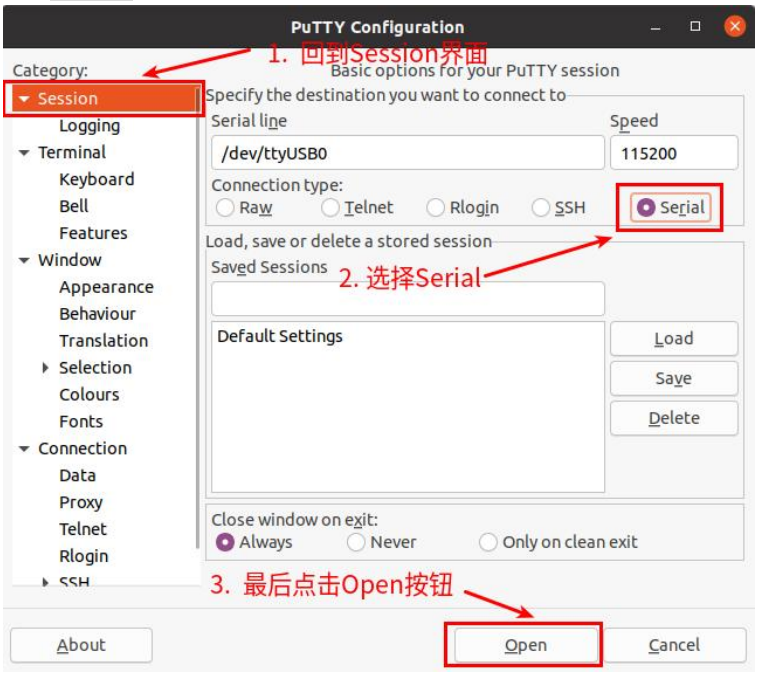
c. 设置 **Flow control** 为 **None**



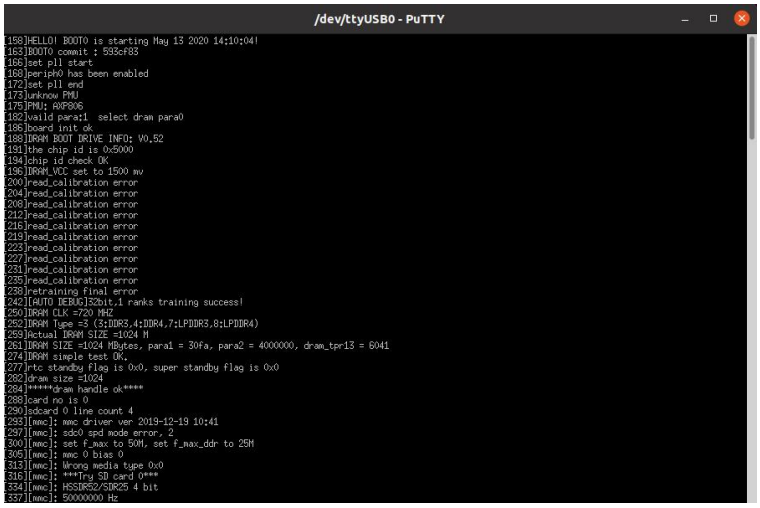
7) 在串口的设置界面设置完后，再回到 Session 界面

a. 首先选择 **Connection type** 为 **Serial**

b. 然后点击 **Open** 按钮连接串口



8) 然后启动开发板，就能从打开的串口终端中看到系统输出的 Log 信息了



2.11.3. Windows 平台调试串口的使用方法

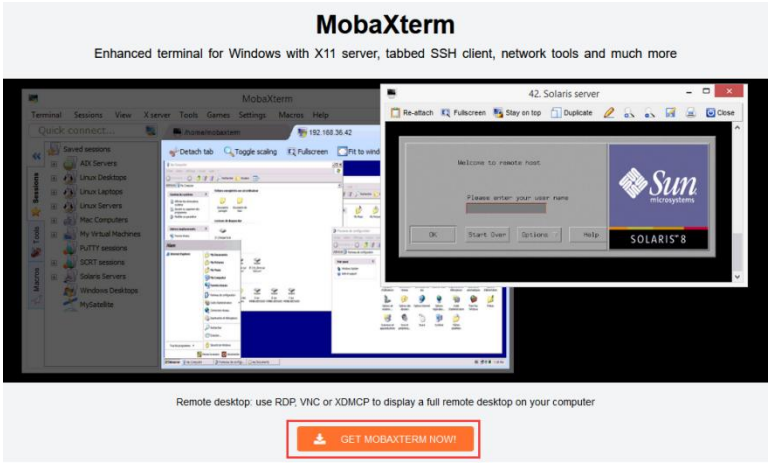
Windows 下可以使用的串口调试软件有很多，如 SecureCRT、MobaXterm 等，下面演示 MobaXterm 的使用方法，这款软件有免费版本，无需购买序列号即可使用。

1) 下载 MobaXterm

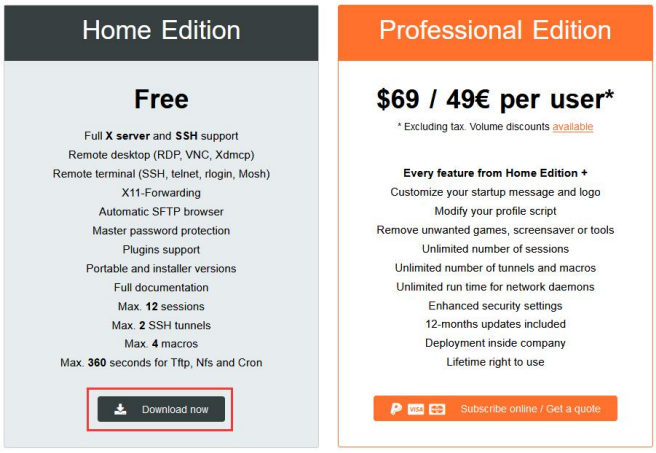
a. 下载 MobaXterm 网址如下

https://mobaxterm.mobatek.net/

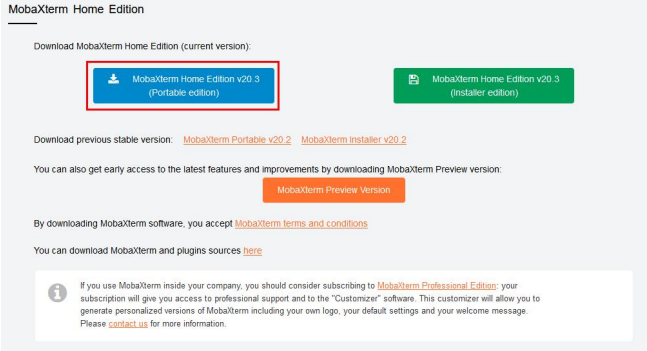
b. 进入 MobaXterm 下载网页后点击 **GET XOBATERM NOW!**



c. 然后选择下载 Home 版本



d. 然后选择 Portable 便携式版本，下载完后无需安装，直接打开就可以使用

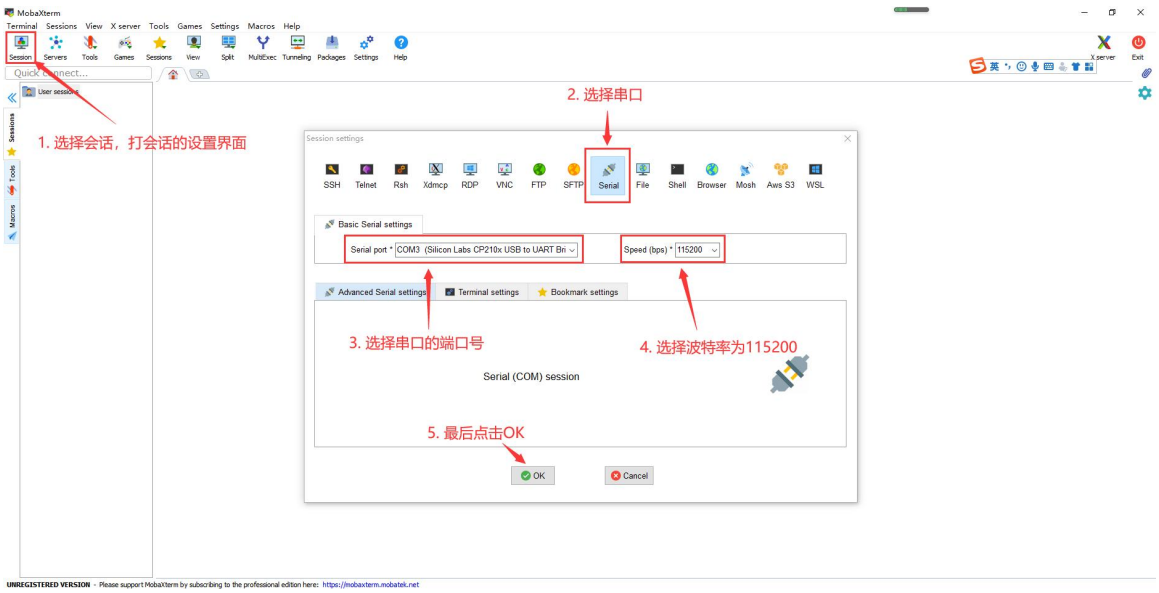


2) 下载完后使用解压缩软件解压下载的压缩包，即可得到 MobaXterm 的可执软件，然后双击打开

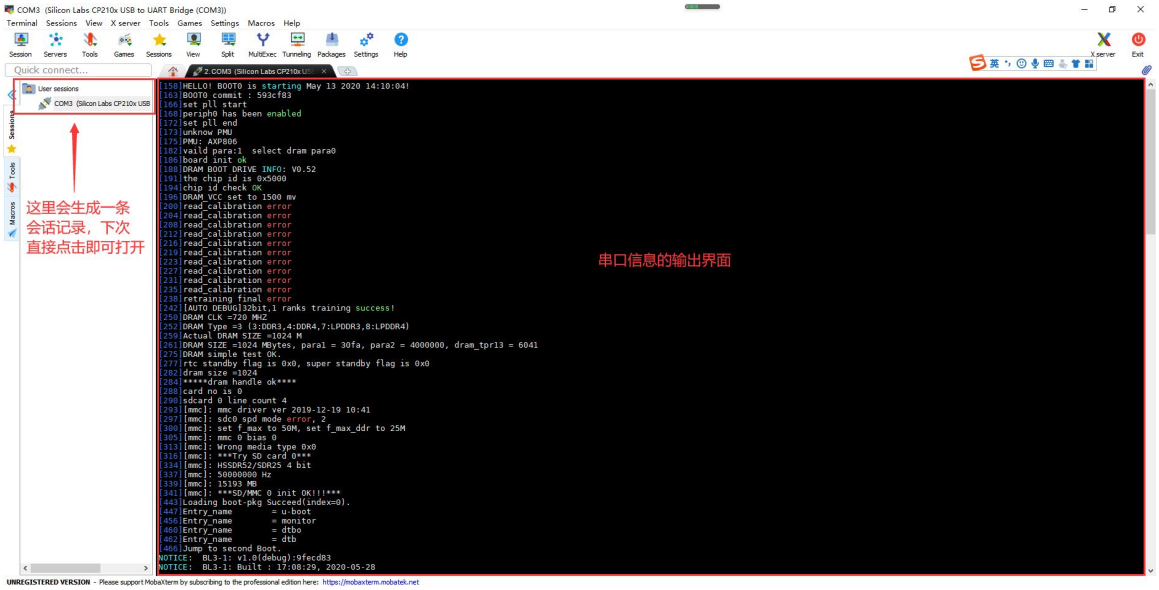


3) 打开软件后，设置串口连接的步骤如下

a. 打开会话的设置界面
b. 选择串口类型
c. 选择串口的端口号（根据实际的情况选择对应的端口号），如果看不到端口号，请使用 **360 驱动大师**扫描安装 USB 转 TTL 串口芯片的驱动
d. 选择串口的波特率为 **115200**
e. 最后点击“**OK**”按钮完成设置



4) 点击“**OK**”按钮后会进入下面的界面，此时启动开发板就能看到串口的输出信息了



2.12. 使用开发板 40 pin 接口中的 5v 引脚供电说明

我们推荐的开发板的供电方式是使用 5V/3A 的 Type C 接口的电源线插到开发板的 Type C 电源接口来供电的。如果需要使用 40 pin 接口中的 5V 引脚来给开发板供电，请确保使用的电源线能满足开发板的供电需求。如果有使用不稳定的情况，

请换回 Type C 电源供电。

1) 首先需要准备一根下图所示的电源线



上图所示的电源线在淘宝可以买到，请自行搜索购买。

2) 使用 40 pin 接口中的 5V 引脚来给开发板供电，电源线的接法如下所示

a. 上图所示的电源线 USB A 口需要插到 5V/3A 的电源适配器接头上（**不建议插到电脑的 USB 接口来供电，如果开发板接的外设过多，使用会不稳定**）

b. 红色的杜邦线需要插到开发板 40 pin 接口的 5V 引脚上

c. 黑色的杜邦线需要插到 40 pin 接口的 GND 引脚上

d. 40 pin 接口 5V 引脚和 GND 引脚在开发板中的位置如下图所示，切记不要接反了



3. Debian/Ubuntu Server 和 Xfce 桌面系统使用说明

3.1. 已支持的 linux 镜像类型和内核版本

Linux 镜像类型	内核版本	桌面版
Ubuntu 22.04 - Jammy	Linux5.15	支持
Debian 12 - Bookworm	Linux5.15	支持
Debian 11 - Bullseye	Linux5.15	支持
Ubuntu 22.04 - Jammy	Linux6.6	支持
Debian 12 - Bookworm	Linux6.6	支持
Debian 11 - Bullseye	Linux6.6	支持

在 **Orange Pi 的资料下载页面**进入对应开发板的下载页面后可以看到下面的下载选项，在下文的描述中，**Ubuntu 镜像和 Debian 镜像一般统称为 Linux 镜像**。



Linux 镜像的命名规则为：

开发板型号_版本号_Linux 发行版类型_发行版代号_服务器或桌面_内核版本

a. **开发板的型号**：都是 **Orangepizero3w**。不同开发板的型号名一般都是不同的，烧录镜像前，请确保所选择镜像的这个型号名和开发板是匹配的。

b. **版本号**：如 **1.x.x**，这个版本号会随着镜像功能的更新而递增，另外开发板 Linux 镜像的版本号最后一个数字都是偶数。

c. **Linux 发行版的类型**：目前支持 **Ubuntu** 和 **Debian**。由于 Ubuntu 源自 Debian，所以两个系统在使用上来说总体区别不大。但部分软件的默认配置和命令的使用上还是有些许区别的，另外 Ubuntu 和 Debian 都各自有维护所支持的软件仓库，在支持的可安装的软件包上也是有些许差异的。这些需要亲自去使用体验才会有比较深刻的认识。有关更多的细节，可以参考下 Ubuntu 和 Debian 官方提供的文档。

d. **发行版代号**：用来区分 Ubuntu 或者 Debian 这样具体的 Linux 发行版的不同版本。其中 **jammy** 是 Ubuntu 发行版，jammy 表示 Ubuntu22.04，不同版本

的最大的区别是新版本的 Ubuntu 系统维护的软件仓库的中的软件很多都比旧版本的 Ubuntu 系统中的要新，比如 Python 和 GCC 编译工具链等。**bookworm** 是 Debian 的具体版本代号，**bookworm** 表示 Debian12。

e. **服务器或桌面**：用来表示系统是否带桌面环境，如果为 **server** 就表示系统没有安装桌面环境，镜像占用的存储空间和资源比较小，主要使用命令行来操作控制系统。如果为 **desktop_gnome** 就表示系统默认安装有 GNOME 桌面环境，镜像占用的存储空间和资源比较大，可以接显示器和鼠标键盘通过界面来操作系统。当然 desktop 版本的系统也可以像 server 版本的系统一样通过命令行来操作。

f. **内核版本**：用来表示 linux 内核的版本号，目前支持 **linux5.15**。

3.2. linux 内核驱动适配情况

功能	Linux5.15	Linux6.6
HDMI 视频	OK	OK
HDMI 音频	OK	OK
TypeC USB2.0	OK	OK
TypeC USB3.0	OK	OK
TypeC DP 显示	OK	OK
TF 卡启动	OK	OK
eMMC	OK	OK
UFS	OK	OK
WIFI	OK	OK
蓝牙	OK	OK
CAM1	OK	OK
CAM2	OK	OK
MIPI LCD	OK	OK
LED 灯	OK	OK
FAN 散热风扇	OK	OK
40 pin GPIO	OK	OK
40 pin I2C	OK	OK
40 pin SPI	OK	OK
40 pin UART	OK	OK
40 pin PWM	OK	OK

温度传感器	OK	OK
硬件看门狗	OK	OK
GPU	仅 Debian bullseye 支持	仅 Debian bullseye 支持
NPU	OK	OK
视频编解码	仅 Debian bullseye 支持	仅 Debian bullseye 支持

3.3. 本手册 linux 命令格式说明

1) 本手册中所有需要在 Linux 系统中输入的命令都会使用下面的方框框起来

```
```

如下所示，黄色方框里内容表示需要特别注意的内容，这里面的命令除外。

```
```

2) 命令前面的提示符类型说明

a. 命令前面提示符指的是下面方框内红色部分的内容，这部分内容不是 linux 命令的一部分，所以在 linux 系统中输入命令时，请不要把红色字体部分的内容也输入进去。

```
orangepi@orangepi:~$ sudo apt update
root@orangepi:~# vim /boot/boot.cmd
test@test:~$ ssh root@192.168.1.xxx
root@test:~# ls
```

b. **orangepi@orangepi:~$** 提示符表示这个命令是在**开发板的 linux 系统**中输入的，提示符最后的 **$** 表示系统当前用户为普通用户，当执行特权命令时，需要加上 **sudo**

c. **root@orangepi:~#** 提示符表示这个命令是在**开发板的 linux 系统**中输入的，提示符最后的 **#** 表示系统当前用户为 root 用户，可以执行任何想要执行的命令

d. **test@test:~$** 提示符表示这个命令是在 Ubuntu PC 或者 Ubuntu 虚拟机中输入的，而不是开发板的 linux 系统中。提示符最后的 **$** 表示系统当前用户为普通用户，当执行特权命令时，需要加上 **sudo**

e. **root@test:~#** 提示符表示这个命令是在 Ubuntu PC 或者 Ubuntu 虚拟机中输入的，而不是开发板的 linux 系统中。提示符最后的 **#** 表示系统当前用户为 root 用户，可以执行任何想要执行的命令

3) 哪些是需要输入的命令？

a. 如下所示，**黑色加粗部分**是需要输入的命令，命令下面的是输出的内容（有些命令有输出，有些可能没有输出），这部分内容是不需要输入的

```
root@orangepi:~# cat /boot/orangepiEnv.txt
verbosity=7
bootlogo=false
console=serial
```

b. 如下所示，有些命令一行写不下会放到下一行，只要黑色加粗的部分就都是需要输入的命令。当这些命令输入到一行的时候，每行最后的“\”是需要去掉的，这个不是命令的一部分。另外命令的不同部分都是有空格的，请别漏了

```
orangepi@orangepi:~$ echo  \
"deb [arch=$(dpkg --print-architecture)  \
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]  \
https://download.docker.com/linux/debian  \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

3.4. linux 系统登录说明

3.4.1. linux 系统默认登录账号和密码

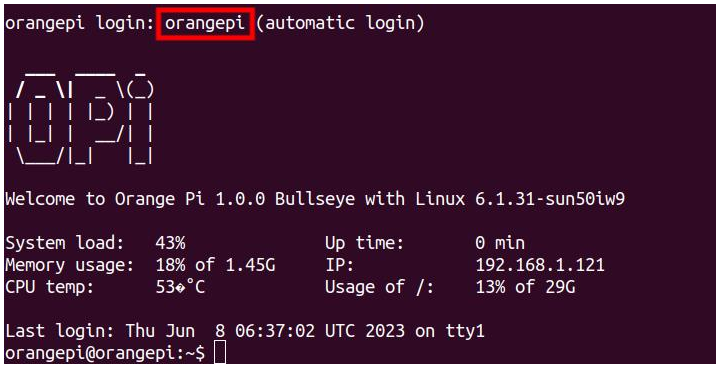
账号	密码
root	**orangepi**
orangepi	**orangepi**

注意，输入密码的时候，屏幕上是不会显示输入的密码的具体内容的，请不要以为是有什么故障，输入完后直接回车即可。

当输入密码提示错误，或者 ssh 连接有问题，请注意，只要使用的是 Orange Pi 提供的 Linux 镜像，就请不要怀疑上面的密码不对，而是要找其它的原因。

3.4.2. 设置 linux 系统终端自动登录的方法

1) linux 系统默认就是自动登录终端的，默认登录的用户名是 **orangepi**



2) 使用下面的命令可以设置 root 用户自动登录终端

```
orangepi@orangepi:~$ sudo auto_login_cli.sh root
```

3) 使用下面的命令可以禁止自动登录终端

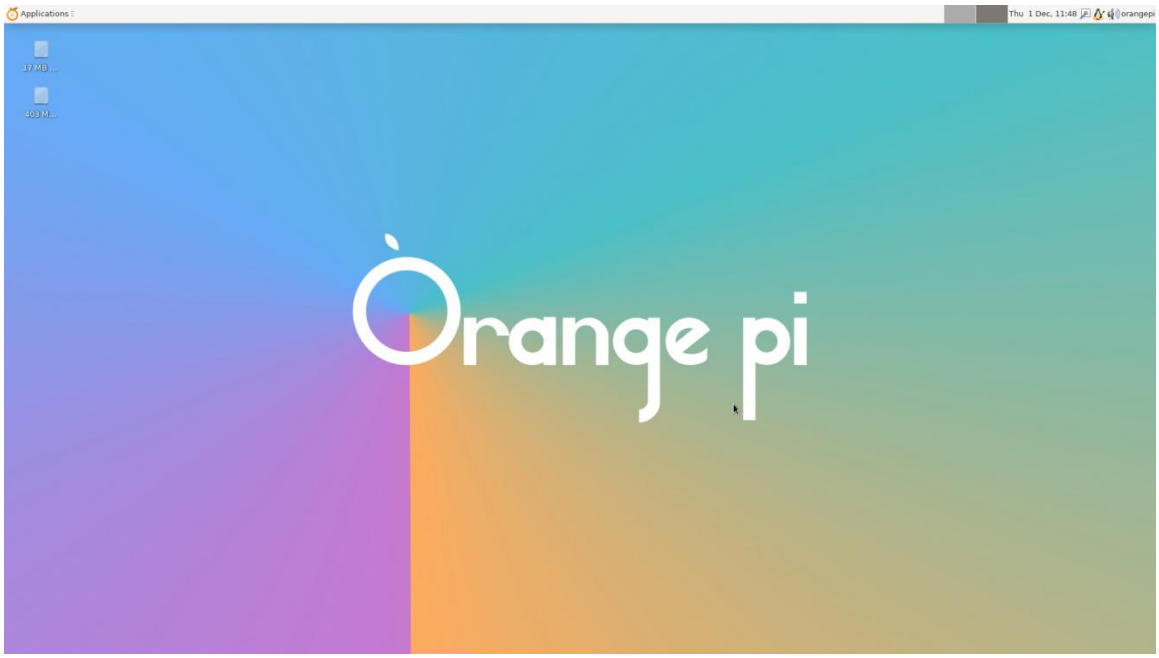
```
orangepi@orangepi:~$ sudo auto_login_cli.sh -d
```

4) 使用下面的命令可以再次设置 orangepi 用户自动登录终端

```
orangepi@orangepi:~$ sudo auto_login_cli.sh orangepi
```

3.4.3. linux 桌面版系统自动登录说明

1) 桌面版系统启动后会自动登录进入桌面，无需输入密码



3.4.4. Linux 桌面版系统 root 用户自动登录的设置方法

1) 执行下面的命令可以设置桌面版系统使用 root 用户自动登录

```
orangepi@orangepi:~$ sudo desktop_login.sh root
```

2) 然后重启系统，就会自动使用 root 用户登录桌面了

3) 执行下面的命令可以再次设置桌面版系统使用 orangepi 用户自动登录

```
orangepi@orangepi:~$ sudo desktop_login.sh orangepi
```

3.5. 板载 LED 灯测试说明

1) 开发板上只有一个红色的 LED 灯，所在位置如下图所示：



2) 红色的 LED 灯在内核启动后会一直闪烁，这是由软件控制的。

3) 设置红灯亮灭和闪烁的方法如下所示：

注意，下面的操作请在 root 用户下进行。

a. 首先进入红灯的设置目录

```
root@orangepi:~# cd /sys/class/leds/status_led
```

b. 设置红灯停止闪烁的命令如下

```
root@orangepi:/sys/class/leds/status_led# echo none > trigger
```

c. 设置红灯常亮的命令如下

```
root@orangepi:/sys/class/leds/status_led# echo default-on > trigger
```

d. 设置红灯闪烁的命令如下

```
root@orangepi:/sys/class/leds/status_led# echo heartbeat > trigger
```

3.6. TF 卡中 linux 系统 rootfs 分区容量操作说明

3.6.1. 第一次启动会自动扩容 TF 卡中 rootfs 分区的容量

1) 将开发板的 Linux 镜像烧录到 TF 卡中后，可以在 **Ubuntu 电脑**中查看下 TF 卡容量的使用情况，步骤如下所示：

注意，这一步不操作是不影响开发板的 Linux 系统自动扩容的。这里只是想说明 TF 卡烧录完 Linux 镜像后，怎么查看 TF 卡容量的方法。

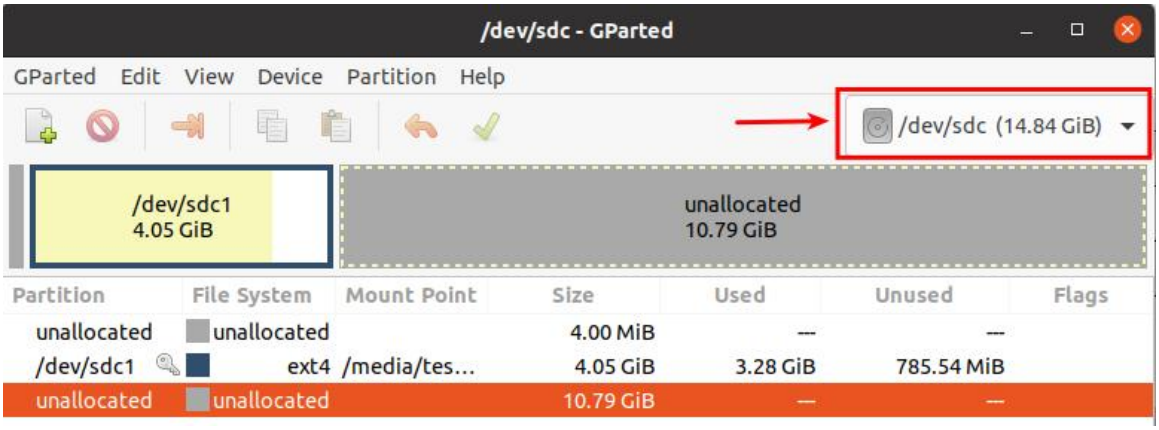
a. 首先在 Ubuntu 电脑中安装下 gparted 这个软件

```
test@test:~$ sudo apt install -y gparted
```

b. 然后打开 gparted

```
test@test:~$ sudo gparted
```

c. 打开 gparted 后在右上角可以选择 TF 卡，然后就可以看到 TF 卡容量的使用情况



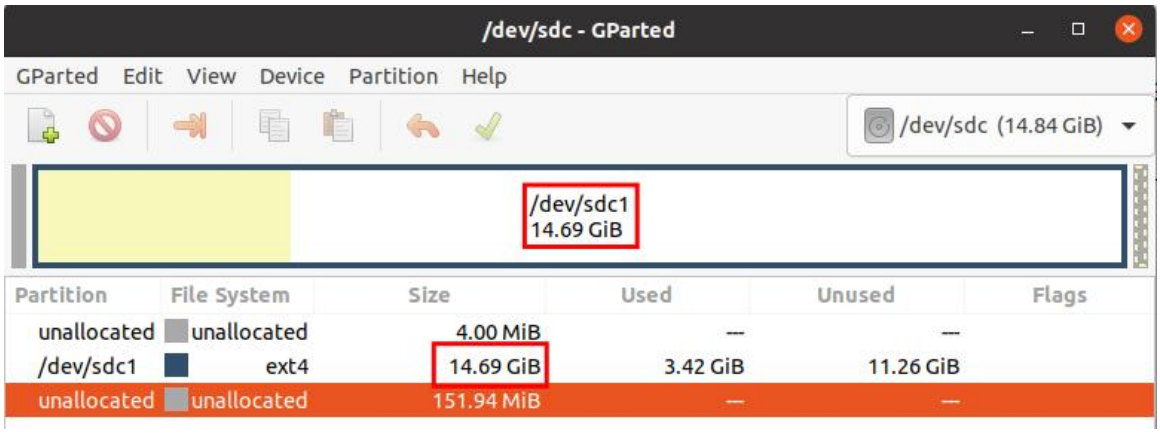
d. 上图显示的是烧录完 Linux 桌面版系统后 TF 卡的情况，可以看到，虽然 TF 卡的总容量是 16GB 的（在 GParted 中显示为 14.84GiB），但是 rootfs 分区（/dev/sdc1）实际只分配了 4.05GiB，还剩下 10.79GiB 未分配

2) 然后可以将烧录好 Linux 系统的 TF 卡插入开发板中启动，TF 卡第一次启动 linux 系统时会通过 **orangepi-resize-filesystem.service** 这个 systemd 服务来调用 **orangepi-resize-filesystem** 脚本自动进行 rootfs 分区的扩容，所以**无需再手动扩容**

3) 登录系统后可以通过 **df -h** 命令来查看 rootfs 的大小，如果和 TF 卡的实际容量一致，说明自动扩容运行正确

```
orangepi@orangepi:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            430M     0  430M   0% /dev
tmpfs           100M  5.6M   95M   6% /run
/dev/mmcblk0p1   15G   915M   14G   7% /
tmpfs           500M     0  500M   0% /dev/shm
```

4) 第一次启动完 Linux 系统后，我们还可以将 TF 卡从开发板中取下来重新插入 **Ubuntu 电脑**，然后再次使用 gparted 查看下 TF 卡的情况，如下图所示，rootfs 分区（/dev/sdc1）的容量已经扩展到了 14.69GiB 了



需要注意的是，linux 系统只有一个 ext4 格式的分区，没有使用单独的 BOOT 分区来存放内核镜像等文件，所以也就不存在 BOOT 分区扩容的问题。

3.6.2. 禁止自动扩容 TF 卡中 rootfs 分区容量的方法

1) 首先在 **Ubuntu 电脑**（Windows 不行）中将开发板的 linux 镜像烧录到 TF 卡中，**然后重新拔插下 TF 卡**

2) 然后 Ubuntu 电脑一般会自动挂载 TF 卡的分区，如果自动挂载正常，使用 ls 命令可以看到下面的输出

```
test@test:~$ ls /media/test/opi_root/
bin  boot  dev  etc  home  lib  lost+found  media  mnt  opt  proc  root  run
sbin  selinux  srv  sys  tmp  usr  var
```

3) 然后在 Ubuntu 电脑中将当前用户切换成 root 用户

```
test@test:~$ sudo -i
```

```
[sudo] test 的密码：
root@test:~#
```

4) 然后进入 TF 卡中的 linux 系统的 root 目录下新建一个名为**.no_rootfs_resize** 的文件

```
root@test:~# cd /media/test/opi_root/
root@test:/media/test/opi_root/# cd root
root@test:/media/test/opi_root/root# touch .no_rootfs_resize
root@test:/media/test/opi_root/root# ls .no_rootfs*
.no_rootfs_resize
```

5) 然后就可以卸载 TF 卡，再拔出 TF 卡插到开发板中启动，linux 系统启动时，当检测到**/root** 目录下有**.no_rootfs_resize** 这个文件就不会再自动扩容 rootfs 了

6) 禁止 rootfs 自动扩容后进入 Linux 系统可以看到 rootfs 分区的总容量只有 4GB（这里测试的是桌面版本的镜像），远小于 TF 卡的实际容量，说明禁止 rootfs 自动扩容成功

```
orangepi@orangepi:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            925M     0  925M   0% /dev
tmpfs           199M  3.2M  196M   2% /run
/dev/mmcblk0p1  4.0G  3.2G  686M  83% /
```

7) 如果需要重新扩容 TF 卡中 rootfs 分区的容量，只需要执行下面的命令，然后重新启动开发板的 Linux 系统即可

注意，请在 root 用户下执行下面的命令。

```
root@orangepi:~# rm /root/.no_rootfs_resize
root@orangepi:~# systemctl enable orangepi-resize-filesystem.service
root@orangepi:~# sudo reboot
```

重启后再次进入开发板的 Linux 系统就可以看到 rootfs 分区已经扩展为 TF 卡的实际容量了

```
root@orangepi:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            925M     0  925M   0% /dev
```

```
tmpfs           199M  3.2M  196M   2% /run
/dev/mmcblk0p1   15G  3.2G   12G  23% /
```

3.6.3. 手动扩容 TF 卡中 rootfs 分区容量的方法

如果 TF 卡的总容量很大，比如为 128GB，不想 Linux 系统 rootfs 分区使用 TF 卡所有的容量，只想分配一部分容量，比如 16GB，给 Linux 系统使用，然后 TF 卡的剩余容量就可以用作其他用途。那么可以使用此小节介绍的内容来手动扩容 TF 中 rootfs 分区的容量。

1) 首先在 **Ubuntu 电脑**（Windows 不行）中将开发板的 linux 镜像烧录到 TF 卡中，**然后重新拔插下 TF 卡**

2) 然后 Ubuntu 电脑一般会自动挂载 TF 卡的分区，如果自动挂载正常，使用 **ls** 命令可以看到下面的输出

```
test@test:~$ ls /media/test/opi_root/
bin  boot  dev  etc  home  lib  lost+found  media  mnt  opt  proc  root  run
sbin  selinux  srv  sys  tmp  usr  var
```

3) 然后在 Ubuntu 电脑中将当前用户切换成 root 用户

```
test@test:~$ sudo -i
[sudo] test 的密码：
root@test:~#
```

4) 然后进入 TF 卡中的 linux 系统的 root 目录下新建一个名为**.no_rootfs_resize** 的文件

```
root@test:~# cd /media/test/opi_root/
root@test:/media/test/opi_root/# cd root
root@test:/media/test/opi_root/root# touch .no_rootfs_resize
root@test:/media/test/opi_root/root# ls .no_rootfs*
.no_rootfs_resize
```

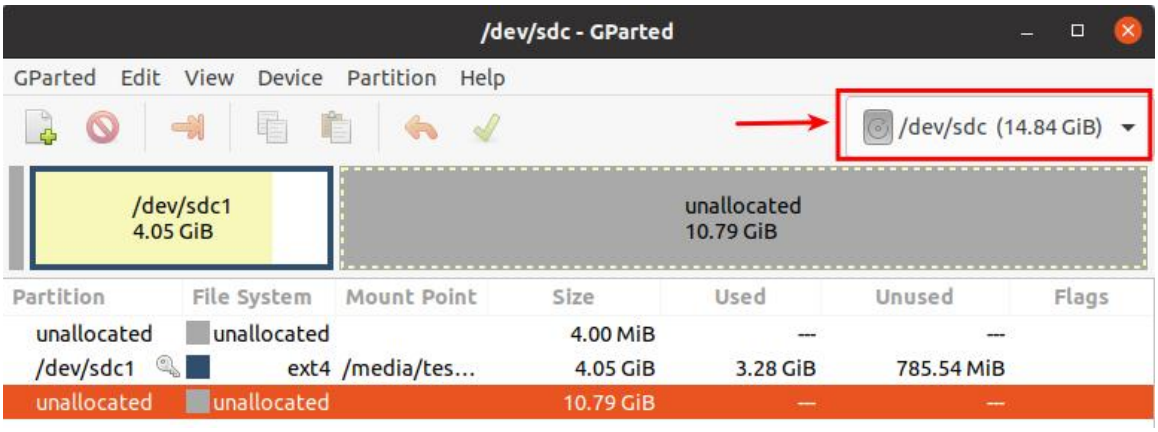
5) 然后在 Ubuntu 电脑中安装下 gparted 这个软件

```
test@test:~$ sudo apt install -y gparted
```

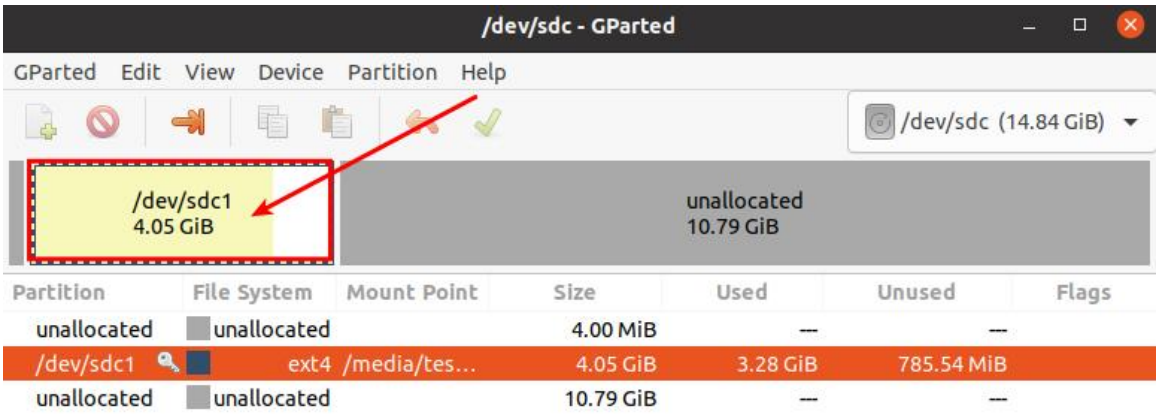
6) 然后打开 gparted

```
test@test:~$ sudo gparted
```

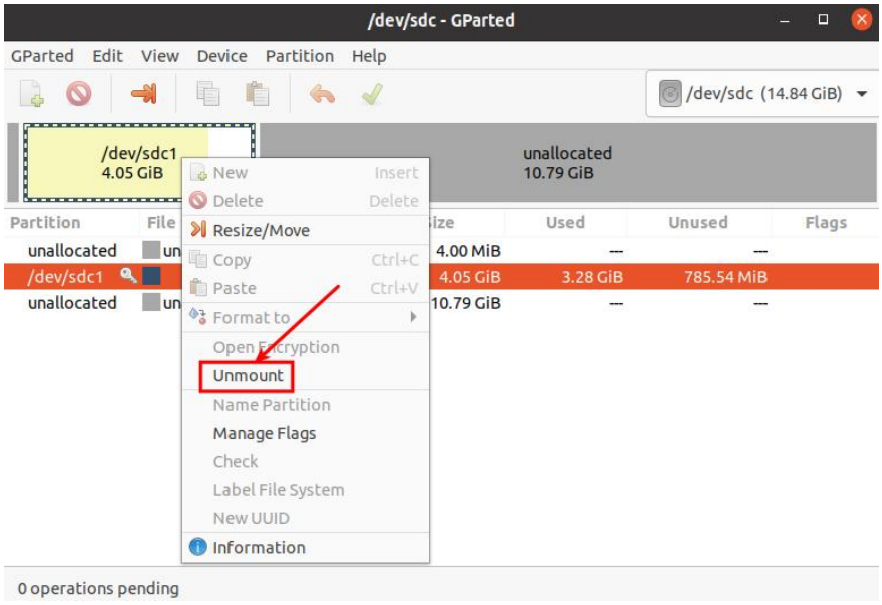
7) 打开 gparted 后在右上角可以选择 TF 卡，然后就可以看到 TF 卡容量的使用情况。下图显示的是烧录完 Linux 桌面版系统后 TF 卡的情况，可以看到，虽然 TF 卡的总容量是 16GB 的（在 GParted 中显示为 14.84GiB），但是 rootfs 分区（/dev/sdc1）实际只分配了 4.05GiB，还剩下 10.79GiB 未分配



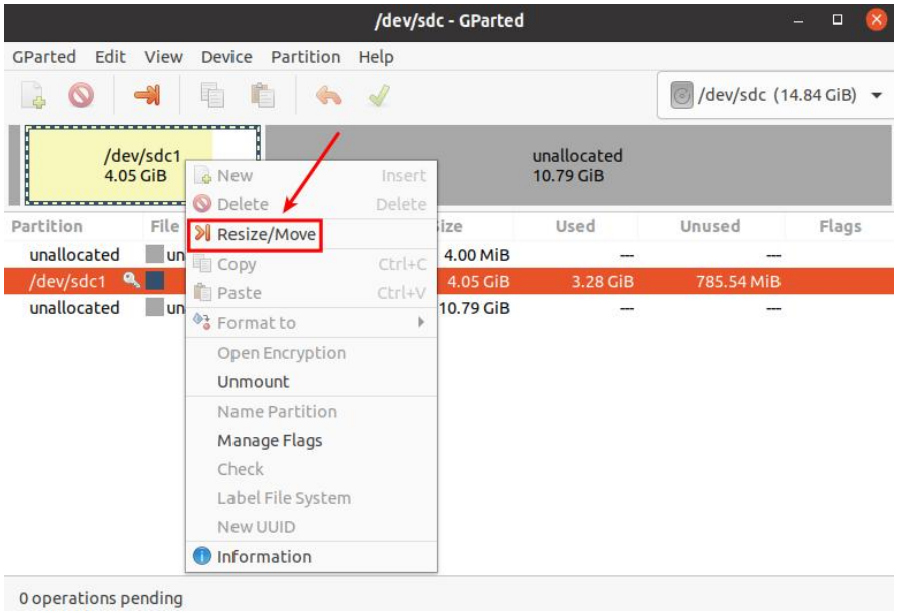
8) 然后选中 rootfs 分区（/dev/sdc1）



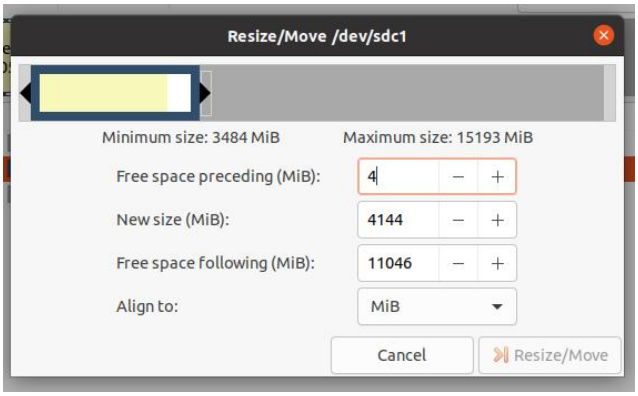
9) 再点击鼠标右键就可以看到下图所示的操作选项，如果 TF 卡已经挂载了，首先需要 Umount 掉 TF 卡的 rootfs 分区



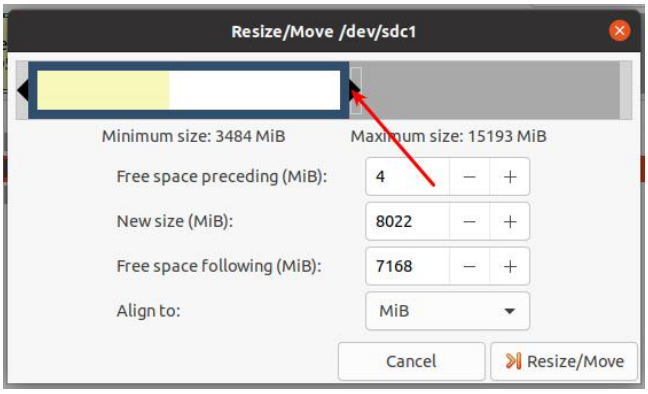
10) 然后再次选中 rootfs 分区，再点击鼠标右键，然后选择 **Resize/Move** 开始扩容 rootfs 分区的大小



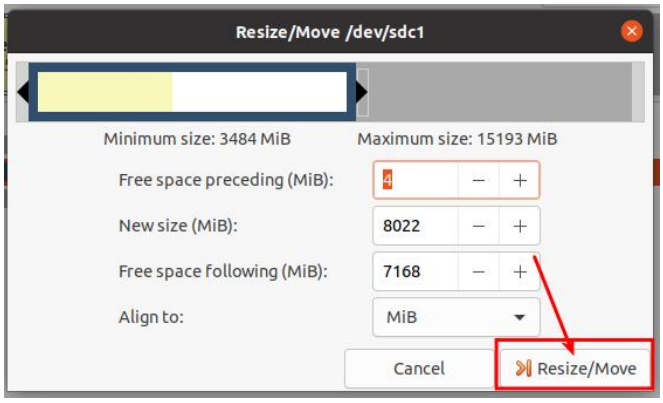
11) **Resize/Move** 选项打开后会弹出下面的设置界面



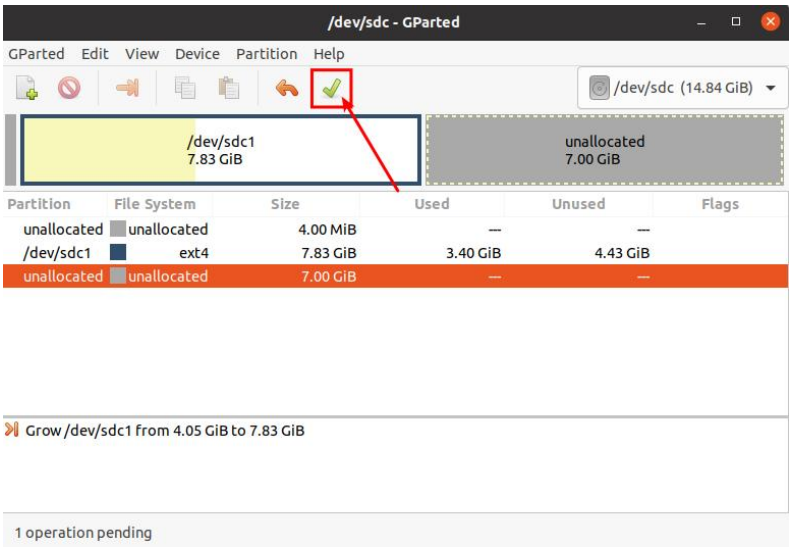
12) 然后可以直接拖动下图所示的位置来设置容量的大小，也可以通过设置 **New size(MiB)**中的数字来设置 rootfs 分区的大小



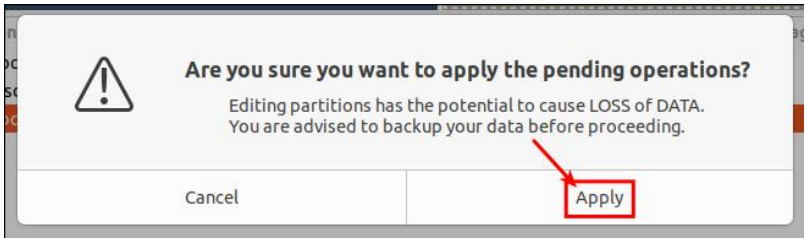
13) 设置好容量后，再点击右下角的 **Resize/Move** 即可



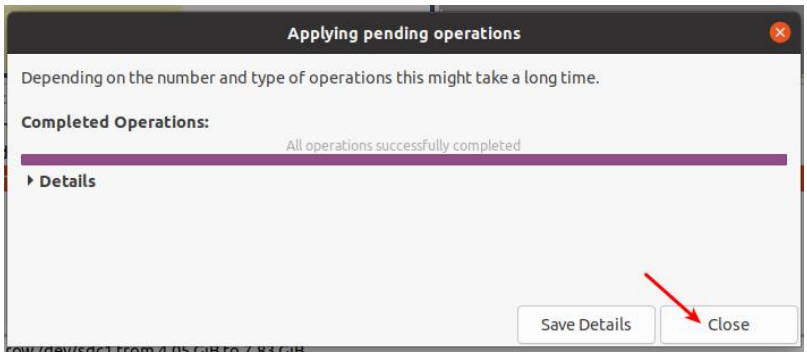
14) 最后确认无误后，再点击下图所示的**绿色** √



15) 然后选择 **Apply**，就会正式开始扩容 rootfs 分区的容量



16) 扩容完成后点击 **Close** 关闭即可



17) 然后就可以把 TF 卡拔下来，再插到开发板中启动，进入开发板的 Linux 系统中后如果使用 **df -h** 命令可以看到 rootfs 分区的大小和前面设置的大小一致的话就说明手动扩容成功

```
root@orangepi:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            925M     0  925M   0% /dev
```

```
tmpfs           199M  3.2M  196M   2% /run
/dev/mmcblk0p1  7.7G  3.2G  4.4G  42% /
```

3.6.4. 缩小 TF 卡中 rootfs 分区容量的方法

在 TF 卡的 Linux 系统中配置好应用程序或者其他的开发环境后，如果想备份下 TF 卡中的 Linux 系统，可以使用此小节的方法先缩小下 rootfs 分区的大小，然后再开始备份。

1) 首先在 **Ubuntu 电脑**（Windows 不行）中插入想要操作的 TF 卡

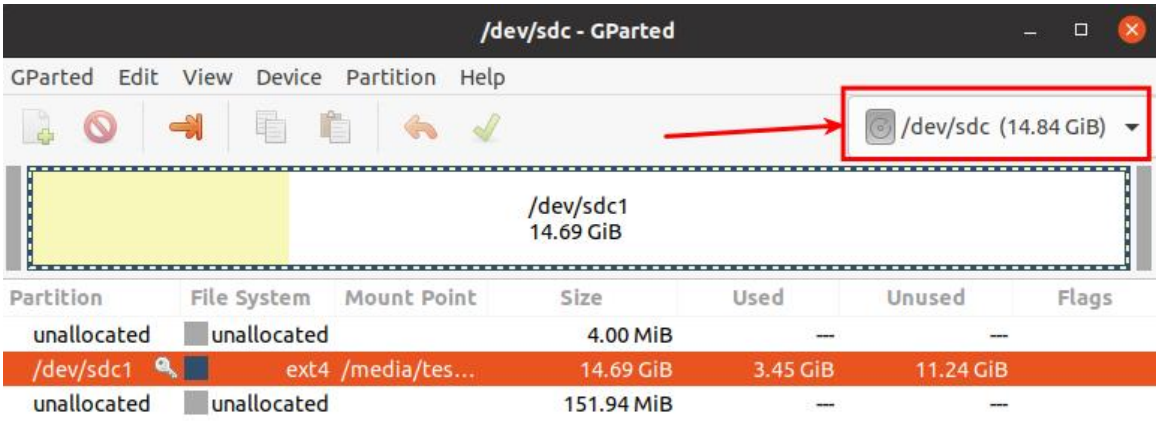
2) 然后在 Ubuntu 电脑中安装下 gparted 这个软件

```
test@test:~$ sudo apt install -y gparted
```

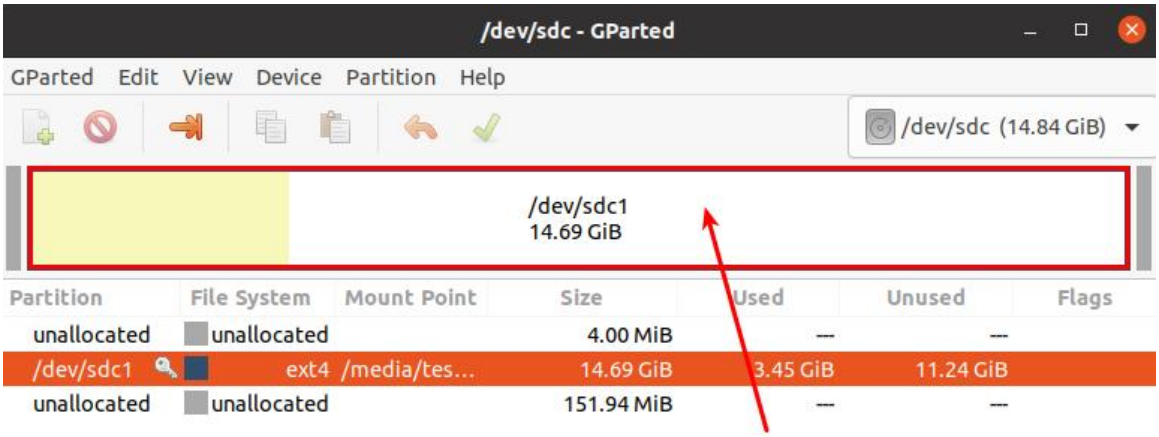
3) 然后打开 gparted

```
test@test:~$ sudo gparted
```

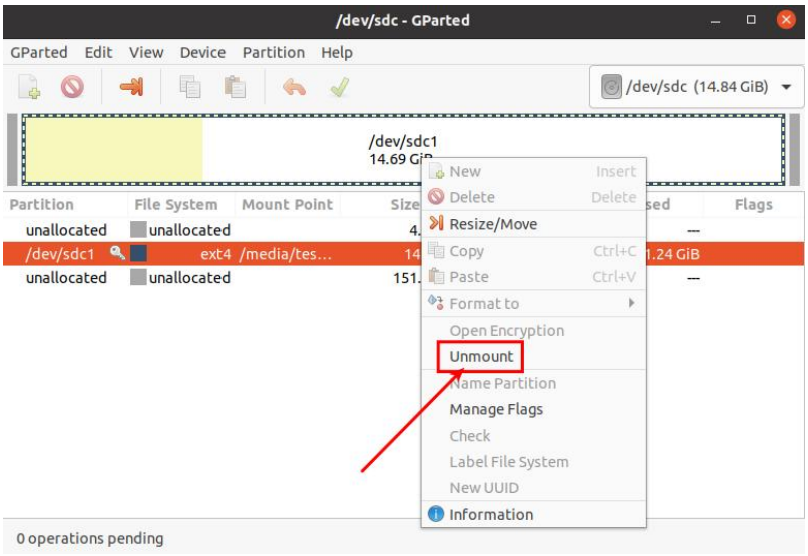
4) 打开 gparted 后在右上角可以选择 TF 卡，然后就可以看到 TF 卡容量的使用情况



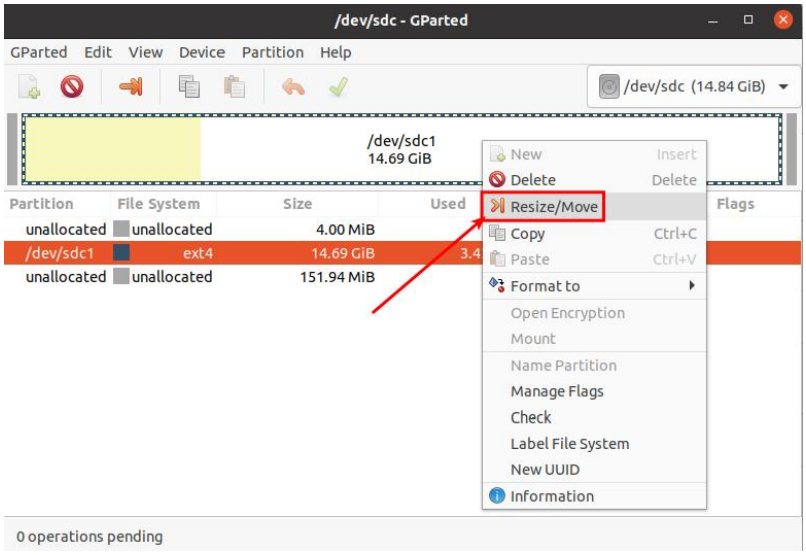
5) 然后选中 rootfs 分区（/dev/sdc1）



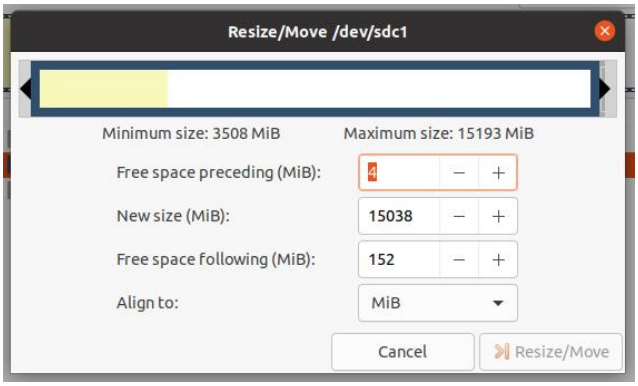
6) 再点击鼠标右键就可以看到下图所示的操作选项，如果 TF 卡已经挂载了，首先需要 Umount 掉 TF 卡的 rootfs 分区



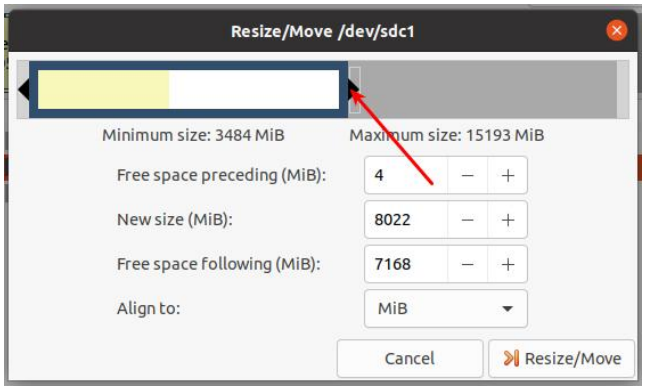
7) 然后再次选中 rootfs 分区，再点击鼠标右键，然后选择 **Resize/Move** 开始设置 rootfs 分区的大小



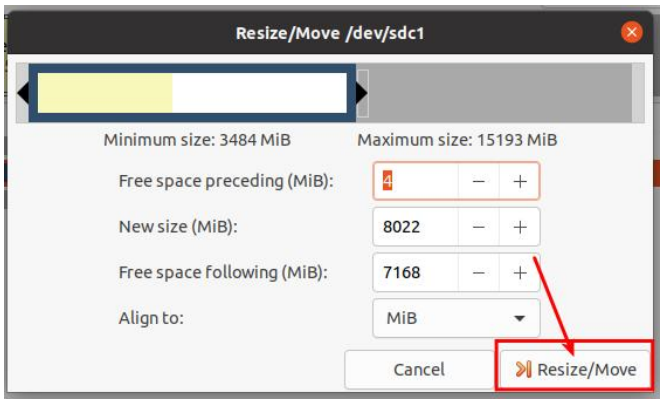
8) **Resize/Move** 选项打开后会弹出下面的设置界面



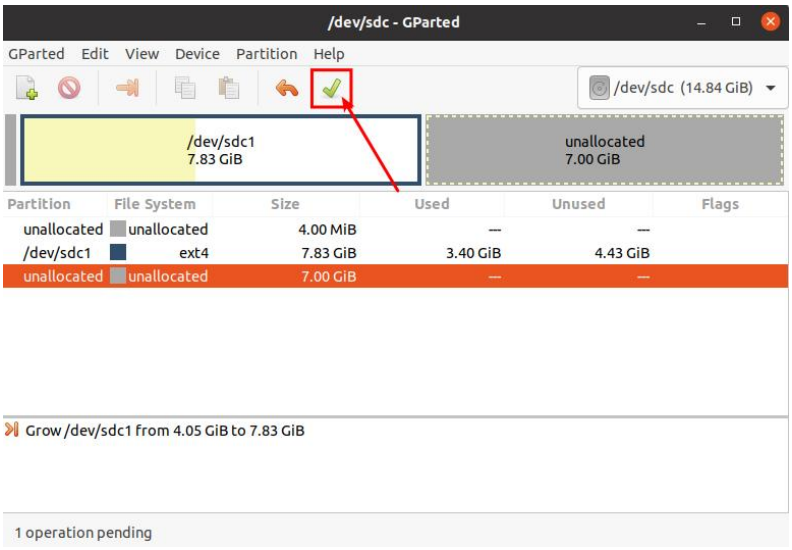
9) 然后可以直接拖动下图所示的位置来设置容量的大小，也可以通过设置 **New sieze(MiB)**中的数字来设置 rootfs 分区的大小



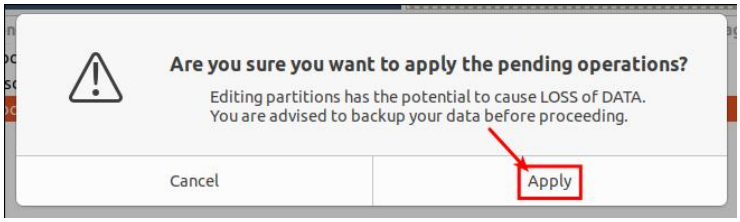
10) 设置好容量后，再点击右下角的 **Resize/Move** 即可



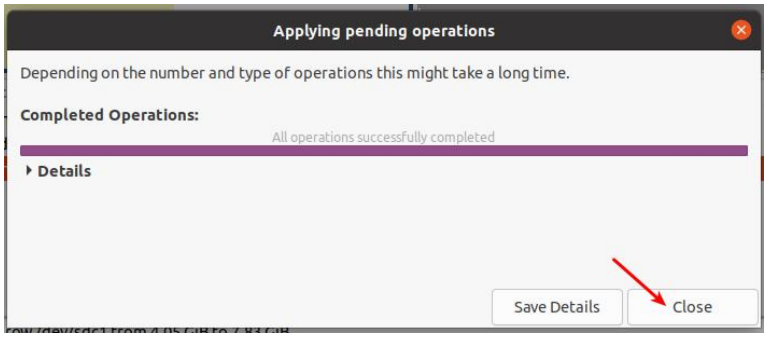
11) 最后确认无误后，再点击下图所示的**绿色** √



12) 然后选择 **Apply**，就会正式开始扩容 rootfs 分区的容量



13) 扩容完成后点击 **Close** 关闭即可



14) 然后就可以把 TF 卡拔下来，再插到开发板中启动，进入开发板的 Linux 系统中后如果使用 **df -h** 命令可以看到 rootfs 分区的大小和前面设置的大小一致的话就说明缩小容量成功

```
root@orangepi:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            925M     0  925M   0% /dev
tmpfs           199M  3.2M  196M   2% /run
/dev/mmcblk0p1  7.7G  3.2G  4.4G  42% /
```

3.7. 网络连接测试

3.7.1. WIFI 连接测试

请不要通过修改/etc/network/interfaces 配置文件的方式来连接 WIFI，通过这种方式连接 WIFI 网络使用会有问题。

3.7.1.1. 服务器版镜像通过命令连接 WIFI

当开发板没有连接以太网，没有连接 HDMI 显示器，只连接了串口时，推荐使用此小节演示的命令来连接 WIFI 网络。因为 nmtui 在某些串口软件（如 minicom）中只能显示字符，无法正常显示图形界面。当然，如果开发板连接了以太网或者 HDMI 显示屏，也可以使用此小节演示的命令来连接 WIFI 网络的。

1) 先登录 linux 系统，有下面两种方式

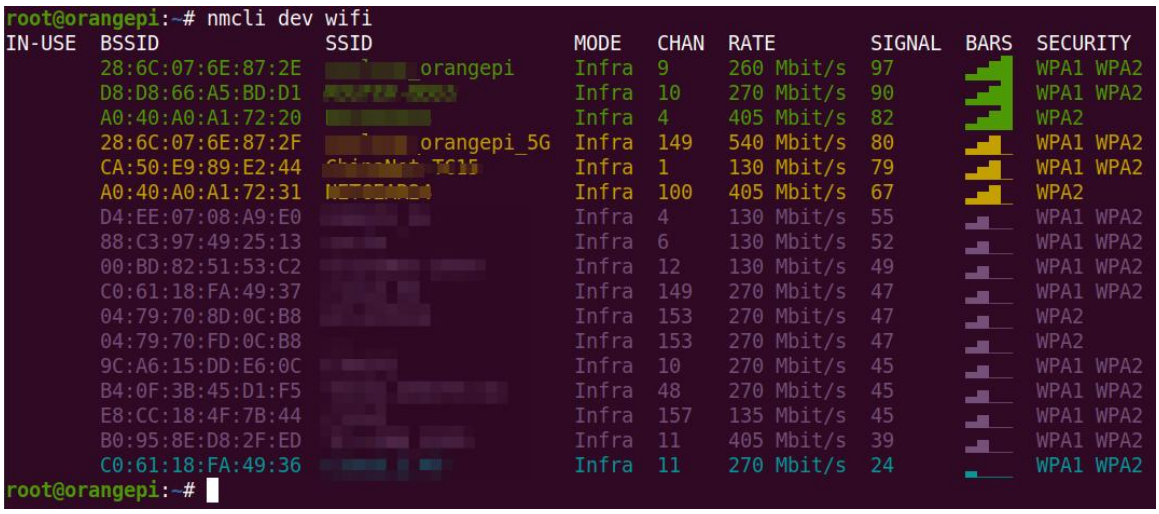
a. 如果开发板连接好了调试串口，可以使用串口终端登录 linux 系统

b. 如果连接了开发板到 HDMI 显示器，可以通过 HDMI 显示的终端登录到 linux

系统

1) 首先使用 **nmcli dev wifi** 命令扫描周围的 WIFI 热点

```
orangepi@orangepi:~$ nmcli dev wifi
```



2) 然后使用 **nmcli** 命令连接扫描到的 WIFI 热点，其中：

a. **wifi_name** 需要换成想连接的 WIFI 热点的名字

b. **wifi_passwd** 需要换成想连接的 WIFI 热点的密码

```
orangepi@orangepi:~$ sudo nmcli dev wifi connect wifi_name password wifi_passwd
Device 'wlan0' successfully activated with 'cf937f88-ca1e-4411-bb50-61f402eef293'.
```

3) 通过 **ip addr show wlan0** 命令可以查看 wifi 的 IP 地址

```
orangepi@orangepi:~$ ip a s wlan0
11: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 23:8c:d6:ae:76:bb brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.11/24 brd 192.168.1.255 scope global dynamic noprefixroute wlan0
       valid_lft 259192sec preferred_lft 259192sec
    inet6 240e:3b7:3240:c3a0:c401:a445:5002:ccdd/64 scope global dynamic noprefixroute
       valid_lft 259192sec preferred_lft 172792sec
    inet6 fe80::42f1:6019:a80e:4c31/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

4) 使用 **ping** 命令可以测试 wifi 网络的连通性，**ping** 命令可以通过 **Ctrl+C** 快捷键来中断运行

```
orangepi@orangepi:~$ ping www.orangepi.org -I wlan0
PING www.orangepi.org (182.92.236.130) from 192.168.1.49 wlan0: 56(84) bytes of data.
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=1 ttl=52 time=43.5 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=2 ttl=52 time=41.3 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=3 ttl=52 time=44.9 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=4 ttl=52 time=45.6 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=5 ttl=52 time=48.8 ms
^C
--- www.orangepi.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 41.321/44.864/48.834/2.484 ms
```

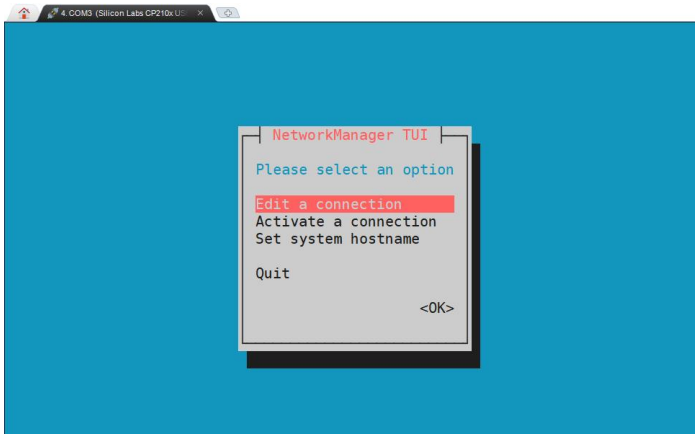
3.7.1.2. 服务器版镜像通过图形化方式连接 WIFI

1) 先登录 linux 系统，有下面两种方式
   a. 如果开发板连接好了调试串口，可以使用串口终端登录 linux 系统（串口软件请使用 MobaXterm，使用 minicom 无法显示图形界面）
   b. 如果连接了开发板到HDMI显示器，可以通过HDMI显示的终端登录到linux系统

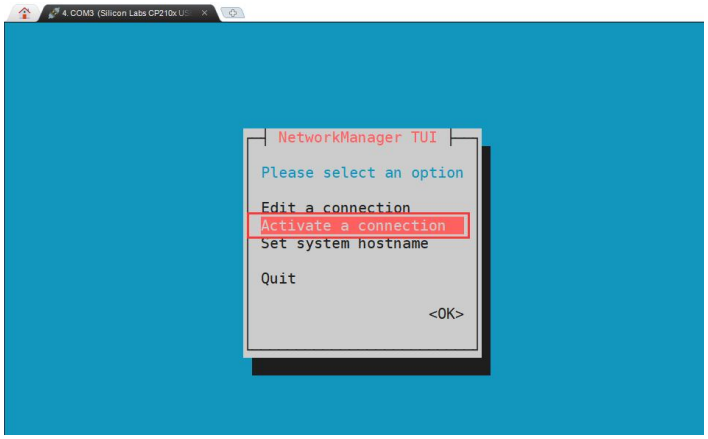
2) 然后在命令行中输入 nmtui 命令打开 wifi 连接的界面

```
orangepi@orangepi:~$ sudo nmtui
```

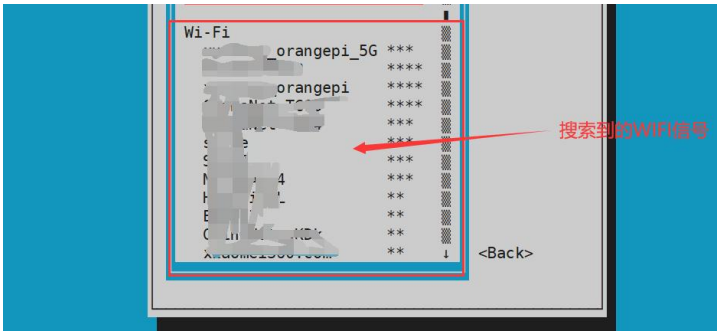
3) 输入 nmtui 命令打开的界面如下所示



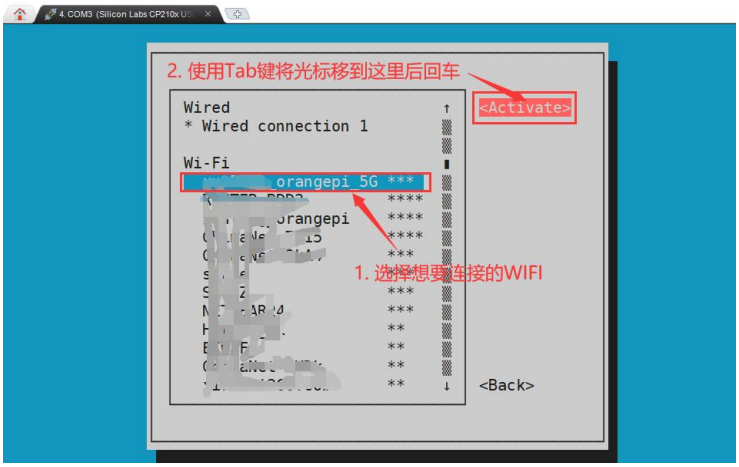
4) 选择 **Activate a connect** 后回车



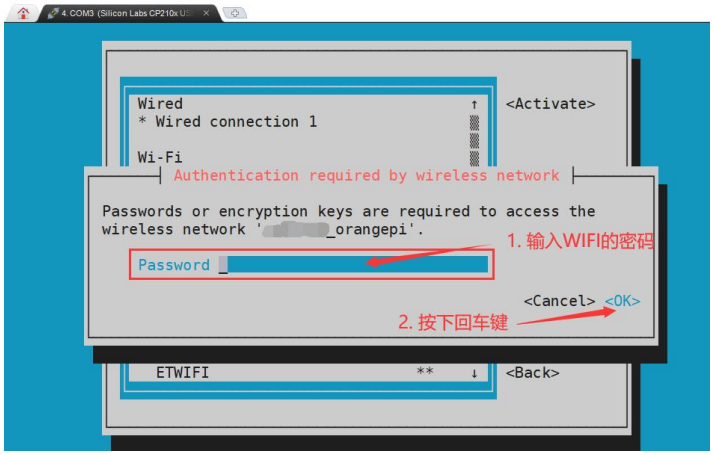
5) 然后就能看到所有搜索到的 WIFI 热点



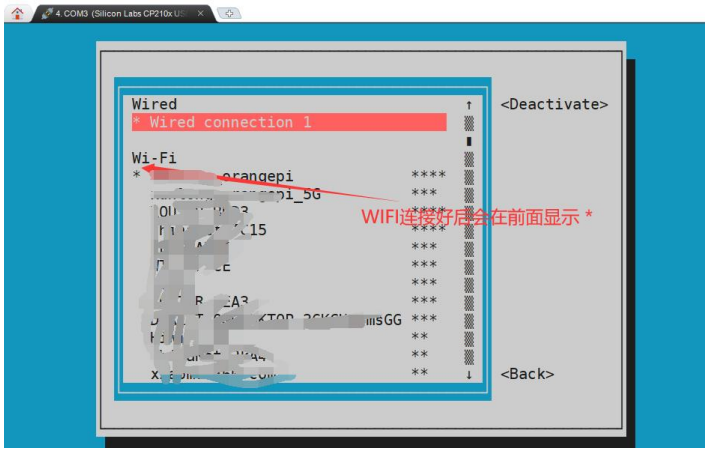
6) 选择想要连接的 WIFI 热点后再使用 Tab 键将光标定位到 **Activate** 后回车



7) 然后会弹出输入密码的对话框，在 **Pssword** 内输入对应的密码然后回车就会开始连接 WIFI



8) WIFI 连接成功后会在已连接的 WIFI 名称前显示一个“*”



9) 通过 **ip a s wlan0** 命令可以查看 wifi 的 IP 地址

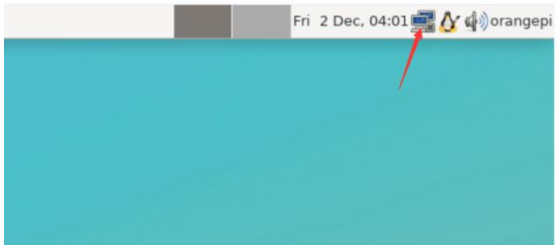
```
orangepi@orangepi:~$ ip a s wlan0
11: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 24:8c:d3:aa:76:bb brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.11/24 brd 192.168.1.255 scope global dynamic noprefixroute wlan0
       valid_lft 259069sec preferred_lft 259069sec
    inet6 240e:3b7:3240:c4a0:c401:a445:5002:ccdd/64 scope global dynamic noprefixroute
       valid_lft 259071sec preferred_lft 172671sec
    inet6 fe80::42f1:6019:a80e:4c31/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

10) 使用 **ping** 命令可以测试 wifi 网络的连通性，**ping** 命令可以通过 **Ctrl+C** 快捷键来中断运行

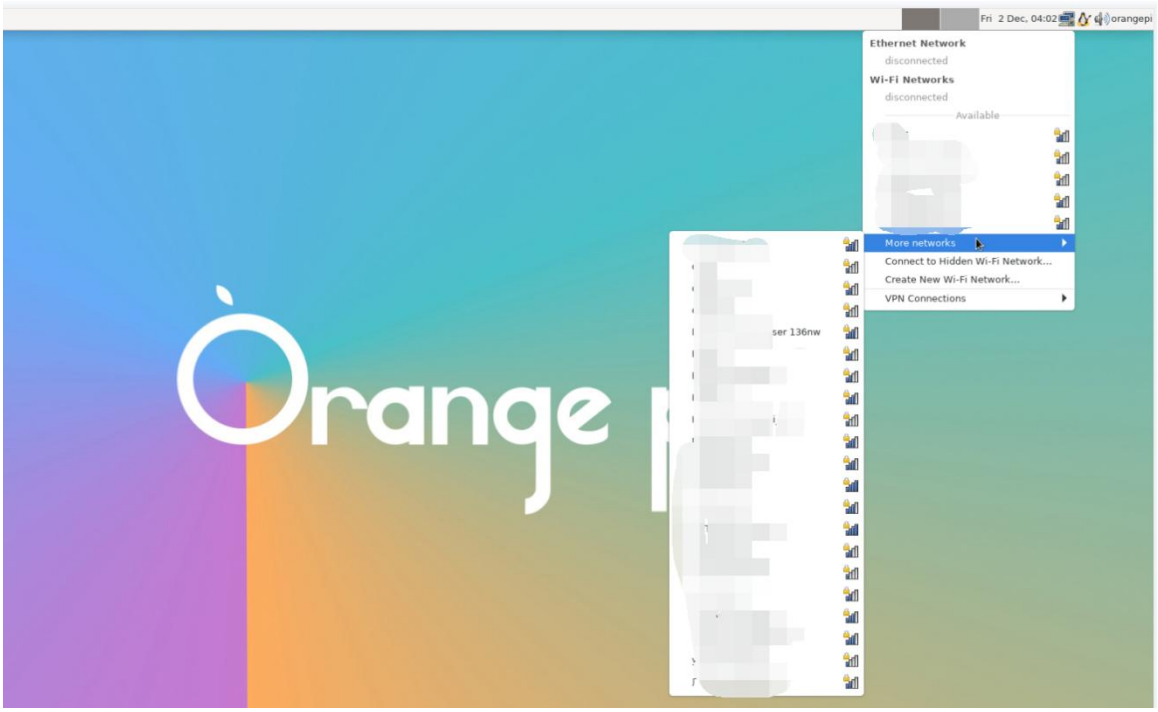
```
orangepi@orangepi:~$ ping www.orangepi.org -I wlan0
PING www.orangepi.org (182.92.236.130) from 192.168.1.49 wlan0: 56(84) bytes of data.
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=1 ttl=52 time=43.5 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=2 ttl=52 time=41.3 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=3 ttl=52 time=44.9 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=4 ttl=52 time=45.6 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=5 ttl=52 time=48.8 ms
^C
--- www.orangepi.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 41.321/44.864/48.834/2.484 ms
```

3.7.1.3. 桌面版镜像的测试方法

1) 点击桌面右上角的网络配置图标



2) 在弹出的下拉框中点击 **More networks** 可以看到所有扫描到的 WIFI 热点，然后选择想要连接的 WIFI 热点



3) 然后输入 WIFI 热点的密码，再点击 **Connect** 就会开始连接 WIFI



3.7.2. 设置静态 IP 地址的方法

请不要通过修改/etc/network/interfaces 配置文件的方式来设置静态 IP 地址。

3.7.2.1. 使用 nmcli 命令来设置静态 IP 地址

1) **请先连接好 WIFI**，然后再开始设置静态 IP 地址

2) 然后通过 **nmcli con show** 命令可以查看网络设备的名字，如下所示

a. **orangepi** 为 WIFI 网络接口的名字（名字不一定相同）

```
orangepi@orangepi:~$ nmcli con show
NAME                UUID                                  TYPE      DEVICE
orangepi            cfc4f922-ae48-46f1-84e1-2f19e9ec5e2a  wifi      wlan0
```

3) 然后输入下面的命令，其中

a. **"orangepi"** 表示设置 WIFI 的静态 IP 地址（通过 **nmcli con show** 命令可以获取到）

b. **ipv4.addresses** 后面是要设置的静态 IP 地址，可以修改为自己想要设置的值

c. **ipv4.gateway** 表示网关的地址

```
orangepi@orangepi:~$ sudo nmcli con mod "orangepi"  \
ipv4.addresses "192.168.1.110/24"  \
ipv4.gateway "192.168.1.1"  \
ipv4.dns "8.8.8.8"  \
ipv4.method "manual"
```

4) 然后重启 linux 系统

```
orangepi@orangepi:~$ sudo reboot
```

5) 然后重新进入 linux 系统使用 **ip addr show wlan0** 命令就可以看到 IP 地址已经设置为想要的值了

```
orangepi@orangepi:~$ ip addr show wlan0
6: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 18:4e:f2:9c:1b:49 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.110/32 brd 192.168.1.110 scope global dynamic noprefixroute wlan0
```

```
    valid_lft 3570sec preferred_lft 3570sec
inet6 fe80::be1a:8847:e5a6:6d52/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
```

3.7.3. 设置 Linux 系统第一次启动自动连接网络的方法

开发板有无线 WIFI，如果想通过 WIFI 来远程登录开发板的 Linux 系统，则需要通过在 HDMI 屏幕或串口中通过命令来连接 WIFI。

但如果 HDMI 屏幕和串口模块都没有，只有 WIFI 可以连接，则可以使用此小节介绍的方法来自动连接 WIFI 并且还能设置 WIFI 的静态 IP 地址或者自动设置以太网口的静态 IP 地址。

要使用此小节的方法，首先需要准备一台 Linux 系统的机器。比如一台安装有 Ubuntu 系统的电脑或者虚拟机。

为什么需要 Linux 系统的机器，因为 TF 卡中烧录的开发板 Linux 系统的根文件系统是 ext4 格式的，Linux 系统的机器可以正常的挂载它，然后对其中的配置文件进行修改。

如果想要在 Windows 系统中来修改，可以使用 Paragon ExtFS for Windows 这款软件，由于此软件需要付费，而目前又没有比较好用的类似的免费软件，这里就不具体演示了。

另外如果尝试 Paragon ExtFS for Windows 这款软件使用有问题请自行解决，我们不答疑。

1) 首先烧录想使用的开发板的 Linux 镜像到 TF 卡中，然后使用读卡器，将烧录好开发板 Linux 镜像的 TF 卡插入安装有 Linux 系统的机器中（比如安装有 Ubuntu 系统的电脑，下面都以 Ubuntu 电脑为例来演示）

2) 当 TF 卡插入 Ubuntu 电脑后，Ubuntu 电脑一般会自动挂载 TF 卡中的 Linux 根文件系统的分区，由下面的命令可以知道，**/media/test/opi_root** 即为 TF 卡中的 Linux 根文件系统挂载的路径

```
test@test:~$ df -h | grep "media"
/dev/sdd1   1.4G   1.2G   167M   88% /media/test/opi_root
test@test:~$ ls /media/test/opi_root
bin  boot  dev  etc  home  lib  lost+found  media  mnt  opt  proc  root  run
sbin  selinux  srv  sys  tmp  usr  var
```

3) 然后进入 TF 卡中烧录的 Linux 系统的**/boot** 目录中

```
test@test:~$ cd /media/test/opi_root/boot/
```

4) 然后将其中的 **orangepi_first_run.txt.template** 复制为 **orangepi_first_run.txt**，通过 orangepi_first_run.txt 配置文件可以设置开发板 Linux 系统第一次启动时自动连接某个 WIFI 热点，也可以设置 WIFI 或者以太网口的静态 IP 地址

```
test@test:/media/test/opi_root/boot$ sudo cp orangepi_first_run.txt.template orangepi_first_run.txt
```

5) 通过下面的命令可以打开 orangepi_first_run.txt 文件，然后就可以查看修改其中的内容

```
test@test:/media/test/opi_root/boot$ sudo vim orangepi_first_run.txt
```

6) orangepi_first_run.txt 文件中的变量使用说明

a. **FR_general_delete_this_file_after_completion** 变量用来设置第一次启动完后是否删除 orangepi_first_run.txt 这个文件，默认为 1，也就是删除，如果设置为 0，第一次启动后会将 orangepi_first_run.txt 重命名为 orangepi_first_run.txt.old，一般保持默认值即可
b. **FR_net_change_defaults** 变量用于设置是否改变默认网络设置，这个必须要设置为 1，否则所有的网络设置都不会生效
c. **FR_net_ethernet_enabled** 变量用来控制是否使能以太网口的配置，如果需要设置以太网口的静态 IP 地址，请设置为 1
d. **FR_net_wifi_enabled** 变量用来控制是否使能 WIFI 的配置，如果需要设置开发板自动连接 WIFI 热点，则必须将其设置为 1，另外请注意，如果此变量设置为 1，则以太网口的设置就会失效。也就是说 WIFI 和以太网口不能同时设置（为什么，因为没必要...）
e. **FR_net_wifi_ssid** 变量用于设置想要连接的 WIFI 热点的名字
f. **FR_net_wifi_key** 变量用于设置想要连接的 WIFI 热点的密码
g. **FR_net_use_static** 变量用于设置是否需要设置 WIFI 或者以太网口的静态 IP 地址
h. **FR_net_static_ip** 变量用于设置静态 IP 的地址，请根据自己的实际情况设置
i. **FR_net_static_gateway** 变量用于设置网关，请根据自己的实际情况设置

7) 下面演示几个具体的设置示例：

a. 比如想要开发板的 Linux 系统第一次启动后自动连接 WIFI 热点，可以这样

设置：

a) 设置 **FR_net_change_defaults** 为 **1**
b) 设置 **FR_net_wifi_enabled** 为 **1**
c) 设置 **FR_net_wifi_ssid** 为想要连接的 WIFI 热点的名字
d) 设置 **FR_net_wifi_key** 为想要连接的 WIFI 热点的密码

b. 比如想要开发板的 Linux 系统第一次启动后自动连接 WIFI 热点，并且设置 WIFI 的 IP 地址为特定的静态 IP 地址（这样当 Linux 系统启动后，可以直接使用设置的静态 IP 地址 ssh 远程登录开发板，无需通过路由器后台来查看开发板的 IP 地址），可以这样设置：

a) 设置 **FR_net_change_defaults** 为 **1**
b) 设置 **FR_net_wifi_enabled** 为 **1**
c) 设置 **FR_net_wifi_ssid** 为想要连接的 WIFI 热点的名字
d) 设置 **FR_net_wifi_key** 为想要连接的 WIFI 热点的密码
e) 设置 **FR_net_use_static** 为 **1**
f) 设置 **FR_net_static_ip** 为想要的 IP 地址
g) 设置 **FR_net_static_gateway** 为对应的网关地址

c. 比如想要开发板的 Linux 系统第一次启动后自动设置以太网口的 IP 地址为想要的静态 IP 地址，可以这样设置：

a) 设置 **FR_net_change_defaults** 为 **1**
b) 设置 **FR_net_ethernet_enabled** 为 **1**
c) 设置 **FR_net_use_static** 为 **1**
d) 设置 **FR_net_static_ip** 为想要的 IP 地址
e) 设置 **FR_net_static_gateway** 为对应的网关地址

8) 修改完 orangepi_first_run.txt 文件后，就可以退出 TF 卡中开发板 Linux 系统的/boot 目录，再卸载 TF 卡，然后就可以将 TF 卡插入开发板中启动了

9) 如果没有设置静态 IP 地址，则还是需要通过路由器后台来查看 IP 地址，如果设置了静态 IP 地址，则可以在电脑上 ping 下设置的静态 IP 地址，如果能 ping 说明系统已经正常启动，并且网络也已设置正确，然后就可以使用设置的 IP 地址 ssh 远程登录开发板的 Linux 系统了

开发板的 Linux 系统第一次启动完后，orangepi_first_run.txt 会被删除或者重

命名为 orangepi_first_run.txt.old，此时就算重新设置 orangepi_first_run.txt 配置文件，然后重新启动开发板的 Linux 系统，orangepi_first_run.txt 中的配置也不会再次生效，因为此配置只在烧录完 Linux 系统后第一次启动才会有作用，这点请特别注意。

3.8. SSH 远程登录开发板

Linux 系统默认都开启了 ssh 远程登录，并且允许 root 用户登录系统。ssh 登录前首先需要确保以太网或者 wifi 网络已连接，然后使用 ip addr 命令或者通过查看路由器的方式获取开发板的 IP 地址。

3.8.1. Ubuntu 下 SSH 远程登录开发板

1) 获取开发板的 IP 地址

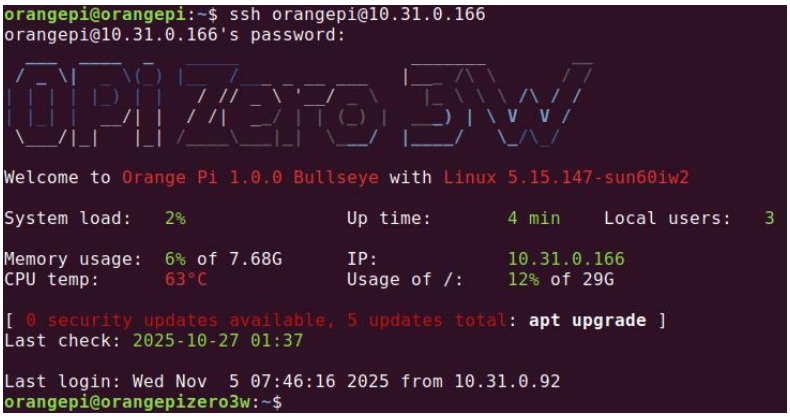
2) 然后就可以通过 ssh 命令远程登录 linux 系统

```
test@test:~$ ssh orangepi@192.168.1.xxx          (需要替换为开发板的 IP 地址)
orangepi@192.168.1.xx's password:                （在这里输入密码，默认密码为 orangepi）
```

注意，输入密码的时候，屏幕上是不会显示输入的密码的具体内容的，请不要以为是有什么故障，输入完后直接回车即可。

如果提示拒绝连接，只要使用的是 Orange Pi 提供的镜像，就请不要怀疑 orangepi 这个密码是不是不对，而是要找其他原因。

3) 成功登录系统后的显示如下图所示



如果 ssh 无法正常登陆 linux 系统，首先请检查下开发板的 IP 地址是否能 ping 通，如果 ping 通没问题的话，可以通过串口或者 HDMI 显示器登录 linux 系统然后在开发板上输入下面的命令后再尝试是否能连接：

root@orangepi:~# **reset_ssh.sh**

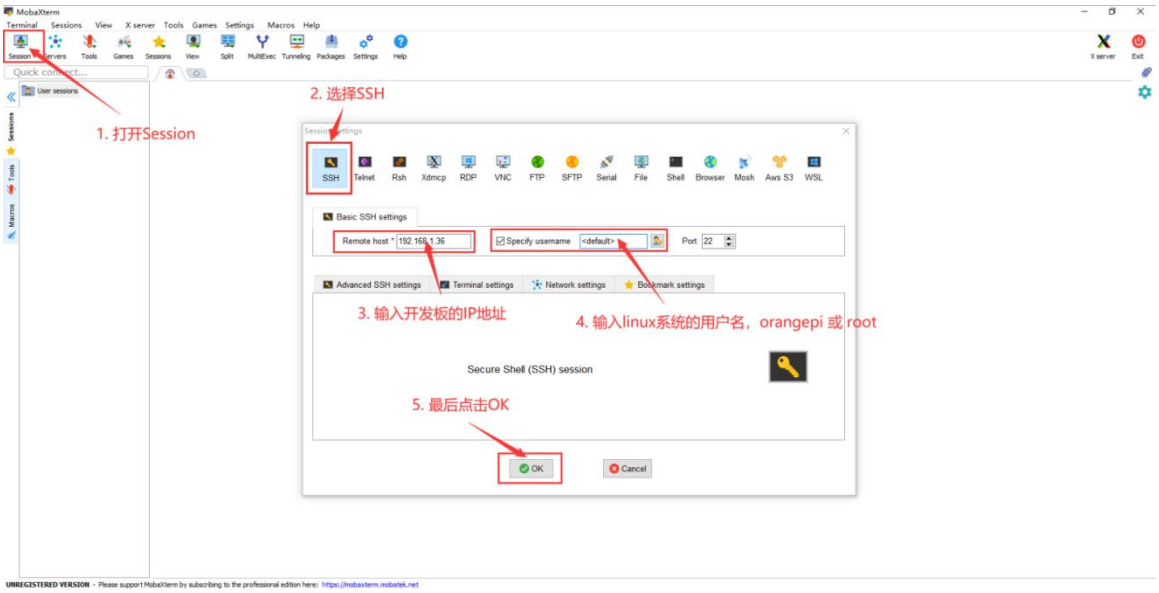
如果还不行，请重烧系统试下。

3.8.2. Windows 下 SSH 远程登录开发板

1) 首先获取开发板的 IP 地址

2) 在 windows 下可以使用 MobaXterm 远程登录开发板，首先新建一个 ssh 会话

a. 打开 **Session**
b. 然后在 **Session Setting** 中选择 **SSH**
c. 然后在 **Remote host** 中输入开发板的 IP 地址
d. 然后在 **Specify username** 中输入 linux 系统的用户名 **root** 或 **orangepi**
e. 最后点击 **OK** 即可

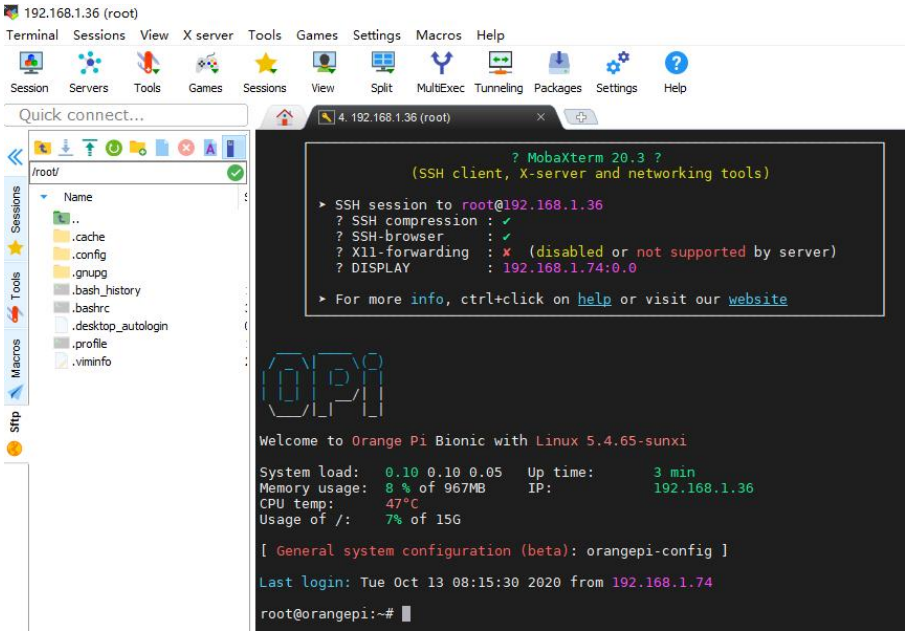


3) 然后会提示输入密码，默认 root 和 orangepi 用户的密码都为 orangepi

注意，输入密码的时候，屏幕上是不会显示输入的密码的具体内容的，请不要以为是有什么故障，输入完后直接回车即可。



4) 成功登录系统后的显示如下图所示



3.9. ADB 的使用方法

3.9.1. 网络 adb 的使用方法

1) 系统启动后请先执行下面的命令

```
orangepi@orangepi:~$ sudo /etc/adb_conf.sh
```

2) 然后请确认下 adbd 已经启动了

```
orangepi@orangepi:~$ ps -ax | grep "adbd"
    808 ?        Sl     0:00 /usr/bin/adbd
   3707 ttyFIQ0  S+     0:00 grep --color=auto adbd
```

3) 然后查看下开发板的 IP 地址，并记下来

4) 然后在 Ubuntu PC 上安装 adb 工具

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y adb
```

5) 然后使用下面的命令连接网络 adb

```
test@test:~$ adb connect 192.168.1.xx:5555        #IP 地址请替换为开发板的 IP 地址
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 192.168.1.xx:5555
test@test:~$ adb devices
List of devices attached
192.168.1.xx:5555          device
```

6) 然后使用下面的命令就可以登录开发板的 linux 系统

```
test@test:~$ adb shell
root@orangepizero3w:/#     <---  看到这个提示符后说明已成功登录开发板
```

7) 使用 adb 上传文件到开发板的命令如下所示

```
test@test:~$ adb push filename /root
filename: 1 file pushed. 3.7 MB/s (1075091 bytes in 0.277s)
```

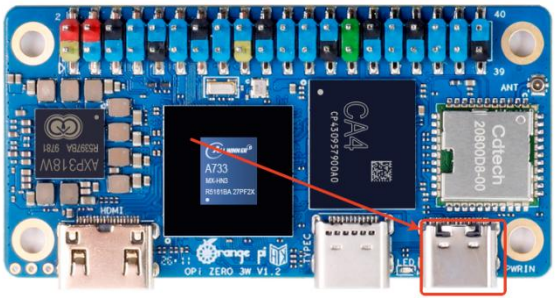
8) 使用 adb 重启开发板的命令如下所示

```
test@test:~$ adb reboot
```

3.9.2. 使用 type-c 数据线连接 adb

1) 准备一根 USB Typc C 接口的数据线， USB 接口一端插入电脑的 USB 接口中，USB Type C 接口一端插入开发板的 TypeC 电源接口中（接口位置请见下面右边图片）。在这种情况下是由电脑的 USB 接口给开发板供电，所以请确保电脑的 USB 接口能提供足够的功率驱动开发板。





2) 然后运行下面的命令将 Type-C 接口设置为 **device** 模式

```
orangepi@orangepi:~$ sudo /etc/adb_conf.sh
```

3) 然后请确认下 adbd 已经启动了

```
orangepi@orangepi:~$ ps -ax | grep "adbd"
    808 ?        Sl     0:00 /usr/bin/adbd
   3707 ttyFIQ0  S+     0:00 grep --color=auto adbd
```

4) 然后在 Ubuntu PC 上安装下 adb 工具

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y adb
```

5) 然后使用下面的命令查看下有没有识别到 adb 设备

```
test@test:~$ adb devices
List of devices attached
e0f9f71bc343c305    device
```

6) 然后使用下面的命令就可以登录开发板的 linux 系统

```
test@test:~$ adb shell
root@orangepizero3w:/#     <---  看到这个提示符后说明已成功登录开发板
```

7) 使用 adb 上传文件到开发板的命令如下所示

```
test@test:~$ adb push filename /root
filename: 1 file pushed. 3.7 MB/s (1075091 bytes in 0.277s)
```

3.10. HDMI 测试

3.10.1. HDMI 显示测试

1) 使用 Mini HDMI 转 HDMI 线连接 Orange Pi 开发板和 HDMI 显示器



2) 启动 linux 系统后如果 HDMI 显示器有图像输出说明 Mini HDMI 接口使用正常

注意，很多笔记本电脑虽然带有 HDMI 接口，但是笔记本的 HDMI 接口一般只有输出功能，并没有 HDMI in 的功能，也就是说并不能将其他设备的 HDMI 输出显示到笔记本的屏幕上。

当想把开发板的 HDMI 接到笔记本电脑 HDMI 接口时，请先确认清楚您的笔记本是支持 HDMI in 的功能。

当 HDMI 没有显示的时候，请先检查下 HDMI 线有没有插紧，确认接线没问题后，可以换一个不同的屏幕试下有没有显示。

3.10.2. HDMI 转 VGA 显示测试

1) 首先需要准备下面的配件

a. HDMI 转 VGA 转换器



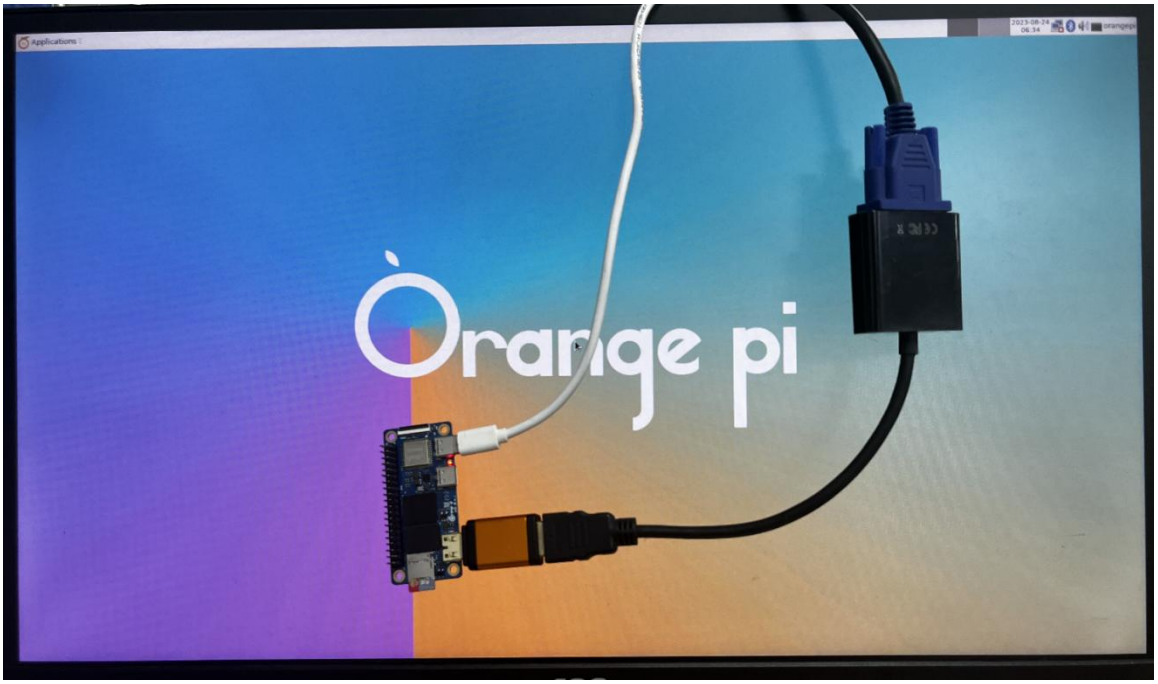
b. 一根 VGA 线





c. 一个支持 VGA 接口的显示器或者电视

2) HDMI 转 VGA 显示测试如下所示

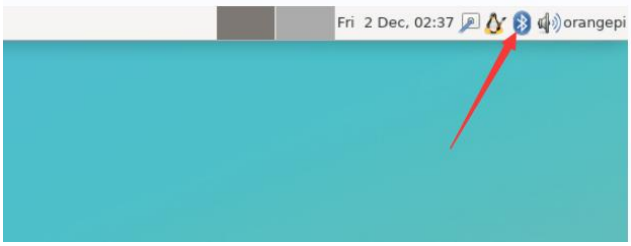


使用 HDMI 转 VGA 显示时，开发板以及开发板的 Linux 系统是不需要做任何设置的，只需要开发板 HDMI 接口能正常显示就可以了。所以如果测试有问题，请检查 HDMI 转 VGA 转换器、VGA 线以及显示器是否有问题。

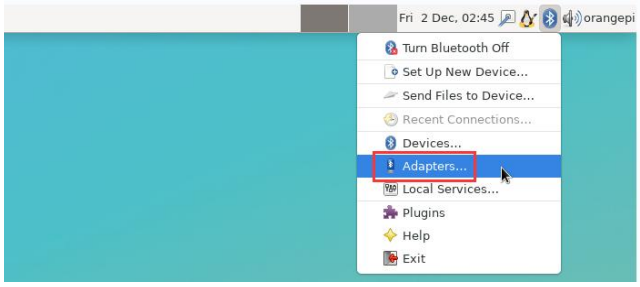
3.11. 蓝牙使用方法

3.11.1. 桌面版镜像的测试方法

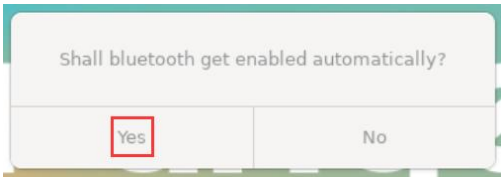
1) 首先点击桌面右上角的蓝牙图标



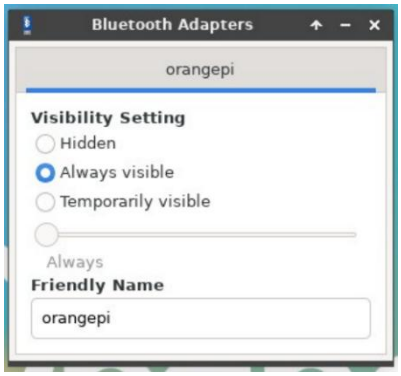
2) 然后选择适配器



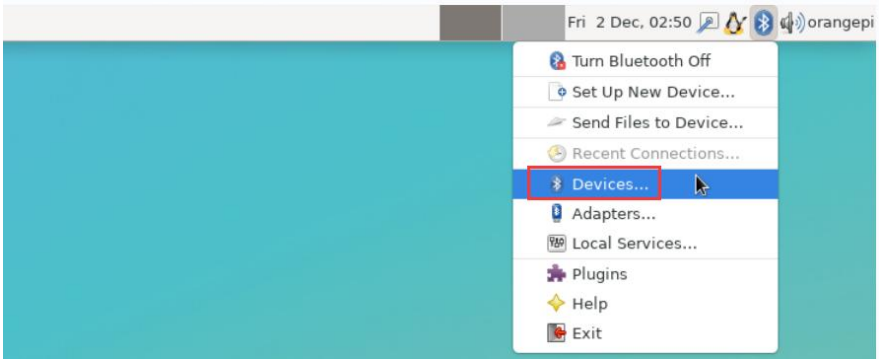
3) 如果有提示下面的界面，请选择 **Yes**



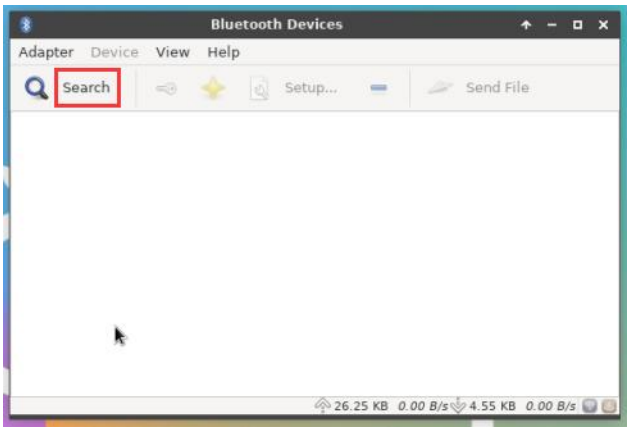
4) 然后在蓝牙的适配器设置界面中设置 **Visibility Setting** 为 **Always visible**，然后关闭即可



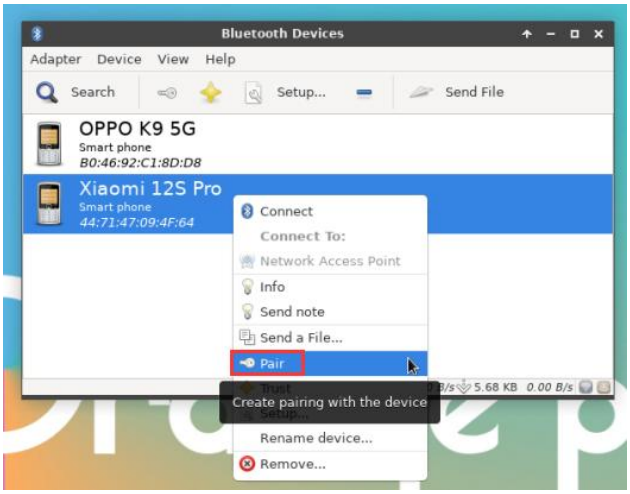
5) 然后打开蓝牙设备的配置界面



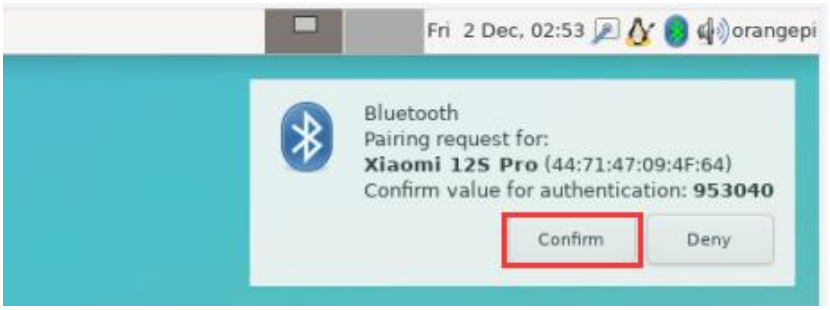
6) 点击 **Search** 即可开始扫描周围的蓝牙设备



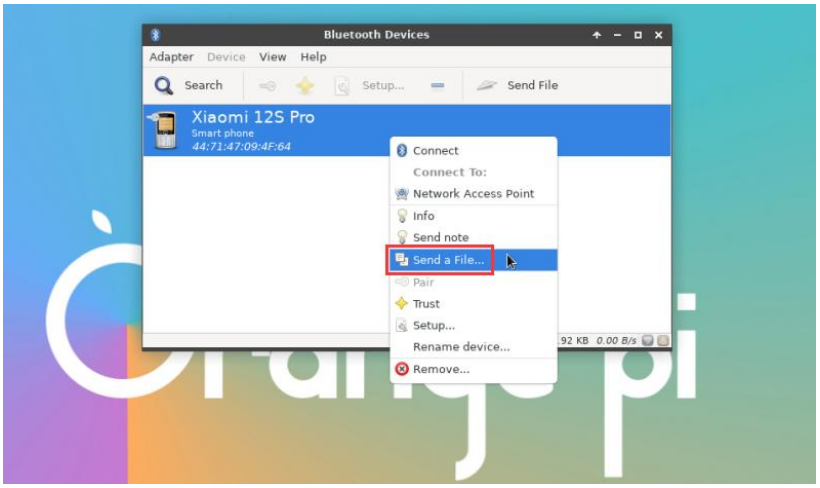
7) 然后选择想要连接的蓝牙设备，再点击鼠标右键就会弹出对此蓝牙设备的操作界面，选择 **Pair** 即可开始配对，这里演示的是和 Android 手机配对



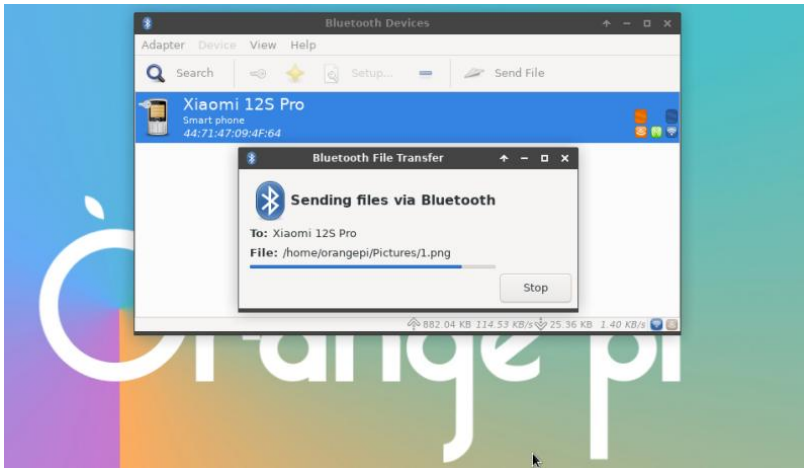
8) 配对时，桌面的右上角会弹出配对确认框，选择 **Confirm** 确认即可，此时手机上也同样需要进行确认



9) 和手机配对完后，可以选择已配对的蓝牙设备，然后右键选择 **Send a File** 即可开始给手机发送一张图片



10) 发送图片的界面如下所示



3.11.2. 服务器版镜像的使用方法

1) 进入系统后首先可以通过 **hciconfig** 命令来查看是否存在蓝牙的设备节点，如果

存在，说明蓝牙初始化正常

```
orangepi@orangepi:~$ sudo apt update && sudo apt install -y bluez
orangepi@orangepi:~$ hciconfig -a
hci0:   Type: Primary  Bus: UART
        BD Address: 3E:61:3D:19:0E:52  ACL MTU: 1021:8  SCO MTU: 240:3
        UP RUNNING
        RX bytes:925 acl:0 sco:0 events:72 errors:0
        TX bytes:5498 acl:0 sco:0 commands:72 errors:0
        Features: 0xbf 0xff 0x8d 0xfe 0xdb 0x3d 0x7b 0xc7
        Packet type: DM1 DM3 DM5 DH1 DH3 DH5 HV1 HV2 HV3
        Link policy: RSWITCH SNIFF
        Link mode: SLAVE ACCEPT
        Name: 'orangepi'
        Class: 0x3c0000
        Service Classes: Rendering, Capturing, Object Transfer, Audio
        Device Class: Miscellaneous,
        HCI Version: 5.0 (0x9)  Revision: 0x400
        LMP Version: 5.0 (0x9)  Subversion: 0x400
        Manufacturer: Spreadtrum Communications Shanghai Ltd (492)
```

2) 使用 **bluetoothctl** 扫描蓝牙设备

```
orangepi@orangepi:~$ sudo bluetoothctl
[NEW] Controller 10:11:12:13:14:15 orangepizero3w [default]
Agent registered
[bluetooth]# power on          #使能控制器
Changing power on succeeded
[bluetooth]# discoverable on      #设置控制器为可被发现的
Changing discoverable on succeeded
[CHG] Controller 10:11:12:13:14:15 Discoverable: yes
[bluetooth]# pairable on     #设置控制器为可配对的
Changing pairable on succeeded
[bluetooth]# scan on          #开始扫描周围的蓝牙设备
Discovery started
[CHG] Controller 10:11:12:13:14:15 Discovering: yes
[NEW] Device 76:60:79:29:B9:31 76-60-79-29-B9-31
```

```
[NEW] Device 9C:2E:A1:42:71:11 小米手机
[NEW] Device DC:72:9B:4C:F4:CF orangepi
[bluetooth]# scan off          #扫描到想连接的蓝牙设备后就可以关闭扫描了，然后记下蓝牙设备的 MAC 地址，这里测试的蓝牙设备为 Android 手机，蓝牙的名字为 orangepi，对应的 MAC 地址为 DC:72:9B:4C:F4:CF
Discovery stopped
[CHG] Controller 10:11:12:13:14:15 Discovering: no
[CHG] Device DC:72:9B:4C:F4:CF RSSI is nil
```

3) 扫描到想配对的设备后就可以进行配对了，配对需要使用设备的 MAC 地址

```
[bluetooth]# pair DC:72:9B:4C:F4:CF       #使用扫描到的蓝牙设备的 MAC 地址进行配对
Attempting to pair with DC:72:9B:4C:F4:CF
[CHG] Device DC:72:9B:4C:F4:CF Connected: yes
Request confirmation
[leeb1m[agent] Confirm passkey 764475 (yes/no): yes   #在这里输入 yes，在手机上也需要确认
[CHG] Device DC:72:9B:4C:F4:CF Modalias: bluetooth:v010Fp107Ed1436
[CHG] Device DC:72:9B:4C:F4:CF UUIDs: 0000046a-0000-1000-8000-00805f9b34fb
[CHG] Device DC:72:9B:4C:F4:CF ServicesResolved: yes
[CHG] Device DC:72:9B:4C:F4:CF Paired: yes
Pairing successful      #提示配对成功
[CHG] Device DC:72:9B:4C:F4:CF ServicesResolved: no
[CHG] Device DC:72:9B:4C:F4:CF Connected: no
```

4) 配对成功后，手机蓝牙界面的显示如下所示



5) 连接蓝牙设备需要安装 **pulseaudio-module-bluetooth** 软件包，然后再启动

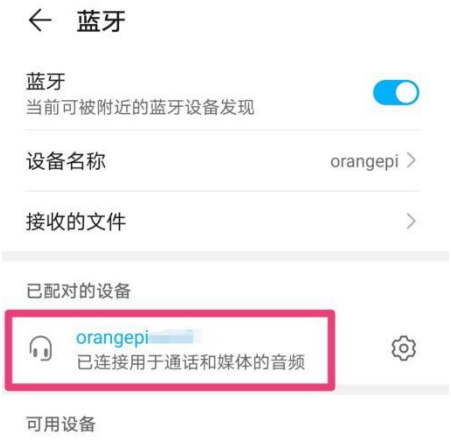
pulseaudio 服务

```
orangepi@orangepi:~$ sudo apt update
orangepi@orangepi:~$ sudo apt -y install pulseaudio-module-bluetooth
orangepi@orangepi:~$ pulseaudio --start
```

6) 连接蓝牙设备的方法

```
orangepi@orangepi:~$ sudo bluetoothctl
Agent registered
[bluetooth]# paired-devices          #查看已配对的蓝牙设备的 MAC 地址
Device DC:72:9B:4C:F4:CF orangepi
[bluetooth]# connect DC:72:9B:4C:F4:CF      #使用 MAC 地址连接蓝牙设备
Attempting to connect to DC:72:9B:4C:F4:CF
[CHG] Device DC:72:9B:4C:F4:CF Connected: yes
Connection successful
[CHG] Device DC:72:9B:4C:F4:CF ServicesResolved: yes
[CHG] Controller 10:11:12:13:14:15 Discoverable: no
[orangepi]#                    #出现这个提示符说明连接成功
```

7) 连接完蓝牙设备后，Android 手机的蓝牙配置界面就可以看到**已连接用于通话和媒体的音频**的提示

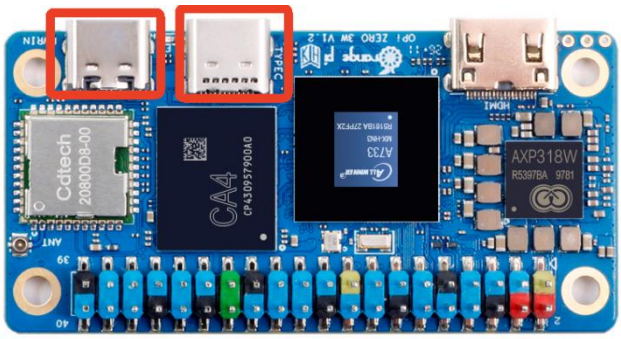


3.12. USB 接口测试

USB 接口是可以接 USB hub 来扩展 USB 接口的数量的。

3.12.1. USB 接口扩展说明

如下图所示，开发板的主板上有两个 Type-C 类型的 USB 接口，是无法直接接 USB Type-A 类型的鼠标、键盘等 USB 设备的。注意只有左边的 Type-C 接口支持供电。



可以准备一根线下图所示的 Type-C 转 USB 线，将其 Type-C 接口的一端插入主板的 Type-C 接口中，然后另一端就可以接鼠标键盘等 USB 设备了，如果觉得一个 USB 接口不够用，还可以通过 USB Hub 来扩展多个 USB 接口。



3.12.2. 连接 USB 鼠标或键盘测试

1) 将 USB 接口的键盘插入 Orange Pi 开发板的 USB 接口中

2) 连接 Orange Pi 开发板到 HDMI 显示器

3) 如果鼠标或键盘能正常操作系统说明 USB 接口使用正常（鼠标只有在桌面版的系统中才能使用）

3.12.3. 连接 USB 存储设备测试

1) 首先将 U 盘或者 USB 移动硬盘插入 Orange Pi 开发板的 USB 接口中

2) 执行下面的命令如果能看到 sdX 的输出说明 U 盘识别成功

```
orangepi@orangepi:~$ cat /proc/partitions | grep "sd*"
major minor  #blocks  name
   8        0   30044160 sda
   8        1   30043119 sda1
```

3) 使用 mount 命令可以将 U 盘挂载到**/mnt** 中，然后就能查看 U 盘中的文件了

```
orangepi@orangepi:~$ sudo mount /dev/sda1 /mnt/
orangepi@orangepi:~$ ls /mnt/
test.txt
```

4) 挂载完后通过 **df -h** 命令就能查看 U 盘的容量使用情况和挂载点

```
orangepi@orangepi:~$ df -h | grep "sd"
/dev/sda1       29G  208K   29G   1% /mnt
```

3.12.4. USB 以太网卡测试

1) 目前**测试过**能用的 USB 以太网卡如下所示，其中 RTL8153 USB 千兆网卡插入开发板的 USB 2.0 Host 接口中测试可以正常使用，但是速率是达不到千兆的，这点请注意

序号	型号
1	RTL8152B USB 百兆网卡
2	RTL8153 USB 千兆网卡

2) 首先将 USB 网卡插入开发板的 USB 接口中，然后在 USB 网卡中插入网线，确保网线能正常上网，如果通过 **dmesg** 命令可以看到下面的 log 信息，说明 USB 网卡识别正常

```
orangepi@orangepi:~$ dmesg | tail
[  121.985016] usb 3-1: USB disconnect, device number 2
[  126.873772] sunxi-ehci 5311000.ehci3-controller: ehci_irq: highspeed device connect
[  127.094054] usb 3-1: new high-speed USB device number 3 using sunxi-ehci
[  127.357472] usb 3-1: reset high-speed USB device number 3 using sunxi-ehci
[  127.557960] r8152 3-1:1.0 eth1: v1.08.9
[  127.602642] r8152 3-1:1.0 enx00e04c362017: renamed from eth1
[  127.731874] IPv6: ADDRCONF(NETDEV_UP): enx00e04c362017: link is not ready
```

```
[    127.763031] IPv6: ADDRCONF(NETDEV_UP): enx00e04c362017: link is not ready
[    129.892465] r8152 3-1:1.0 enx00e04c362017: carrier on
[    129.892583] IPv6: ADDRCONF(NETDEV_CHANGE): enx00e04c362017: link becomes ready
```

3) 然后通过 ifconfig 命令可以看到 USB 网卡的设备节点，以及自动分配的 IP 地址

```
orangepi@orangepi:~$ sudo ifconfig
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.177  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::681f:d293:4bc5:e9fd  prefixlen 64  scopeid 0x20<link>
        ether 00:e0:4c:36:20:17  txqueuelen 1000  (Ethernet)
        RX packets 1849  bytes 134590 (134.5 KB)
        RX errors 0  dropped 125  overruns 0  frame 0
        TX packets 33  bytes 2834 (2.8 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

4) 测试网络连通性的命令如下（注意 eth1 需替换为实际的网络设备）

```
orangepi@orangepi:~$ ping www.baidu.com -I eth1
PING www.a.shifen.com (14.215.177.38) from 192.168.1.12 eth0: 56(84) bytes of data.
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=1 ttl=56 time=6.74 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=2 ttl=56 time=6.80 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=3 ttl=56 time=6.26 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=4 ttl=56 time=7.27 ms
^C
--- www.a.shifen.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 6.260/6.770/7.275/0.373 ms
```

3.12.5. USB 摄像头测试

1) 首先将 USB 摄像头插入到 Orange Pi 开发板的 USB 接口中

2) 然后通过 lsmod 命令可以看到内核自动加载了下面的模块

```
orangepi@orangepi:~$ lsmod
Module                  Size  Used by
uvcvideo              106496  0
```

3) 通过 v4l2-ctl 命令可以看到 USB 摄像头的设备节点信息为/dev/video0

```
orangepi@orangepi:~$ sudo apt update
orangepi@orangepi:~$ sudo apt install -y v4l-utils
orangepi@orangepi:~$ v4l2-ctl --list-devices
USB 2.0 Camera (usb-sunxi-ehci-1):
    /dev/video0
```

注意 v4l2 中的 l 是小写字母 l，不是数字 1。
另外 video 的序号不一定都是 video0，请以实际看到的为准。

4) 使用 fswebcam 测试 USB 摄像头

a. 安装 fswebcam

```
orangepi@orangepi:~$ sudo apt update
orangepi@orangepi:~$ sudo apt-get install -y fswebcam
```

b. 安装完 fswebcam 后可以使用下面的命令来拍照

a) -d 选项用于指定 USB 摄像头的设备节点
b) --no-banner 用于去除照片的水印
c) -r 选项用于指定照片的分辨率
d) -S 选项用设置于跳过前面的帧数
e) ./image.jpg 用于设置生成的照片的名字和路径

```
orangepi@orangepi:~$ sudo fswebcam -d /dev/video0   \
--no-banner -r 1280x720 -S 5 ./image.jpg
```

c. 在服务器版的 linux 系统中，拍完照后可以使用 scp 命令将拍好的图片传到 Ubuntu PC 上镜像观看

```
orangepi@orangepi:~$ scp image.jpg test@192.168.1.55:/home/test（根据实际情况修改 IP 地址和路径）
```

d. 在桌面版的 linux 系统中，可以通过 HDMI 显示器直接查看拍摄的图片

5) 使用 mjpg-streamer 测试 USB 摄像头

a. 下载 mjpg-streamer

a) Github 的下载地址：

```
orangepi@orangepi:~$ git clone https://github.com/jacksonliam/mjpg-streamer
```

b) Gitee 的镜像下载地址为：

```
orangepi@orangepi:~$ git clone https://gitee.com/leeboby/mjpg-streamer
```

b. 安装依赖的软件包

a) Ubuntu 系统

```
orangepi@orangepi:~$ sudo apt-get install -y cmake libjpeg8-dev
```

b) Debian 系统

```
orangepi@orangepi:~$ sudo apt-get install -y cmake libjpeg62-turbo-dev
```

c. 编译安装 mjpg-streamer

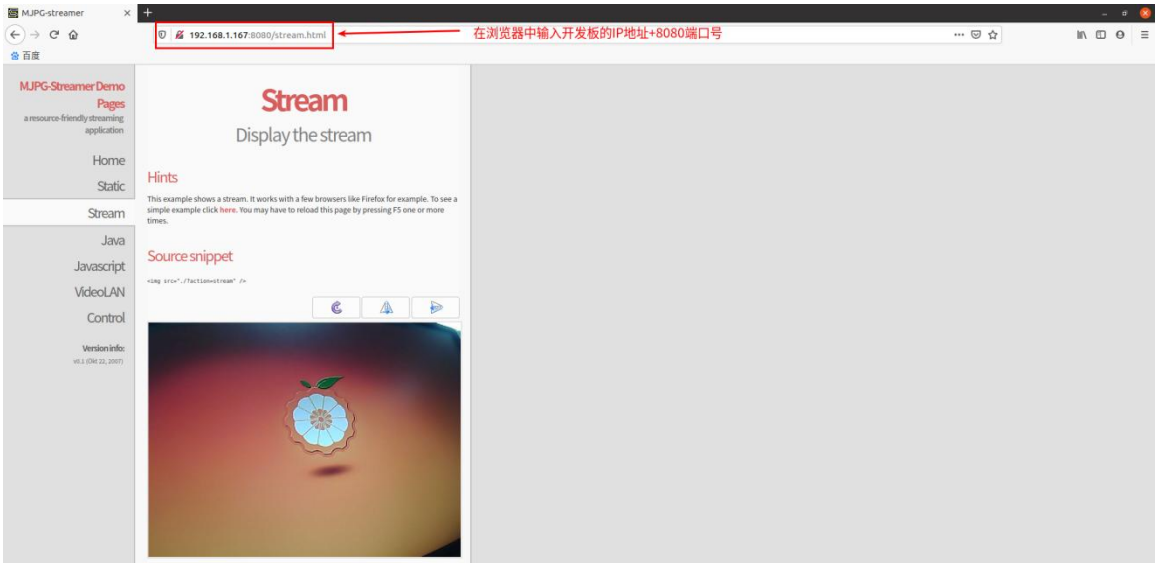
```
orangepi@orangepi:~$ cd mjpg-streamer/mjpg-streamer-experimental
orangepi@orangepi:~/mjpg-streamer/mjpg-streamer-experimental$ make -j4
orangepi@orangepi:~/mjpg-streamer/mjpg-streamer-experimental$ sudo make install
```

d. 然后输入下面的命令启动 mjpg_streamer

注意，video 的序号不一定都是 video0，请以实际看到的为准。

```
orangepi@orangepi:~/mjpg-streamer/mjpg-streamer-experimental$ export LD_LIBRARY_PATH=.
orangepi@orangepi:~/mjpg-streamer/mjpg-streamer-experimental$ sudo ./mjpg_streamer -i "./input_uvc.so -d \
/dev/video0 -u -f 30" -o "./output_http.so -w ./www"
```

e. 然后在和开发板同一局域网的 Ubuntu PC 或者 Windows PC 或者手机的浏览器中输入【开发板的 IP 地址:8080】就能看到摄像头输出的视频了



3.13. 温度传感器

1) A733 包含多个温度传感器，可通过系统提供的/sys/class/thermal/thermal_zoneX 接口获取，其中：

a. type：表示传感器类型（如 CPU、GPU、DDR 等）

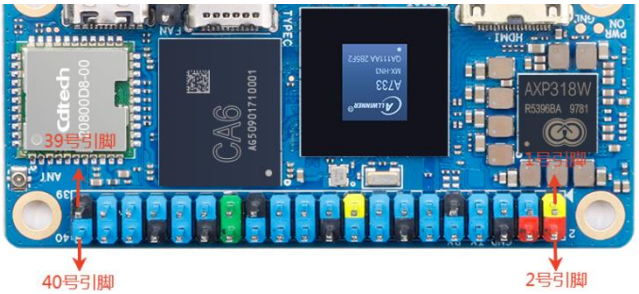
b. temp：表示温度值（单位为毫摄氏度，需除以 1000）

2) 系统还提供了 **check_temp.sh** 脚本，可一次性列出所有温度传感器及对应温度：

```
orangepi@orangepi:~$ check_temp.sh
ZONE              TYPE                 TEMP(°C)
thermal_zone5     cpul_idle_zone       48.7
thermal_zone3     cpul_thermal_zone    48.7
thermal_zone1     ddr_thermal_zone     48.7
thermal_zone6     cpub_idle_zone       48.4
thermal_zone4     gpu_thermal_zone     48.0
thermal_zone0     cpub_thermal_zone    47.9
thermal_zone2     npu_thermal_zone     47.5
thermal_zone7     skin_zone            33.7
orangepi@orangepi:~$
```

3. 14. 40 pin 接口引脚说明

1) Orange Pi Zero 3w 开发板 40 pin 接口引脚的顺序请参考开发板上的丝印图



2) 开发板 40 pin 接口引脚的功能如下表所示

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PB3	35	3	4		5V		
		TWI0-SCK	PB2	34	5	6		GND		
		PWM0-0	PB4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PB1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PB6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PB8	TWI1-SDA	
			GND		39	40	39	PB7	TWI1-SCL	PWM0-9

3) 40 pin 接口中总共有 **28** 个 GPIO 口，所有 GPIO 口的电压都是 **3.3v**

3.15. 安装 wiringOP 的方法

注意，Orange Pi 发布的 linux 镜像中已经预装了 wiringOP，除非 wiringOP 的代码有更新，否则无需重新下载编译安装，直接使用即可。

编译好的 wiringOP 的 deb 包在 orangepi-build 中的存放路径为：
orangepi-build/external/cache/debs/arm64/wiringpi_x.xx.deb

进入系统后可以运行下 gpio readall 命令，如果能看到下面的输出，说明 wiringOP 已经预装并且能正常使用。

```
orangepi@orangepizero3w:~$ gpio readall
 +------+-----+----------+--------+---+  ZERO3W  +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |   35 |   0 |    SDA.0 |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
 |   34 |   1 |    SCL.0 |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
 |   36 |   2 |  PWM0-0  |    OFF | 0 |  7 || 8  | 0 | ALT2   | TXD.0    | 3   | 41   |
 |      |     |      GND |        |   |  9 || 10 | 0 | ALT2   | RXD.0    | 4   | 42   |
 |   32 |   5 |      PB0 |  ALT14 | 1 | 11 || 12 | 0 | OFF    | PB5      | 6   | 37   |
 |   33 |   7 |      PB1 |  ALT14 | 1 | 13 || 14 |   |        | GND      |     |      |
 |  137 |   8 |      PE9 |    OFF | 0 | 15 || 16 | 0 | OFF    | PL2      | 9   | 354  |
 |      |     |     3.3V |        |   | 17 || 18 | 0 | OFF    | PL3      | 10  | 355  |
 |  130 |  11 |  MOSI.3  |    OFF | 0 | 19 || 20 |   |        | GND      |     |      |
 |  131 |  12 |  MISO.3  |    OFF | 0 | 21 || 22 | 0 | OFF    | PD0      | 13  | 96   |
 |  129 |  14 |  SCLK.3  |    OFF | 0 | 23 || 24 | 0 | OFF    | CE.0     | 15  | 128  |
 |      |     |      GND |        |   | 25 || 26 | 0 | OFF    | PE4      | 16  | 132  |
 |  357 |  17 |      PL5 |    OFF | 0 | 27 || 28 | 0 | OFF    | PL4      | 18  | 356  |
 |  140 |  19 |     PE12 |    OFF | 0 | 29 || 30 |   |        | GND      |     |      |
 |  141 |  20 |     PE13 |    OFF | 0 | 31 || 32 | 0 | OFF    | PD1      | 21  | 97   |
 |   99 |  22 |      PD3 |    OFF | 0 | 33 || 34 |   |        | GND      |     |      |
 |   38 |  23 |      PB6 |    OFF | 0 | 35 || 36 | 0 | OFF    | PD2      | 24  | 98   |
 |  100 |  25 |      PD4 |    OFF | 0 | 37 || 38 | 0 | OFF    | PB8      | 26  | 40   |
 |      |     |      GND |        |   | 39 || 40 | 0 | OFF    | PB7      | 27  | 39   |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+  ZERO3W  +---+--------+----------+-----+------+
```

1) 下载 wiringOP 的代码

```
orangepi@orangepi:~$ sudo apt update
orangepi@orangepi:~$ sudo apt install -y git
orangepi@orangepi:~$ git clone https://github.com/orangepi-xunlong/wiringOP.git -b next
```

注意，源码需要下载 wiringOP next 分支的代码，请别漏了-b next 这个参数。
如果从 GitHub 下载代码有问题，可以直接使用 Linux 镜像中自带的 wiringOP 源码，存放位置为：/usr/src/wiringOP。

2) 编译安装 wiringOP

```
orangepi@orangepi:~$ cd wiringOP
orangepi@orangepi:~/wiringOP$ sudo ./build clean
orangepi@orangepi:~/wiringOP$ sudo ./build
```

3) 测试 gpio readall 命令的输出如下

```
orangepi@orangepizero3w:~$ gpio readall
 +------+-----+----------+--------+---+  ZERO3W  +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  |   Name   | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |   35 |   0 |   SDA.0  |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
 |   34 |   1 |   SCL.0  |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
 |   36 |   2 |  PWM0-0  |    OFF | 0 |  7 || 8  | 0 | ALT2   | TXD.0    | 3   | 41   |
 |      |     |      GND |        |   |  9 || 10 | 0 | ALT2   | RXD.0    | 4   | 42   |
 |   32 |   5 |      PB0 |  ALT14 | 1 | 11 || 12 | 0 | OFF    | PB5      | 6   | 37   |
 |   33 |   7 |      PB1 |  ALT14 | 1 | 13 || 14 |   |        | GND      |     |      |
 |  137 |   8 |      PE9 |    OFF | 0 | 15 || 16 | 0 | OFF    | PL2      | 9   | 354  |
 |      |     |     3.3V |        |   | 17 || 18 | 0 | OFF    | PL3      | 10  | 355  |
 |  130 |  11 |  MOSI.3  |    OFF | 0 | 19 || 20 |   |        | GND      |     |      |
 |  131 |  12 |  MISO.3  |    OFF | 0 | 21 || 22 | 0 | OFF    | PD0      | 13  | 96   |
 |  129 |  14 |  SCLK.3  |    OFF | 0 | 23 || 24 | 0 | OFF    | CE.0     | 15  | 128  |
 |      |     |      GND |        |   | 25 || 26 | 0 | OFF    | PE4      | 16  | 132  |
 |  357 |  17 |      PL5 |    OFF | 0 | 27 || 28 | 0 | OFF    | PL4      | 18  | 356  |
 |  140 |  19 |     PE12 |    OFF | 0 | 29 || 30 |   |        | GND      |     |      |
 |  141 |  20 |     PE13 |    OFF | 0 | 31 || 32 | 0 | OFF    | PD1      | 21  | 97   |
 |   99 |  22 |      PD3 |    OFF | 0 | 33 || 34 |   |        | GND      |     |      |
 |   38 |  23 |      PB6 |    OFF | 0 | 35 || 36 | 0 | OFF    | PD2      | 24  | 98   |
 |  100 |  25 |      PD4 |    OFF | 0 | 37 || 38 | 0 | OFF    | PB8      | 26  | 40   |
 |      |     |      GND |        |   | 39 || 40 | 0 | OFF    | PB7      | 27  | 39   |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  |   Name   | wPi | GPIO |
 +------+-----+----------+--------+---+  ZERO3W  +---+--------+----------+-----+------+
```

3.16. 40pin 接口 GPIO、I2C、UART、SPI 和 PWM 测试

3.16.1. 40pin GPIO 口测试

1) 下面以 7 号引脚——对应 GPIO 为 PB4——对应 wPi 序号为 2——为例演示如何设置 GPIO 口的高低电平

```
orangepi@orangepizero3w:~$ gpio readall
 +------+-----+----------+--------+---+  ZERO3W  +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  |   Name   | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |   35 |   0 |   SDA.0  |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
 |   34 |   1 |   SCL.0  |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
 |   36 |   2 |  PWM0-0  |    OFF | 0 |  7 || 8  | 0 | ALT2   | TXD.0    | 3   | 41   |
 |      |     |      GND |        |   |  9 || 10 | 0 | ALT2   | RXD.0    | 4   | 42   |
```

2) 首先设置 GPIO 口为输出模式，其中第三个参数需要输入引脚对应的 wPi 的序号

```
orangepi@orangepi:~/wiringOP$ gpio mode 2 out
```

3) 然后设置 GPIO 口输出低电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 0v，说明设置低电平成功

```
orangepi@orangepi:~/wiringOP$ gpio write 2 0
```

使用 gpio readall 可以看到 7 号引脚的值(V)变为了 0

```
orangepi@orangepizero3w:~$ gpio readall
 +------+-----+----------+--------+---+---+  ZERO3W  +---+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  |   Name   | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |   35 |   0 |    SDA.0 |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
 |   34 |   1 |    SCL.0 |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
 |   36 |   2 |  PWM0-0  |    OUT | 0 |  7 || 8  | 0 | ALT2   | TXD.0    | 3   | 41   |
 |      |     |      GND |        |   |  9 || 10 | 0 | ALT2   | RXD.0    | 4   | 42   |
```

4) 然后设置 GPIO 口输出高电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 3.3v，说明设置高电平成功

```
orangepi@orangepi:~/wiringOP$ gpio write 2 1
```

使用 gpio readall 可以看到 7 号引脚的值(V)变为了 1

```
orangepi@orangepizero3w:~$ gpio readall
 +------+-----+----------+--------+---+---+  ZERO3W  +---+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  |   Name   | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |   35 |   0 |    SDA.0 |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
 |   34 |   1 |    SCL.0 |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
 |   36 |   2 |  PWM0-0  |    OUT | 1 |  7 || 8  | 0 | ALT2   | TXD.0    | 3   | 41   |
 |      |     |      GND |        |   |  9 || 10 | 0 | ALT2   | RXD.0    | 4   | 42   |
```

5) 其他引脚的设置方法类似，只需修改 wPi 的序号为引脚对应的序号即可

3.16.2. 40 pin GPIO 口上下拉电阻的设置方法

1) 下面以 7 号引脚——对应 GPIO 为 PB4——对应 wPi 序号为 2——为例演示如何设置 GPIO 口的上下拉电阻

```
orangepi@orangepizero3w:~$ gpio readall
 +------+-----+----------+--------+---+---+  ZERO3W  +---+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  |   Name   | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |   35 |   0 |    SDA.0 |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
 |   34 |   1 |    SCL.0 |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
 |   36 |   2 |  PWM0-0  |    OFF | 0 |  7 || 8  | 0 | ALT2   | TXD.0    | 3   | 41   |
 |      |     |      GND |        |   |  9 || 10 | 0 | ALT2   | RXD.0    | 4   | 42   |
```

2) 首先需要设置 GPIO 口为输入模式，其中第三个参数需要输入引脚对应的 wPi 的序号

```
orangepi@orangepi:~/wiringOP$ gpio mode 2 in
```

3) 设置为输入模式后，执行下面的命令可以设置 GPIO 口为上拉模式

```
orangepi@orangepi:~/wiringOP$ gpio mode 2 up
```

4) 然后输入下面的命令读取 GPIO 口的电平，如果电平为 1，说明上拉模式设置成功

```
orangepi@orangepi:~/wiringOP$ gpio read 2
1
```

5) 然后执行下面的命令可以设置 GPIO 口为下拉模式

```
orangepi@orangepi:~/wiringOP$ gpio mode 2 down
```

6) 然后输入下面的命令读取 GPIO 口的电平，如果电平为 0，说明下拉模式设置成功

```
orangepi@orangepi:~/wiringOP$ gpio read 2
0
```

3.16.3. 40 pin SPI 测试

1) 由下图可知，Orange Pi Zero 3w 可用的 SPI 为 SPI3。

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PB3	35	3	4		5V		
		TWI0-SCK	PB2	34	5	6		GND		
		PWM0-0	PB4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PB1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PB6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PB8	TWI1-SDA	
			GND		39	40	39	PB7	TWI1-SCL	PWM0-9

2) SPI3 在 40pin 中对应的引脚如下表所示。

	SPI3 对应 40pin
MOSI	19 号引脚
MISO	21 号引脚

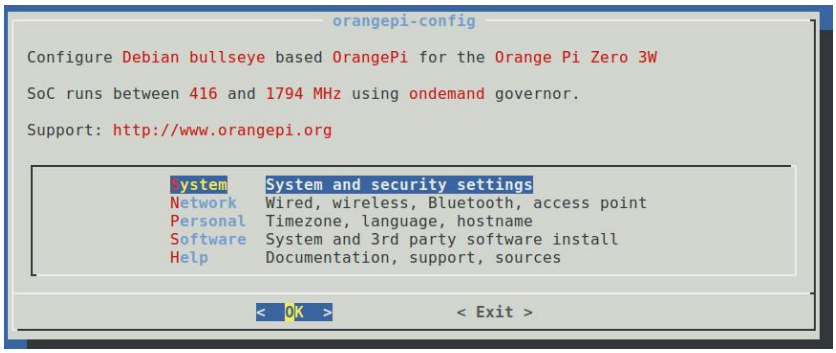
CLK	23 号引脚
CS0	24 号引脚
CS1	26 号引脚

3) 在 linux 系统中，40 pin 中的 SPI 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

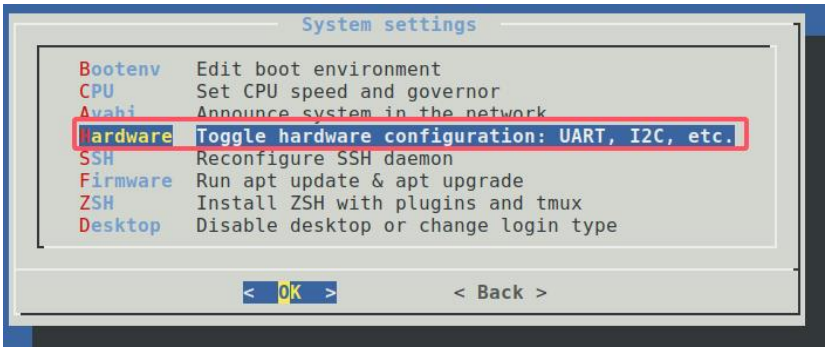
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**



c. 然后选择 **Hardware**



d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中打开的 SPI 的配置



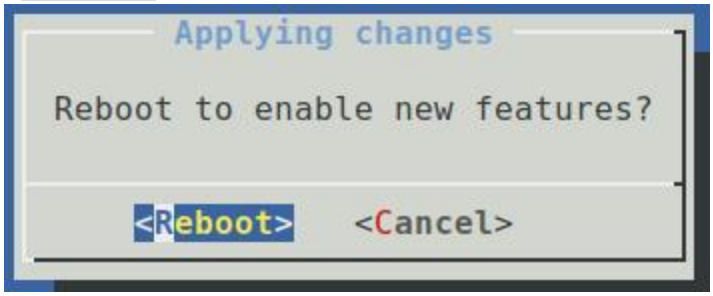
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



4) 然后查看下 linux 系统中是否存在 **spidevx.x** 的设备节点，如果存在，说明 SPI 的配置已经生效了

```
orangepi@orangepi:~$ ls /dev/spidev*
/dev/spidev3.0
```

5) 先不短接 SPI3 的 mosi 和 miso 两个引脚，运行 spidev_test 的输出结果如下所示，可以看到 TX 和 RX 的数据不一致

```
orangepi@orangepi:~$ sudo spidev_test -v -D /dev/spidev3.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D    | ......@.....................
RX | FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF    | ................................
```

6) 然后短接 SPI3 的 mosi 和 miso 两个引脚再运行 spidev_test 的输出如下，可以看到发送和接收的数据一样

```
orangepi@orangepi:~$ sudo spidev_test -v -D /dev/spidev3.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D    | ......@.....................
```

```
RX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D  | ......@....▒.....................▒.
```

3. 16. 4. 40 pin I2C 测试

1) 由下图可知，Orange Pi Zero 3w 可用的 i2c 为 i2c0、i2c1、i2c2 和 i2c3 共四组 i2c 总线。

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PB3	35	3	4		5V		
		TWI0-SCK	PB2	34	5	6		GND		
		PWM0-0	PB4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PB1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PB6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PB8	TWI1-SDA	
			GND		39	40	39	PB7	TWI1-SCL	PWM0-9

2) 四组 I2C 总线在 40pin 中对应的引脚如下表所示。

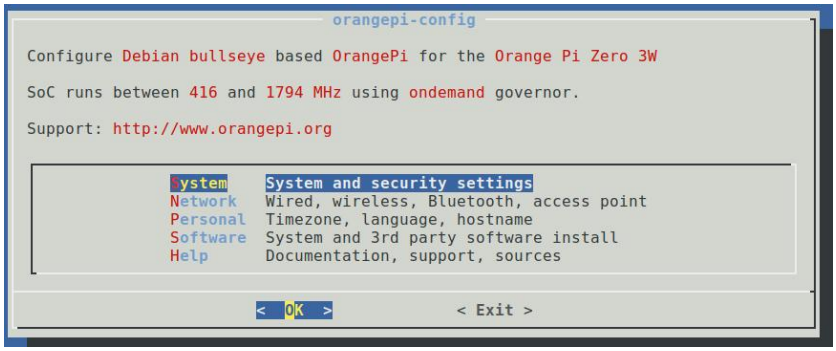
I2C 总线	SDA 对应 40pin	SCL 对应 40pin	dtbo 对应配置
I2C0	3 号引脚	5 号引脚	i2c0
I2C1	38 号引脚	40 号引脚	i2c1
I2C2	19 号引脚	23 号引脚	i2c2
I2C3	26 号引脚	21 号引脚	i2c3

3) 在 linux 系统中，40 pin 中的 I2C 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

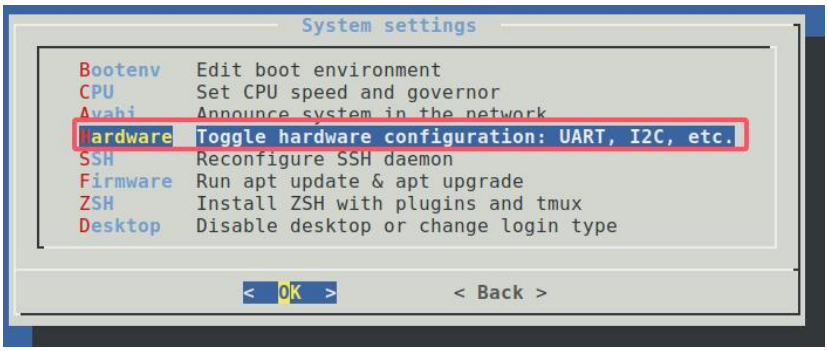
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**



c. 然后选择 **Hardware**



d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中打开的 I2C 的配置



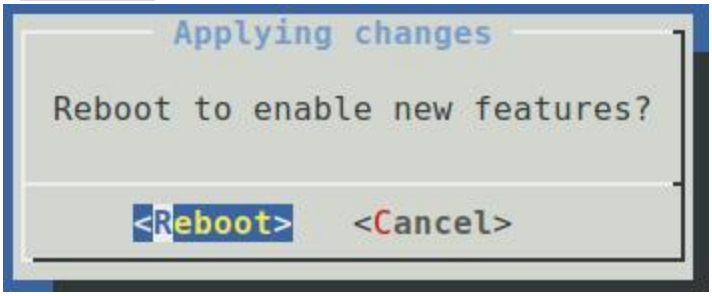
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



4) 启动 linux 系统后，先确认下/dev 下存在 i2c 的设备节点

```
orangepi@orangepi:~$ ls /dev/i2c-*
/dev/i2c-0  /dev/i2c-1  /dev/i2c-13  /dev/i2c-15  /dev/i2c-2  /dev/i2c-20
```

```
/dev/i2c-3   /dev/i2c-5   /dev/i2c-7   /dev/i2c-8   /dev/i2c-9
```

5) 然后开始测试 i2c，首先安装 i2c-tools

```
orangepi@orangepi:~$ sudo apt-get update
orangepi@orangepi:~$ sudo apt-get install -y i2c-tools
```

6) 然后在 40 pin 接头的 i2c 引脚上接一个 i2c 设备

5V 和 3.3V 引脚请根据具体的 i2c 设备进行选择，不同的 i2c 设备需要的电压值可能不同。

7) 然后使用 **i2cdetect -y** 命令如果能检测到连接的 i2c 设备的地址，就说明 i2c 能正常使用

```
orangepi@orangepi:~$ sudo i2cdetect -y -r 0      #i2c0 的命令
orangepi@orangepi:~$ sudo i2cdetect -y -r 1      #i2c1 的命令
orangepi@orangepi:~$ sudo i2cdetect -y -r 2      #i2c2 的命令
orangepi@orangepi:~$ sudo i2cdetect -y -r 3      #i2c3 的命令
```

3.16.5. 40 pin 的 UART 测试

1) 由下表可知，Orange Pi Zero 3w 可用的 uart 为 uart2、uart6 和 uart7 共三组 UART 总线。

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PB3	35	3	4		5V		
		TWI0-SCK	PB2	34	5	6		GND		
		PWM0-0	PB4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PB1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PB6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PB8	TWI1-SDA	
			GND		39	40	39	PB7	TWI1-SCL	PWM0-9

2) 三组 UART 总线在 40 pin 中对应的引脚如下表所示。

UART 总线	RX 对应 40pin	TX 对应 40pin	dtbo 对应配置
UART2	13 号引脚	11 号引脚	uart2
UART6	23 号引脚	24 号引脚	uart6
UART7	18 号引脚	16 号引脚	uart7

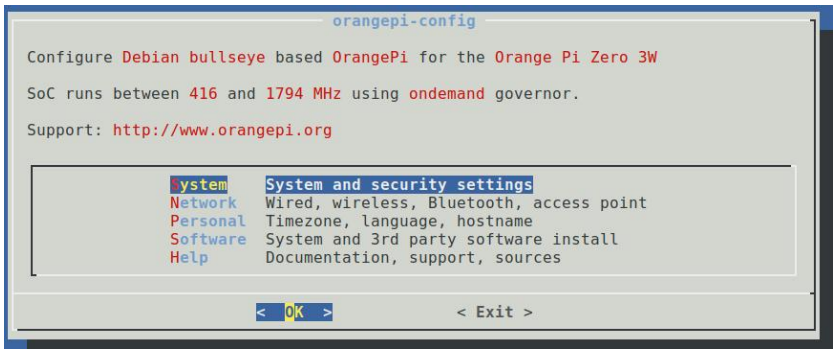
3) 在 linux 系统中，40 pin 中的 UART 默认都是关闭的，需要手动打开才能使用。

详细步骤如下所示：

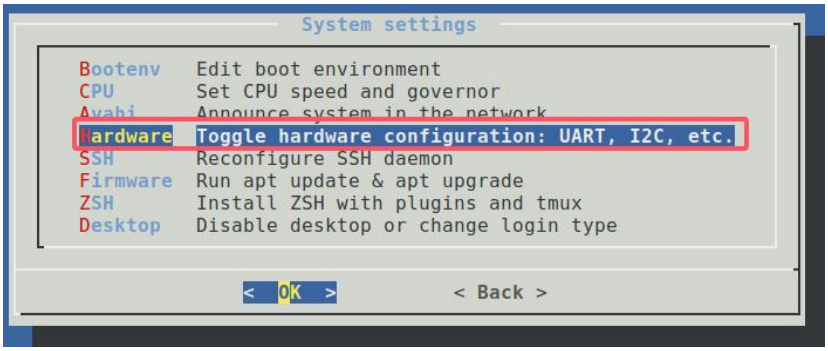
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**



c. 然后选择 **Hardware**



d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中打开的 UART 的配置



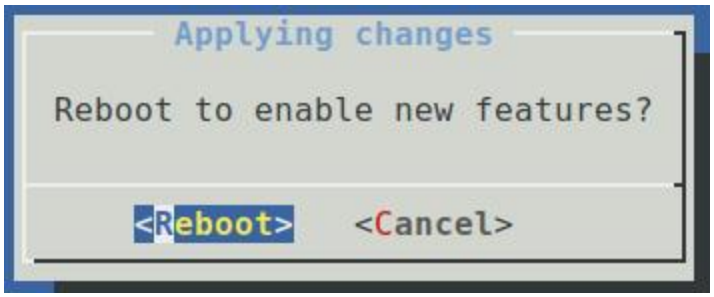
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



4) 进入 linux 系统后，先确认下**/dev** 下是否存在 uart 的设备节点

```
orangepi@orangepi:~$ ls /dev/ttyS*
/dev/ttyS0   /dev/ttyS1   /dev/ttyS2   /dev/ttyS6   /dev/ttyS7
```

5) 然后开始测试 uart 接口，先使用杜邦线短接要测试的 uart 接口的 rx 和 tx

6) 使用 **gpio serial** 命令测试串口的回环功能如下所示，如果能看到下面的打印，说明串口通信正常

注意，gpio serial /dev/ttySx 命令中最后的 x 需要替换为对应的 uart 设备节点的序号。

```
orangepi@orangepi:~$ gpio serial /dev/ttySx

Out:    0:  ->    0
Out:    1:  ->    1
Out:    2:  ->    2
Out:    3:  ->    3^C
```

3.16.6. 使用/sys/class/pwm/测试 PWM 的方法

1) 由下表可知，Orange Pi Zero 3w 可用的 pwm 有 pwm0_0、pwm0_1、pwm0_2、pwm0_3、pwm0_4、pwm0_8 和 pwm0_9 共 7 路 PWM

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PB3	35	3	4		5V		
		TWI0-SCK	PB2	34	5	6		GND		
		PWM0-0	PB4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PB1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PB6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PB8	TWI1-SDA	
			GND		39	40	39	PB7	TWI1-SCL	PWM0-9

2) PWM 在 40pin 中对应的引脚如下表所示。

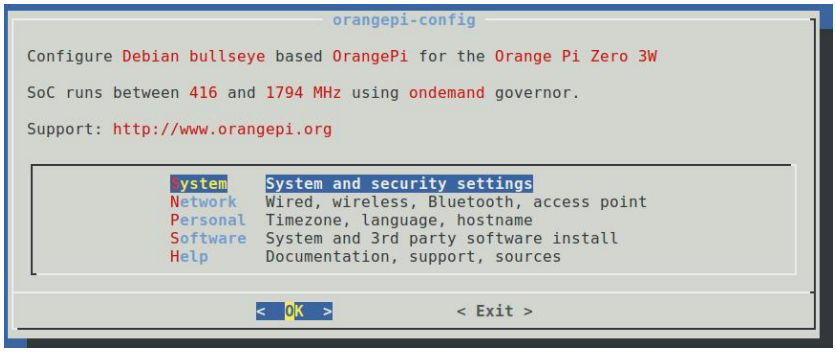
PWM 总线	对应 40pin	dtbo 对应配置
PWM0-0	7 号引脚	pwm0
PWM0-1	32 号引脚	pwm1
PWM0-2	36 号引脚	pwm2
PWM0-3	33 号引脚	pwm3
PWM0-4	37 号引脚	pwm4
PWM0-8	35 号引脚	pwm8
PWM0-9	40 号引脚	pwm9

3) 在 linux 系统中，40 pin 中的 PWM 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

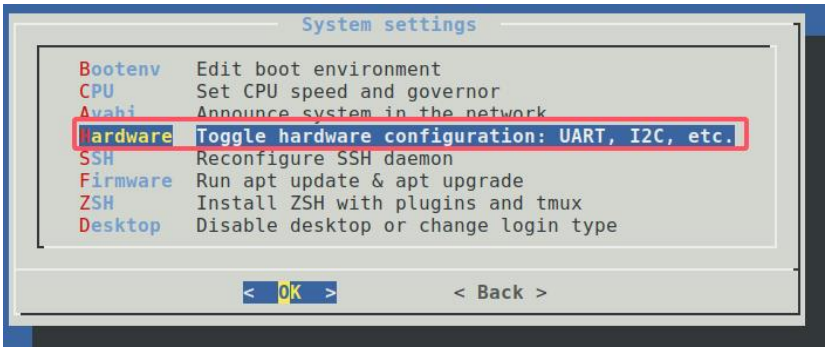
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

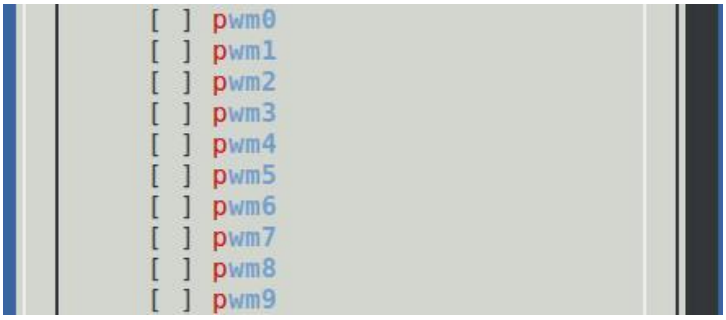
b. 然后选择 **System**



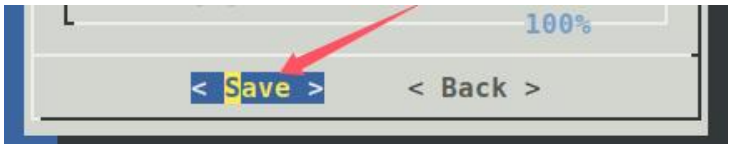
c. 然后选择 **Hardware**



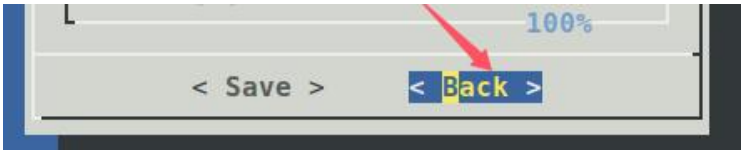
d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中打开的 PWM 的配置



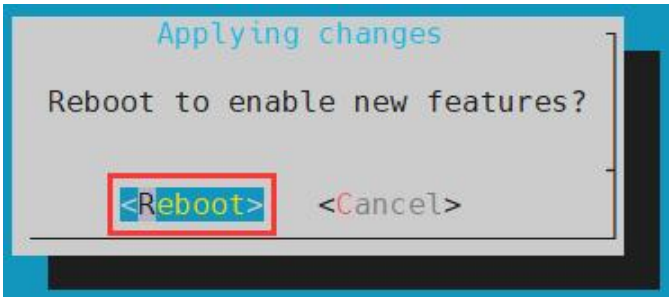
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



4) 重启后就可以开始 PWM 的测试

下面的命令请在 root 用户下执行。

a. 在命令行中输入下面的命令可以让 pwm0 输出一个 50Hz 的方波

```
root@orangepi:~# echo 0 > /sys/class/pwm/pwmchip0/export
root@orangepi:~# echo 20000000 > /sys/class/pwm/pwmchip0/pwm0/period
root@orangepi:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable
```

b. 在命令行中输入下面的命令可以让 pwm1 输出一个 50Hz 的方波

```
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/export
root@orangepi:~# echo 20000000 > /sys/class/pwm/pwmchip0/pwm1/period
root@orangepi:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm1/duty_cycle
```

```
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/pwm1/enable
```

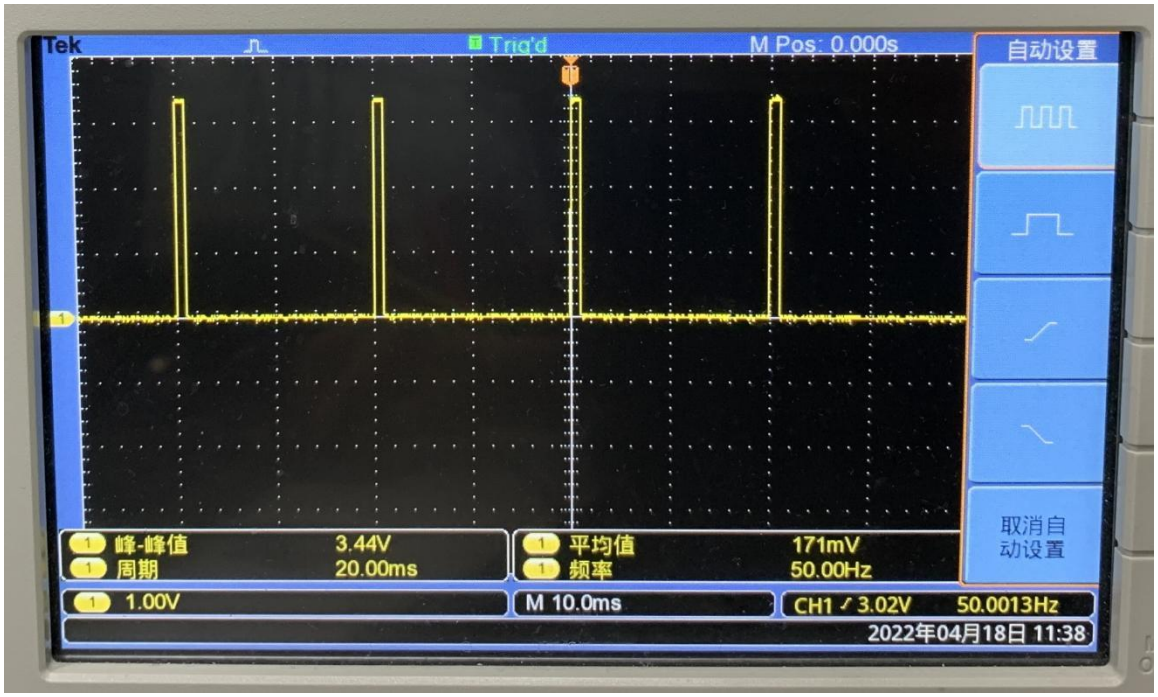
c. 在命令行中输入下面的命令可以让 pwm2 输出一个 50Hz 的方波

```
root@orangepi:~# echo 2 > /sys/class/pwm/pwmchip0/export
root@orangepi:~# echo 20000000 > /sys/class/pwm/pwmchip0/pwm2/period
root@orangepi:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm2/duty_cycle
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/pwm2/enable
```

d. 在命令行中输入下面的命令可以让 pwm3 输出一个 50Hz 的方波

```
root@orangepi:~# echo 3 > /sys/class/pwm/pwmchip0/export
root@orangepi:~# echo 20000000 > /sys/class/pwm/pwmchip0/pwm3/period
root@orangepi:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm3/duty_cycle
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/pwm3/enable
```

e. 其它的 pwm 的测试方法都是类似的，这里不再赘述



3.17. wiringOP-Python 的安装使用方法

wiringOP-Python 是 wiringOP 的 Python 语言版本的库，用于在 Python 程序中操作开发板的 GPIO、I2C、SPI 和 UART 等硬件资源。

另外请注意下面所有的命令都是在 root 用户下操作的。

3.17.1. wiringOP-Python 的安装方法

1) 首先安装依赖包

```
root@orangepi:~# sudo apt-get update
root@orangepi:~# sudo apt-get -y install git swig python3-dev python3-setuptools
```

2) 然后使用下面的命令下载 wiringOP-Python 的源码

注意，下面的 git clone--recursive 命令会自动下载 wiringOP 的源码，因为 wiringOP-Python 是依赖 wiringOP 的。请确保下载过程没有因为网络问题而报错。

如果从 GitHub 下载代码有问题，可以直接使用 Linux 镜像中自带的 wiringOP-Python 源码，存放位置为：/usr/src/wiringOP-Python。

```
root@orangepi:~# git clone --recursive https://github.com/orangepi-xunlong/wiringOP-Python -b next
root@orangepi:~# cd wiringOP-Python
root@orangepi:~/wiringOP-Python# git submodule update --init --remote
```

3) 然后使用下面的命令编译 wiringOP-Python 并将其安装到开发板的 Linux 系统中

```
root@orangepi:~# cd wiringOP-Python
root@orangepi:~/wiringOP-Python# python3 generate-bindings.py > bindings.i
root@orangepi:~/wiringOP-Python# sudo python3 setup.py install
```

4) 然后输入下面的命令，如果有帮助信息输出，说明 wiringOP-Python 安装成功，按下 **q** 键可以退出帮助信息的界面

```
root@orangepi:~/wiringOP-Python# python3 -c "import wiringpi;  help(wiringpi)"
Help on module wiringpi:

NAME
    wiringpi

DESCRIPTION
    # This file was automatically generated by SWIG (http://www.swig.org).
    # Version 4.0.2
    #
    # Do not make changes to this file unless you know what you are doing--modify
    # the SWIG interface file instead.
```

5) 在 python 命令行下测试 wiringOP-Python 是否安装成功的步骤如下所示：

a. 首先使用 python3 命令进入 python3 的命令行模式

```
root@orangepi:~# python3
```

b. 然后导入 wiringpi 的 python 模块

```
>>> import wiringpi;
```

c. 最后输入下面的命令可以查看下 wiringOP-Python 的帮助信息，按下 **q** 键可以退出帮助信息的界面

```
>>> help(wiringpi)
Help on module wiringpi:

NAME
    wiringpi

DESCRIPTION
    # This file was automatically generated by SWIG (http://www.swig.org).
    # Version 4.0.2
    #
    # Do not make changes to this file unless you know what you are doing--modify
    # the SWIG interface file instead.

CLASSES
    builtins.object
        GPIO
        I2C
        Serial
        nes

    class GPIO(builtins.object)
     |  GPIO(pinmode=0)
     |

>>>
```

3.17.2. 40 pin GPIO 口测试

wiringOP-Python 跟 wiringOP 一样，也是可以通过指定 wPi 号来确定操作哪一个 GPIO 引脚，因为 wiringOP-Python 中没有查看 wPi 号的命令，所以只能通过 wiringOP 中的 gpio 命令来查看板子 wPi 号与物理引脚的对应关系。

```
orangepi@orangepizero3w:~$ gpio readall
 +------+-----+----------+--------+---+  ZERO3W  +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |   35 |   0 |   SDA.0  |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
 |   34 |   1 |   SCL.0  |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
 |   36 |   2 |  PWM0-0  |    OFF | 0 |  7 || 8  | 0 | ALT2   | TXD.0    | 3   | 41   |
 |      |     |      GND |        |   |  9 || 10 | 0 | ALT2   | RXD.0    | 4   | 42   |
 |   32 |   5 |      PB0 |  ALT14 | 1 | 11 || 12 | 0 | OFF    | PB5      | 6   | 37   |
 |   33 |   7 |      PB1 |  ALT14 | 1 | 13 || 14 |   |        | GND      |     |      |
 |  137 |   8 |      PE9 |    OFF | 0 | 15 || 16 | 0 | OFF    | PL2      | 9   | 354  |
 |      |     |     3.3V |        |   | 17 || 18 | 0 | OFF    | PL3      | 10  | 355  |
 |  130 |  11 |  MOSI.3  |    OFF | 0 | 19 || 20 |   |        | GND      |     |      |
 |  131 |  12 |  MISO.3  |    OFF | 0 | 21 || 22 | 0 | OFF    | PD0      | 13  | 96   |
 |  129 |  14 |  SCLK.3  |    OFF | 0 | 23 || 24 | 0 | OFF    | CE.0     | 15  | 128  |
 |      |     |      GND |        |   | 25 || 26 | 0 | OFF    | PE4      | 16  | 132  |
 |  357 |  17 |      PL5 |    OFF | 0 | 27 || 28 | 0 | OFF    | PL4      | 18  | 356  |
 |  140 |  19 |     PE12 |    OFF | 0 | 29 || 30 |   |        | GND      |     |      |
 |  141 |  20 |     PE13 |    OFF | 0 | 31 || 32 | 0 | OFF    | PD1      | 21  | 97   |
 |   99 |  22 |      PD3 |    OFF | 0 | 33 || 34 |   |        | GND      |     |      |
 |   38 |  23 |      PB6 |    OFF | 0 | 35 || 36 | 0 | OFF    | PD2      | 24  | 98   |
 |  100 |  25 |      PD4 |    OFF | 0 | 37 || 38 | 0 | OFF    | PB8      | 26  | 40   |
 |      |     |      GND |        |   | 39 || 40 | 0 | OFF    | PB7      | 27  | 39   |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+  ZERO3W  +---+--------+----------+-----+------+
```

1) 下面以 7 号引脚——对应 GPIO 为 PB4 ——对应 wPi 序号为 2——为例演示如何设置 GPIO 口的高低电平

```
orangepi@orangepizero3w:~$ gpio readall
 +------+-----+----------+--------+---+  ZERO3W  +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |   35 |   0 |   SDA.0  |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
 |   34 |   1 |   SCL.0  |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
 |   36 |   2 |  PWM0-0  |    OFF | 0 |  7 || 8  | 0 | ALT2   | TXD.0    | 3   | 41   |
 |      |     |      GND |        |   |  9 || 10 | 0 | ALT2   | RXD.0    | 4   | 42   |
```

2) 直接用命令测试的步骤如下所示：

a. 首先设置 GPIO 口为输出模式，其中 **pinMode** 函数的第一个参数是引脚对应的 wPi 的序号，第二个参数是 GPIO 的模式

```
root@orangepi:~/wiringOP-Python# python3 -c "import wiringpi; \
from wiringpi import GPIO; wiringpi.wiringPiSetup() ; \
wiringpi.pinMode(2, GPIO.OUTPUT) ;  "
```

b. 然后设置 GPIO 口输出低电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 0v，说明设置低电平成功

```
root@orangepi:~/wiringOP-Python# python3 -c "import wiringpi; \
 from wiringpi import GPIO; wiringpi.wiringPiSetup() ;\
wiringpi.digitalWrite(2, GPIO.LOW)"
```

c. 然后设置 GPIO 口输出高电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 3.3v，说明设置高电平成功

```
root@orangepi:~/wiringOP-Python# python3 -c "import wiringpi; \
 from wiringpi import GPIO; wiringpi.wiringPiSetup() ;\
wiringpi.digitalWrite(2, GPIO.HIGH)"
```

3) 在 python3 的命令行中测试的步骤如下所示：

a. 首先使用 python3 命令进入 python3 的命令行模式

```
root@orangepi:~# python3
```

b. 然后导入 wiringpi 的 python 模块

```
>>> import wiringpi
>>> from wiringpi import GPIO
```

c. 然后设置 GPIO 口为输出模式，其中 **pinMode** 函数的第一个参数是引脚对应的 wPi 的序号，第二个参数是 GPIO 的模式

```
>>> wiringpi.wiringPiSetup()
0
>>> wiringpi.pinMode(2, GPIO.OUTPUT)
```

d. 然后设置 GPIO 口输出低电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 0v，说明设置低电平成功

```
>>> wiringpi.digitalWrite(2, GPIO.LOW)
```

e. 然后设置 GPIO 口输出高电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 3.3v，说明设置高电平成功

```
>>> wiringpi.digitalWrite(2, GPIO.HIGH)
```

4) wiringOP-Python 在 python 代码中设置 GPIO 高低电平的方法可以参考下 examples 中的 **blink.py** 测试程序，**blink.py** 测试程序会设置开发板 40 pin 中所有的 GPIO 口的电压不断的高低变化

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# ls blink.py
blink.py
```

```
root@orangepi:~/wiringOP-Python/examples# python3 blink.py
```

3.17.3. 40 pin SPI 测试

1) 由下图可知，Orange Pi Zero 3w 可用的 SPI 为 SPI3。

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PB3	35	3	4		5V		
		TWI0-SCK	PB2	34	5	6		GND		
		PWM0-0	PB4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PB1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PB6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PB8	TWI1-SDA	
			GND		39	40	39	PB7	TWI1-SCL	PWM0-9

2) SPI3 在 40pin 对应的引脚如下表所示

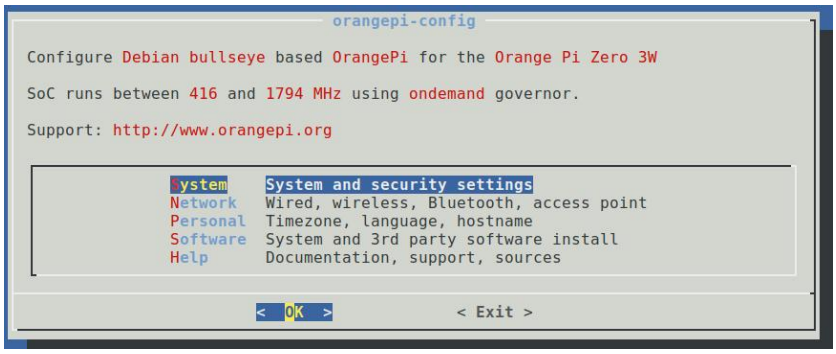
	SPI3 对应 40pin
MOSI	**19 号引脚**
MISO	**21 号引脚**
CLK	**23 号引脚**
CS0	**24 号引脚**
CS1	**26 号引脚**

3) 在 linux 系统中，40 pin 中的 SPI 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

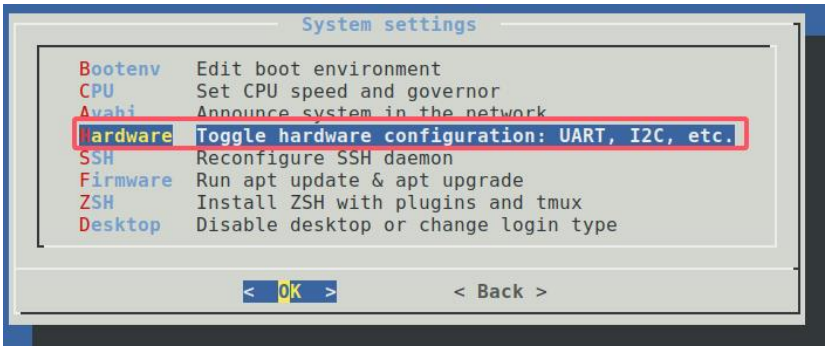
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**



c. 然后选择 **Hardware**



d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中打开的 SPI 的配置



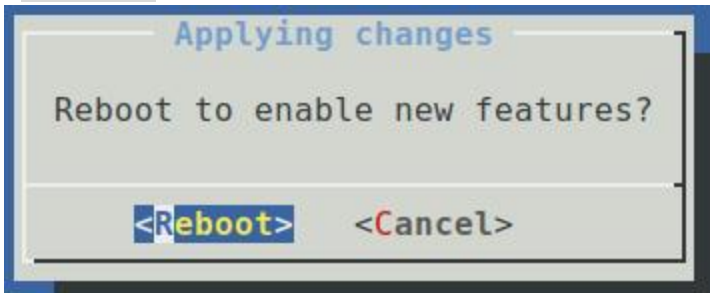
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



4) 然后查看下 linux 系统中是否存在 **spidevx.x** 的设备节点，如果存在，说明 SPI 的配置已经生效了

```
orangepi@orangepi:~$ ls /dev/spidev*
/dev/spidev3.0
```

5) 然后可以使用 examples 中的 **spidev_test.py** 程序测试下 SPI 的回环功能，**spidev_test.py** 程序需要指定下面的两个参数：

a. **--channel**：指定 SPI 的通道号

b. **--port**：指定 SPI 的端口号

6) 先不短接 SPI 的 mosi 和 miso 两个引脚，运行 spidev_test.py 的输出结果如下所示，可以看到 TX 和 RX 的数据不一致

--channel 和--port 参数后面的 x 需要替换为具体 SPI 的通道号和 SPI 的端口号。

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# python3 spidev_test.py \
--channel x --port x
spi mode: 0x0
max speed: 500000 Hz (500 KHz)
Opening device /dev/spidev1.1
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D    |......@.........|
RX | FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF    |................|
```

7) 然后使用杜邦线短接 SPI 的 txd 和 rxd 两个引脚再运行 spidev_test.py 的输出如下，可以看到发送和接收的数据一样，说明 SPI 回环测试正常

--channel 和--port 参数后面的 x 需要替换为具体 SPI 的通道号和 SPI 的端口号。

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# python3 spidev_test.py \
--channel x --port x
spi mode: 0x0
max speed: 500000 Hz (500 KHz)
Opening device /dev/spidev1.1
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D    |......@.........|
RX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D    |......@.........|
```

3.17.4. 40 pin I2C 测试

1) 由下表可知，Orange Pi Zero 3w 可用的 i2c 为 i2c0、i2c1、i2c2 和 i2c3 共四组 i2c 总线

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PB3	35	3	4		5V		
		TWI0-SCK	PB2	34	5	6		GND		
		PWM0-0	PB4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PB1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PB6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PB8	TWI1-SDA	
			GND		39	40	39	PB7	TWI1-SCL	PWM0-9

2) 三组 I2C 总线在 40pin 中对应的引脚如下表所示。

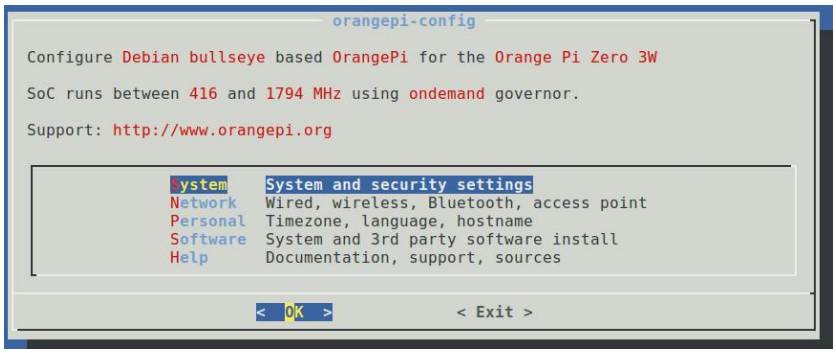
I2C 总线	SDA 对应 40pin	SCL 对应 40pin	dtbo 对应配置
I2C0	**3 号引脚**	**5 号引脚**	**i2c0**
I2C1	**38 号引脚**	**40 号引脚**	**i2c1**
I2C2	**19 号引脚**	**23 号引脚**	**i2c2**
I2C3	**26 号引脚**	**21 号引脚**	**i2c3**

3) 在 linux 系统中，40 pin 中的 I2C 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

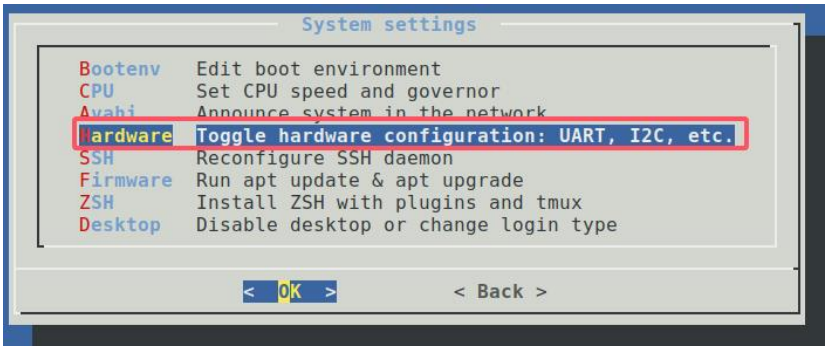
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**



c. 然后选择 **Hardware**



d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中打开的 I2C 的配置



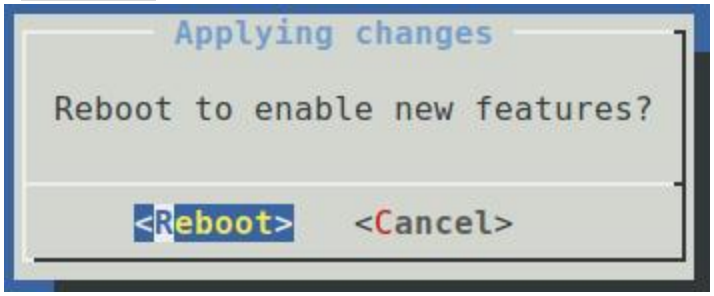
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



4) 启动 linux 系统后，先确认下/dev 下存在 i2c 的设备节点

```
orangepi@orangepi:~$ ls /dev/i2c-*
/dev/i2c-0  /dev/i2c-1  /dev/i2c-13  /dev/i2c-15  /dev/i2c-2  /dev/i2c-20
/dev/i2c-3  /dev/i2c-5  /dev/i2c-7  /dev/i2c-8  /dev/i2c-9
```

5) 然后开始测试 i2c，首先安装下 i2c-tools

```
orangepi@orangepi:~$ sudo apt-get update
orangepi@orangepi:~$ sudo apt-get install -y i2c-tools
```

6) 然后在 40 pin 接头的 i2c 引脚上接一个 i2c 设备，这里以 DS1307 RTC 模块为例



7) 然后使用 **i2cdetect -y** 命令如果能检测到连接的 i2c 设备的地址，就说明 i2c 设备连接正确

```
orangepi@orangepi:~$ sudo i2cdetect -y -r 0      #i2c0 的命令
orangepi@orangepi:~$ sudo i2cdetect -y -r 1      #i2c1 的命令
orangepi@orangepi:~$ sudo i2cdetect -y -r 2      #i2c2 的命令
orangepi@orangepi:~$ sudo i2cdetect -y -r 3      #i2c3 的命令
```

8) 然后可以运行 **examples** 中的 **ds1307.py** 测试程序读取 RTC 的时间

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# python3 ds1307.py --device \
"/dev/i2c-1"
Thu 2022-06-16  04:35:46
Thu 2022-06-16  04:35:47
Thu 2022-06-16  04:35:48
^C
exit
```

3.17.5. 40 pin 的 UART 测试

1) 由下表可知，Orange Pi Zero 3w 可用的 uart 为 uart2、uart6 和 uart7 共三组 UART 总线。

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PB3	35	3	4		5V		
		TWI0-SCK	PB2	34	5	6		GND		
		PWM0-0	PB4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PB1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PB6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PB8	TWI1-SDA	
			GND		39	40	39	PB7	TWI1-SCL	PWM0-9

2) 六组 UART 总线在 40 pin 中对应的引脚如下表所示。

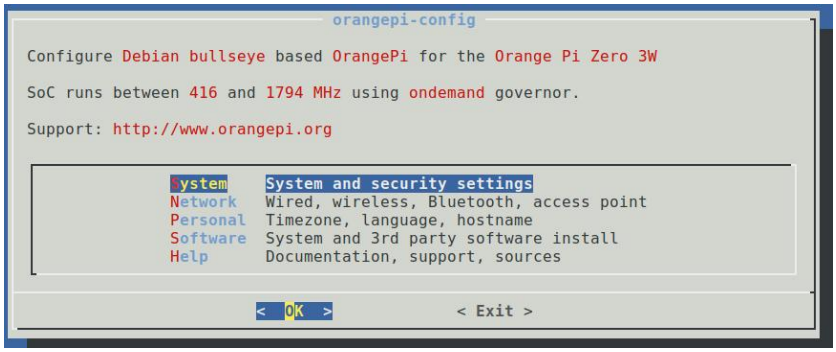
UART 总线	RX 对应 40pin	TX 对应 40pin	dtbo 对应配置
UART2	**13 号引脚**	**11 号引脚**	**uart2**
UART6	**23 号引脚**	**24 号引脚**	**uart6**
UART7	**18 号引脚**	**16 号引脚**	**uart7**

3) 在 linux 系统中，40 pin 中的 UART 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

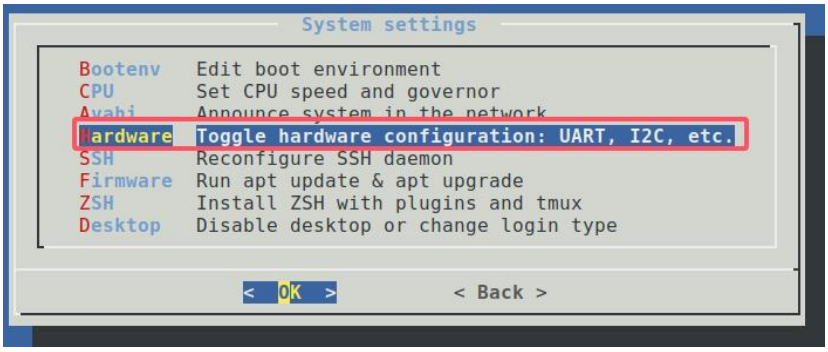
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**



c. 然后选择 **Hardware**



d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中打开的 UART 的配置



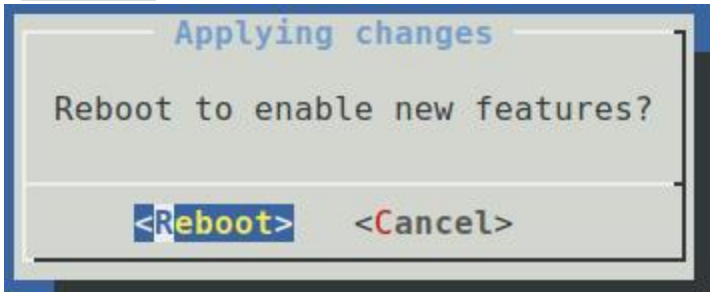
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



4) 进入 linux 系统后，先确认下**/dev** 下是否存在 uart 的设备节点

```
orangepi@orangepi:~$ ls /dev/ttyS*
/dev/ttyS0   /dev/ttyS1   /dev/ttyS2   /dev/ttyS6   /dev/ttyS7
```

5) 然后开始测试 uart 接口，先使用杜邦线短接要测试的 uart 接口的 rx 和 tx

6) 最后可以运行 examples 中的 **serialTest.py** 程序来测试下串口的回环功能，如果能看到下面的打印，说明串口回环测试正常

注意，gpio serial /dev/ttySx 命令中最后的 x 需要替换为对应的 uart 设备节点的序号。

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# python3 serialTest.py --device /dev/ttySx
```

```
Out:    0: ->    0
Out:    1: ->    1
Out:    2: ->    2
Out:    3: ->    3
Out:    4:^C
exit
```

3.18. 硬件看门狗测试

Orange Pi 发布的 linux 系统中预装了 watchdog_test 程序，可以直接测试。

运行 watchdog_test 程序的方法如下所示：

a. 第二个参数 10 表示看门狗的计数时间，如果这个时间内没有喂狗，系统会重启
b. 我们可以通过按下键盘上的任意键（ESC 除外）来喂狗，喂狗后，程序会打印一行 keep alive 表示喂狗成功

```
orangepi@orangepi:~$ sudo watchdog_test 10
open success
options is 33152,identity is sunxi-wdt
put_usr return,if 0,success:0
The old reset time is: 16
return ENOTTY,if -1,success:0
return ENOTTY,if -1,success:0
put_user return,if 0,success:0
put_usr return,if 0,success:0
keep alive
keep alive
keep alive
```

3.19. 查看 A733 芯片的 chipid

查看 A733 芯片 chipid 的命令如下所示，每个芯片的 chipid 都是不同的，所以可以使用 chipid 来区分多个开发板。

```
orangepi@orangepi:~# cat /sys/class/sunxi_info/sys_info |grep sunxi_serial
```

```
sunxi_serial          : 147d208d0161172000007c0000000000
```

3.20. Python 相关说明

3.20.1. Python 源码编译安装的方法

如果使用的 Ubuntu 或者 Debian 系统软件仓库中的 Python 版本不符合开发的要求，想要使用最新版本的 Python，可以使用下面的方法下载 Python 的源码包来编译安装最新版本的 Python。

下面演示的是编译安装 Python3.9 的最新版本，如果要编译安装其他的版本的 Python，方法也是一样的（需要下载想要安装的 Python 对应的源码）。

1) 首先安装编译 Python 需要的依赖包

```
orangepi@orangepi:~$ sudo apt-get update
orangepi@orangepi:~$ sudo apt-get install -y build-essential zlib1g-dev   \
libncurses5-dev libgdbm-dev libnss3-dev libssl-dev libsqlite3-dev   \
libreadline-dev libffi-dev curl libbz2-dev
```

2) 然后下载最新版本的 Python3.9 源码并解压

```
orangepi@orangepi:~$ wget   \
https://www.python.org/ftp/python/3.9.10/Python-3.9.10.tgz
orangepi@orangepi:~$ tar xvf Python-3.9.10.tgz
```

3) 然后运行配置命令

```
orangepi@orangepi:~$ cd Python-3.9.10
orangepi@orangepi:~/Python-3.9.10$ ./configure --enable-optimizations
```

4) 然后编译安装 Python3.9，编译时间大概需要半个小时左右

```
orangepi@orangepi:~/Python-3.9.10$ make -j4
orangepi@orangepi:~/Python-3.9.10$ sudo make altinstall
```

5) 安装完后可以使用下面的命令查看下刚安装的 Python 的版本号

```
orangepi@orangepi:~/Python-3.9.10$ python3.9 --version
Python 3.9.10
```

6) 然后更新下 pip

```
orangepi@orangepi:~$ /usr/local/bin/python3.9 -m pip install --upgrade pip
```

3.20.2. Python 更换 pip 源的方法

Linux 系统 pip 默认使用的源为 Python 官方的源，但是国内访问 Python 官方的源速度是很慢的，并且经常会由于网络原因导致 Python 软件包安装失败。所以在使用 pip 安装 Python 库时，请记得更换下 pip 源。

1) 首先安装下 **python3-pip**

```
orangepi@orangepi:~$ sudo apt-get update
orangepi@orangepi:~$ sudo apt-get install -y python3-pip
```

2) Linux 下永久更换 pip 源的方法

a. 先新建**~/.pip** 目录，然后添加 **pip.conf** 配置文件，并在其中设置 pip 的源为清华源

```
orangepi@orangepi:~$ mkdir -p ~/.pip
orangepi@orangepi:~$ cat <<EOF > ~/.pip/pip.conf
[global]
timeout = 6000
index-url = https://pypi.tuna.tsinghua.edu.cn/simple
trusted-host = pypi.tuna.tsinghua.edu.cn
EOF
```

b. 然后使用 pip3 安装 Python 库速度就会很快了

3) Linux 下临时更换 pip 源的方法，其中的**<packagename>**需要替换为具体的包名

```
orangepi@orangepi:~$ pip3 install <packagename> -i  \
https://pypi.tuna.tsinghua.edu.cn/simple --trusted-host pypi.tuna.tsinghua.edu.cn
```

3.21. 安装 Docker 的方法

Orange Pi 提供的 linux 镜像已经预装了 Docker，只是 Docker 服务默认没有打开。使用 **enable_docker.sh** 脚本可以使能 docker 服务，然后就可以开始使用 docker 命令了，并且在下次启动系统时也会自动启动 docker 服务。

```
orangepi@orangepi:~$ enable_docker.sh
```

可以使用下面的命令测试下 docker，如果能运行 **hello-world** 说明 docker 能正常使用了。

```
orangepi@orangepi:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
256ab8fe8778: Pull complete
Digest:
sha256:7f0a9f93b4aa3022c3a4c147a449ef11e0941a1fd0bf4a8e6c9408b2600777c5
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
.....
```

使用 docker 命令时，如果提示 **permission denied**，请将当前用户加入到 docker 用户组，这样不需要 sudo 就能运行 docker 命令了。

```
orangepi@orangepi:~$ sudo usermod -aG docker $USER
```

注意：需要退出重新登录系统才能生效，重启系统也可以。

3.22. Home Assistant 的安装方法

注意，这里只会提供在 Ubuntu 或者 Debian 系统中安装 Home Assistant 的方法，Home Assistant 详细的使用方法请参考官方文档或者相应的书籍。

3.22.1. 通过 docker 安装

1) 首先请安装好 docker，并确保 docker 能正常运行。docker 的安装步骤请参考**安装 Docker 的方法**一节的说明。

2) 然后可以搜索下 Home Assistant 的 docker 镜像

```
orangepi@orangepi:~$ docker search homeassistant
```

3) 然后使用下面的命令下载 Home Assistant 的 docker 镜像到本地，镜像大小大概有 1GB 多，下载时间会比较长，请耐心等待下载完成

```
orangepi@orangepi:~$ docker pull homeassistant/home-assistant
Using default tag: latest
latest: Pulling from homeassistant/home-assistant
be307f383ecc: Downloading
5fbc4c07ac88: Download complete
..... (省略部分输出)
3cc6a1510c9f: Pull complete
7a4e4d5b979f: Pull complete
Digest:
sha256:81d381f5008c082a37da97d8b08dd8b358dae7ecf49e62ce3ef1eeaefc4381bb
Status: Downloaded newer image for homeassistant/home-assistant:latest
docker.io/homeassistant/home-assistant:latest
```

4) 然后可以使用下面的命令查看下刚下载的 Home Assistant 的 docker 镜像

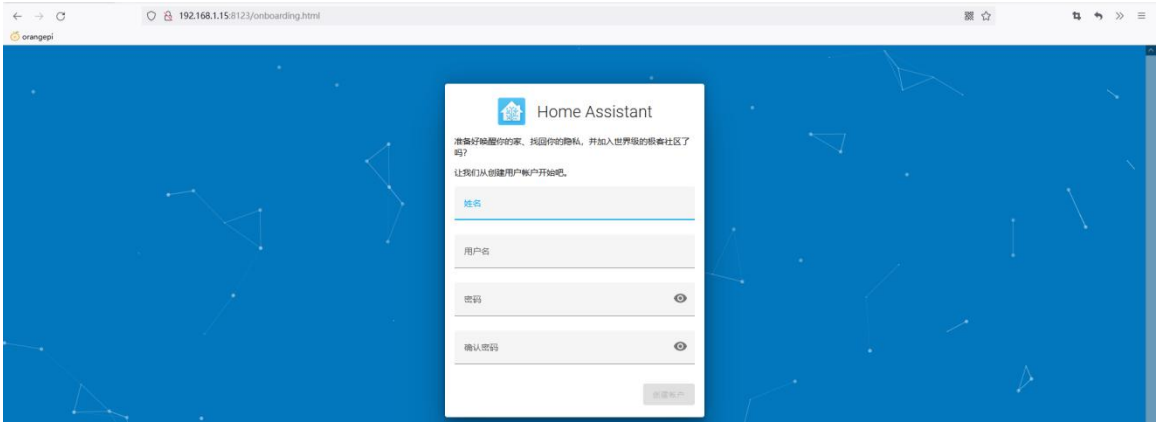
```
orangepi@orangepi:~$ docker images homeassistant/home-assistant
REPOSITORY                  TAG       IMAGE ID       CREATED        SIZE
homeassistant/home-assistant  latest    bfa0ab9e1cf5   2 months ago   1.17GB
```

5) 此时就可以运行 Home Assistant 的 docker 容器了

```
orangepi@orangepi:~$ docker run -d \
  --name homeassistant \
  --privileged \
  --restart=unless-stopped \
  -e TZ=Asia/Shanghai \
  -v /home/orangepi/home-assistant:/config \
  --network=host \
  homeassistant/home-assistant:latest
```

6) 然后在浏览器中输入【开发板的 IP 地址:8123】就能看到 Home Assistant 的界面

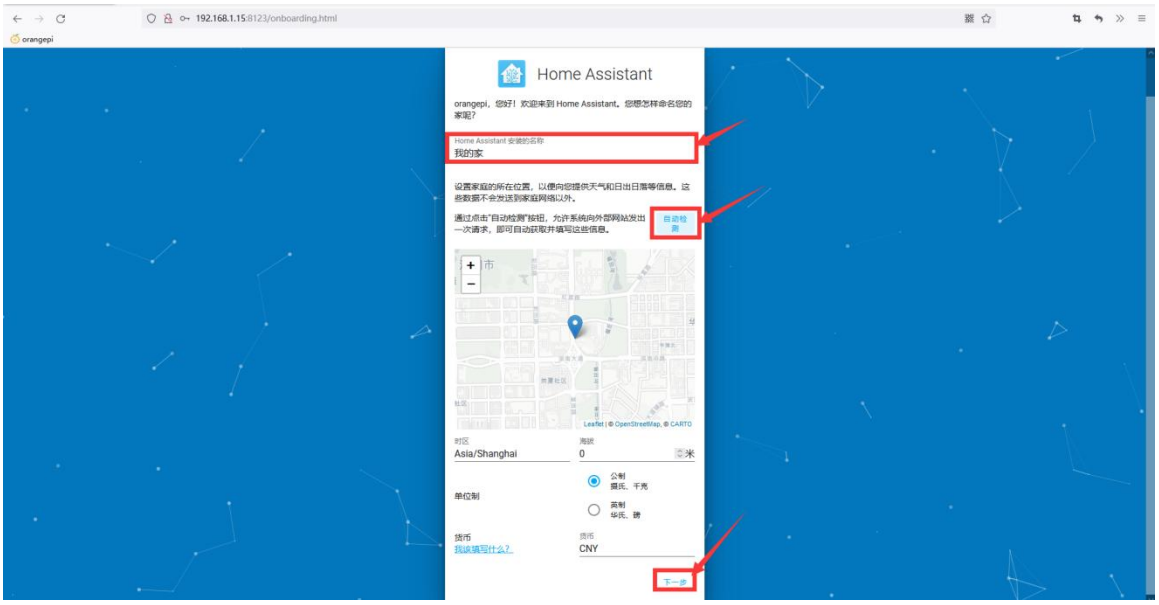
Home Assistant 容器的启动需要一段时间，如果下面的界面没有正常显示，请等待几秒钟再刷新。如果等待一分钟以上还没有正常显示下面的界面说明 Home Assistant 安装有问题，此时需要去检查前面的安装设置过程是否有问题了。



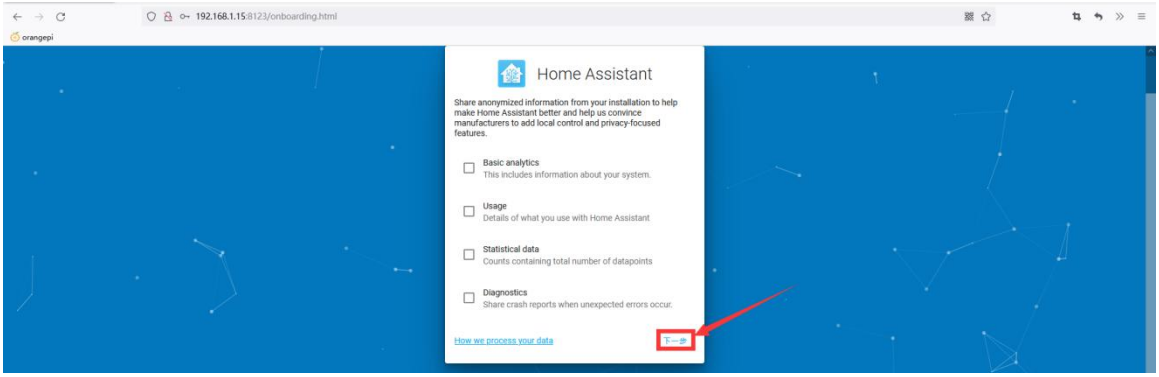
7) 然后输入**姓名、用户名**和**密码**再点击**创建账号**



8) 然后按照界面提示根据自己的喜好设置，再点击下一步



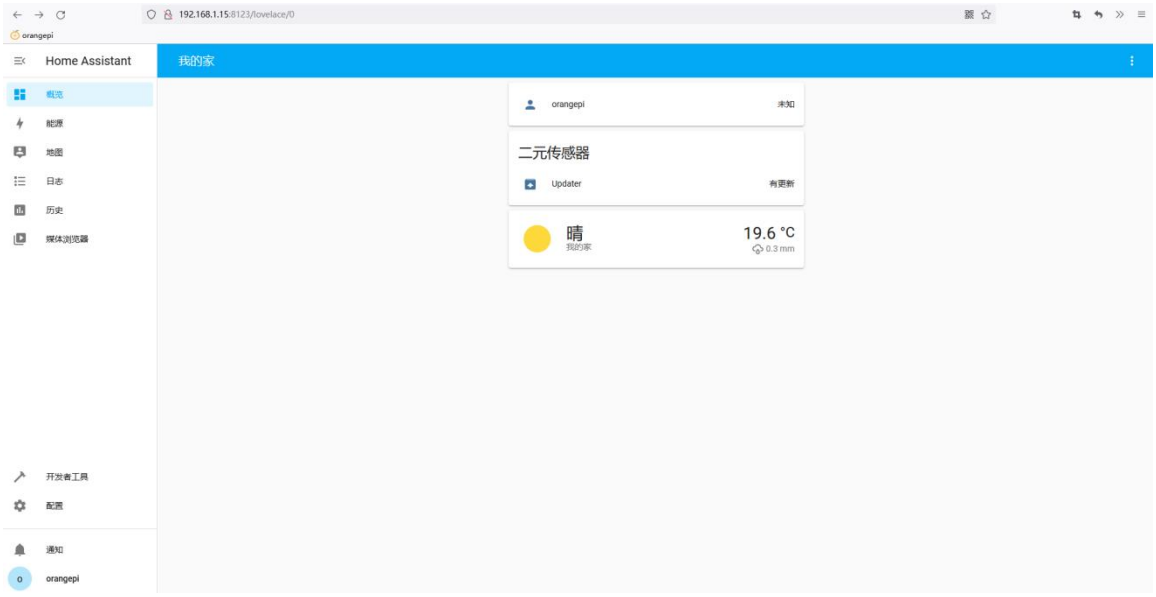
9) 然后点击下一步



10) 然后点击完成



11) Home Assistant 最终显示的主界面如下图所示



12) 停止 Home Assistant 容器的方法

a. 查看 docker 容器的命令如下所示

```
orangepi@orangepi:~$ docker ps -a
```

b. 停止 Home Assistant 容器的命令如下所示

```
orangepi@orangepi:~$ docker stop homeassistant
```

c. 删除 Home Assistant 容器的命令如下所示

```
orangepi@orangepi:~$ docker rm homeassistant
```

3.22.2. 通过 python 安装

安装前请先更换下 pip 的源为国内源，加快 Python 包的安装速度，配置方法见 Python 更换 pip 源的方法一节的说明。

1) 首先安装依赖包

```
orangepi@orangepi:~$ sudo apt-get update
orangepi@orangepi:~$ sudo apt-get install -y python3 python3-dev python3-venv    \
python3-pip libffi-dev libssl-dev libjpeg-dev zlib1g-dev autoconf build-essential    \
libopenjp2-7 libtiff5 libturbojpeg0-dev tzdata
```

如果是 debian12 请使用下面的命令：

```
orangepi@orangepi:~$ sudo apt-get update
```

```
orangepi@orangepi:~$ sudo apt-get install -y python3 python3-dev python3-venv   \
python3-pip libffi-dev libssl-dev libjpeg-dev zlib1g-dev autoconf build-essential   \
libopenjp2-7 libturbojpeg0-dev tzdata
```

2) 然后需要编译安装 Python3.9，方法请参考 **Python 源码编译安装的方法**一节

Ubuntu Jammy 默认的 Python 版本就是 Python3.10，所以无需编译安装。
Debian Bullseye 默认的 Python 版本就是 Python3.9，所以无需编译安装。
Debian Bookworm 默认的 Python 版本就是 Python3.11，所以也无需编译安装。

3) 然后创建 Python 虚拟环境

Debian Bookworm 中是 python3.11，请记得替换对应的命令。

```
orangepi@orangepi:~$ sudo mkdir /srv/homeassistant
orangepi@orangepi:~$ sudo chown orangepi:orangepi /srv/homeassistant
orangepi@orangepi:~$ cd /srv/homeassistant
orangepi@orangepi:/srv/homeassistant$ python3.9 -m venv .
orangepi@orangepi:/srv/homeassistant$ source bin/activate
(homeassistant) orangepi@orangepi:/srv/homeassistant$
```

4) 然后安装需要的 Python 包

```
(homeassistant) orangepi@orangepi:/srv/homeassistant$ python3 -m pip install wheel
```

5) 然后就可以安装 Home Assistant Core

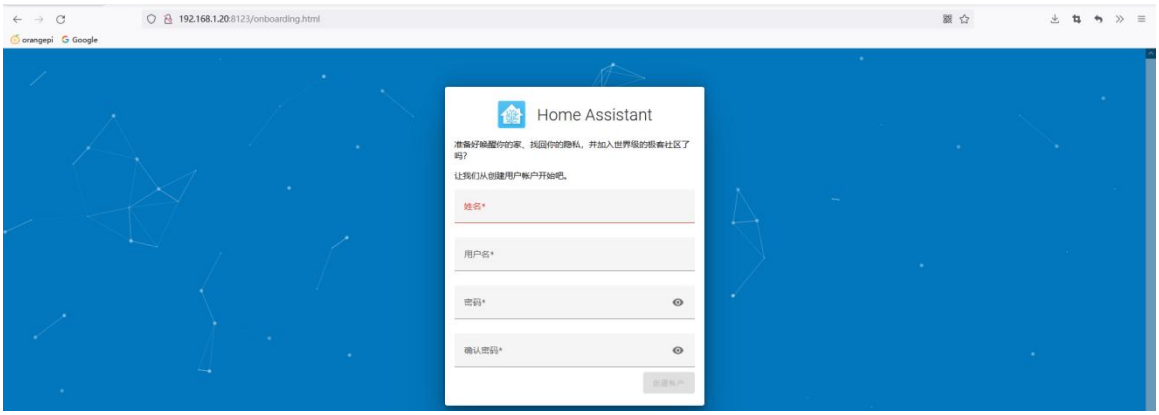
```
(homeassistant) orangepi@orangepi:/srv/homeassistant$ pip3 install homeassistant
```

6) 然后输入下面的命令就可以运行 Home Assistant Core

```
(homeassistant) orangepi@orangepi:/srv/homeassistant$ hass -v
```

7) 然后在浏览器中输入**【开发板的 IP 地址:8123】**就能看到 Home Assistant 的界面

第一次运行 hass 命令时，会下载安装和缓存一些运行必须的库和依赖包。这个过程可能会花费几分钟的时间。注意，此时在浏览器中是无法看到 Home Assistant 的界面的，请等待一段时间后再刷新下。



3.23. OpenCV 的安装方法

3.23.1. 使用 apt 来安装 OpenCV

1) 安装命令如下所示

```
orangepi@orangepi:~$ sudo apt-get update
orangepi@orangepi:~$ sudo apt-get install -y libopencv-dev python3-opencv
```

2) 然后使用下面的命令打印 OpenCV 的版本号输出正常，说明 OpenCV 安装成功

a. Ubuntu22.04 中 OpenCV 的版本如下所示：

```
orangepi@orangepi:~$ python3 -c "import cv2; print(cv2.__version__)"
4.5.4
```

b. Debian11 中 OpenCV 的版本如下所示：

```
orangepi@orangepi:~$ python3 -c "import cv2; print(cv2.__version__)"
4.5.1
```

c. Debian12 中 OpenCV 的版本如下所示：

```
orangepi@orangepi:~$ python3 -c "import cv2; print(cv2.__version__)"
4.6.0
```

3.24. 设置中文环境以及安装中文输入法

注意，安装中文输入法前请确保开发板使用的 Linux 系统为桌面版系统。

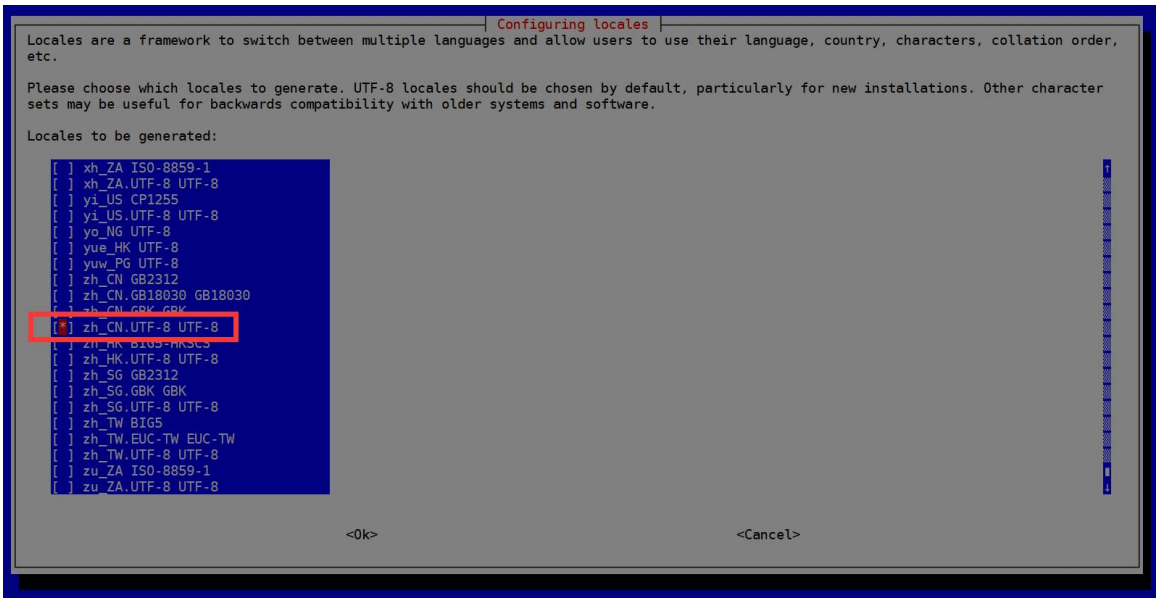
3.24.1. Debian 系统的安装方法

1) 首先设置默认 **locale** 为中文

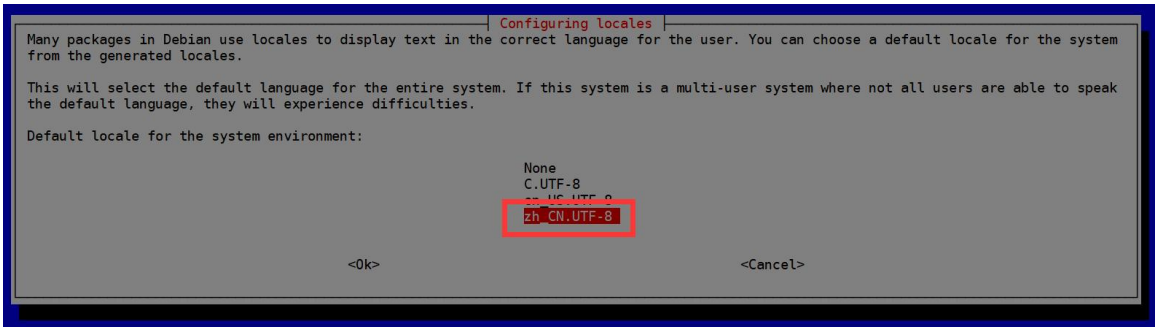
a. 输入下面的命令可以开始配置 **locale**

```
orangepi@orangepi:~$ sudo dpkg-reconfigure locales
```

b. 然后在弹出的界面中选择 **zh_CN.UTF-8 UTF-8**（通过键盘上的上下方向按键来上下移动，通过空格键来选择，最后通过 Tab 键可以将光标移动到 **<OK>**，然后回车即可）



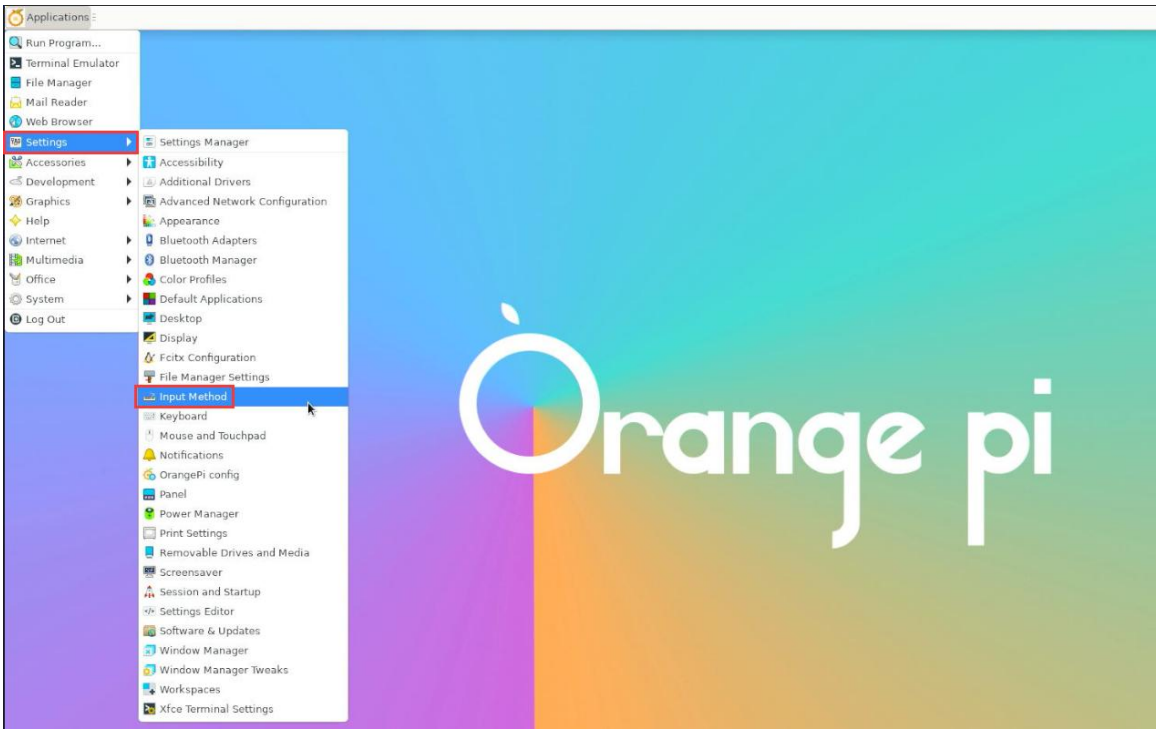
c. 然后设置默认 **locale** 为 **zh_CN.UTF-8**



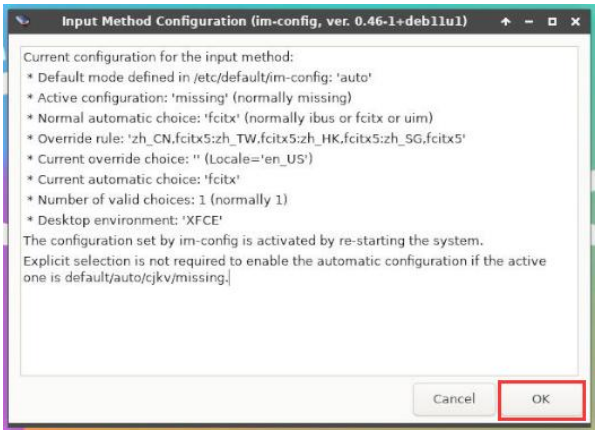
d. 退出界面后就会开始 **locale** 的设置，命令行显示的输出如下所示

```
orangepi@orangepi:~$ sudo dpkg-reconfigure locales
Generating locales (this might take a while)...
  en_US.UTF-8... done
  zh_CN.UTF-8... done
Generation complete.
```

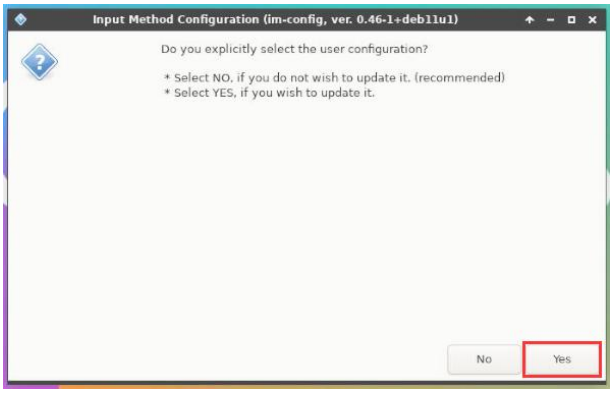
2) 然后打开 **Input Method**



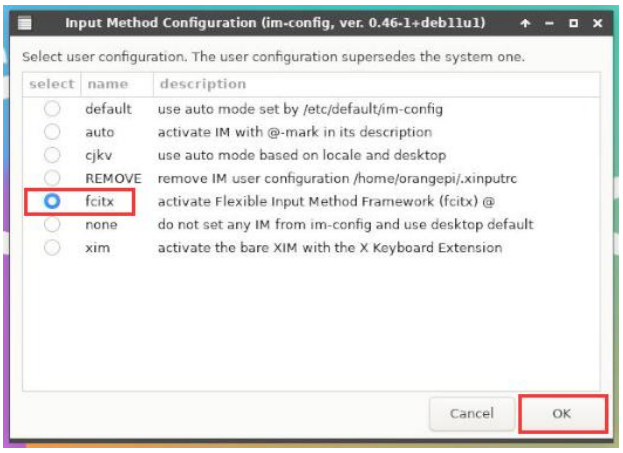
3) 然后选择 **OK**



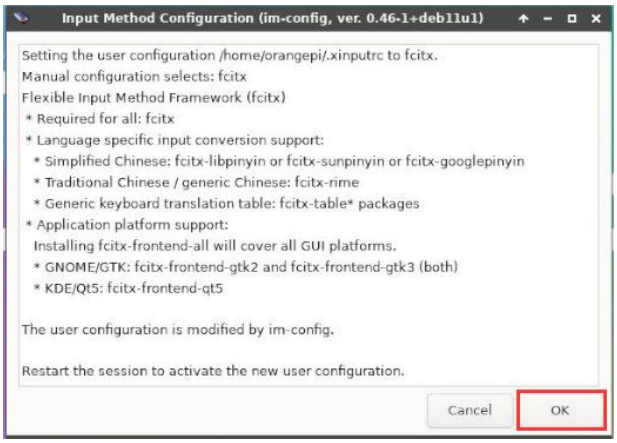
4) 然后选择 **Yes**



5) 然后选择 **fcitx**

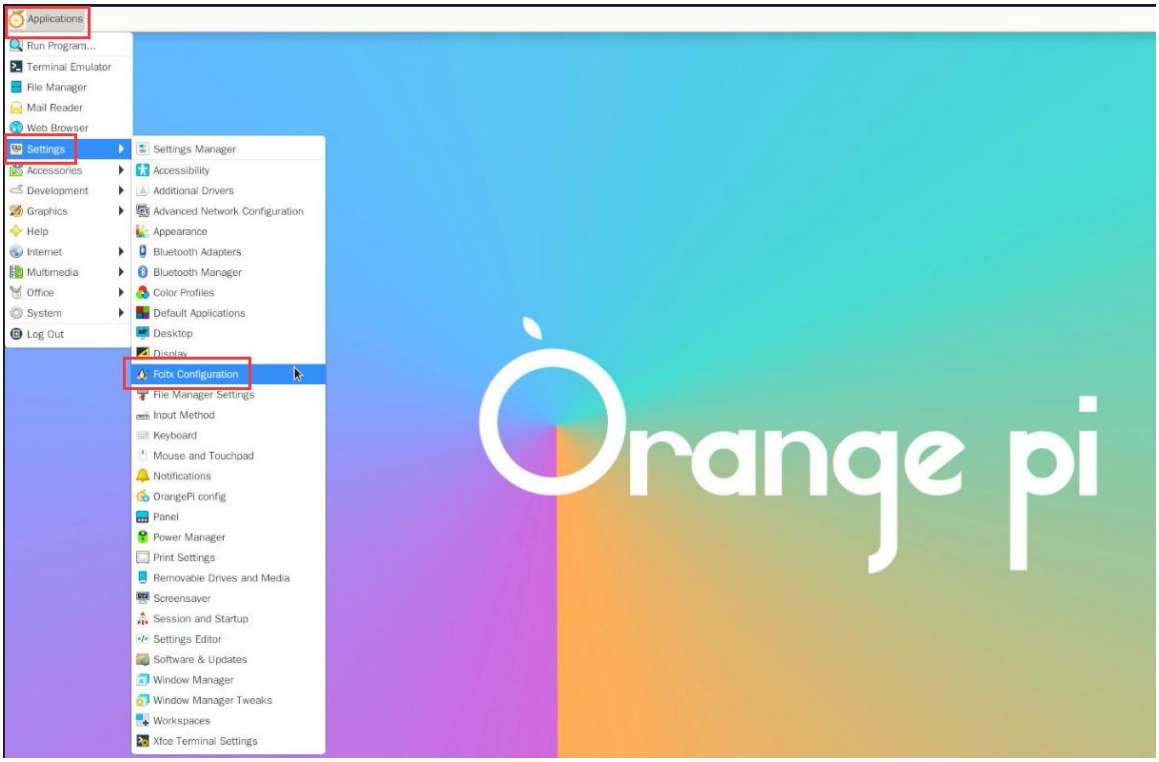


6) 然后选择 **OK**

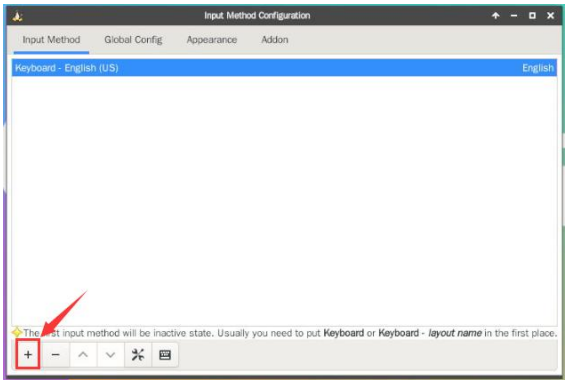


7) **然后重启 Linux 系统才能使配置生效**

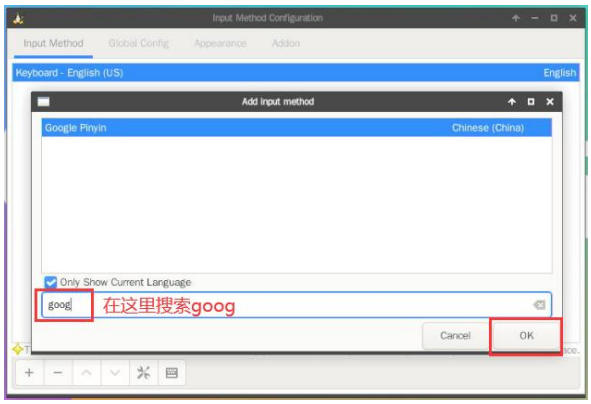
8) 然后打开 **Fcitx configuration**



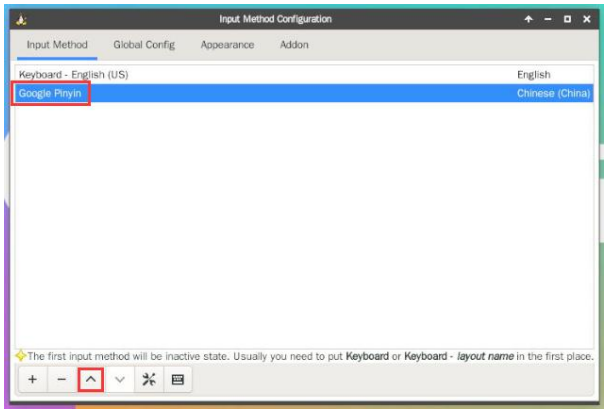
9) 然后点击下图所示位置的+号

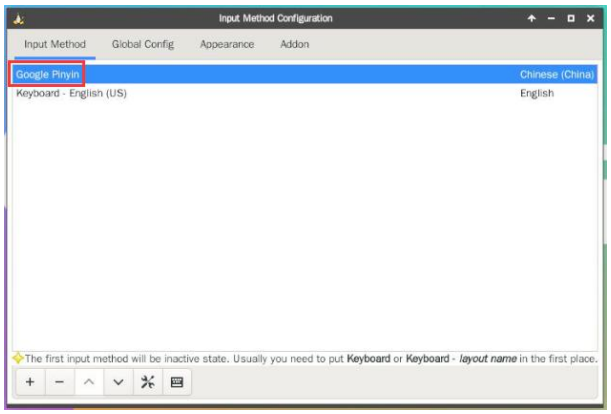


10) 然后搜索 **Google Pinyin** 再点击 **OK**



11) 然后将 **Google Pinyin** 放到最前面

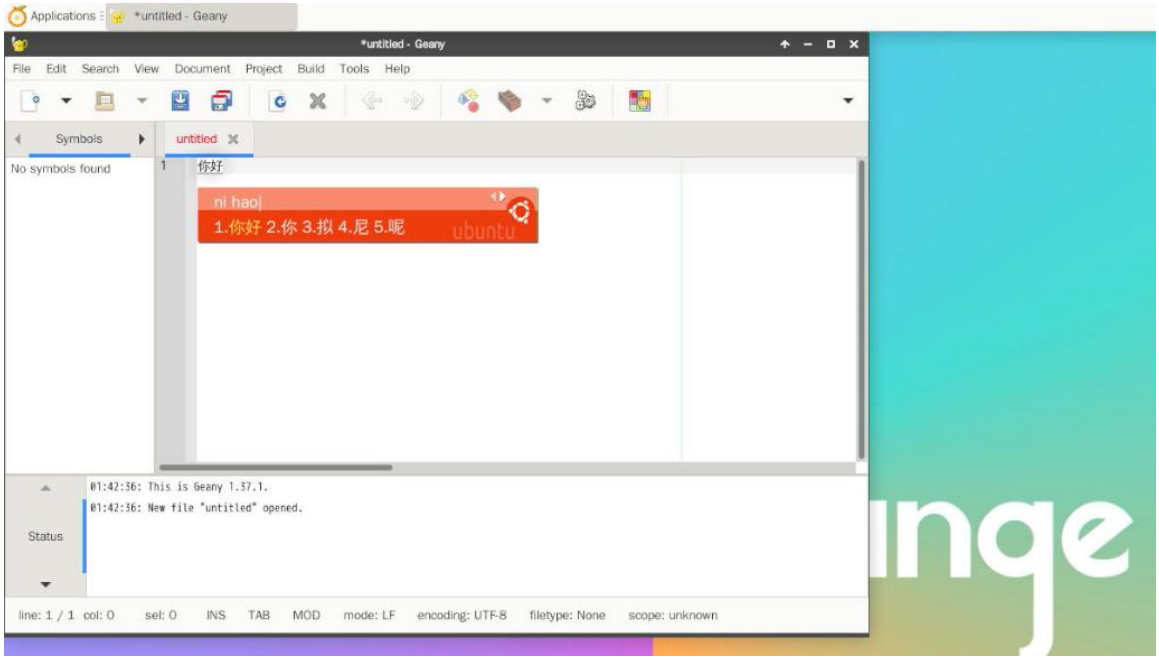




12) 然后打开 **Geany** 这个编辑器测试下中文输入法



13) 中文输入法测试如下所示

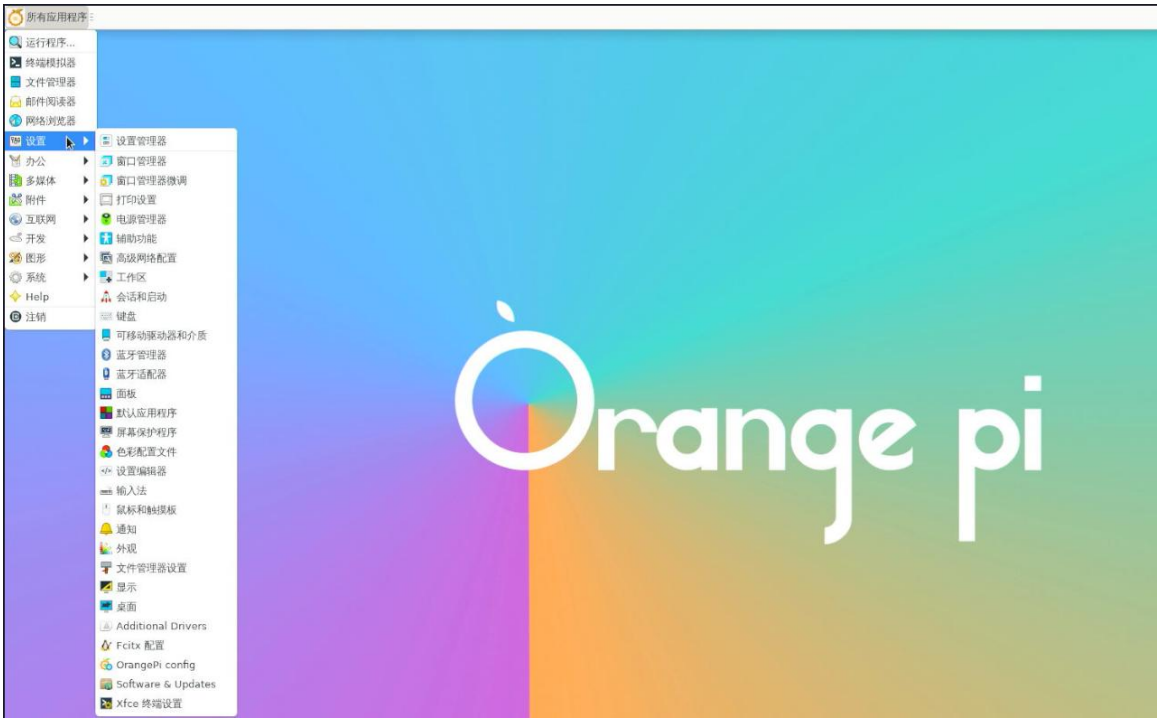


14) 通过 **Ctrl+Space** 快捷键可以切换中英文输入法

15) 如果需要整个系统都显示为中文，可以将**/etc/default/locale** 中的变量都设置为 **zh_CN.UTF-8**

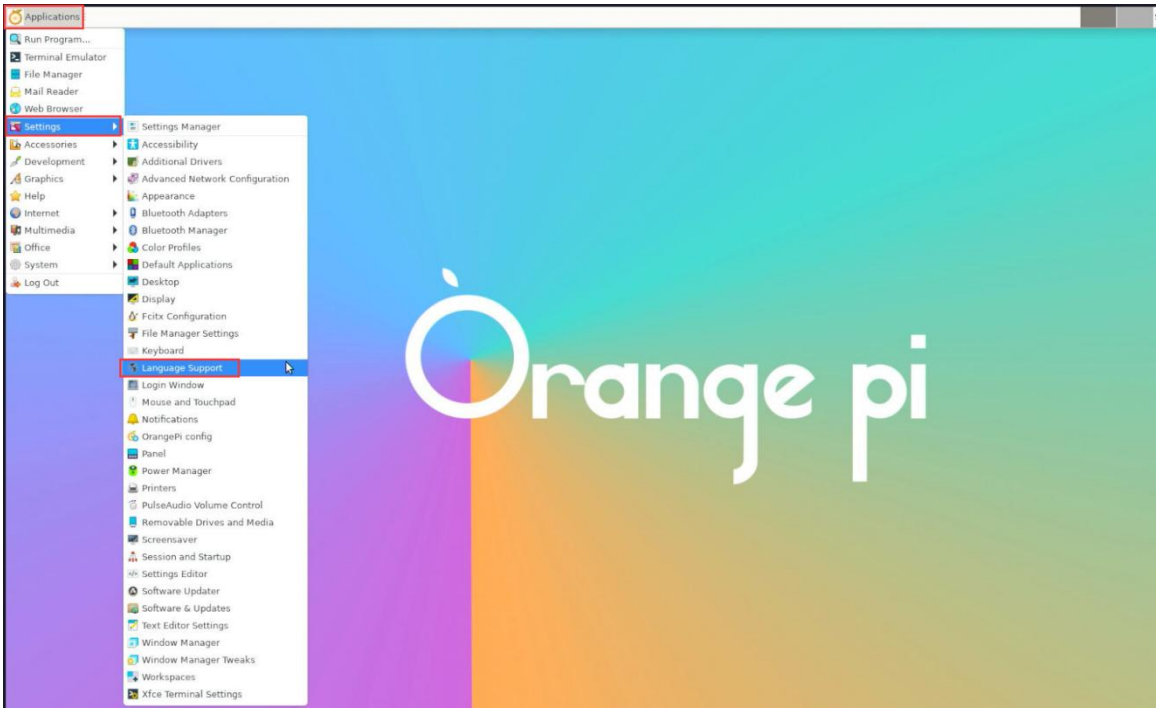
```
orangepi@orangepi:~$ sudo vim /etc/default/locale
#    File generated by update-locale
LC_MESSAGES=zh_CN.UTF-8
LANG=zh_CN.UTF-8
LANGUAGE=zh_CN.UTF-8
```

16) 然后**重启系统**就能看到系统显示为中文了



3. 24. 2. Ubuntu 22.04 系统的安装方法

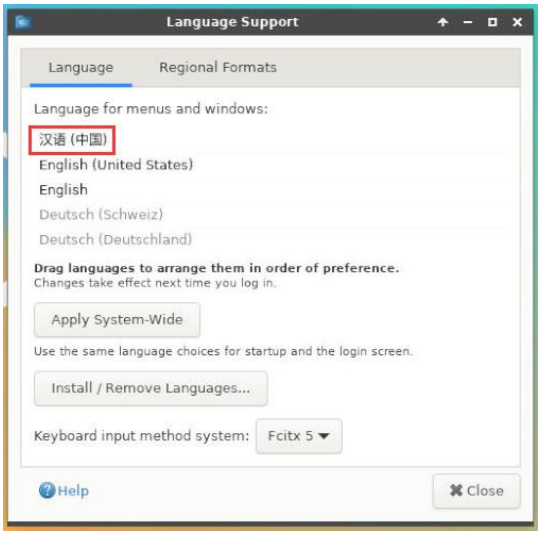
1) 首先打开 **Language Support**



2) 然后找到**汉语（中国）**选项

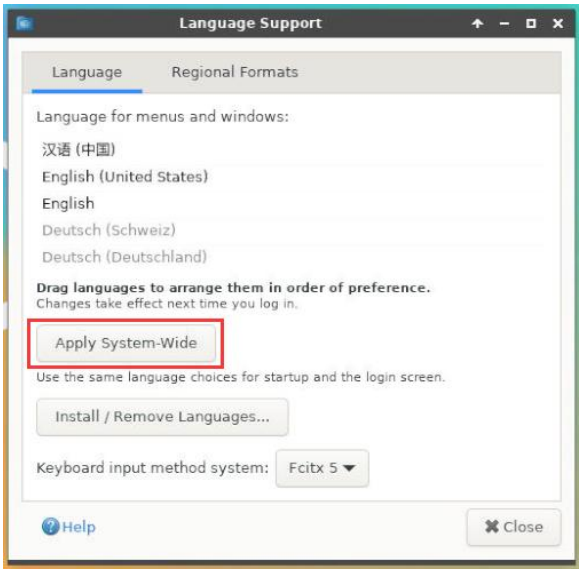


3) 然后请使用鼠标左键选中**汉语（中国）**并按住不动，然后往上将其拖到最开始的位置，拖完后的显示如下图所示：



注意，这一步不是很好拖动的，请耐心多试几次。

4) 然后选择 **Apply System-Wide** 将中文设置应用到整个系统

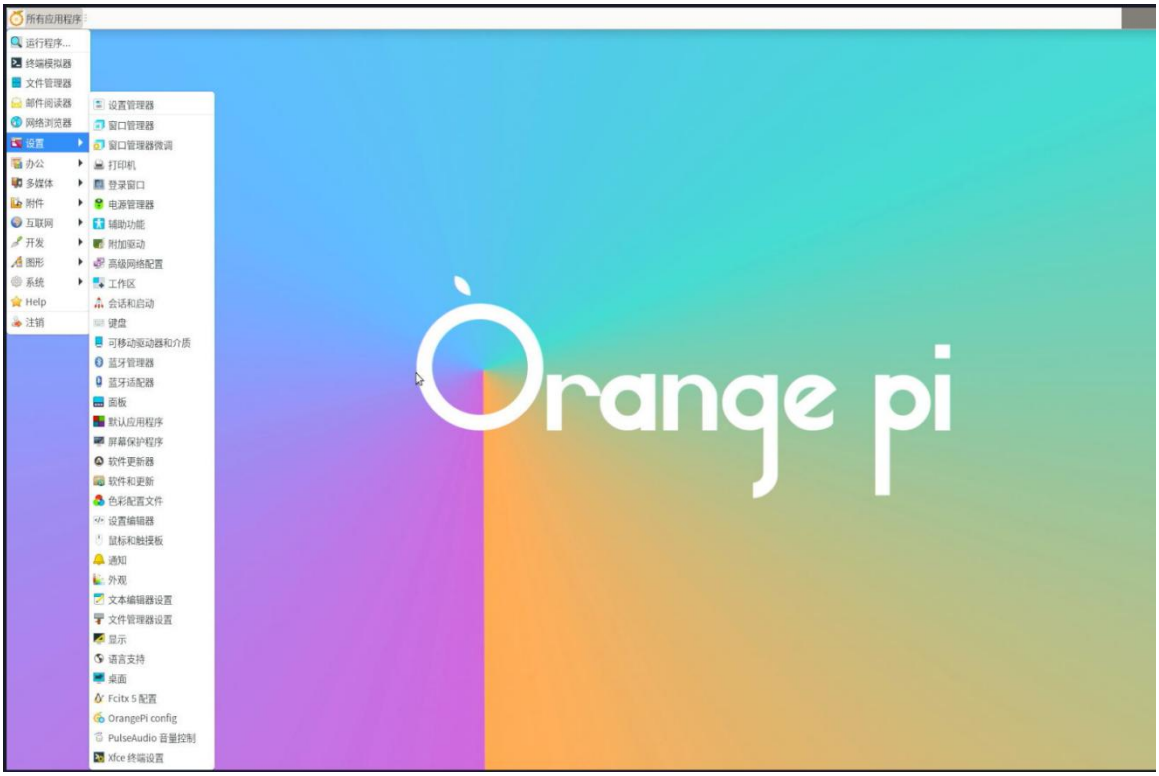


5) **然后重启 Linux 系统使配置生效**

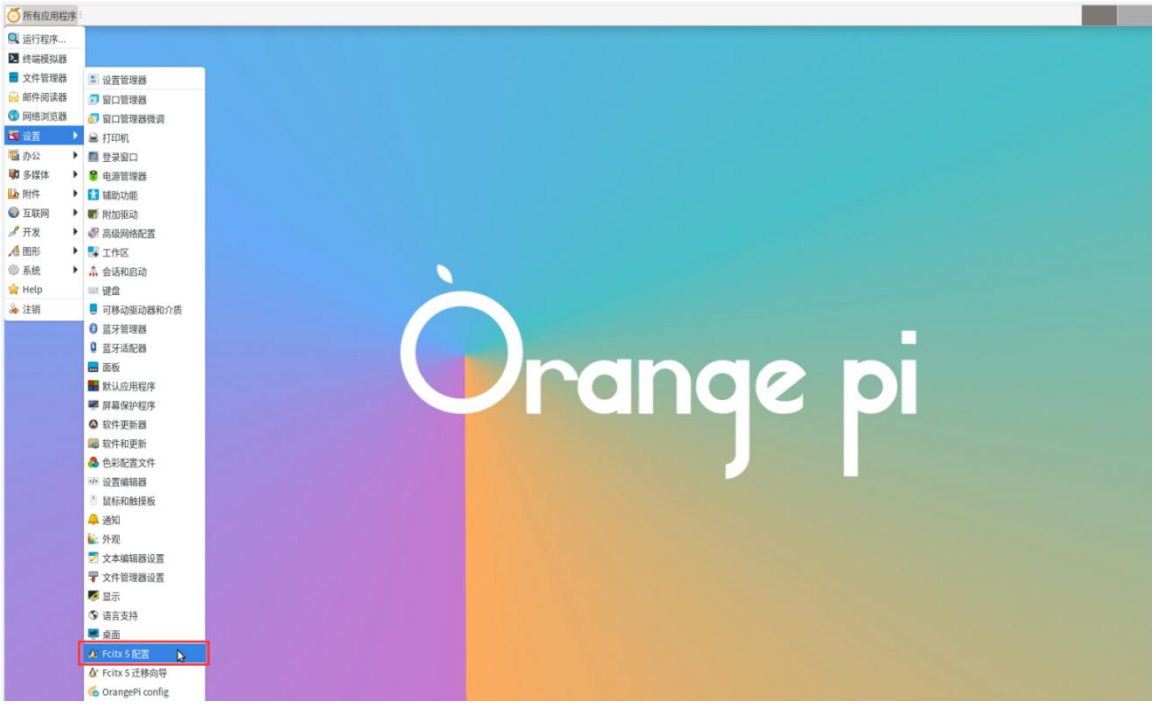
6) 重新进入系统后，在下面的界面请选择**不要再次询问我**，然后请根据自己的喜好决定标准文件夹是否也要更新为中文



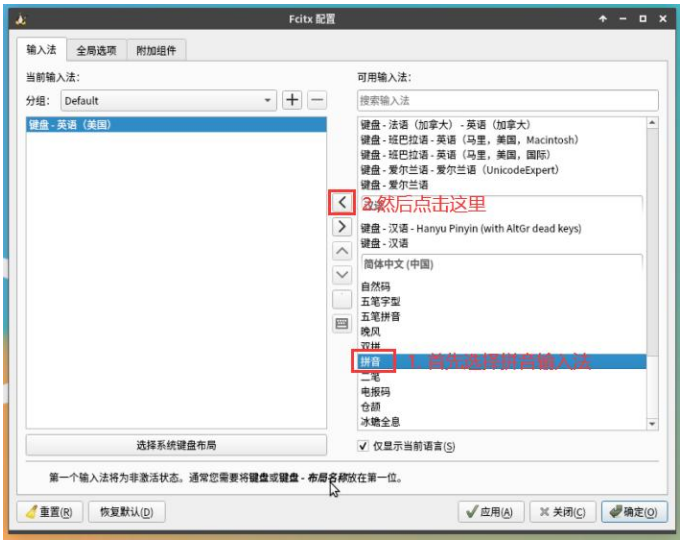
7) 然后可以看到桌面都显示为中文了



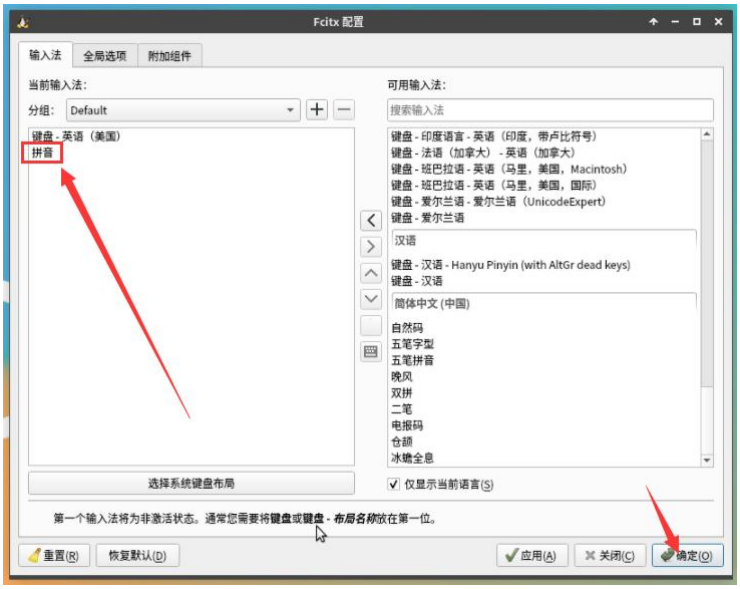
8) 然后打开 Fcitx5 配置程序



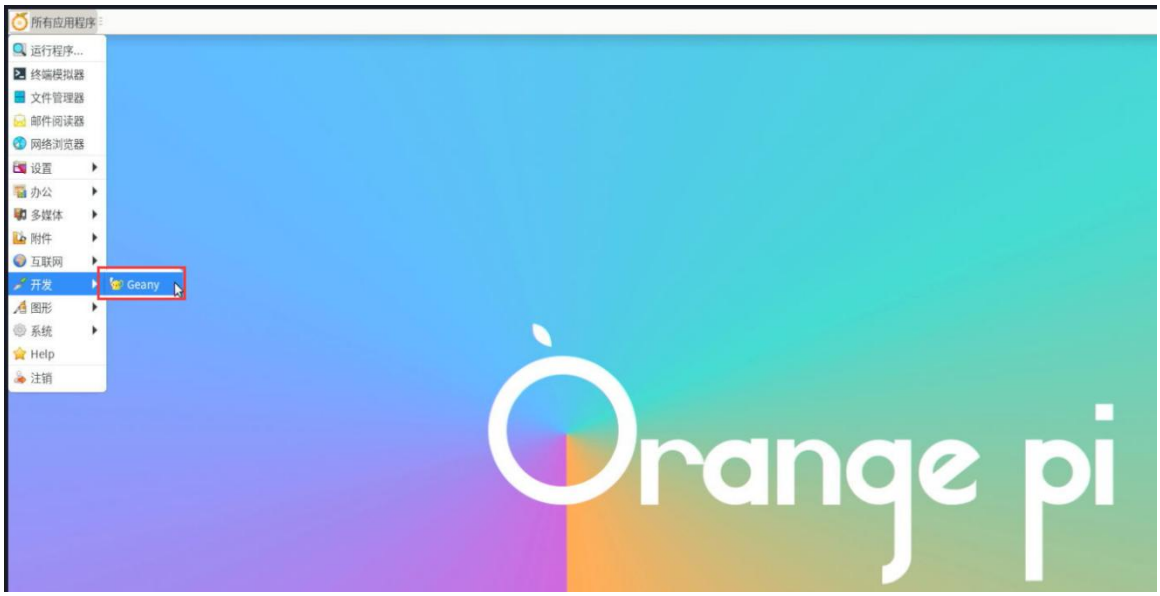
9) 然后选择使用拼音输入法



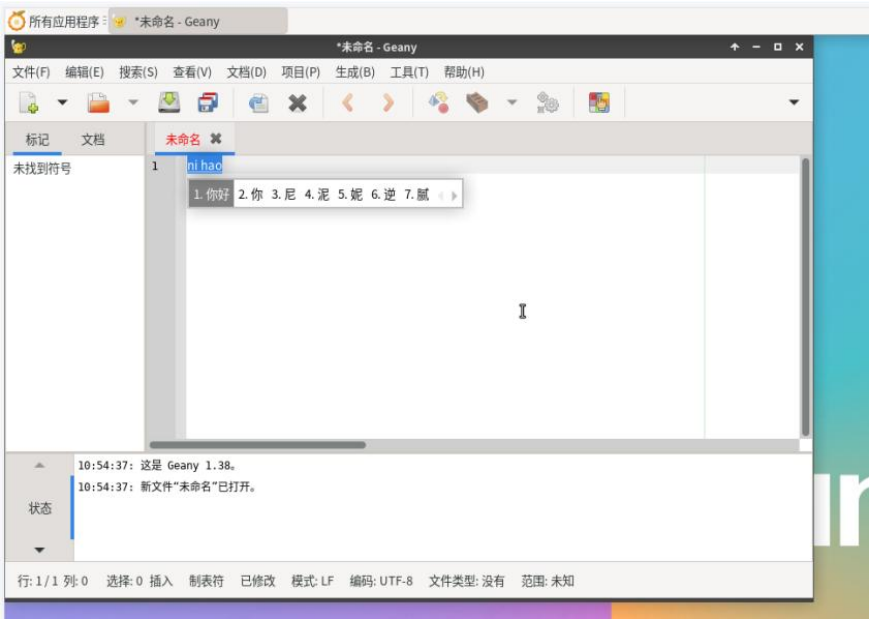
10) 选择后的界面如下所示，再点击确定即可



11) 然后我们可以打开 **Geany** 测试下中文输入法，打开方式如下图所示



12) 打开 **Geany** 后，默认还是英文输入法，我们可以通过 **Ctrl+Space** 快捷键来切换成中文输入法，然后就能输入中文了



3.25. 宝塔 Linux 面板的安装方法

宝塔 Linux 面板是提升运维效率的服务器管理软件，支持一键 LAMP/LNMP/集群/监控/网站/FTP/数据库/JAVA 等 100 多项服务器管理功能（摘抄自宝塔官网）

1) 首先需要扩展下**/tmp** 空间的大小，设置完后需要**重启下开发板的 linux 系统**，命令如下所示：

```
orangepi@orangepi:~$ sudo sed -i 's/nosuid/&,size=2G/' /etc/fstab
orangepi@orangepi:~$ sudo reboot
```

2) 重启后，可以看到**/tmp** 空间的大小变为 2G 了

```
orangepi@orangepi:~$ df -h | grep "/tmp"
tmpfs                2.0G     12K  2.0G     1% /tmp
```

3) 然后在 linux 系统中输入下面的命令就可以开始宝塔的安装

```
orangepi@orangepi:~$ sudo install_bt_panel.sh
```

4) 然后宝塔安装程序会提醒是否安装 **Bt-Panel** 到**/www** 文件夹，此时输入 **y** 即可

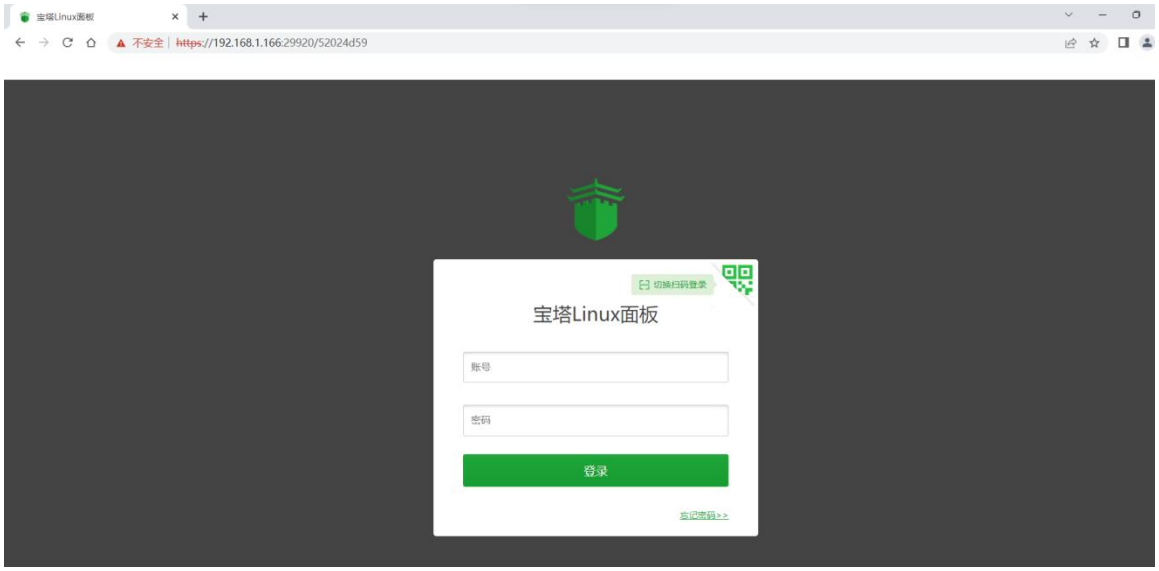
```
+----------------------------------------------------------------------
| Bt-WebPanel FOR CentOS/Ubuntu/Debian
+----------------------------------------------------------------------
| Copyright © 2015-2099 BT-SOFT(http://www.bt.cn) All rights reserved.
+----------------------------------------------------------------------
| The WebPanel URL will be http://SERVER_IP:8888 when installed.
+----------------------------------------------------------------------

Do you want to install Bt-Panel to the /www directory now?(y/n): y
```

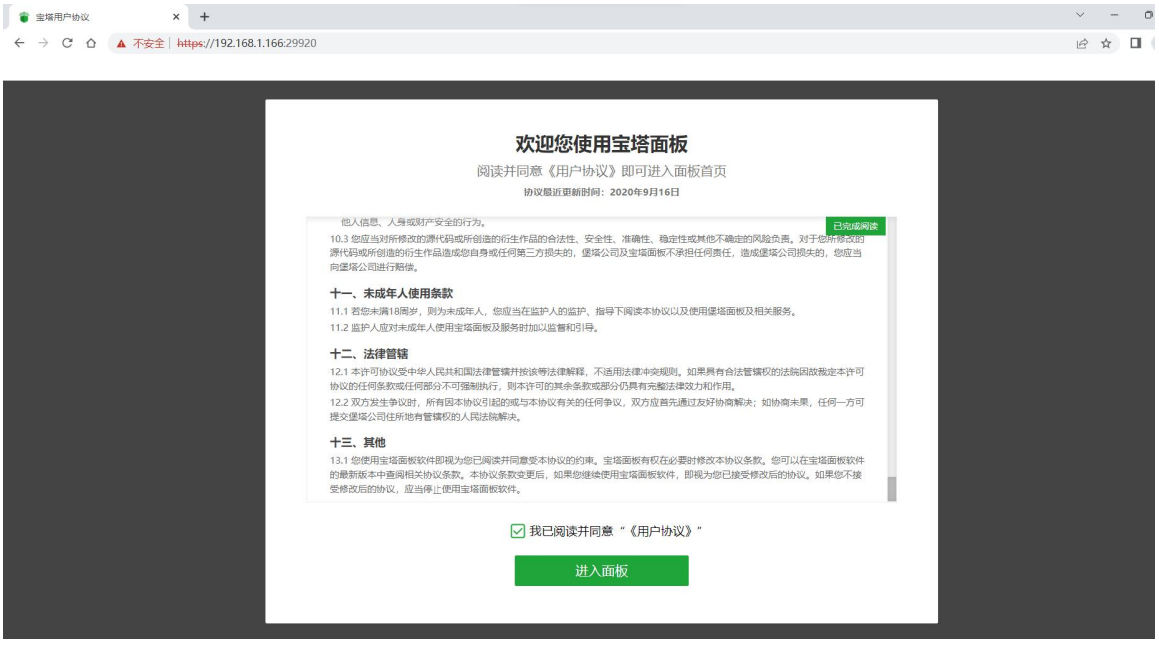
5) 然后要做的就是耐心等待，当看到终端输出下面的打印信息时，说明宝塔已经安装完成，整个安装过程大约耗时 34 分钟，根据网络速度的不同可能会有一些差别

```
==================================================================
Congratulations! Installed successfully!
==================================================================
外网面板地址: https://183.15.204.194:29920/52024d59
内网面板地址: https://192.168.1.166:29920/52024d59
username: 4qhagfrc
password: 27b2d026
If you cannot access the panel,
release the following panel port [29920] in the security group
若无法访问面板，请检查防火墙/安全组是否有放行面板[29920]端口
因已开启面板自签证书，访问面板会提示不匹配证书，请参考以下链接配置证书
https://www.bt.cn/bbs/thread-105443-1-1.html
==================================================================
Time consumed: 34 Minute!
orangepi@orangepi:~$
```

6) 此时在浏览器中输入上面显示的**面板地址**就可以打开宝塔 Linux 面板的登录界面，然后在对应的位置输入上图显示的 **username** 和 **password** 就可以登录进宝塔

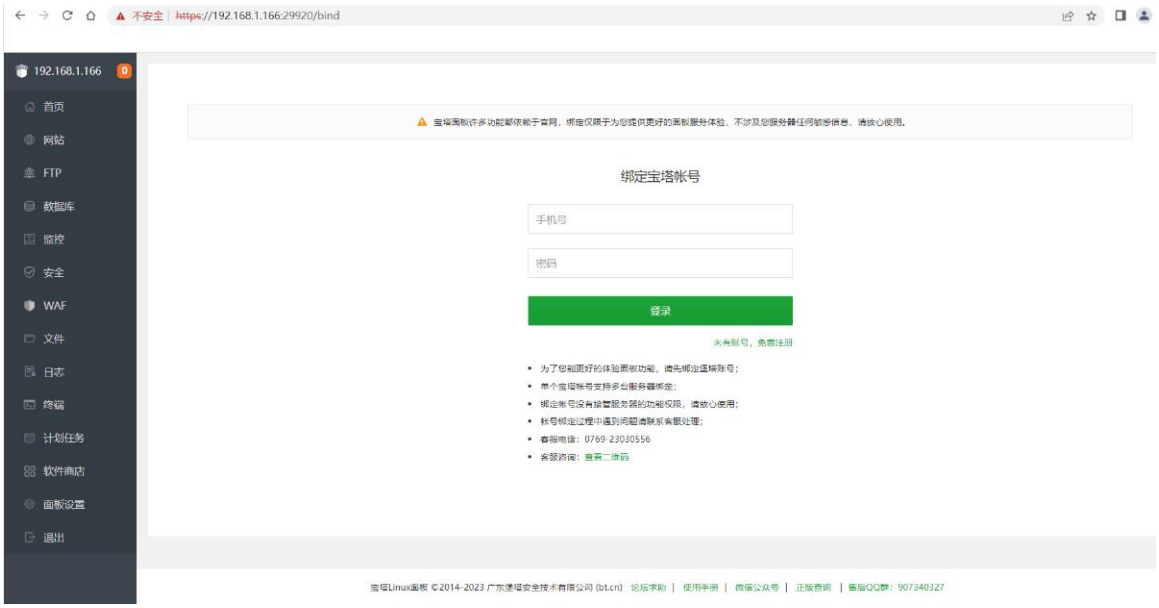


7) 成功登录宝塔后的会弹出下面的欢迎界面，首先请将中间的用户须知阅读完拖到最下面，然后就可以选择“我已同意并阅读《用户协议》”，接着点击“进入面板”就可以进入宝塔了

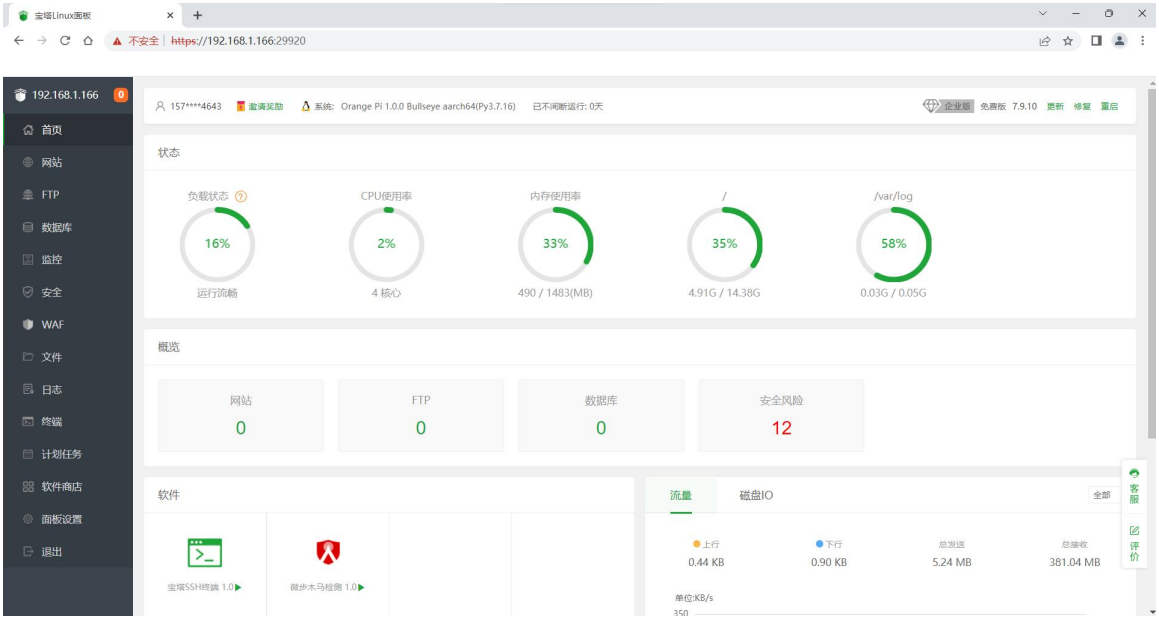


8) 进入宝塔后首先会提示需要绑定宝塔官网的账号，如果没有账号可以去宝塔的官

网（**https://www.bt.cn**）注册一个



9) 最终显示的界面如下图所示，可以很直观的看到开发板 Linux 系统的一些状态信息，比如负载状态、CPU 的使用率、内存使用率和存储空间的使用情况等



10) 宝塔的更多功能可以参考下面资料自行探索

使用手册：**http://docs.bt.cn**
论坛地址：**https://www.bt.cn/bbs**
GitHub 链接：**https://github.com/aaPanel/BaoTa**

3.26. QT 的安装方法

1) 使用下面的脚本可以安装 QT5 和 QT Creator

```
orangepi@orangepi:~$ install_qt.sh
```

2) 安装完后会自动打印 QT 的版本号

a. Ubuntu22.04 自带的 QT 版本为 **5.15.3**

```
orangepi@orangepi:~$ install_qt.sh
.....
QMake version 3.1
Using Qt version 5.15.3 in /usr/lib/aarch64-linux-gnu
```

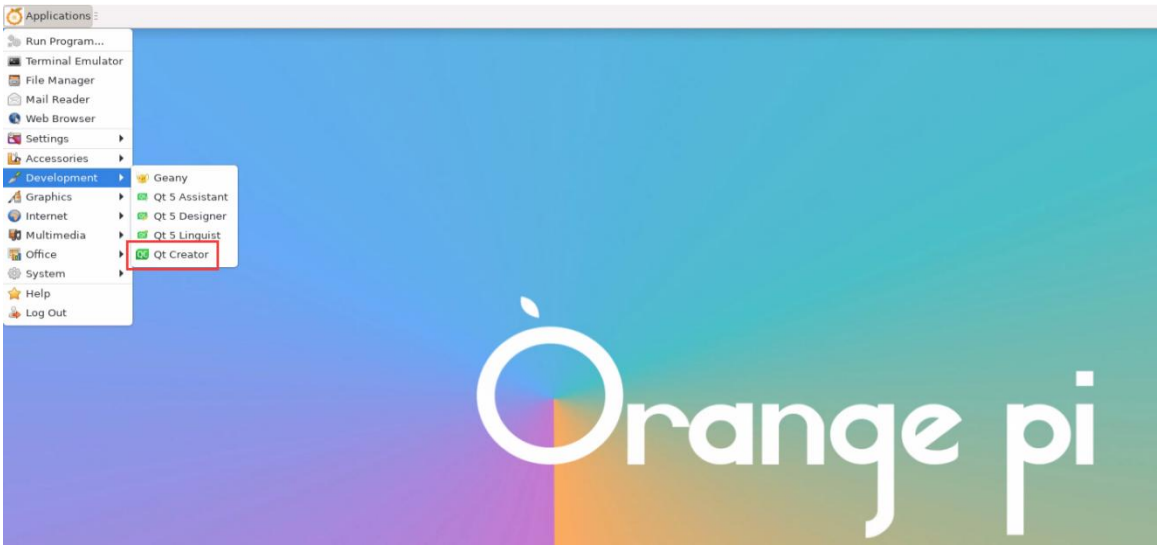
b. Debian11 自带的 QT 版本为 **5.15.2**

```
orangepi@orangepi:~$ install_qt.sh
.....
QMake version 3.1
Using Qt version 5.15.2 in /usr/lib/aarch64-linux-gnu
```

c. Debian12 自带的 QT 版本为 **5.15.8**

```
orangepi@orangepi:~$ install_qt.sh
.....
QMake version 3.1
Using Qt version 5.15.8 in /usr/lib/aarch64-linux-gnu
```

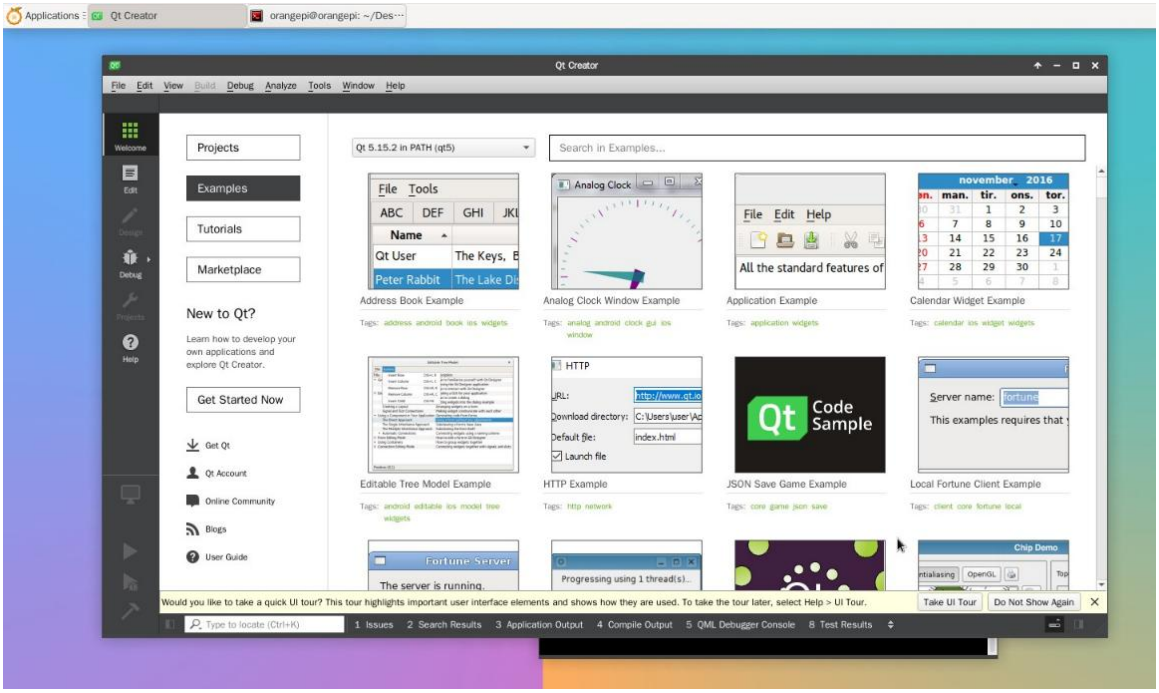
3) 然后在 **Applications** 列表中就可以看到 QT Creator 的启动图标



也可以使用下面的命令打开 QT Creator

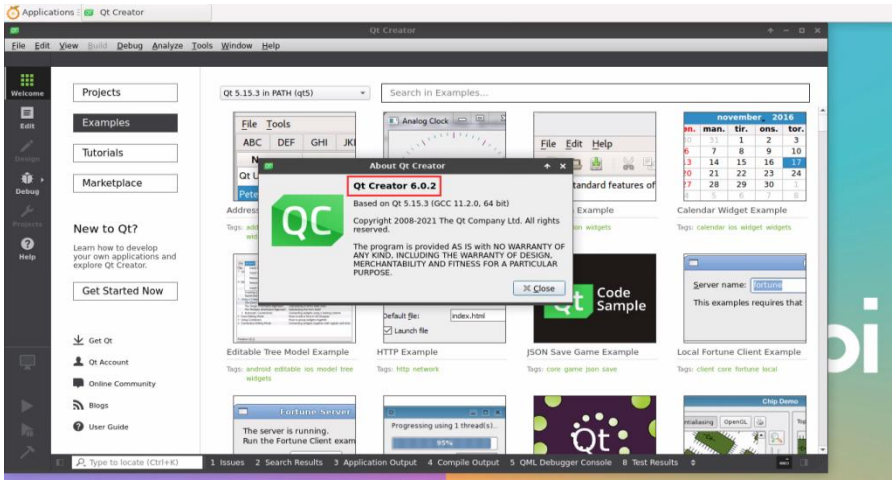
```
orangepi@orangepi:~$ qtcreator
```

4) QT Creator 打开后的界面如下所示

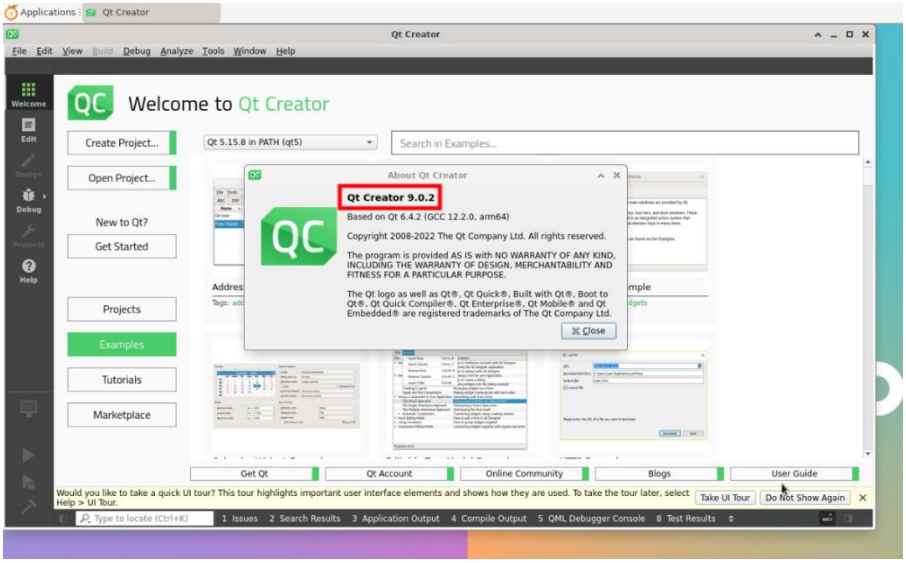


5) QT Creator 的版本如下所示

a. QT Creator 在 **Ubuntu22.04** 中的默认版本如下所示

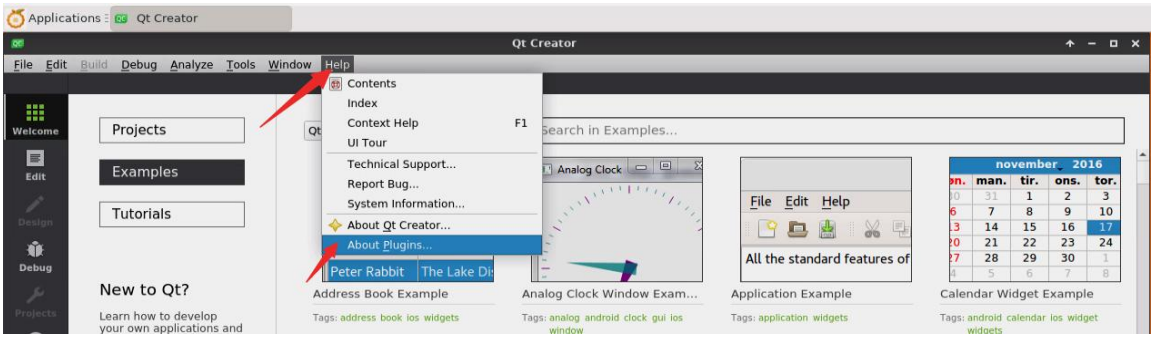


b. QT Creator 在 **Debian12** 中的默认版本如下所示

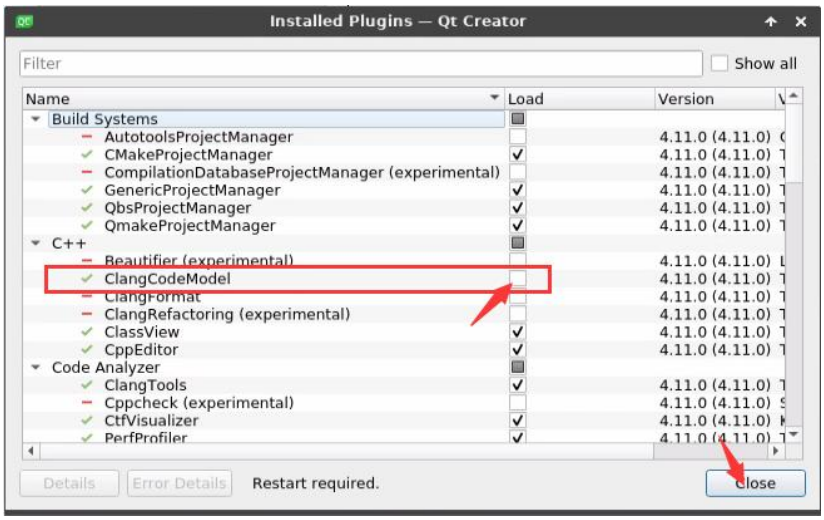


6) 然后设置下 QT

a. 首先打开 **Help->About Plugins...**



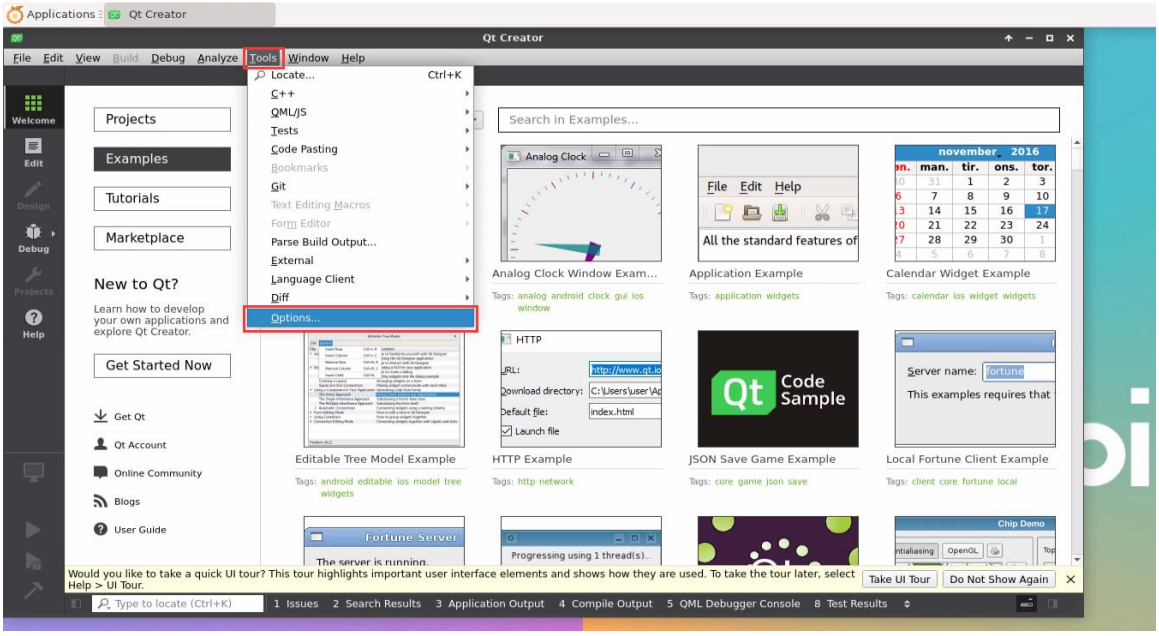
b. 然后去掉 **ClangCodeModel** 的那个勾

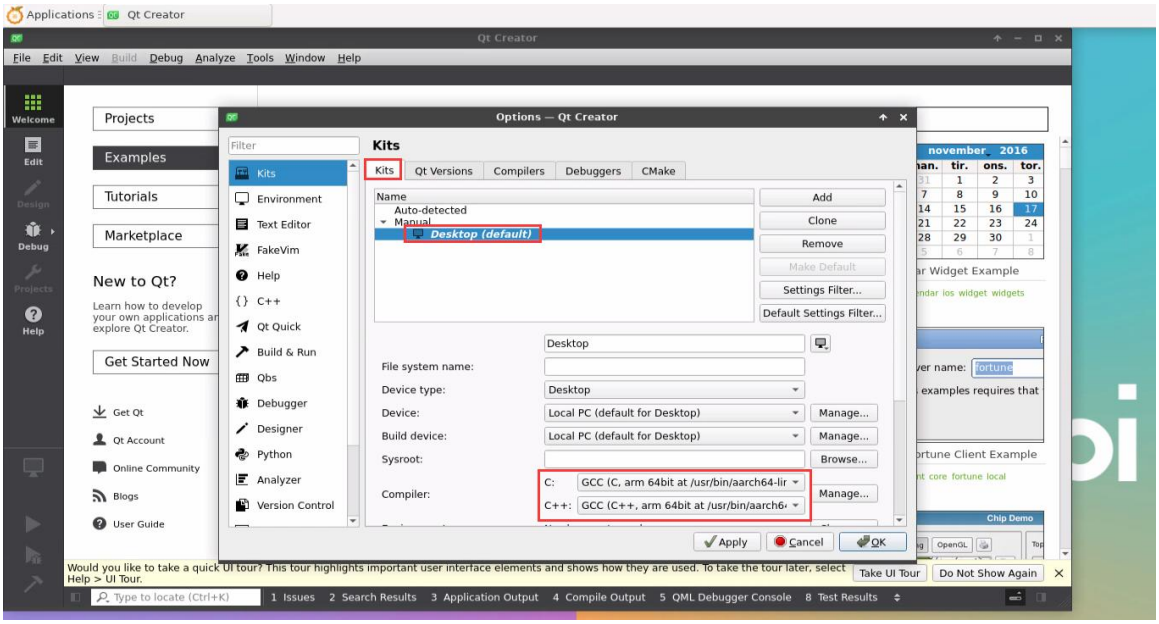


c. **设置完后需要重启下 QT Creator**

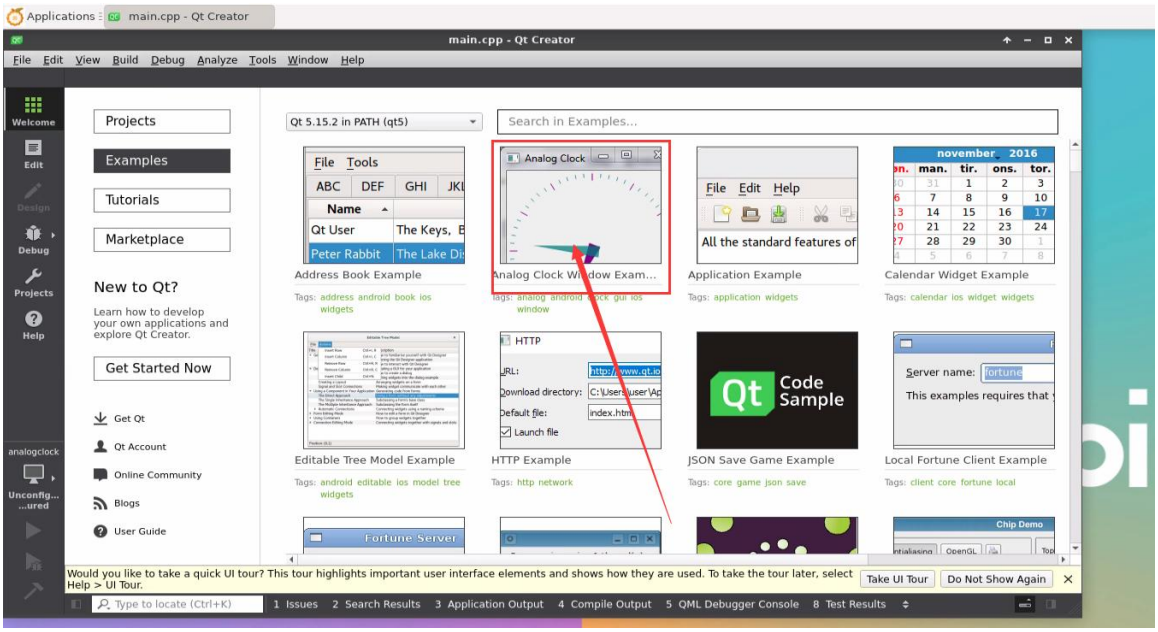
d. 然后确保 QT Creator 使用的 GCC 编译器，如果默认为 Clang，请修改为 GCC

Debian12 请跳过这步。

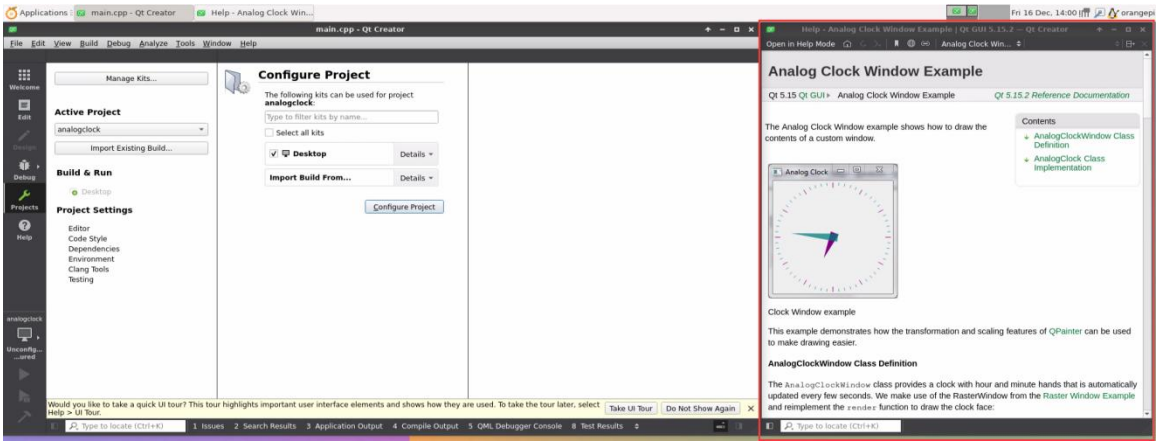




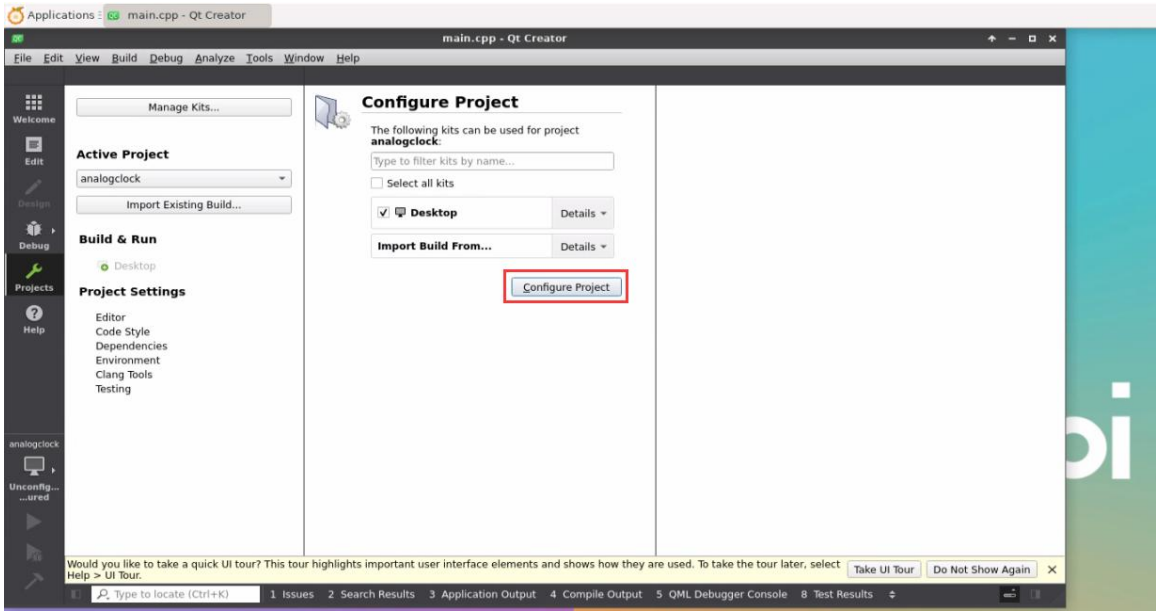
7) 然后就可以打开一个示例代码



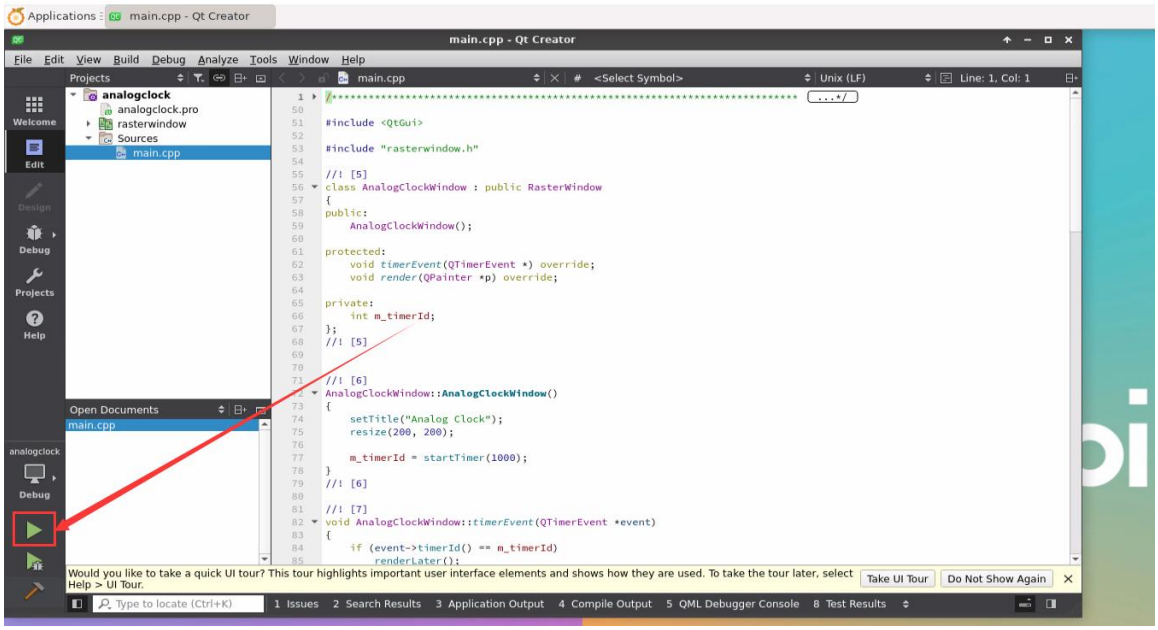
8) 点击示例代码后会自动打开对应的说明文档，可以仔细看下其中的使用说明



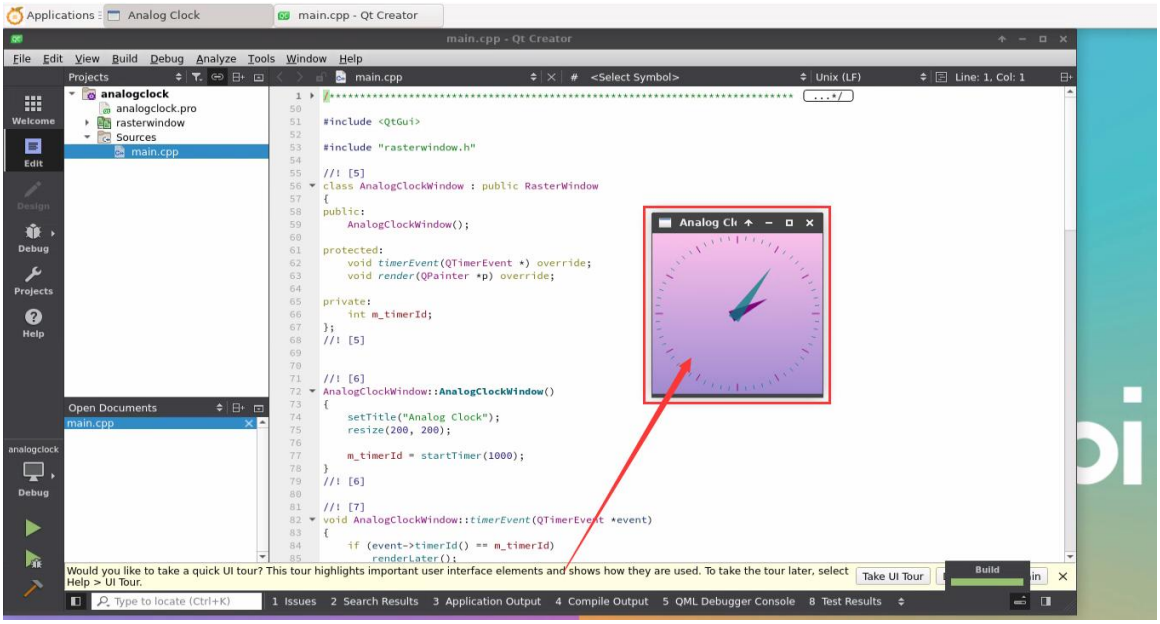
9) 然后点击下 **Configure Project**



10) 然后点击左下角的绿色三角形编译运行下示例代码



11) 等待一段时间后，会弹出下图所示的界面，此时就说明 QT 能正常编译运行



12) 参考资料

https://wiki.qt.io/Install_Qt_5_on_Ubuntu
https://download.qt.io/archive/qtcreator
https://download.qt.io/archive/qt

3.27. ROS 安装方法

3.27.1. Ubuntu22.04 安装 ROS 2 Humble 的方法

1) 使用 **install_ros.sh** 脚本可以安装 ros2

```
orangepi@orangepi:~$ install_ros.sh ros2
```

2) **install_ros.sh** 脚本安装完 ros2 后会自动运行下 **ros2 -h** 命令，如果能看到下面的打印，说明 ros2 安装完成

```
usage: ros2 [-h] Call `ros2 <command> -h` for more detailed usage. ...

ros2 is an extensible command-line tool for ROS 2.

optional arguments:
  -h, --help            show this help message and exit
```

```
Commands:
  action         Various action related sub-commands
  bag            Various rosbag related sub-commands
  component      Various component related sub-commands
  daemon         Various daemon related sub-commands
  doctor         Check ROS setup and other potential issues
  interface      Show information about ROS interfaces
  launch         Run a launch file
  lifecycle      Various lifecycle related sub-commands
  multicast      Various multicast related sub-commands
  node           Various node related sub-commands
  param          Various param related sub-commands
  pkg            Various package related sub-commands
  run            Run a package specific executable
  security       Various security related sub-commands
  service        Various service related sub-commands
  topic          Various topic related sub-commands
  wtf            Use `wtf` as alias to `doctor`

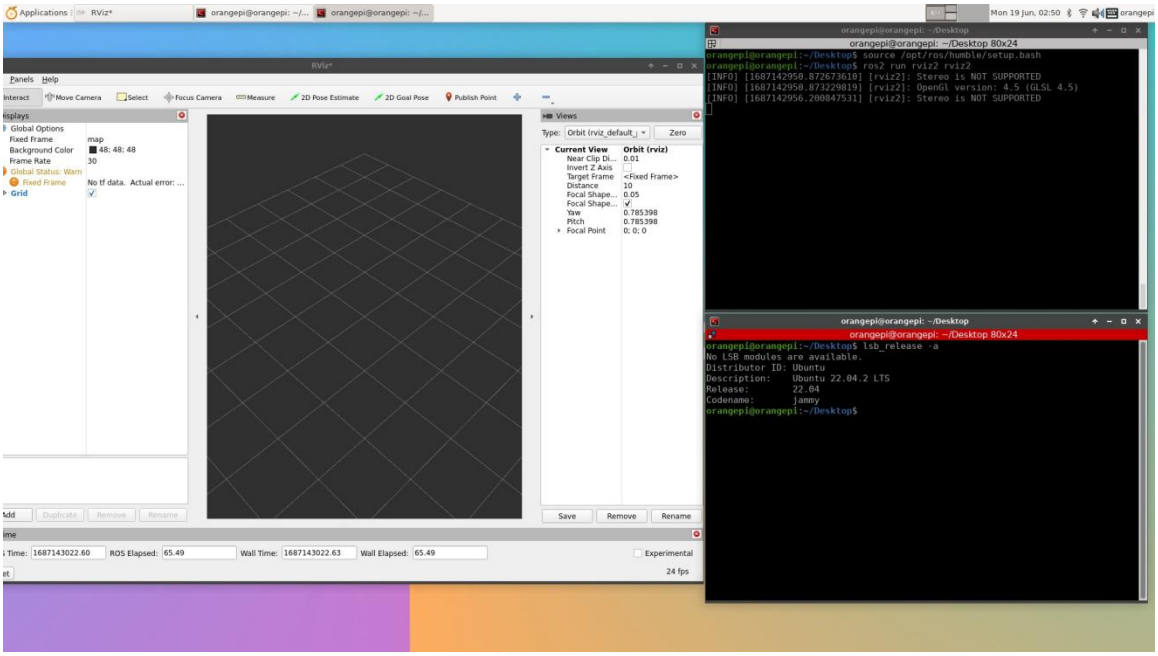
  Call `ros2 <command> -h` for more detailed usage.
```

3) 然后可以使用 **test_ros.sh** 脚本测试下 ROS 2 是否安装成功，如果能看到下面的打印，说明 ROS 2 能正常运行

```
orangepi@orangepi:~$ test_ros.sh
[INFO] [1671174101.200091527] [talker]: Publishing: 'Hello World: 1'
[INFO] [1671174101.235661048] [listener]: I heard: [Hello World: 1]
[INFO] [1671174102.199572327] [talker]: Publishing: 'Hello World: 2'
[INFO] [1671174102.204196299] [listener]: I heard: [Hello World: 2]
[INFO] [1671174103.199580322] [talker]: Publishing: 'Hello World: 3'
[INFO] [1671174103.204019965] [listener]: I heard: [Hello World: 3]
```

4) 运行下面的命令可以打开 rviz2

```
orangepi@orangepi:~$ source /opt/ros/humble/setup.bash
orangepi@orangepi:~$ ros2 run rviz2 rviz2
```



5) 参考文档

http://docs.ros.org/en/humble/index.html
http://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debians.html

3.28. 安装内核头文件的方法

1) OPi 发布的 Linux 镜像默认自带了内核头文件的 deb 包，存放的位置为**/opt/**

```
orangepi@orangepi:~$ ls /opt/linux-headers*
/opt/linux-headers-xxx-sun60iw2_x.x.x_arm64.deb
```

2) 使用下面的命令可以安装内核头文件的 deb 包

```
orangepi@orangepi:~$ sudo dpkg -i /opt/linux-headers*.deb
```

3) 安装完后在**/usr/src** 下就能看到内核头文件所在的文件夹

```
orangepi@orangepi:~$ ls /usr/src
linux-headers-x.x.x
```

4) 然后可以编译下 Linux 镜像中自带的 hello 内核模块的源码，hello 模块的源码在**/usr/src/hello** 中，进入此目录后，然后使用 make 命令编译即可。

```
orangepi@orangepi:~$ cd /usr/src/hello/
```

```
orangepi@orangepi:/usr/src/hello$ sudo make
make -C /lib/modules/5.15.147-sun60iw2/build M=/usr/src/hello modules
make[1]: Entering directory '/usr/src/linux-headers-5.15.147-sun60iw2'
  CC [M]  /usr/src/hello/hello.o
  MODPOST /usr/src/hello/Module.symvers
  CC [M]  /usr/src/hello/hello.mod.o
  LD [M]  /usr/src/hello/hello.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.15.147-sun60iw2'
```

5) 编译完后会生成 **hello.ko** 内核模块

```
orangepi@orangepi:/usr/src/hello$ ls *.ko
hello.ko
```

6) 使用 **insmod** 命令可以将 **hello.ko** 内核模块插入内核中

```
orangepi@orangepi:/usr/src/hello$ sudo insmod hello.ko
```

7) 然后使用 **demsg** 命令可以查看下 **hello.ko** 内核模块的输出，如果能看到下面的输出说明 **hello.ko** 内核模块加载正确

```
orangepi@orangepi:/usr/src/hello$ sudo dmesg | grep "Hello"
[ 2871.893988] Hello Orange Pi -- init
```

8) 使用 **rmmod** 命令可以卸载 **hello.ko** 内核模块

```
orangepi@orangepi:/usr/src/hello$ sudo rmmod hello
orangepi@orangepi:/usr/src/hello$ dmesg | grep "Hello"
[ 2871.893988] Hello Orange Pi -- init
[ 3173.800892] Hello Orange Pi -- exit
```

3.29. Linux 系统支持的部分编程语言测试

3.29.1. Debian Bullseye 系统

1) Debian Bullseye 默认安装有 gcc 编译工具链，可以直接在开发板的 Linux 系统中编译 C 语言的程序

a. gcc 的版本如下所示

```
orangepi@orangepi:~$ gcc --version
```

```
gcc (Debian 10.2.1-6) 10.2.1 20210110
Copyright (C) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR
PURPOSE.
```

b. 编写 C 语言的 **hello_world.c** 程序

```
orangepi@orangepi:~$ vim hello_world.c
#include <stdio.h>

int main(void)
{
        printf("Hello World!\n");

        return 0;
}
```

c. 然后编译运行 **hello_world.c**

```
orangepi@orangepi:~$ gcc -o hello_world hello_world.c
orangepi@orangepi:~$ ./hello_world
Hello World!
```

2) Debian Bullseye 默认安装有 Python3

a. Python 具体版本如下所示

```
orangepi@orangepi:~$ python3
Python 3.9.2 (default, Feb 28 2021, 17:03:44)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

使用 Ctrl+D 快捷键可退出 python 的交互模式。

b. 编写 Python 语言的 **hello_world.py** 程序

```
orangepi@orangepi:~$ vim hello_world.py
print('Hello World!')
```

c. 运行 **hello_world.py** 的结果如下所示

```
orangepi@orangepi:~$ python3 hello_world.py
Hello World!
```

3) Debian Bullseye 默认没有安装 Java 的编译工具和运行环境

a. 可以使用下面的命令安装 openjdk，Debian Bullseye 中最新版本为 openjdk-17

```
orangepi@orangepi:~$ sudo apt install -y openjdk-17-jdk
```

b. 安装完后可以查看下 Java 的版本

```
orangepi@orangepi:~$ java --version
```

c. 编写 Java 版本的 **hello_world.java**

```
orangepi@orangepi:~$ vim hello_world.java
public class hello_world
{
        public static void main(String[] args)
        {
                System.out.println("Hello World!");
        }
}
```

d. 然后编译运行 **hello_world.java**

```
orangepi@orangepi:~$ javac hello_world.java
orangepi@orangepi:~$ java hello_world
Hello World!
```

3.29.2. Debian Bookworm 系统

1) Debian Bookworm 默认安装有 gcc 编译工具链，可以直接在开发板的 Linux 系统中编译 C 语言的程序

a. gcc 的版本如下所示

```
orangepi@orangepi:~$ gcc --version
gcc (Debian 12.2.0-14) 12.2.0
Copyright (C) 2022 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

b. 编写 C 语言的 **hello_world.c** 程序

```
orangepi@orangepi:~$ vim hello_world.c
#include <stdio.h>

int main(void)
{
```

```
        printf("Hello World!\n");

        return 0;
}
```

c. 然后编译运行 **hello_world.c**

```
orangepi@orangepi:~$ gcc -o hello_world hello_world.c
orangepi@orangepi:~$ ./hello_world
Hello World!
```

2) Debian Bookworm 默认安装有 Python3

a. Python 具体版本如下所示

```
orangepi@orangepi:~$ python3
Python 3.11.2 (main, Apr 28 2025, 14:11:48) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

使用 Ctrl+D 快捷键可退出 python 的交互模式。

b. 编写 Python 语言的 **hello_world.py** 程序

```
orangepi@orangepi:~$ vim hello_world.py
print('Hello World!')
```

c. 运行 **hello_world.py** 的结果如下所示

```
orangepi@orangepi:~$ python3 hello_world.py
Hello World!
```

3) Debian Bookworm 默认没有安装 Java 的编译工具和运行环境

a. 可以使用下面的命令安装 openjdk，Debian Bookworm 中最新版本为 openjdk-17

```
orangepi@orangepi:~$ sudo apt install -y openjdk-17-jdk
```

b. 安装完后可以查看下 Java 的版本

```
orangepi@orangepi:~$ java --version
```

c. 编写 Java 版本的 **hello_world.java**

```
orangepi@orangepi:~$ vim hello_world.java
public class hello_world
{
        public static void main(String[] args)
        {
```

```
                System.out.println("Hello World!");
        }
}
```

d. 然后编译运行 **hello_world.java**

```
orangepi@orangepi:~$ javac hello_world.java
orangepi@orangepi:~$ java hello_world
Hello World!
```

3.29.3. Ubuntu Jammy 系统

1) Ubuntu Jammy 默认安装有 gcc 编译工具链，可以直接在开发板的 Linux 系统中编译 C 语言的程序

a. gcc 的版本如下所示

```
orangepi@orangepi:~$ gcc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR
 PURPOSE.
```

b. 编写 C 语言的 **hello_world.c** 程序

```
orangepi@orangepi:~$ vim hello_world.c
#include <stdio.h>

int main(void)
{
        printf("Hello World!\n");

        return 0;
}
```

c. 然后编译运行 **hello_world.c**

```
orangepi@orangepi:~$ gcc -o hello_world hello_world.c
orangepi@orangepi:~$ ./hello_world
Hello World!
```

2) Ubuntu Jammy 默认安装有 Python3

a. Python3 具体版本如下所示

```
orangepi@orangepi:~$ python3
Python 3.10.12 (main, Jul 29 2024, 16:56:48) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

使用 Ctrl+D 快捷键可退出 python 的交互模式。

b. 编写 Python 语言的 **hello_world.py** 程序

```
orangepi@orangepi:~$ vim hello_world.py
print('Hello World!')
```

c. 运行 **hello_world.py** 的结果如下所示

```
orangepi@orangepi:~$ python3 hello_world.py
Hello World!
```

3) Ubuntu Jammy 默认没有安装 Java 的编译工具和运行环境

a. 可以使用下面的命令安装 openjdk-18

```
orangepi@orangepi:~$ sudo apt install -y openjdk-18-jdk
```

b. 安装完后可以查看下 Java 的版本

```
orangepi@orangepi:~$ java --version
openjdk 18.0.2-ea 2022-07-19
OpenJDK Runtime Environment (build 18.0.2-ea+9-Ubuntu-222.04)
OpenJDK 64-Bit Server VM (build 18.0.2-ea+9-Ubuntu-222.04, mixed mode, sharing)
```

c. 编写 Java 版本的 **hello_world.java**

```
orangepi@orangepi:~$ vim hello_world.java
public class hello_world
{
        public static void main(String[] args)
        {
                System.out.println("Hello World!");
        }
}
```

d. 然后编译运行 **hello_world.java**

```
orangepi@orangepi:~$ javac hello_world.java
orangepi@orangepi:~$ java hello_world
Hello World!
```

3.30. 上传文件到开发板 Linux 系统中的方法

3.30.1. 在 Ubuntu PC 中上传文件到开发板 Linux 系统中的方法

3.30.1.1. 使用 scp 命令上传文件的方法

1) 使用 scp 命令可以在 Ubuntu PC 中上传文件到开发板的 Linux 系统中，具体命令如下所示

a. **file_path：**需要替换为要上传文件的路径
b. **orangepi：**为开发板 linux 系统的用户名，也可以替换成其它的，比如 root
c. **192.168.xx.xx:** 为开发板的 IP 地址，请根据实际情况进行修改
d. **/home/orangepi:** 开发板 linux 系统中的路径，也可以修改为其它的路径

```
test@test:~$ scp    file_path    orangepi@192.168.xx.xx:/home/orangepi/
```

2) 如果要上传文件夹，需要加上-r 参数

```
test@test:~$ scp    -r    dir_path    orangepi@192.168.xx.xx:/home/orangepi/
```

3) scp 还有更多的用法，请使用下面的命令查看 man 手册

```
test@test:~$ man scp
```

3.30.1.2. 使用 filezilla 上传文件的方法

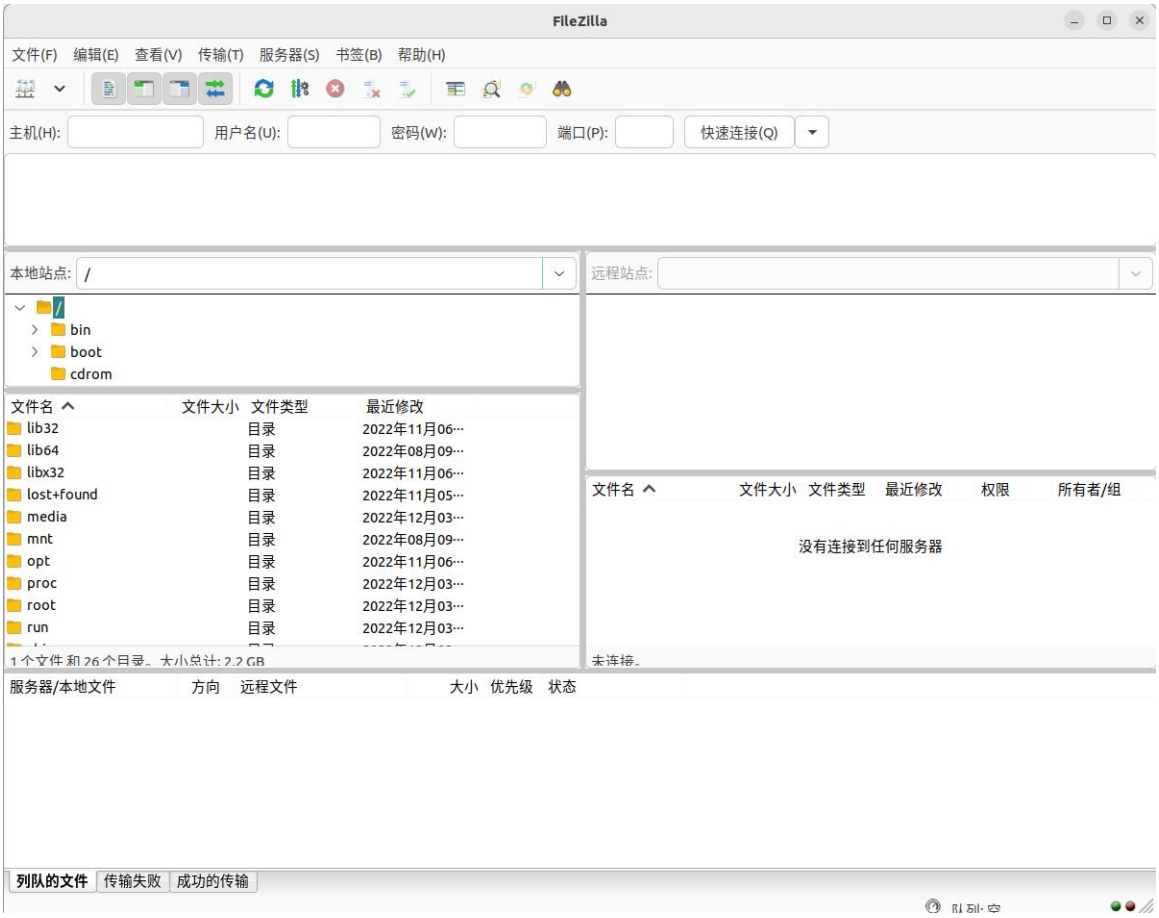
1) 首先在 Ubuntu PC 中安装 filezilla

```
test@test:~$ sudo apt install -y filezilla
```

2) 然后使用下面的命令打开 filezilla

```
test@test:~$ filezilla
```

3) filezilla 打开后的界面如下所示，此时右边远程站点下面显示的是空的



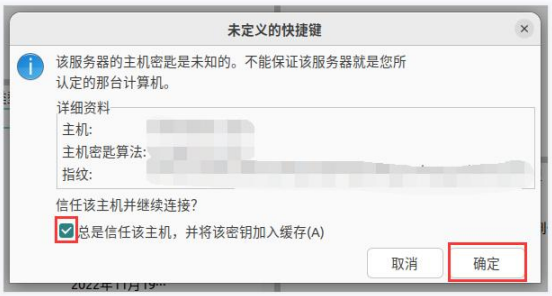
4) 连接开发板的方法如下图所示



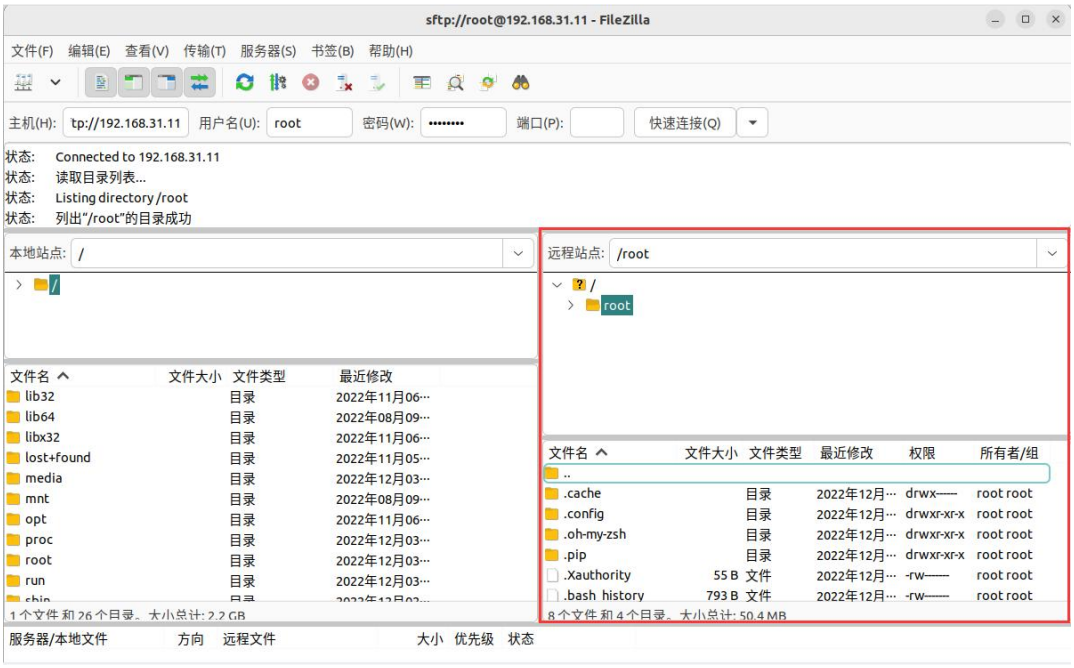
5) 然后选择**保存密码**，再点击**确定**



6) 然后选择**总是信任该主机**，再点击**确定**

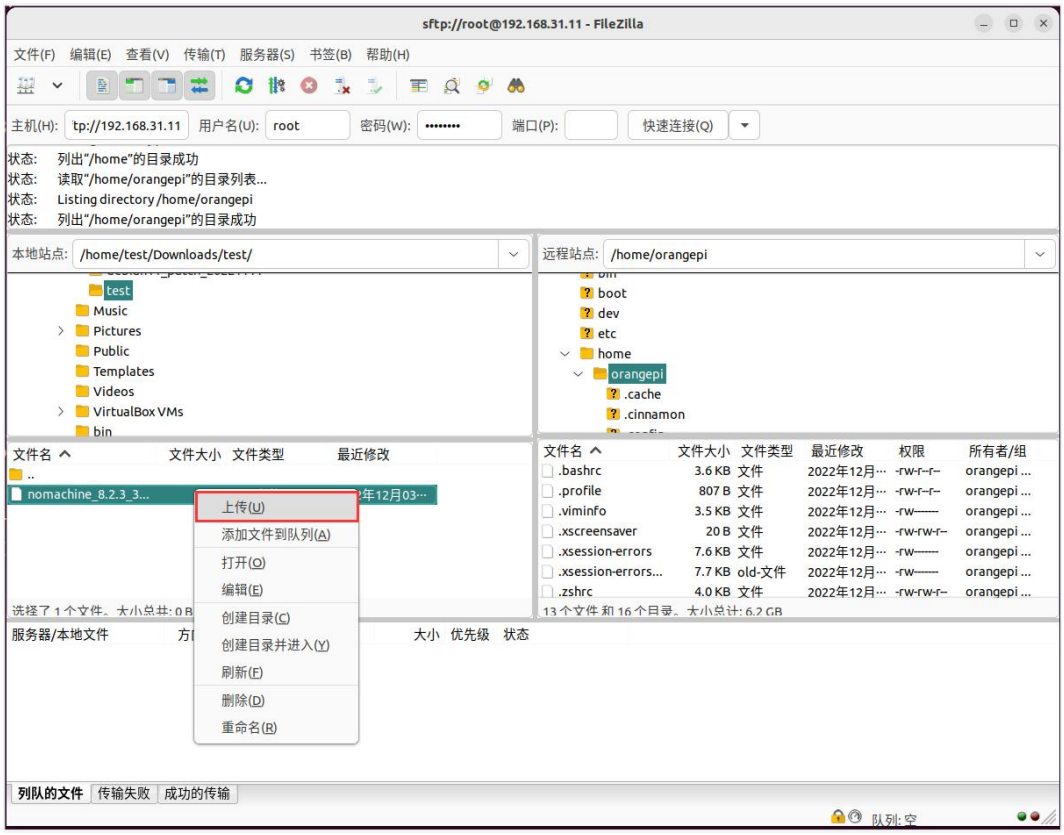


7) 连接成功后在 filezilla 软件的右边就可以看到开发板 linux 文件系统的目录结构了



8) 然后在 filezilla 软件的右边选择要上传到开发板中的路径，再在 filezilla 软件的左边选中 Ubuntu PC 中要上传的文件，再点击鼠标右键，再点击上传选项就会开始上

传文件到开发板中了。



9) 上传完成后就可以去开发板 linux 系统中的对应路径中查看上传的文件了

10) 上传文件夹的方法和上传文件的方法是一样的，这里就不再赘述了

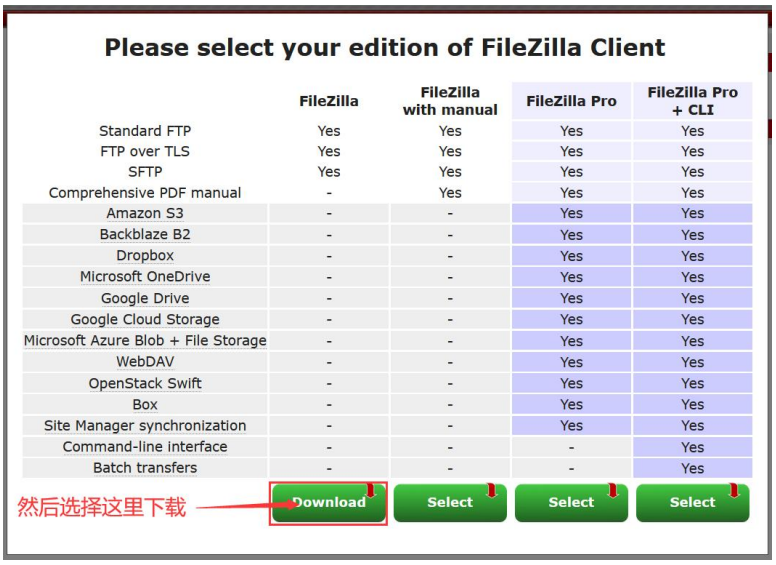
3.30.2. 在 Windows PC 中上传文件到开发板 Linux 系统中的方法

3.30.2.1. 使用 filezilla 上传文件的方法

1) 首先下载 filezilla 软件 Windows 版本的安装文件，下载链接如下所示

https://filezilla-project.org/download.php?type=client

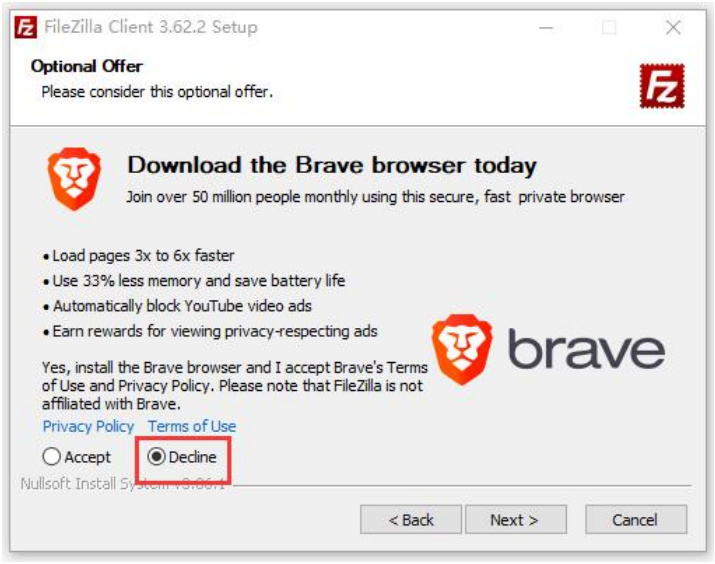




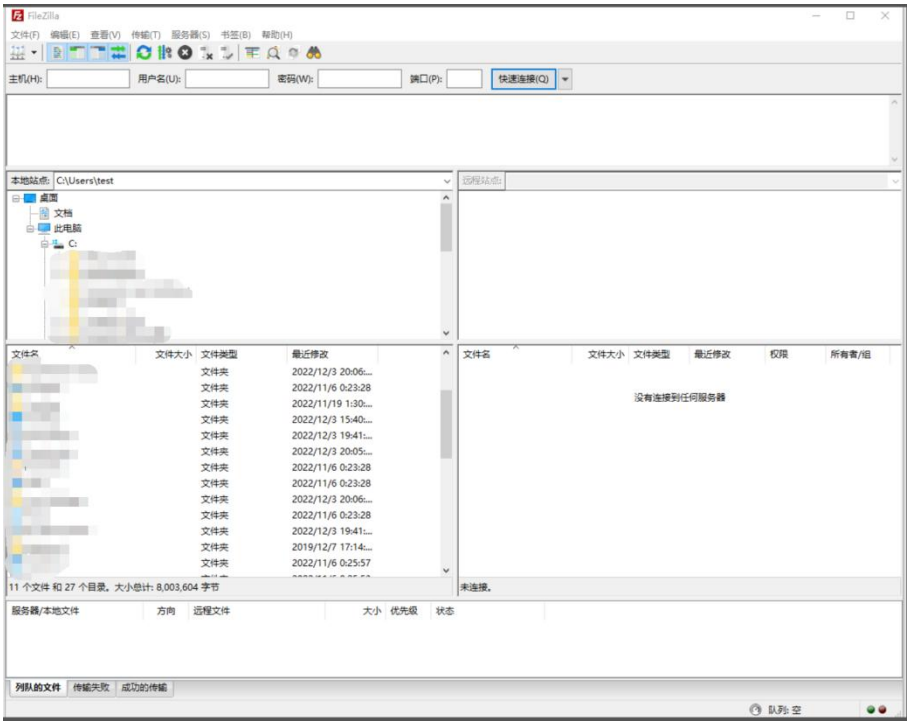
2) 下载的安装包如下所示，然后双击直接安装即可

FileZilla_Server_1.5.1_win64-setup.exe

安装过程中，下面的安装界面请选择 `Decline`，然后再选择 `Next>`



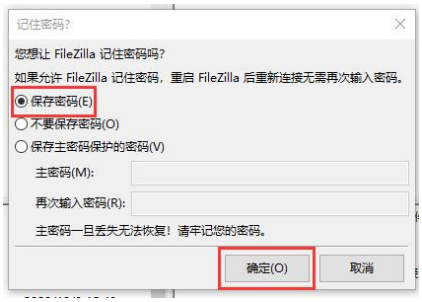
3) filezilla 打开后的界面如下所示，此时右边远程站点下面显示的是空的



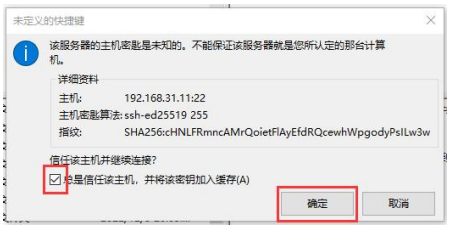
4) 连接开发板的方法如下图所示：



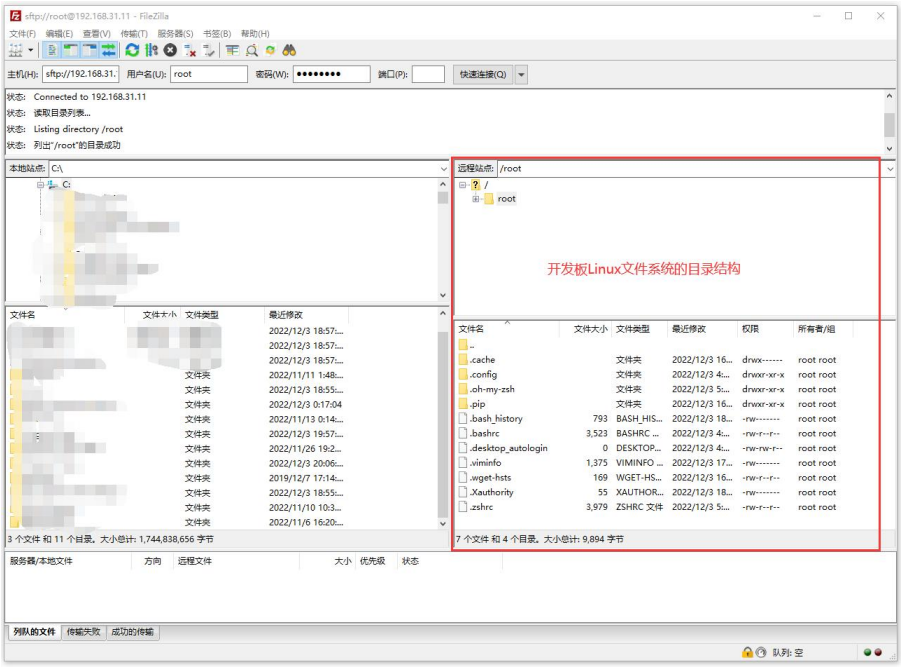
5) 然后选择**保存密码**，再点击**确定**



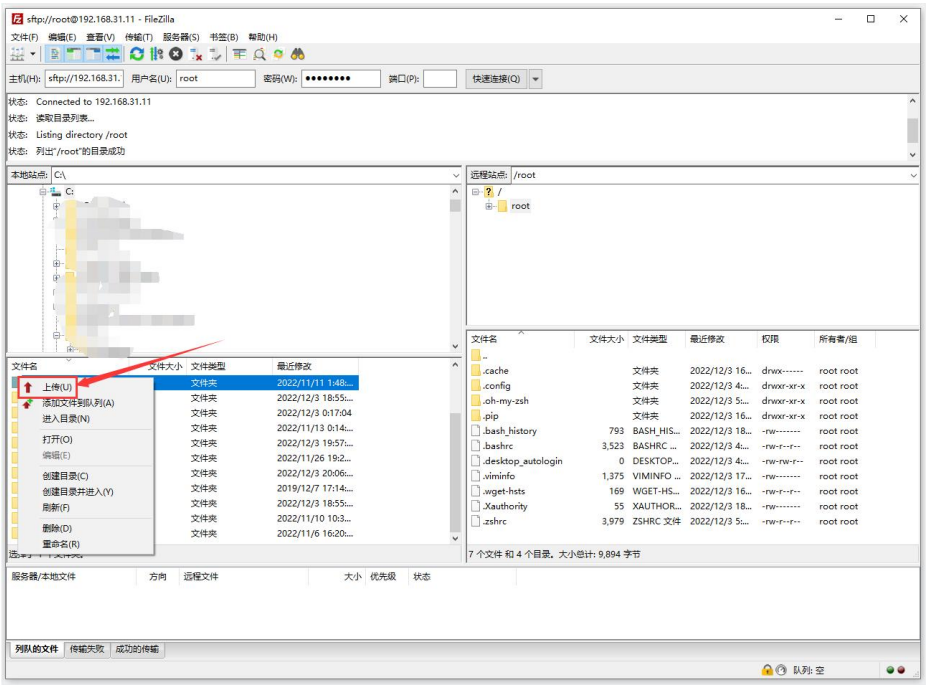
6) 然后选择**总是信任该主机**，再点击**确定**



7) 连接成功后在 filezilla 软件的右边就可以看到开发板 linux 文件系统的目录结构了



8) 然后在 filezilla 软件的右边选择要上传到开发板中的路径，再在 filezilla 软件的左边选中 Windows PC 中要上传的文件，再点击鼠标右键，再点击上传选项就会开始上传文件到开发板中了



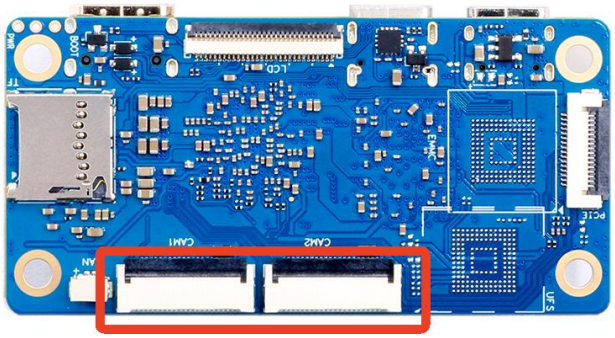
9) 上传完成后就可以去开发板 linux 系统中的对应路径中查看上传的文件了

10) 上传文件夹的方法和上传文件的方法是一样的，这里就不再赘述了

3.31. IMX219 MIPI 摄像头的使用方法

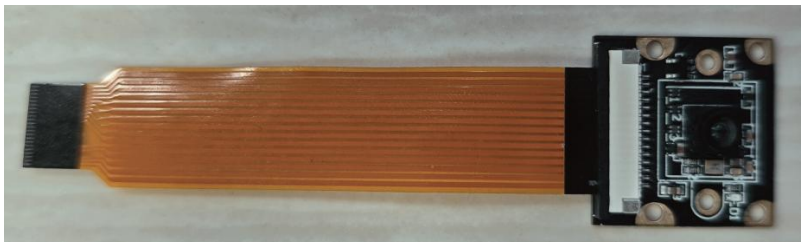
3.31.1. 摄像头的安装方法

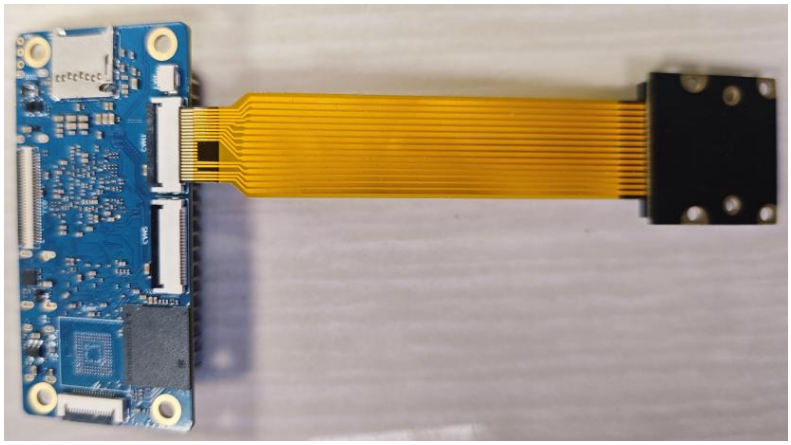
1) 开发板上总共有 2 个摄像头接口，目前只支持接 IMX219 摄像头，CAM1 和 CAM2 的位置如下所示：



2) 摄像头接口规格是 22PIN 0.5mm 间距，需要通过 22PIN 转 15PIN 的排线将

IMX219 摄像头连接到在开发板上，CAM1 和 CAM2 接法都是一样的





3.31.2. 通过 Gstreamer 打开摄像头的方法

注意，目前只有 Debian bullseye 系统支持这个方法，其它版本系统暂时不支持。

1) 首先安装下摄像头驱动

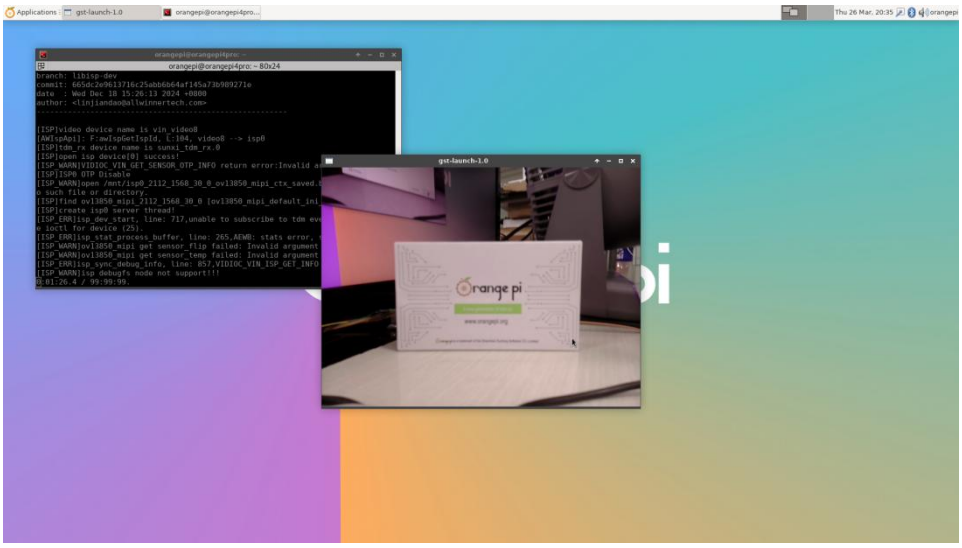
```
orangepi@orangepi:~$ sudo modprobe vin_v4l2
```

2) 然后执行下面的命令确认摄像头的设备节点，当前连接了两个 MIPI 摄像头，所以会显示有两个节点。

```
orangepi@orangepi:~$ ls /dev/video*
/dev/video0   /dev/video8
```

3) 系统预装有 **test_camera.sh** 脚本，可以执行测试脚本打开摄像头。

```
orangepi@orangepi:~$ test_camera.sh /dev/video0     #打开 CAM1
orangepi@orangepi:~$ test_camera.sh /dev/video8     #打开 CAM2
```



3.31.3. 通过 OpenCV 程序打开 MIPI 摄像头的方法

1) 首先需要安装下依赖包。

```
orangepi@orangepi:~$ sudo apt install python3-pip python3-opencv libopencv-dev
orangepi@orangepi:~$ sudo apt install python3-pybind11 python3-dev
```

2) 然后进入 **v4l2_opencv_demo** 所在的目录，可以看到包含有下面的文件。

```
orangepi@orangepi:~$ cd /opt/v4l2_opencv_demo/
orangepi@orangepi:/opt/v4l2_opencv_demo$ ls
Makefile  test_camera.cpp  test_camera.py  v4l2_camera_bind.cpp
v4l2_camera.cpp  v4l2_camera.h
```

3) 将 **v4l2_opencv_demo** 拥有者改为普通用户，这样普通用户才能操作文件.

```
orangepi@orangepi:~$ sudo chown orangepi:orangepi /opt/v4l2_opencv_demo/ -R
```

4) 示例文件说明：

a. **Makefile**：构建 C++示例程序的编译规则文件。
b. **test_camera.cpp**：C++测试程序，使用 OpenCV 实时预览摄像头。
c. **test_camera.py** ：Python 示例程序，使用 OpenCV 预览摄像头画面。
d. **v4l2_camera_bind.cpp**： 使用 pybind11 封装 V4L2Camera 类，供 Python 调用。
e. **v4l2_camera.cpp** ：V4L2Camera 类实现，封装摄像头初始化与帧采集逻辑。
f. **v4l2_camera.h** ： V4L2Camera 类头文件，定义接口与成员变量。

5) 然后安装下摄像头驱动

```
orangepi@orangepi:~$ sudo modprobe vin_v4l2
```

6) 然后执行下面的命令确认摄像头的设备节点，当前连接了两个 MIPI 摄像头，所以会显示有两个节点。

```
orangepi@orangepi:~$ ls /dev/video*
/dev/video0  /dev/video8
```

7) 使用 C++示例打开 MIPI 摄像头的步骤如下：

a. 首先执行下面的命令编译 C++示例。

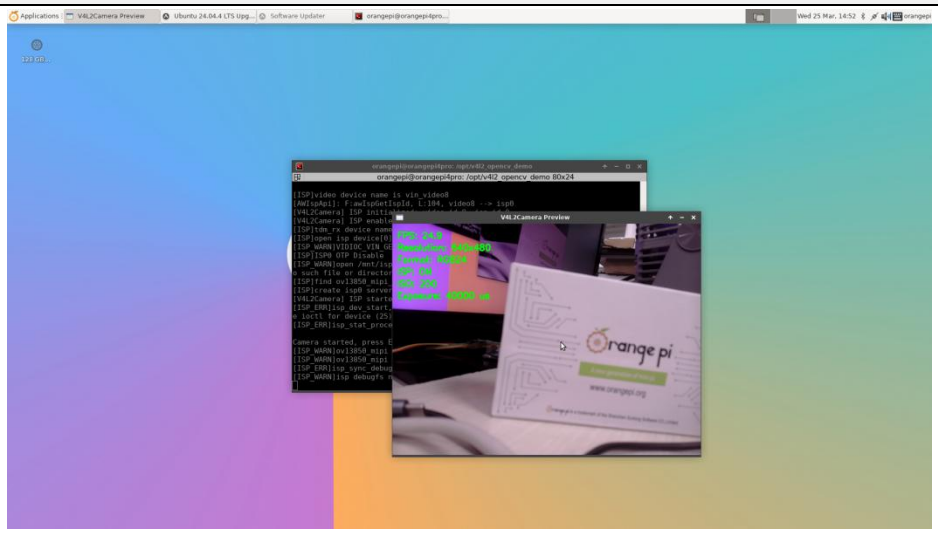
```
orangepi@orangepi:~$ cd /opt/v4l2_opencv_demo/
orangepi@orangepi:/opt/v4l2_opencv_demo$ make clean && make
```

b. 编译完成后在当前目录会生成 **cam_test** 可执行文件。

```
orangepi@orangepi:/opt/v4l2_opencv_demo$ ls cam_test
cam_test
```

c. 然后执行 **cam_test** 程序打开 MIPI 摄像头（注意摄像头节点根据实际情况选择）：

```
orangepi@orangepi:/opt/v4l2_opencv_demo$ ./cam_test /dev/video0     #打开 CAM1
orangepi@orangepi:/opt/v4l2_opencv_demo$ ./cam_test /dev/video8     #打开 CAM2
```



d. 摄像头分辨率可通过修改 **test_camera.cpp** 文件中的以下代码设置，修改后注意需要重新编译代码。

```
int main(int argc, char* argv[])
{
```

```
const char* device = "/dev/video8";
int width = 640;
int height = 480;
```

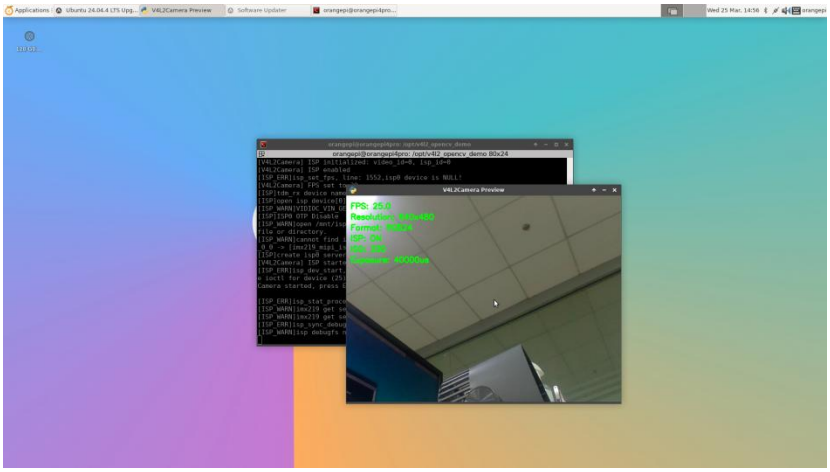
8) 使用 Python 示例打开 MIPI 摄像头的步骤如下：

a. 首先执行下面的命令编译 **python_module**

```
orangepi@orangepi:~$ cd /opt/v4l2_opencv_demo/
orangepi@orangepi:/opt/v4l2_opencv_demo$ make python_module
```

b. 然后执行 test_camera.py 脚本打开 MIPI 摄像头。

```
orangepi@orangepi:/opt/v4l2_opencv_demo$ python3 test_camera.py -d /dev/video0    #打开 CAM1
orangepi@orangepi:/opt/v4l2_opencv_demo$ python3 test_camera.py -d /dev/video8    #打开 CAM2
```



c. 摄像头分辨率可通过修改 test_camera.py 文件中的以下代码设置：

```
parser.add_argument('--width', '-W', type=int, default=640,
                    help='Image width (default: 640)')
parser.add_argument('--height', '-H', type=int, default=480,
                    help='Image height (default: 480)')
```

3.32. 10.1 寸 MIPI LCD 屏幕的使用方法

3.32.1. 10.1 寸 MIPI 屏幕的组装方法

1) 首先准备需要的配件

a. 10.1 寸 MIPI LCD 显示屏+触摸屏



b. 屏幕转接板+31pin 转 40pin 排线



c. 30pin MIPI 排线



d. 12pin 触摸屏排线



2) 按照下图将 12pin 触摸屏排线、31pin 转 40pin 排线、30pin MIPI 排线接到屏幕转接板上，注意**触摸屏排线蓝色的绝缘面朝下**，其它两根排线绝缘面朝上，如果接错会导致无显示或者不能触摸的问题



3) 按照下图将连接好排线的转接板置于 MIPI LCD 屏上面，并通过 31pin 转 40pin 排线连接 MIPI LCD 屏与转接板



4) 然后通过 12pin 触摸屏排线连接触摸屏与转接板，注意绝缘面的朝向



5) 最后通过 30pin MIPI 排线连接到开发板的 LCD 接口上



3. 32. 2. 打开 10.1 寸 MIPI LCD 屏幕配置的方法

1) Linux 镜像默认是没有打开 mipi lcd 屏幕的配置的，如果需要使用 mipi lcd 屏幕，需要手动打开才行。

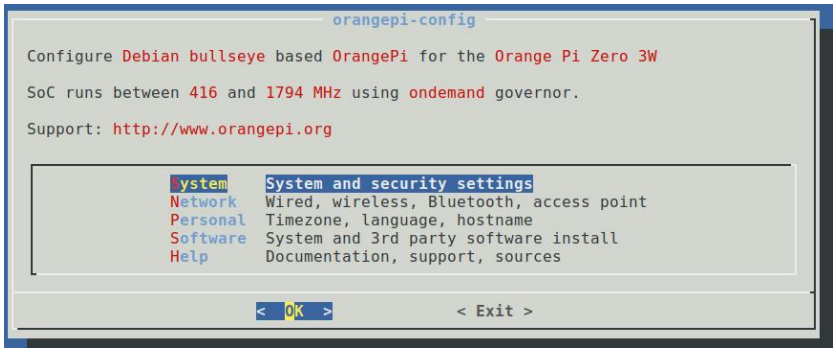
2) 开发板上 mipi lcd 屏幕的接口如下图所示：

3) 打开 mipi lcd 配置的方法如下所示：

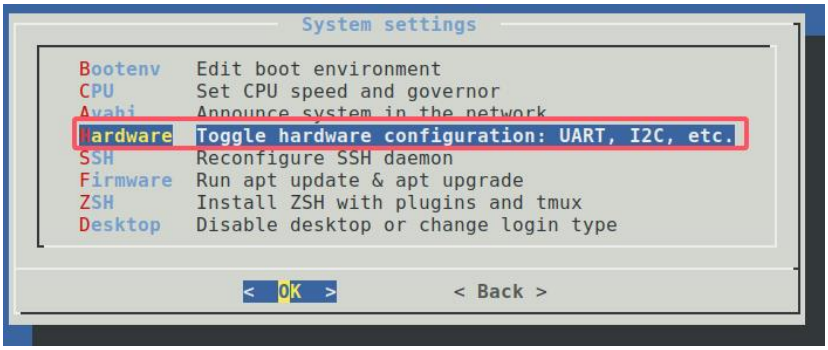
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

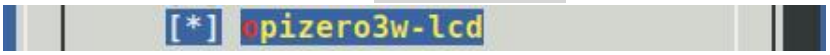
b. 然后选择 **System**



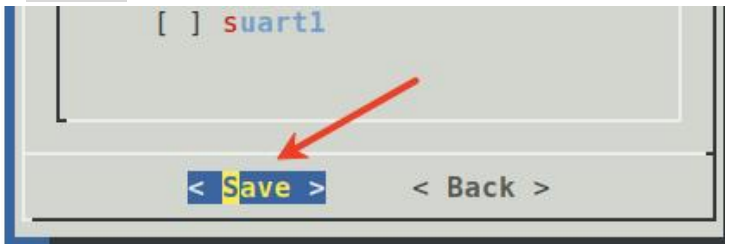
c. 然后选择 **Hardware**



d. 然后使用键盘的方向键定位到 **opizero3w-lcd**，再使用**空格**选中



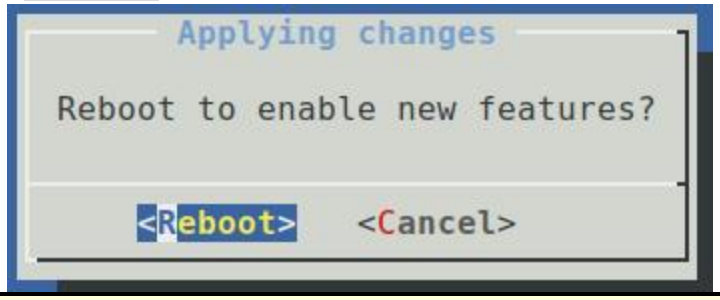
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



上面的设置最终会在**/boot/orangepiEnv.txt** 中加入 **overlays=opizero3w-lcd**。设置完后可以先检查下。如果不存在这行配置，那么设置就是有问题。

如果觉得使用 **orangepi-config** 比较麻烦，也可以使用 **vim** 编辑器打开 **/boot/orangepiEnv.txt**，然后加入 **overlays=opizero3w-lcd** 这行配置也是可以。

orangepi@orangepi:~$ **cat /boot/orangepiEnv.txt | grep "lcd"**

```
overlays=opizero3w-lcd      #示例配置
```

4) 然后重启 Linux 系统

5) 重启后可以看到 lcd 屏幕的显示如下所示（默认为竖屏）：



3.32.3. 服务器版镜像旋转显示方向的方法

1) 在**/boot/orangepiEnv.txt** 中加入 **extraargs=fbcon=rotate:要旋转的方向**这句配置就可以设置服务器版本的 linux 系统显示的方向，其中 **fbcon=rotate:**后面的数字可以设置为：

a. 0: 正常屏（默认为竖屏）
b. 1: 顺时钟转 90 度
c. 2: 翻转 180 度
d. 3: 顺时钟转 270 度

```
orangepi@orangepi:~$ sudo vim /boot/orangepiEnv.txt
overlays=lcd
extraargs=fbcon=rotate:3
```

2) 然后重启 linux 系统就能看到 lcd 屏幕显示的方向已经旋转了

3. 33. Debian bullseye 系统硬解测试说明

注意，目前只有 Debian bullseye 系统支持硬解，其它版本系统暂时不支持。

1) VLC 播放器和 Gstreamer 都支持硬件解码，目前测试过的格式如下所示：

a. 视频硬件解码 - HEVC
b. 视频硬件解码 - H.264
c. 视频硬件解码 - VP8
d. 视频硬件解码 - VP9
e. 视频硬件解码 - MPEG2

2) 首先准备测试视频，测试视频可以从下面网站上下载，或者从官方工具的 Videos 目录下载。

```
http://bbb3d.renderfarming.net/download.html
```

3) 然后在 Debian 系统的桌面中打开命令行终端，再运行下面的 gstreamer 命令。

```
orangepi@orangepi:~$ gst-play-1.0 bbb_sunflower_1080p_60fps_normal.mp4
```

4) 或者使用下面的命令也可以。

```
orangepi@orangepi:~$ gst-launch-1.0 filesrc location=bbb_sunflower_1080p_60fps_normal.mp4 ! qtdemux ! h264parse ! omxh264dec ! xvimagesink
```

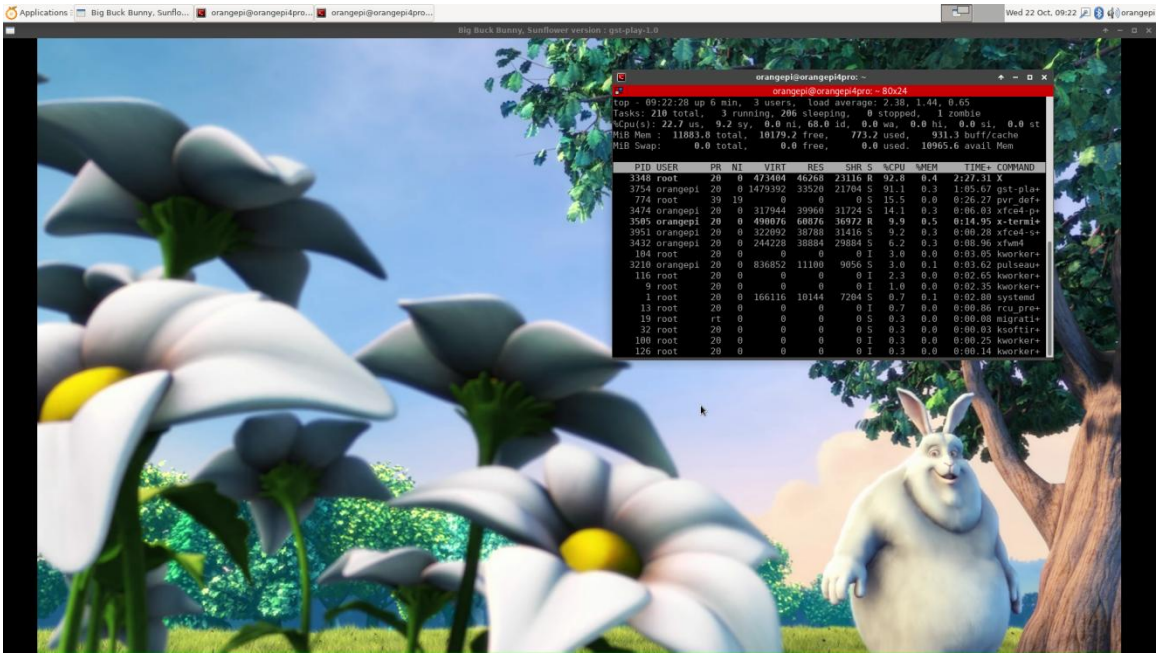
5) 如果使用 VLC 播放器，直接右键点击视频文件选择 VLC 播放器打开即可。

6) 可以通过下面的命令验证是否调用到硬件解码，如果看到中断数在不断增加，说明已经在用硬件解码了。

```
orangepi@orangepi:~# cat /proc/interrupts |grep cedar_dev
```



7) 在硬件解码的情况下，执行 htop 命令可以看到 CPU 占用率较低。



3. 34. Debian bullseye 系统 GPU 测试说明

注意，目前只有 Debian bullseye 系统支持 GPU 加速，其它版本系统暂时不支持。

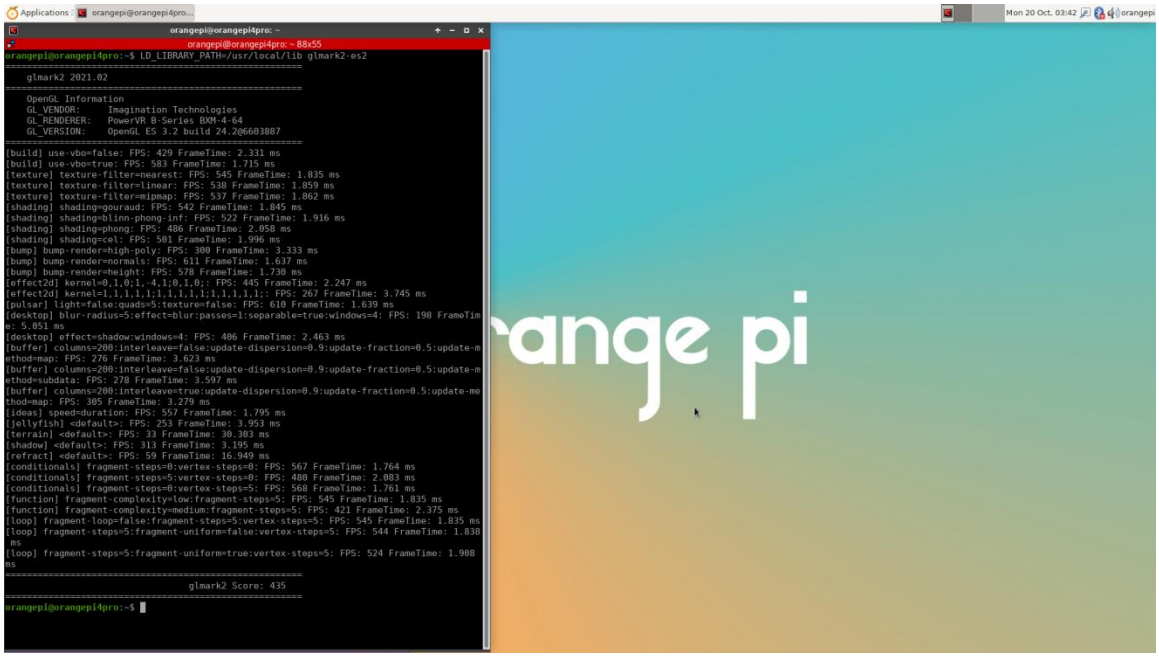
1) 首先在 Debian 系统的桌面中打开命令行终端，然后运行 **glmark2-es2**，下面的命令二选一即可：

注意，是要在桌面中运行下面的命令。

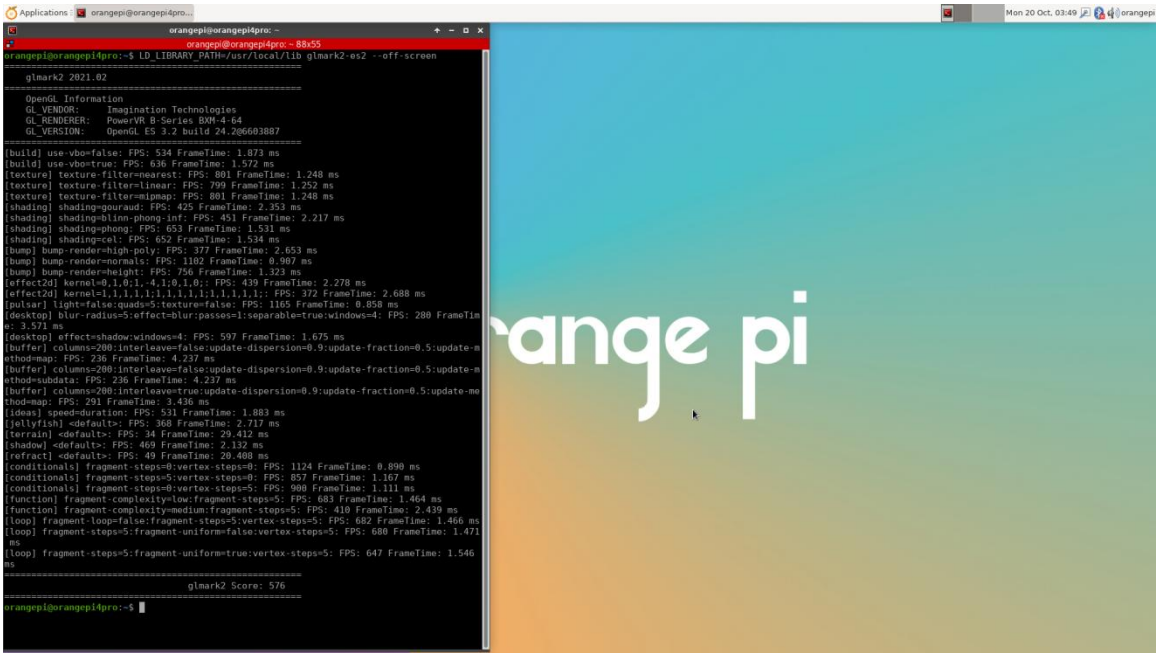
```
orangepi@orangepi:~$ LD_LIBRARY_PATH=/usr/local/lib glmark2-es2
orangepi@orangepi:~$ LD_LIBRARY_PATH=/usr/local/lib glmark2-es2 --off-screen
```

2) **glmark2-es2** 是 OpenGL (ES) 2.0 的基准测试工具，使用 glmark2 可以测试下 GPU OpenGL ES 2.0 的性能

a. **glmark2-es2** 命令的测试分数如下所示



b. **glmark2-es2 --off-screen** 命令的测试分数如下所示



3.35. NPU 的使用说明

3.35.1. PC 端开发环境配置

3.35.1.1. 安装 Docker 环境

1) 首先在 Ubuntu 电脑中安装下 docker 软件，安装完 docker 后如果使用 docker -v 命令能看到版本号说明安装成功。

```
$ sudo apt update
$ sudo apt install -y docker.io
$ docker -v
```

2) 然后从全志网盘下载 ACUITY docker 压缩包并解压。

```
$ unzip docker_images_v2.0.x.zip
```

3) 然后执行下面的命令导入包含全志 NPU 开发环境的 docker 镜像。

```
$ cd docker_images_v2.0.x
$ unzip ubuntu-npu_v2.0.10.tar.zip
$ sudo docker load -i ubuntu-npu_v2.0.10.tar
```

4) 然后从 Orange Pi 官网的官方工具中下载 **ai-sdk.tar.gz** 压缩包，并解压到 docker_data 目录。

```
$ mkdir docker_data
$ tar -xvf ai-sdk.tar.gz -C docker_data
```

5) 然后创建并进入 docker 容器。

```
$ cd docker_data
$ sudo docker run --ipc=host -itd -v ${PWD}:/workspace --name allwinner_v2.0.10
ubuntu-npu:v2.0.10 /bin/bash
$ sudo docker exec -it allwinner_v2.0.10 /bin/bash
```

3.35.1.2. 模型转换和量化

1) 在开始模型转换前，需要配置模型编译脚本

```
# cd /workspace/ai-sdk/models
# source env.sh v3
# cp ../scripts/* .
```

2) 在 **ai-sdk/models** 目录中提供了多个框架模型的转换示例。下面以 **yolov5s** 模型为例，介绍模型部署脚本的使用方法。

3) **yolov5s** 的目录结构：

```
# tree yolov5s-sim
yolov5s-sim
|-- channel_mean_value.txt
|-- convert_export.sh
|-- dataset.txt
|-- images
|   |-- COCO_train2014_000000000529.jpg
|   |-- COCO_train2014_000000001183.jpg
|   |-- COCO_train2014_000000002349.jpg
|   |-- COCO_train2014_000000003685.jpg
|   |-- COCO_train2014_000000004463.jpg
|   `-- dog.jpg
|-- inference_compare.sh
|-- inputs_outputs.txt
`-- yolov5s-sim.onnx      #yolov5s 的 onnx 模型文件，已把输入尺寸固定为(1,3,640,640)

1 directory, 12 files
```

4) 生成 uint8 量化的 NBG 文件。

a. 导入模型。

```
# ./pegasus_import.sh yolov5s-sim
```

b. 修改 yolov5s-sim_inputmeta.yml 文件的 **scale** 为 0.00392157

```
# vim yolov5s-sim/yolov5s-sim_inputmeta.yml
```

c. 执行量化，默认使用 uint8 量化。

```
# ./pegasus_quantize.sh yolov5s-sim uint8
```

d. 运行仿真验证。

```
# ./pegasus_inference.sh yolov5s-sim uint8
```

e. 导出 NPU 可用的 NBG 模型文件

```
# ./pegasus_export_ovx.sh yolov5s-sim uint8
```

f. 执行完成后，量化生成的 NBG 文件路径如下：

```
yolov5s-sim/wksp/yolov5s-sim_uint8_nbg_unify/network_binary.nb
```

3.35.2. 板端模型部署

3.35.2.1. 板端环境准备

1) 首先开发板端需要安装 opencv 和 cmake。

```
orangepi@orangepi:~$ sudo apt update
orangepi@orangepi:~$ sudo apt install libopencv-dev cmake
```

2) 然后从 Orange Pi 官网的官方工具中下载 **ai-sdk.tar.gz** 压缩包。

3) 然后将压缩包上传到开发板，并解压。

```
orangepi@orangepi:~$ tar -xvf ai-sdk.tar.gz
```

4) 压缩包解压后，目录结构如下所示。

```
orangepi@orangepi:~$ cd ai-sdk
orangepi@orangepi:~/ai-sdk$ ls
examples machinfo Makefile models scripts tests tools unified-android
unified-tina viplite-android viplite-tina
```

3.35.2.2. 运行 yolov5 目标检测示例

1）首先编译 yolov5 示例。

```
orangepi@orangepi:~/ai-sdk$ cd examples/yolov5
orangepi@orangepi:~/ai-sdk/examples/yolov5$ mkdir build
orangepi@orangepi:~/ai-sdk/examples/yolov5$ cd build
orangepi@orangepi:~/ai-sdk/examples/yolov5/build$ cmake ..
```

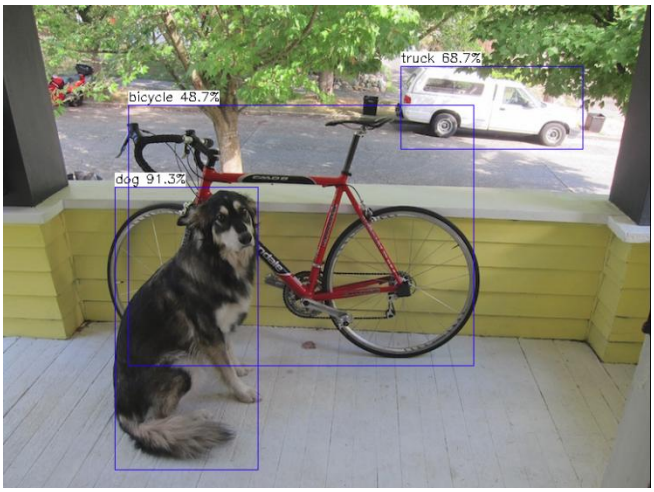
```
orangepi@orangepi:~/ai-sdk/examples/yolov5/build$ make
```

2）然后执行下面的命令运行示例。

```
orangepi@orangepi:~/ai-sdk/examples/yolov5/build$ ./yolov5 ../model/v3/yolov5.nb ../input_data/dog.jpg ./yolov5 nbg input
VIPLite driver software version 2.0.3.2-AW-2024-08-30
yolov5_preprocess.cpp run.
yolov5_postprocess.cpp run.
detection num: 3
16:  87%, [ 134,  216,  308,  552], dog
 2:  54%, [ 463,   77,  693,  170], car
 1:  49%, [ 160,  131,  562,  423], bicycle
```

3）输出结果会保存为当前目录的 result.png

```
orangepi@orangepi:~/ai-sdk/examples/yolov5/build$ ls result.png
result.png
```



3.35.2.3. 运行 resnet50 图像分类示例

1）首先编译 resnet50 示例。

```
orangepi@orangepi:~/ai-sdk$ cd examples/resnet50
orangepi@orangepi:~/ai-sdk/examples/resnet50$ mkdir build
orangepi@orangepi:~/ai-sdk/examples/resnet50$ cd build
orangepi@orangepi:~/ai-sdk/examples/resnet50/build$ cmake ..
orangepi@orangepi:~/ai-sdk/examples/resnet50/build$ make
```

2）然后执行下面的命令运行示例。

```
orangepi@orangepi:~/ai-sdk/examples/resnet50/build$ ./resnet50 ../model/v3/resnet50.nb ../input_data/dog_224_224.jpg
```

3）输出结果信息如下所示，输出了模型预测的前 5 名，其中最可能的类别是 collie

```
orangepi@orangepi:~/ai-sdk/examples/resnet50/build$ ./resnet50 ../model/v3/resnet50.nb ../input_data/dog_224_224.jpg
...
========== top5 ==========
class id: 231, prob: 10.750525, label: collie
class id: 160, prob: 10.616143, label: Afghan hound, Afghan
class id: 169, prob: 10.481762, label: borzoi, Russian wolfhound
class id: 230, prob: 10.347381, label: Shetland sheepdog, Shetland sheep dog, Shetland
class id: 176, prob: 6.987841, label: Saluki, gazelle hound
class_postprocess success.
```

3.35.2.4. 运行 struct2depth 深度检测示例

1）首先编译 struct2depth 示例。

```
orangepi@orangepi:~/ai-sdk$ cd examples/struct2depth
orangepi@orangepi:~/ai-sdk/examples/struct2depth$ mkdir build
orangepi@orangepi:~/ai-sdk/examples/struct2depth$ cd build
orangepi@orangepi:~/ai-sdk/examples/struct2depth/build$ cmake ..
orangepi@orangepi:~/ai-sdk/examples/struct2depth/build$ make
```

2）然后执行下面的命令运行示例。

```
orangepi@orangepi:~/ai-sdk/examples/struct2depth/build$ ./struct2depth -b ../model/v3/struct2depth.nb -i ../input_data/0015.jpg
```

3）模型推理的深度信息将会保存到 jpg 和 txt 文件中。

```
orangepi@orangepi:~/ai-sdk/examples/struct2depth/build$ ls disp_* output_*
disp_color.jpg  disp_show.jpg  output_1.txt  output_3.txt
disp_gray.jpg   output_0.txt   output_2.txt
```

3.35.2.5. 运行 transformer_cls 图像分类示例

1）首先编译 transformer_cls 示例。

```
orangepi@orangepi:~/ai-sdk$ cd examples/transformer_cls
orangepi@orangepi:~/ai-sdk/examples/transformer_cls$ mkdir build
orangepi@orangepi:~/ai-sdk/examples/transformer_cls$ cd build
orangepi@orangepi:~/ai-sdk/examples/transformer_cls/build$ cmake ..
orangepi@orangepi:~/ai-sdk/examples/transformer_cls/build$ make
```

2）然后执行下面的命令运行示例。

```
orangepi@orangepi:~/ai-sdk/examples/transformer_cls/build$ ./transformer_cls ../model/v3/transformer_cls.nb ../input_data/cat_224_224.jpg
```

3）图像分类输出结果如下所示。

```
orangepi@orangepi:~/ai-sdk/examples/transformer_cls/build$ ./transformer_cls ../model/v3/transformer_cls.nb ../input_data/cat_224_224.jpg
./transformer_cls nbg input
VIPLite driver software version 2.0.3.2-AW-2024-08-30
get jpeg success.
trans data success.
awnn_set_input_buffers success.
awnn_run success.
class_postprocess.cpp run.
========== top5 ==========
class id: 281, prob: 8.171444, label: tabby, tabby cat
class id: 285, prob: 6.455214, label: Egyptian cat
class id: 282, prob: 6.139260, label: tiger cat
class id: 287, prob: 3.832908, label: lynx, catamount
class id: 756, prob: 3.263942, label: rain barrel
class_postprocess success.
```

3.35.2.6. 运行 yolact 目标检测分割示例

1）首先编译 yolact 示例。

```
orangepi@orangepi:~/ai-sdk$ cd examples/yolact
orangepi@orangepi:~/ai-sdk/examples/yolact$ mkdir build
```

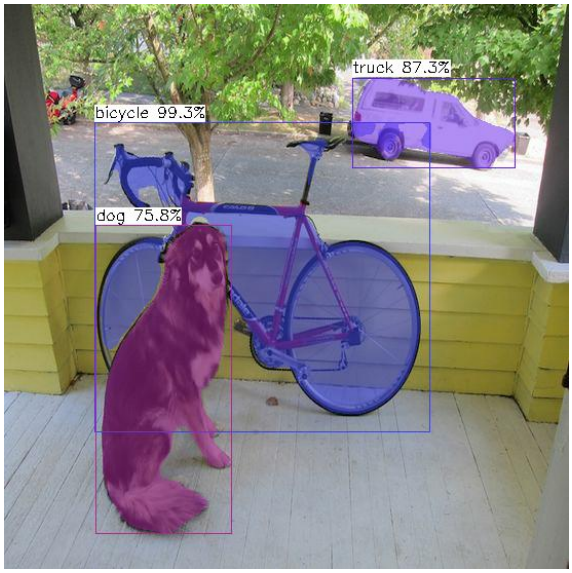
```
orangepi@orangepi:~/ai-sdk/examples/yolact$ cd build
orangepi@orangepi:~/ai-sdk/examples/yolact/build$ cmake ..
orangepi@orangepi:~/ai-sdk/examples/yolact/build$ make
```

2）然后执行下面的命令运行示例。

```
orangepi@orangepi:~/ai-sdk/examples/yolact/build$ ./yolact ../model/v3/yolact.nb ../input_data/dog_550_550.jpg
```

3）输出结果会保存为当前目录的 result.png

```
orangepi@orangepi:~/ai-sdk/examples/yolact/build$ ls result.png
result.png
```



3.35.2.7. 运行 deepspeech2 语音识别示例

1）首先编译 deepspeech2 示例。

```
orangepi@orangepi:~/ai-sdk$ cd examples/deepspeech2
orangepi@orangepi:~/ai-sdk/examples/deepspeech2$ mkdir build
orangepi@orangepi:~/ai-sdk/examples/deepspeech2$ cd build
orangepi@orangepi:~/ai-sdk/examples/deepspeech2/build$ cmake ..
orangepi@orangepi:~/ai-sdk/examples/deepspeech2/build$ make
```

2）然后执行下面的命令运行示例。

```
orangepi@orangepi:~/ai-sdk/examples/deepspeech2/build$ ./deepspeech2    ../model/v3/deepspeech2.nb ../input_data/1188-133604-0010.flac_756_161_1.tensor
```

3）输出结果如下所示。

```
orangepi@orangepi:~/ai-sdk/deepspeech2/build$ ./deepspeech2  ../model/v3/deepspeech2.nb ../input_data/1188-133604-0010.flac_756_161_1.tensor
input tensor_file = ../input_data/1188-133604-0010.flac_756_161_1.tensor
VIPLite driver software version 2.0.3.2-AW-2024-08-30
Original array: but  in  thiss  vvingyet copiiedd ffrom  turrrnerr  yoou haavve the  ttwo  prrincciiplleess  brrouught  oout perrfeccttllly
Modified array : but in this vingyet copied from turner you have the two principles brought out perfectly
```

3.35.2.8. 运行 chineseocr 文字识别示例

1）首先编译 chineseocr 示例。

```
orangepi@orangepi:~/ai-sdk$ cd examples/chineseocr
orangepi@orangepi:~/ai-sdk/examples/chineseocr$ mkdir build
orangepi@orangepi:~/ai-sdk/examples/chineseocr$ cd build
orangepi@orangepi:~/ai-sdk/examples/chineseocr/build$ cmake ..
orangepi@orangepi:~/ai-sdk/examples/chineseocr/build$ make
```

2）然后执行下面的命令运行示例。

```
orangepi@orangepi:~/ai-sdk/examples/chineseocr/build$ ./chineseocr -d ../model/v3/ -1 dbnet_1024 -2 angle_net -3 crnn_lite_lstm_256 -4 keys.txt -i ../input_data/1.jpg
```

3）输出结果如下所示，可以看到识别到了图片中的文字。

```
orangepi@orangepi:~/ai-sdk/examples/chineseocr/build$ ./chineseocr -d ../model/v3/ -1 dbnet_1024 -2 angle_net -3 crnn_lite_lstm_256 -4 keys.txt -i ../input_data/1.jpg
...
=====End detect=====
FullDetectTime(903.447417ms)
我们在全志科技
AI
芯片
起飞 12345666666 !

run finished.
~CrnnNet.
~AngleNet.
```

```
~DbNet.
~NpuUint.
```

3.36. 烧写 linux 镜像到 eMMC 的方法

注意，开发板可以通过 TF 卡启动，也可以通过 eMMC 启动，TF 卡的优先级要高于 eMMC。也就是说，如果开发板插了 TF 卡，并且 TF 卡中是有系统的，那么默认就会启动 TF 卡中系统，而不会启动 eMMC 中的系统。

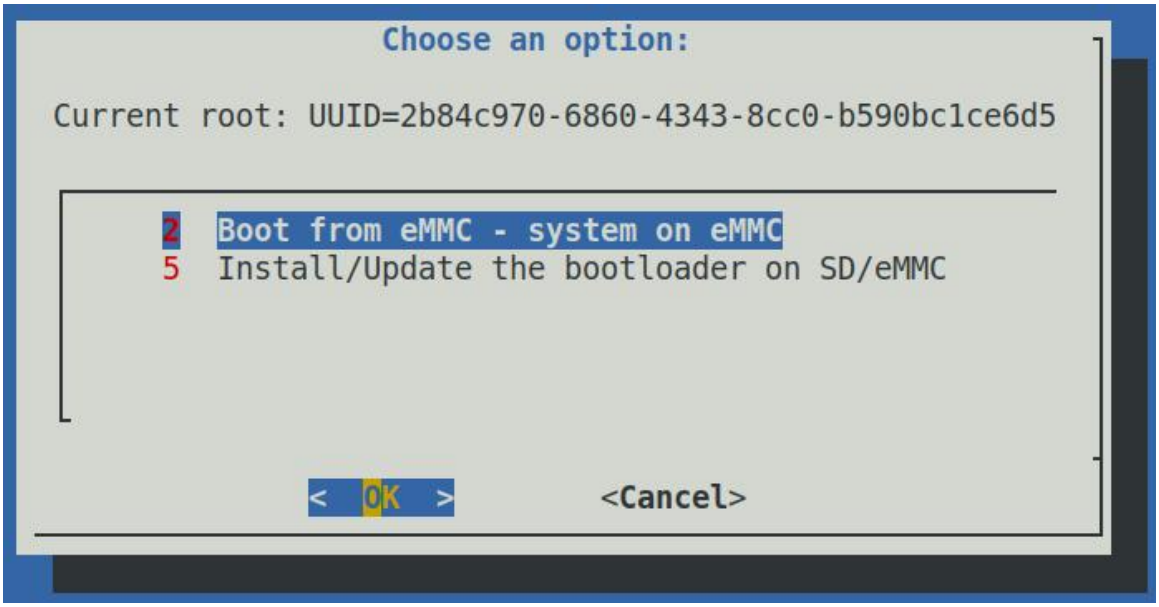
在进行下面操作前，请确保开发板贴有 eMMC。

1) 烧录 linux 镜像到 eMMC 中需要借助 TF 卡来完成，首先将 linux 镜像烧录到 TF 卡上，然后启动开发板进入 linux 系统

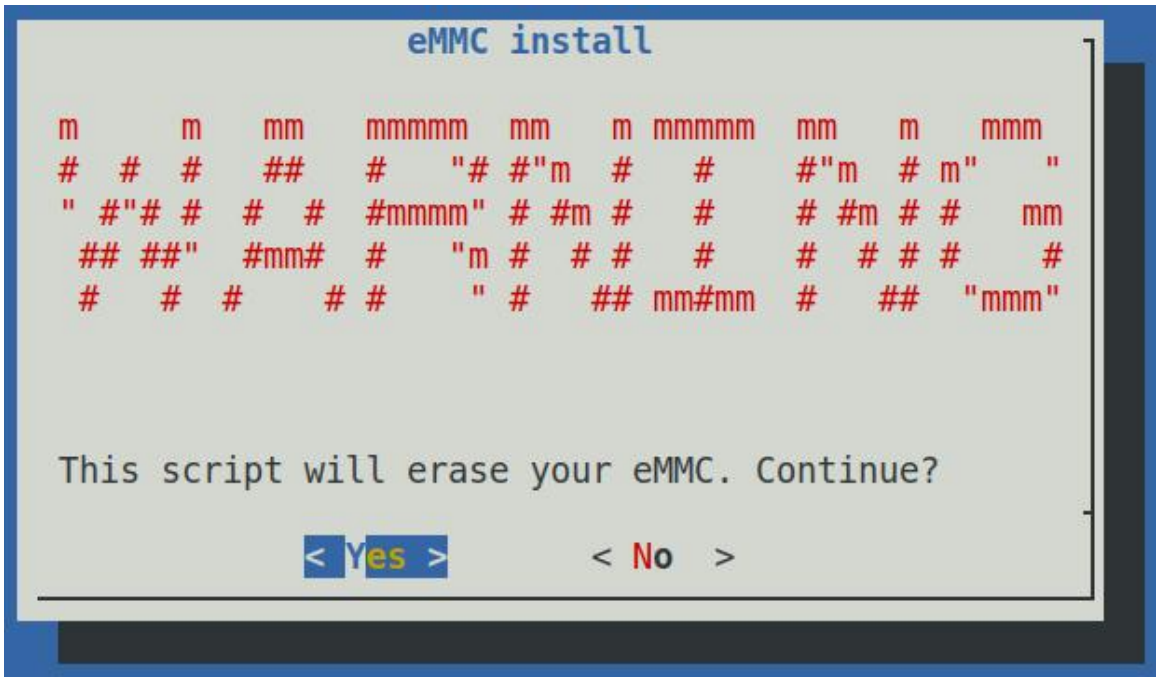
2) 然后运行 **nand-sata-install** 脚本，**记得加 sudo 权限**

```
orangepi@orangepi:~$ sudo nand-sata-install
```

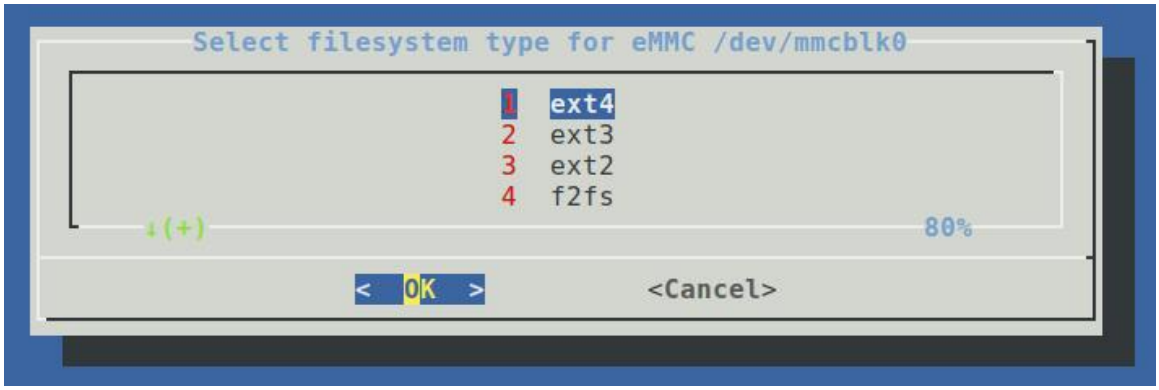
3) 然后选择 **2 Boot from eMMC - system on eMMC**



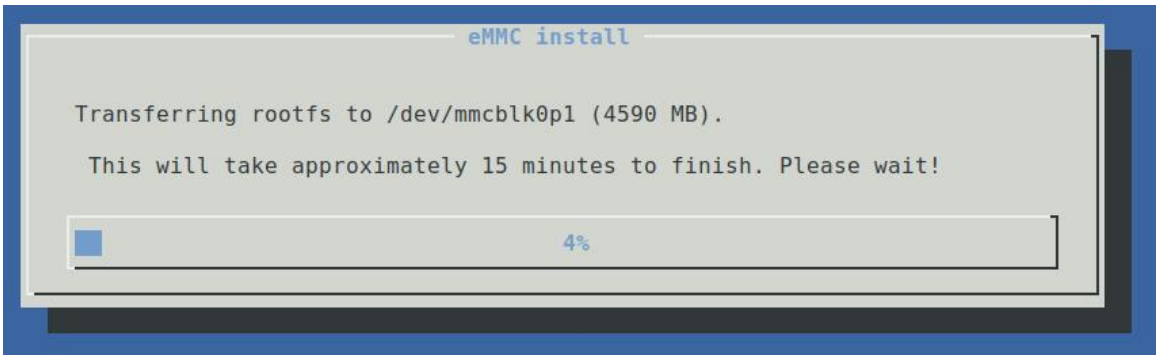
4) 然后会弹出一个警告，脚本将会擦除 eMMC 上的所有数据，选择**<Yes>**继续



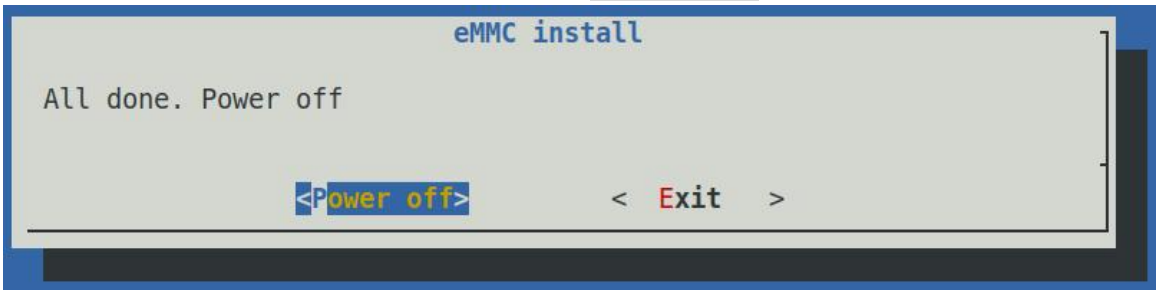
5) 然后会提示选择文件系统的类型，支持 ext2/3/4、f2fs 以及 btrfs 五种文件系统



6) 然后会开始格式化 eMMC，格式化完 eMMC 后就会开始烧录 linux 镜像到 eMMC 中



7) 烧录完后会提示下面的选项，可以选择**<Power off>**直接关机



8) 然后拔出 TF 卡，再重新上电，就会启动 eMMC 中的 linux 系统了

3.37. 烧写 linux 镜像到 UFS 的方法

注意，开发板可以通过 TF 卡启动，也可以通过 UFS 启动，TF 卡的优先级要高于 UFS。也就是说，如果开发板插了 TF 卡，并且 TF 卡中是有系统的，那么默认就会启动 TF 卡中系统，而不会启动 UFS 中的系统。

在进行下面操作前，请确保开发板贴有 UFS。

1) 烧录 linux 镜像到 UFS 中需要借助 TF 卡来完成，首先将 linux 镜像烧录到 TF 卡上，然后启动开发板进入 linux 系统

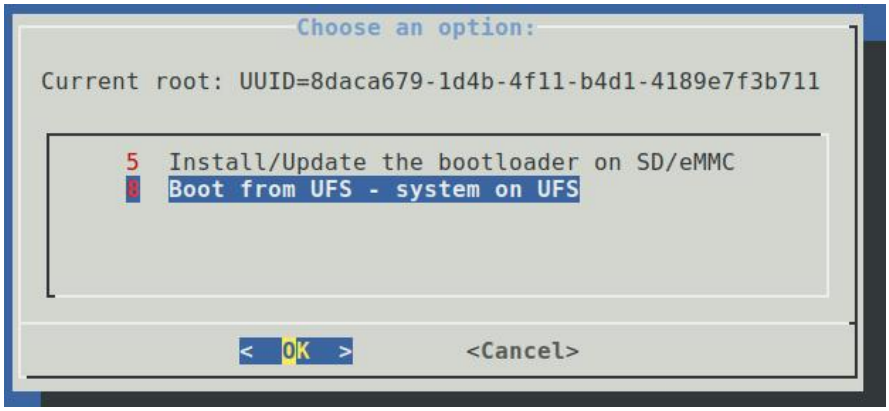
2) 然后给 UFS 创建一个分区，注意需要确认下 UFS 的设备节点，这里是 **sda**

```
orangepi@orangepi:~$ sudo parted --script /dev/sda mklabel msdos mkpart primary ext4 65536s 100%
```

3) 然后运行 **nand-sata-install** 脚本，**记得加 sudo 权限**

```
orangepi@orangepi:~$ sudo nand-sata-install
```

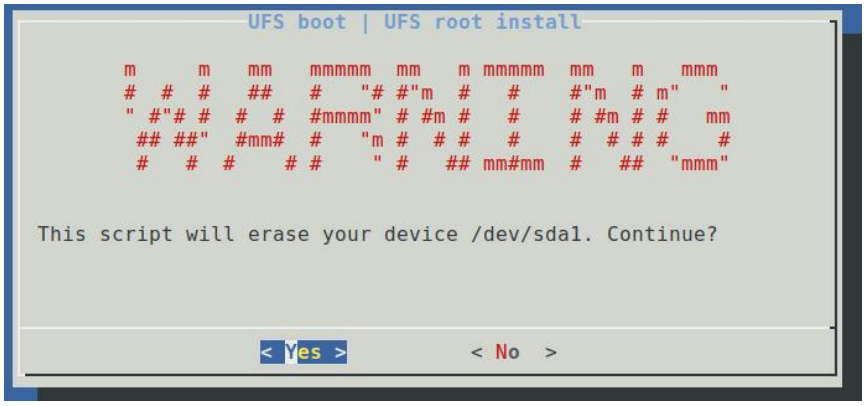
4) 然后选择 **8 Boot from UFS - system on UFS**



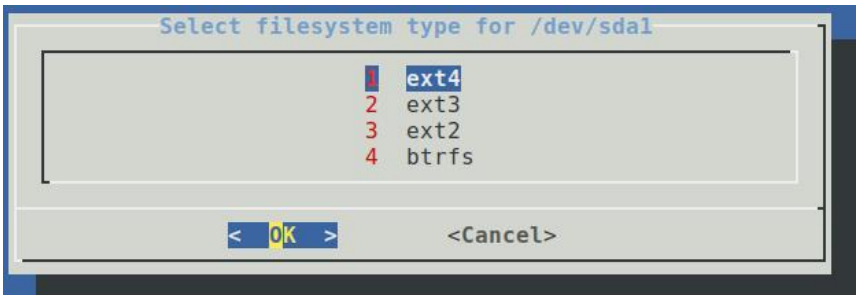
5) 选择**<OK>**继续



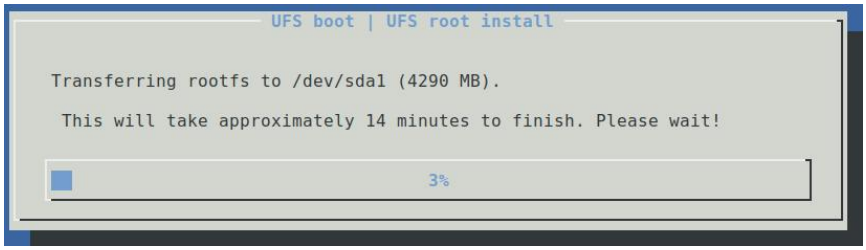
9) 然后会弹出一个警告，脚本将会擦除 UFS 上的所有数据，选择**<Yes>**继续



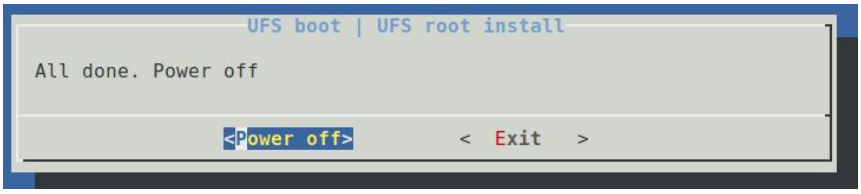
6) 然后会提示选择文件系统的类型，支持 ext2/3/4、f2fs 以及 btrfs 五种文件系统



7) 然后会开始格式化 UFS，格式化完 UFS 后就会开始烧录 linux 镜像到 UFS 中



8) 烧录完后会提示下面的选项，可以选择**<Power off>**直接关机



9) 然后拔出 TF 卡，再重新上电，就会启动 UFS 中的 linux 系统了

3.38. 系统备份脚本 opi-bkimg 的使用方法

opi-bkimg 是为 Orange Pi 系列开发板设计的系统备份脚本，它可以帮助用户将当前运行的 Linux 系统完整地备份到一个镜像文件中，以便后续恢复或迁移到其他设备上。

在开始备份前，请确保系统中留有足够的空间，否则会备份失败。

1) 运行 **opi-bkimg** 脚本开始备份系统。

```
orangepi@orangepi:~$ sudo opi-bkimg
```

2) 如果系统中没有 **opi-bkimg** 脚本，可以自行从下面的仓库下载。

https://gitee.com/orangepi-xunlong/opi-bkimg

3) 脚本会检查根文件系统大小并创建一个大小合适的镜像文件，随后使用 **rsync** 命令将系统数据复制到镜像文件中，具体过程如下所示。

```
orangepi@orangepi:~$ sudo opi-bkimg
[ o.k. ] Starting backup to image [ /mnt/backup.img ]
[ o.k. ] Current rootfs size [ 2561 MiB ]
[ o.k. ] Creating blank image for rootfs [ 3500 MiB ]
[ .... ] dd: 3.42GiB [ 126MiB/s] [=========================================>] 100%
[ o.k. ] Creating partitions [ root: ext4 ]
[ .... ] Creating rootfs [ ext4 on /dev/loop1p1 ]
[ .... ] Copying file to [ / ] [################################] 100%
[ .... ] Re-enabling [ orangepi-resize-filesystem ]
[ o.k. ] Writing U-boot bootloader [ /dev/loop1 ]
[ o.k. ] Unmounting [ /mnt/opi-bkimg.iaioMi/rootfs ]
[ o.k. ] Backup completed [ /mnt/backup.img ]
orangepi@orangepi:~$
```

4) 当脚本显示 **Backup completed** 消息说明备份结束，镜像生成路径如下所示，

```
orangepi@orangepi:~$ ls /mnt/
backup.img
```

5) **backup.img** 即为可以烧录的镜像文件。

3. 39. Linux Overlayroot 的使用方法

在某些用户场景中，系统经常通过直接断电而非执行 poweroff 命令关机，这可能导致文件系统损坏，从而无法正常启动。为防止此类问题，建议启用 overlayroot 功能，将根文件系统挂载为只读。这样可以确保即使在异常断电情况下，原始文件系统不受影响，从而提高系统的稳定性和可靠性。

Overlayroot 是一种基于 Overlayfs 的技术，可将 Linux 根文件系统（/）挂载为只读，并通过叠加层（overlay）提供可写存储。用户的所有修改（如文件写入、配置变更等）都会被重定向到可写层，而原始根文件系统保持不变，适用于嵌入式设备、服务器等对系统稳定性要求高的场景。

Overlayroot 由两层组成：只读层（lowerdir）和可写层（upperdir）。只读层是原始根文件系统（/），以只读模式挂载；可写层用于存储用户修改的数据，与只读层合并后形成最终的根文件系统视图，使用户在操作时感知不到底层的分层结

构。

可写层可存放于 tmpfs 或块设备。使用 tmpfs 时，所有修改存于内存，重启后丢失，系统恢复至原始状态；使用块设备（如 U 盘、TF 卡分区）时，数据可持久化存储，重启后不会丢失。

3.39.1. 配置 Overlayroot 为 tmpfs 模式

该模式下，系统所有修改都会存放于内存(tmpfs)，重启后所有更改将丢失，系统恢复到初始状态。

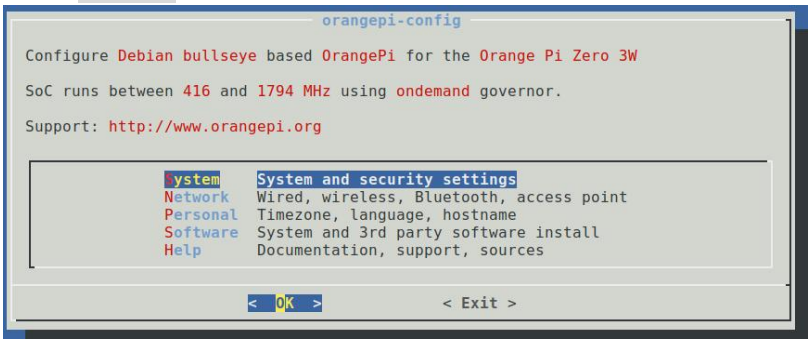
1) Linux 系统中 Overlayroot 默认是关闭的，需要手动打开才能使用。打开步骤如下所示：

a. 首先请确保开发板已连接至有线或无线网络，具体方法可参考网络连接测试章节

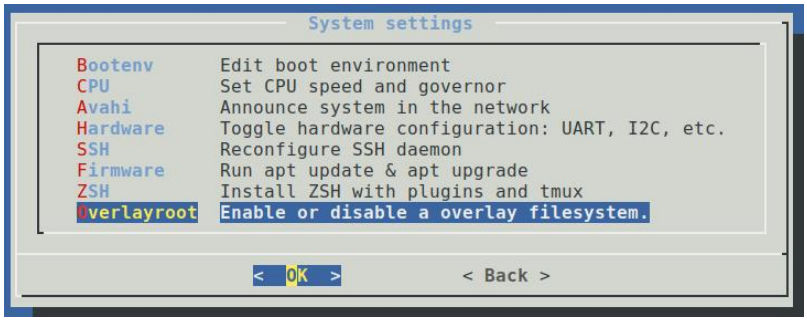
b. 然后运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

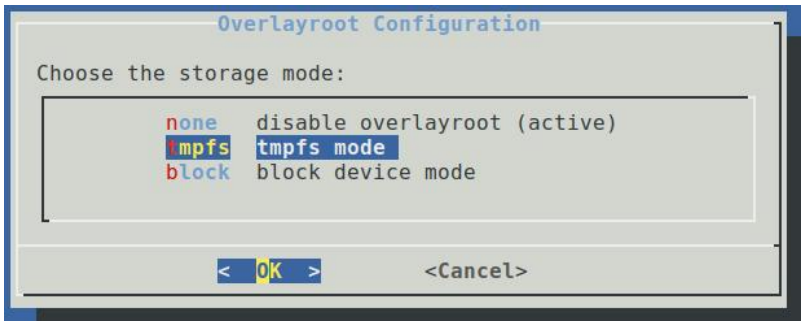
c. 然后选择 **System**



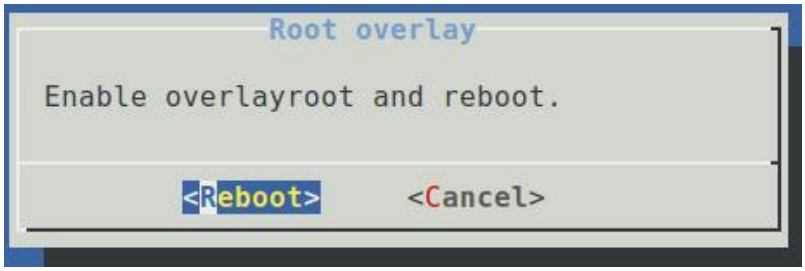
d. 然后选择 **Overlayroot**



e. 然后选择 **tmpfs** 回车



f. 然后选择**<Reboot>**重启系统使配置生效



2) 然后输入 **mount | grep root** 命令可以看到文件系统的挂载信息。

```
orangepi@orangepi:~$ mount |grep root
/dev/mmcblk0p1 on /media/root-ro type ext4 (ro,relatime)
tmpfs-root on /media/root-rw type tmpfs (rw,relatime)
overlayroot on / type overlay (rw,relatime,lowerdir=/media/root-ro,upperdir=/media/root-rw/overlay,workdir=/media/root-rw/overlay-workdir/_)
overlayroot on /var/log.hdd type overlay (rw,relatime,lowerdir=/media/root-ro,upperdir=/media/root-rw/overlay,workdir=/media/root-rw/overlay-workdir/_)
orangepi@orangepi:~$
```

通过挂载信息可见，文件系统分区/dev/mmcblkXp1 以只读模式挂载至/media/root-ro，tmpfs 作为可写层挂载至/media/root-rw，说明 overlayroot 的配置已经生效。

由于 tmpfs 为内存文件系统，重启后所有修改都会丢失。

3.39.2. 配置 Overlayroot 为块设备存储模式

该模式下，可写层使用具体存储设备（如 TF 卡分区 /dev/mmcblkXp2），重启后数据不会丢失。

请注意，打开了 Overlayroot 后，对/boot 分区的修改（如更新内核或修改/boot/orangepiEnv.txt）不会生效。原因是块设备模式下修改的实际上是覆盖层中的/boot，而 Uboot 在启动时读取的是真实分区中的文件，因此修改不会生效。要使修改生效，需要禁用 Overlayroot 后再进行这些操作。

1) Linux 系统中 Overlayroot 默认是关闭的，需要手动打开才能使用。打开步骤如下所示：

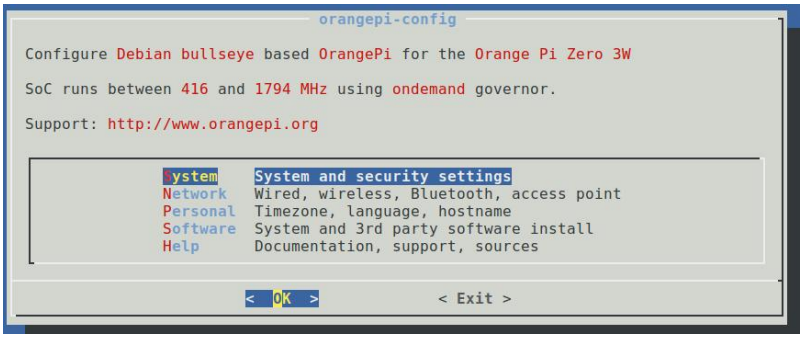
a. 首先请确保开发板已连接至有线或无线网络，具体方法可参考网络连接测试

章节

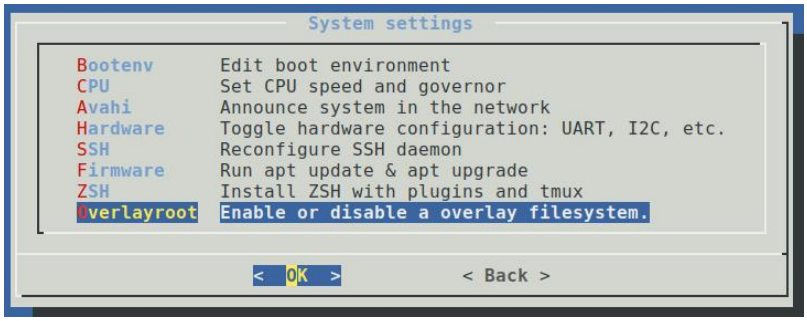
a. 然后运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

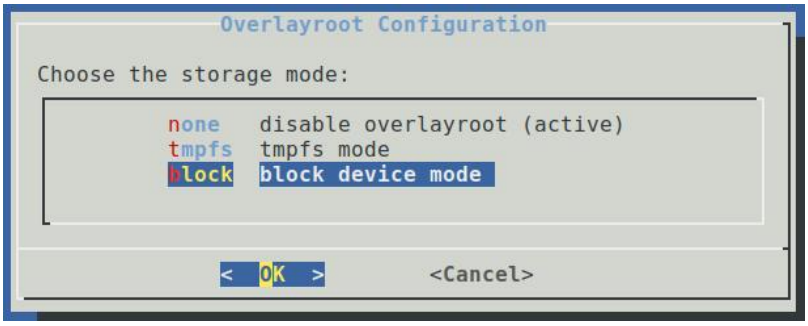
b. 然后选择 **System**



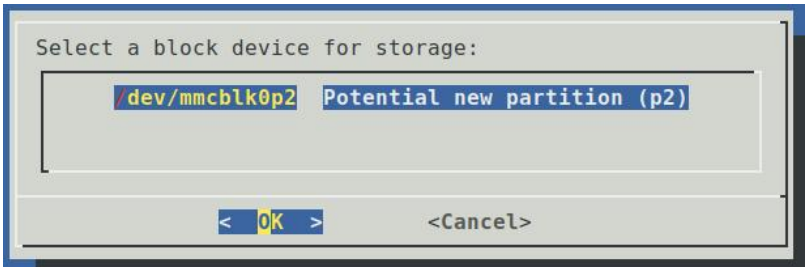
c. 然后选择 **Overlayroot**



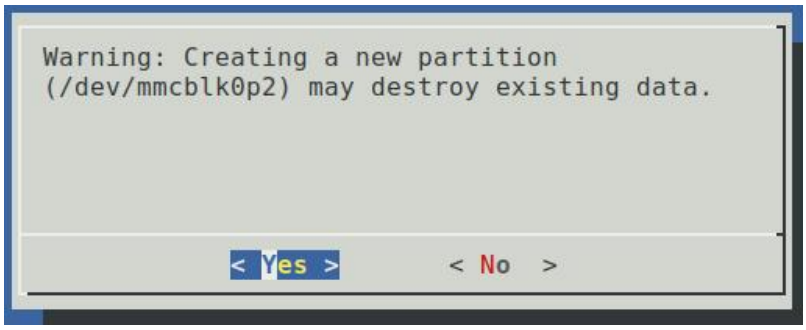
d. 然后选择 **block** 回车



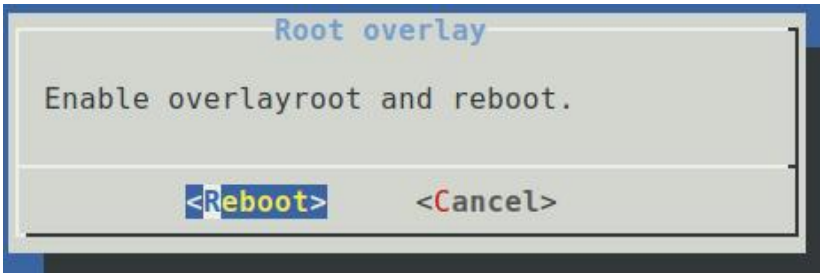
e. 然后会创建一个 Overlay 可写层分区。这里需要选择 **Overlay 可写层**分区所在的存储设备。如果开发板没有插入其他的 USB 设备，默认就只有一个选项，表示在 TF 卡中创建分区。分区的大小为：TF 的总容量减去 2 倍 rootfs 所占用的容量。



f. 然后选择**<Yes>**自动创建并格式化分区



g. 然后选择**<Reboot>**重启系统使配置生效



2) 然后输入 **mount | grep root** 命令可以看到文件系统的挂载信息。

```
orangepi@orangepi:~$ mount |grep root
/dev/mmcblk0p1 on /media/root-ro type ext4 (ro,relatime)
/dev/mmcblk0p2 on /media/root-rw type ext4 (rw,relatime)
overlayroot on / type overlay (rw,relatime,lowerdir=/media/root-ro,upperdir=/media/root-rw/overlay,workdir=/media/root-rw/overlay-workdir/_)
overlayroot on /var/log.hdd type overlay (rw,relatime,lowerdir=/media/root-ro,upperdir=/media/root-rw/overlay,workdir=/media/root-rw/overlay-workdir/_)
orangepi@orangepi:~$
```

通过挂载信息可以看到，文件系统分区/dev/mmcblkXp1 以只读模式挂载至/media/root-ro， /dev/mmcblkXp2 作为可写层挂载至/media/root-rw，说明overlayroot 的配置已经生效。

3) 因为用户所有的修改都是重定向到**/media/root-rw** 目录的，所以可以通过删除**/media/root-rw/**下的所有内容来达到恢复出厂设置的目的（需要重启才会生效）。

```
orangepi@orangepi:~$ sudo rm -rf /media/root-rw/*
orangepi@orangepi:~$ reboot
```

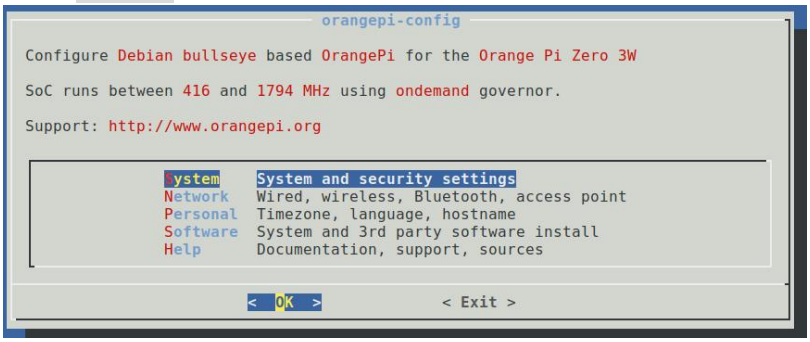
3.39.3. 禁用 Overlayroot 的方法

1) 禁用 Overlayroot 的步骤如下所示：

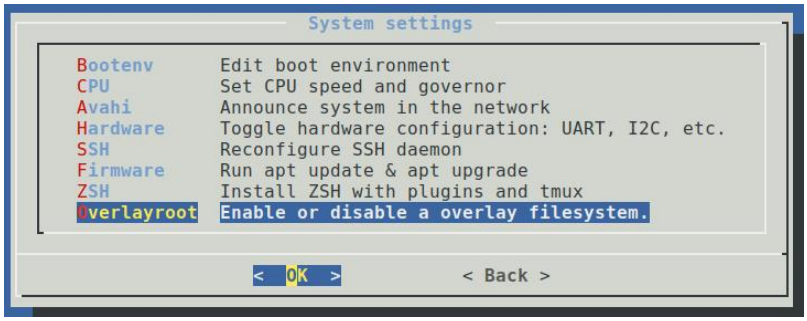
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

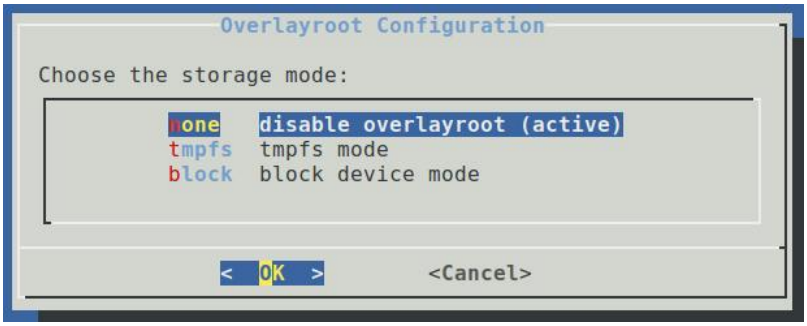
b. 然后选择 **System**



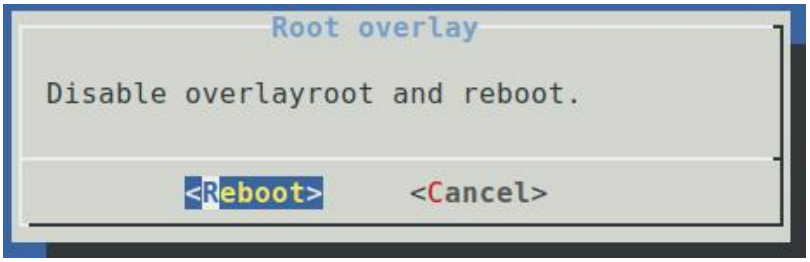
c. 然后选择 **Overlayroot**



d. 然后选择 **none**，然后回车



e. 然后选择**<Reboot>**重启系统使配置生效



2) 如果觉得使用 orangepi-config 的方式比较繁琐，也可以使用下面的步骤。

a. 首先需要重新以可写模式挂载 **root-ro** 目录。

```
orangepi@orangepi:~$ sudo mount -o remount,rw /media/root-ro
```

b. 然后再修改只读分区下的**/etc/overlayroot.conf** 文件，将 overlayroot 变量设置为空，然后重启系统。

```
orangepi@orangepi:~$ sudo vim /media/root-ro/etc/overlayroot.conf
overlayroot=""
```

3.39.4. 通过 opi-bkimg 备份 overlayroot 镜像的方法

上面方法只适用于单台设备本地操作，在某些情况下可能对系统造成损坏，更推荐使用本小节的方法，通过备份生成一个支持 overlayroot 的镜像，这样可以方便烧录到其它设备中。在开始备份前，请确保系统当前未开启 overlayroot 功能，如已启用，请按照上一小节的方法将其禁用。

opi-bkimg 是为 Orange Pi 系列开发板设计的系统备份脚本，它可以帮助用户将当前运行的 Linux 系统完整地备份到一个镜像文件中，以便后续恢复或迁移到其他设备上。

在开始备份前，请确保系统中留有足够的空间，否则会备份失败。

1) 运行 **opi-bkimg** 脚本开始备份系统，注意需要指定 **overlayfs** 参数。

```
orangepi@orangepi:~$ sudo opi-bkimg overlayfs
```

2) 如果系统中没有 **opi-bkimg** 脚本，可以自行从下面的仓库下载。

```
https://gitee.com/orangepi-xunlong/opi-bkimg
```

3) 脚本会检查根文件系统大小并创建一个大小合适的镜像文件，随后使用 **rsync** 命令将系统数据复制到镜像文件中，具体过程如下所示。

```
orangepi@orangepi:~$ sudo opi-bkimg overlayfs
[ o.k. ] Starting backup to image [ /mnt/backup.img ]
[ o.k. ] Current rootfs size [ 5331 MiB ]
[ o.k. ] Overlayfs size [ 256MB ]
[ o.k. ] Creating blank image for rootfs [ 7500 MiB ]
[ .... ] dd: 7.32GiB [ 224MiB/s] [=========================================>] 100%
[ o.k. ] Creating partitions [ root: ext4, overlay: ext4 ]
[ .... ] Creating rootfs [ ext4 on /dev/loop0p1 ]
[ .... ] Creating overlayfs [ ext4 on /dev/loop0p2 ]
[ .... ] Copying 178549 file to [ / ] [##############################] 100%
[ .... ] Re-enabling [ orangepi-resize-filesystem ]
[ .... ] Enabling [ setup-overlay ]
[ o.k. ] Writing U-boot bootloader [ /dev/loop0 ]
[ o.k. ] Backup completed [ /mnt/backup.img ]
[ o.k. ] Unmounting [ /mnt/opi-bkimg.khPthG/rootfs ]
orangepi@orangepi:~$
```

4) 当脚本显示 **Backup completed** 信息说明备份结束，镜像生成路径如下所示。

```
orangepi@orangepi:~$ ls /mnt/
backup.img
```

5) **backup.img** 即为可以直接烧录的镜像文件，将该镜像烧录至 TF 卡或 eMMC 后并上电启动，系统会自动完成初始化，期间会经历一次重启的过程，系统重启完成后执行 df -lh 命令，如果能输出与下图类似的信息说明系统 overlayroot 初始化成功。

```
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            99M  4.6M   94M   5% /run
/dev/mmcblk0p1  2.7G  2.1G  550M  80% /media/root-ro
/dev/mmcblk0p2   25G  652K   25G   1% /media/root-rw
overlayroot      25G  652K   25G   1% /
tmpfs           491M     0  491M   0% /dev/shm
tmpfs           5.0M  4.0K  5.0M   1% /run/lock
tmpfs           491M     0  491M   0% /tmp
/dev/zram1       47M  2.1M   42M   5% /var/log
tmpfs            99M     0   99M   0% /run/user/1000
```

3.40. 关机和重启开发板的方法

1) 在 Linux 系统运行的过程中，如果直接拔掉电源断电，可能会导致文件系统丢失某些数据，建议断电前先使用 **poweroff** 命令关闭开发板的 Linux 系统，然后再拔掉电源

```
orangepi@orangepi:~$ sudo poweroff
```

2) 使用 **reboot** 命令即可重启开发板中的 Linux 系统

```
orangepi@orangepi:~$ sudo reboot
```

4. Linux SDK——orangepi-build 使用说明

4.1. 编译系统需求

Linux SDK，即 **orangepi-build**，只支持在安装有 **Ubuntu 22.04** 的 X64 电脑上运行，所以下载 orangepi-build 前，请首先确保自己电脑已安装的 Ubuntu 版本是 Ubuntu 22.04。查看电脑已安装的 Ubuntu 版本的命令如下所示，如果 Release 字段显示的不是 **22.04**，说明当前使用的 Ubuntu 版本不符合要求，请更换系统后再进行下面的操作。

```
test@test:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04 LTS
Release:        22.04
Codename:       jammy
```

如果电脑安装的是 Windows 系统，没有安装有 Ubuntu 22.04 的电脑，可以考虑使用 **VirtualBox** 或者 **VMware** 来在 Windows 系统中安装一个 Ubuntu 22.04 虚拟机。但是请注意，不要在 WSL 虚拟机上编译 orangepi-build，因为 orangepi-build 没有在 WSL 虚拟机中测试过，所以无法确保能正常在 WSL 中使用 orangepi-build，另外请不要在**开发板**的 Linux 系统中使用 orangepi-build。Ubuntu 22.04 **amd64** 版本的安装镜像下载地址为：

```
https://mirrors.tuna.tsinghua.edu.cn/ubuntu-releases/22.04/ubuntu-22.04-desktop-amd64.iso
```

在电脑中或者虚拟机中安装完 Ubuntu 22.04 后，请先设置 Ubuntu 22.04 的软件源为清华源（或者其它你觉得速度快的国内源），不然后面安装软件的时候很容易由于网络原因而出错。替换清华源的步骤如下所示：

a. 替换清华源的方法参考这个网页的说明即可。

```
https://mirrors.tuna.tsinghua.edu.cn/help/ubuntu/
```

b. 注意 Ubuntu 版本需要切换到 22.04。



Ubuntu 镜像使用帮助

Ubuntu 的软件源配置文件是 /etc/apt/sources.list。将系统自带的该文件做个备份，将该文件替换为下面内容，即可使用 TUNA 的软件源镜像。

选择你的ubuntu版本: 22.04 LTS

```
# 默认注释了源码镜像以提高 apt update 速度，如有需要可自行取消注释
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse

# 预发布软件源，不建议启用
# deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
```

c. 需要替换的**/etc/apt/sources.list** 文件的内容为：

```
test@test:~$ sudo mv /etc/apt/sources.list cat /etc/apt/sources.list.bak
test@test:~$ sudo vim /etc/apt/sources.list
# 默认注释了源码镜像以提高 apt update 速度，如有需要可自行取消注释
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse


# 预发布软件源，不建议启用
# deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
```

d. 替换完后需要更新下包信息，并确保没有报错。

```
test@test:~$ sudo apt-get update
```

e. 另外，由于内核和 U-boot 等源码都是存放在 GitHub 上的，所以编译镜像的时候请确保电脑能正常从 GitHub 下载代码，这点是非常重要的。

4.2. 获取 linux sdk 的源码

4.2.1. 下载 orangepi-build

linux sdk 指的是 orangepi-build 这套代码，orangepi-build 是基于 armbian build 编译系统修改而来的，使用 orangepi-build 可以编译出多个版本的 linux 镜像。使用下面的命令可以下载 orangepi-build 的代码：

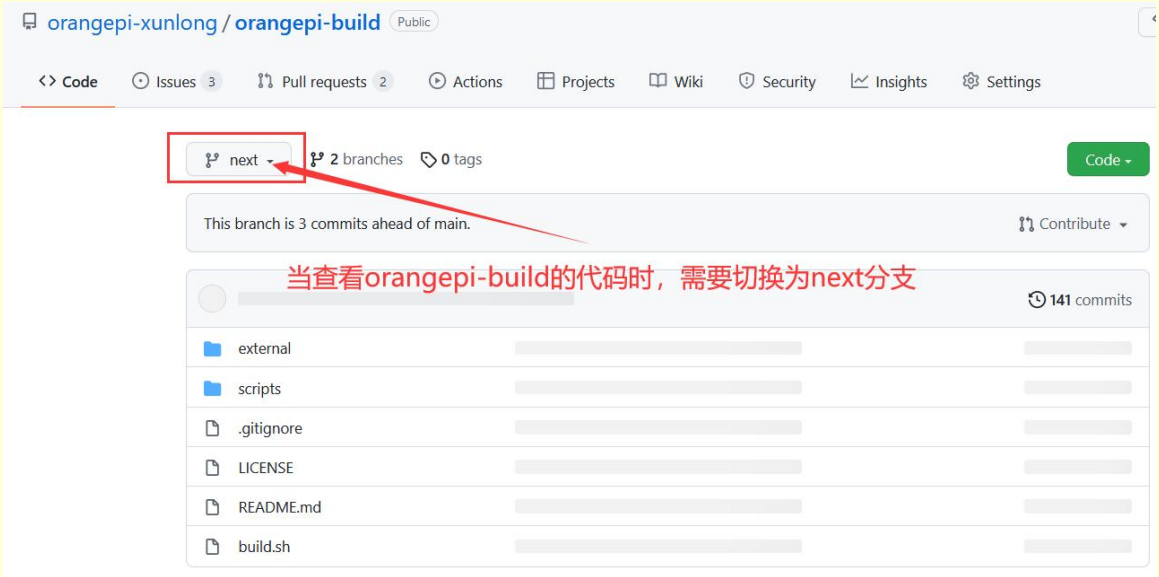
a. 从 github 下载的命令如下所示：

```
test@test:~$ sudo apt-get update && sudo apt-get install -y git
test@test:~$ git clone https://github.com/orangepi-xunlong/orangepi-build.git -b next
```

b. 从 gitee 下载的命令如下所示：

```
test@test:~$ sudo apt-get update && sudo apt-get install -y git
test@test:~$ git clone https://gitee.com/orangepi-xunlong/orangepi-build.git -b next
```

注意，使用 A733 Soc 的开发板是需要下载 orangepi-build 的 next 分支源码的，上面的 git clone 命令需要指定 orangepi-build 源码的分支为 next。



通过 git clone 命令下载 orangepi-build 的代码是不需要输入 github 账号的用户名和密码的（下载本手册中的其他代码也是一样的），如果如输入 git clone 命令后 Ubuntu PC 提示需要输入 github 账号的用户名和密码，一般都是 git clone 后面的 orangepi-build 仓库的地址输入错误了，请仔细检查命令拼写是否有错误，而不是

以为我们这里忘了提供 github 账号的用户名和密码。

A733 系列开发板当前使用的 u-boot 和 linux 内核版本如下所示：

分支	u-boot 版本	linux 内核版本
legacy	**u-boot v2018.05**	**linux5.15**
current	**u-boot v2018.05**	**Linux6.6**

这里所说的分支和 orangepi-build 源代码的分支不是同一个东西，请不要搞混了。此分支主要是用来区分不同内核源码版本的。

目前全志提供的 linux5.15 bsp 内核我们定义为 legacy 分支，linux6.6 bsp 内核我们定义为 current 分支。

orangepi-build 下载完后会包含下面的文件和文件夹：

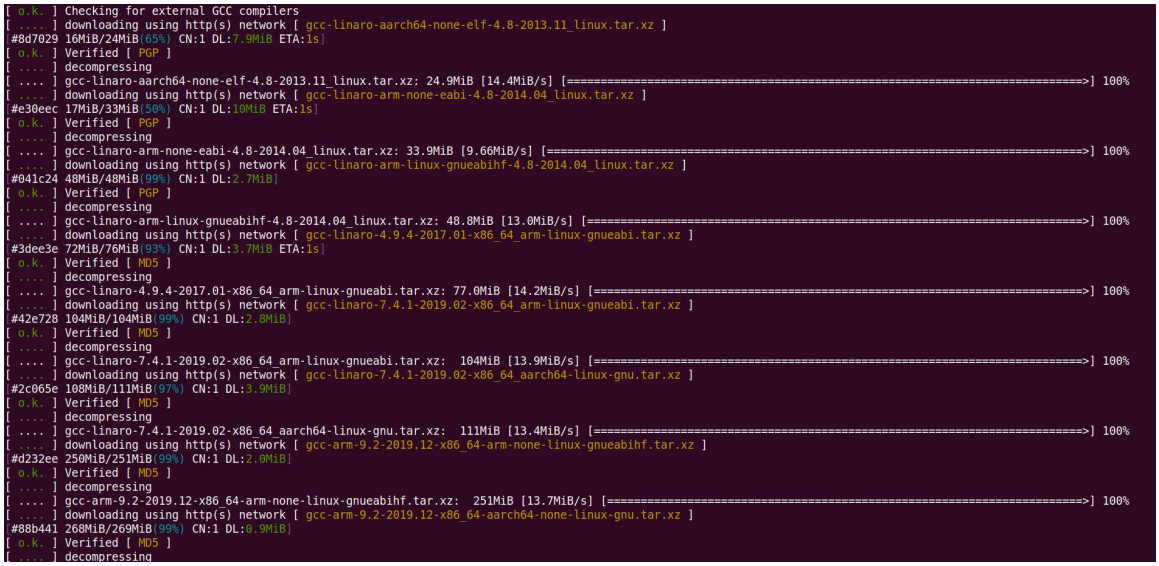
a. **build.sh**: 编译启动脚本
b. **external**: 包含编译镜像需要用的配置文件、特定的脚本以及部分程序的源码等
c. **LICENSE**: GPL 2 许可证文件
d. README.md: orangepi-build 说明文件
e. **scripts**: 编译 linux 镜像的通用脚本

```
test@test:~/orangepi-build$ ls
build.sh  external  LICENSE  README.md  scripts
```

如果是从 github 下载的 orangepi-build 的代码，下载完后你可能会发现 orangepi-build 中并没有包含 u-boot 和 linux 内核的源码，也没有编译 u-boot 和 linux 内核需要用到交叉编译工具链，这是正常的，因为这些东西都存放在其它单独的 github 仓库或者某些服务器上了（下文会详述其地址）。orangepi-build 在脚本和配置文件中会指定 u-boot、linux 内核和交叉编译工具链的地址，运行 orangepi-build 时，当其发现本地没有这些东西，会自动去相应的地方下载的。

4.2.2. 下载交叉编译工具链

orangepi-build 第一次运行的时候会自动下载交叉编译工具链放在 **toolchains** 文件夹中，每次运行 orangepi-build 的 build.sh 脚本后，都会检查 **toolchains** 中的交叉编译工具链是否都存在，如果不存在则会重新开始下载，如果存在则直接使用，不会重复下载。



交叉编译工具链在中国境内的镜像网址为清华大学的开源软件镜像站：

https://mirrors.tuna.tsinghua.edu.cn/armbian-releases/_toolchain/

toolchains 下载完后会包含多个版本的交叉编译工具链：

```
test@test:~/orangepi-build$ ls toolchains/
gcc-arm-11.2-2022.02-x86_64-aarch64-none-linux-gnu
gcc-linaro-4.9.4-2017.01-x86_64_aarch64-linux-gnu
gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi
gcc-arm-11.2-2022.02-x86_64-arm-none-linux-gnueabihf
gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi
gcc-linaro-aarch64-none-elf-4.8-2013.11_linux
gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu
gcc-linaro-5.5.0-2017.10-x86_64_arm-linux-gnueabihf
gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux
gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf
gcc-linaro-7.4.1-2019.02-x86_64_aarch64-linux-gnu
gcc-linaro-arm-none-eabi-4.8-2014.04_linux
```

编译 A733 Linux 内核源码使用的交叉编译工具链为：

a. linux5.15

gcc-arm-11.2-2022.02-x86_64-aarch64-none-linux-gnu

编译 A733 u-boot 源码使用的交叉编译工具链为：

a. v2018.05

```
gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi
```

4.2.3. orangepi-build 完整目录结构说明

2) orangepi-build 仓库下载完后并不包含 linux 内核、u-boot 的源码以及交叉编译工具链，linux 内核和 u-boot 的源码存放在独立的 git 仓库中

a. linux 内核源码存放的 git 仓库如下：

a) Github

```
https://github.com/orangepi-xunlong/linux-orangepi/tree/orange-pi-5.15-sun60iw2
```

b) Gitee

```
https://gitee.com/orangepi-xunlong/orange-pi-5.15-sun60iw2
```

b. u-boot 源码存放的 git 仓库如下：

a) Github

```
https://github.com/orangepi-xunlong/u-boot-orangepi/tree/v2018.05-sun60iw2
```

b) Gitee

```
https://gitee.com/orangepi-xunlong/u-boot-orangepi/tree/v2018.05-sun60iw2
```

3) orangepi-build 第一次运行的时候会去下载交叉编译工具链、u-boot 和 linux 内核源码，成功编译完一次 linux 镜像后在 orangepi-build 中可以看到的文件和文件夹有

a. **build.sh**: 编译启动脚本
b. **external**: 包含编译镜像需要用的配置文件、特定功能的脚本以及部分程序的源码，编译镜像过程中缓存的 rootfs 压缩包也存放在 external 中
c. **kernel**: 存放 linux 内核的源码
d. **LICENSE**: GPL 2 许可证文件
e. **README.md**: orangepi-build 说明文件
f. **output**: 存放编译生成的 u-boot、linux 等 deb 包、编译日志以及编译生成的镜像等文件
g. **scripts**: 编译 linux 镜像的通用脚本
h. **toolchains**: 存放交叉编译工具链
i. **u-boot**: 存放 u-boot 的源码
j. **userpatches**: 存放编译脚本需要用到的配置文件

```
test@test:~/orangepi-build$ ls
build.sh  external  kernel  LICENSE  output  README.md  scripts  toolchains  u-boot  userpatches
```

4.3. 编译 u-boot

如果从 github 下载代码很慢，可以在./build.sh 后面加入 GITEE_SERVER=yes 这个参数，这样在编译的时候都会从 gitee 网站去下载。

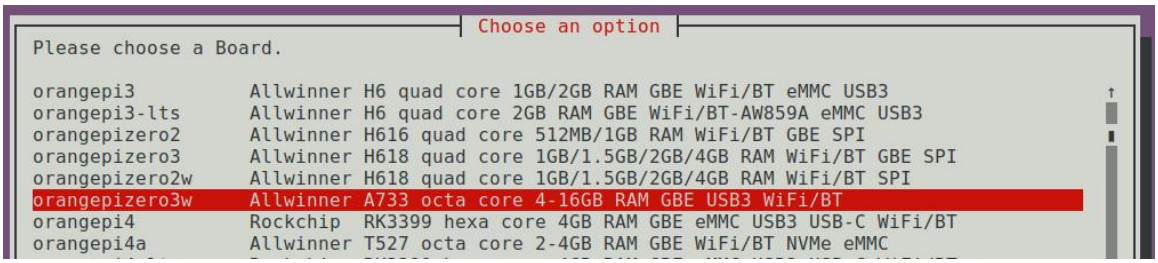
加入 GITEE_SERVER=yes 参数的编译命令如下所示:

test@test:~/orangepi-build$ **sudo ./build.sh GITEE_SERVER=yes**

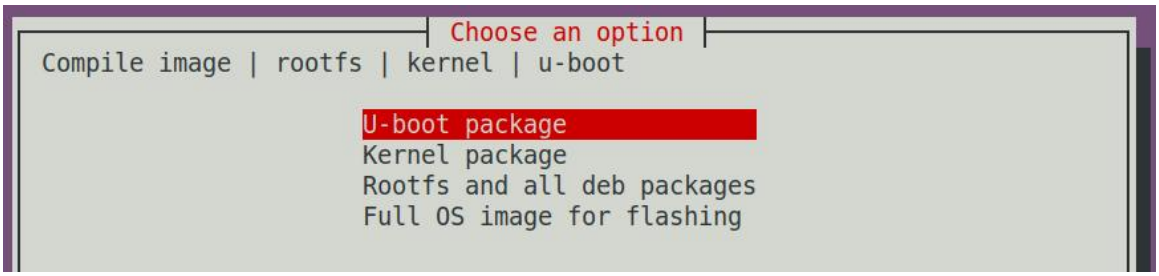
1) 运行 build.sh 脚本，记得加 sudo 权限

test@test:~/orangepi-build$ **sudo ./build.sh**

2) 接着选择开发板的型号



3) 选择 **U-boot package**，然后回车



4) 然后就会开始编译 u-boot，编译 current 分支时提示的部分信息说明如下所示：

a. u-boot 源码的版本

[o.k.] Compiling u-boot [**v2018.05**]

b. 交叉编译工具链的版本

[o.k.] Compiler version [**aarch64-linux-gnu-gcc 11**]

c. 编译生成的 u-boot deb 包的路径

[o.k.] Target directory [**orangepi-build/output/debs/u-boot**]

d. 编译生成的 u-boot deb 包的包名

```
[ o.k. ] File name [ linux-u-boot-current-orangepizero3w_x.x.x_arm64.deb ]
```

e. 编译使用的时间

```
[ o.k. ] Runtime [ 1 min ]
```

f. 重复编译 u-boot 的命令，使用下面的命令无需通过图形界面选择，可以直接开始编译 u-boot

```
[ o.k. ] Repeat Build Options [ sudo ./build.sh    BOARD=orangepizero3w
BRANCH=current BUILD_OPT=u-boot ]
```

5) 查看编译生成的 u-boot deb 包

```
test@test:~/orangepi-build$ ls output/debs/u-boot/
linux-u-boot-current-orangepizero3w_x.x.x_arm64.deb
```

6) orangepi-bulid 编译系统编译 u-boot 源码时首先会将 u-boot 的源码和 github 服务器的 u-boot 源码进行同步，所以如果想修改 u-boot 的源码，首先需要关闭源码的下载更新功能（**需要完整编译过一次 u-boot 后才能关闭这个功能，否则会提示找不到 u-boot 的源码**），否则所作的修改都会被还原，方法如下：

设置 **userpatches/config-default.conf** 中的 IGNORE_UPDATES 变量为“yes”

```
test@test:~/orangepi-build$ vim userpatches/config-default.conf
.....
IGNORE_UPDATES="yes"
.....
```

7) 调试 u-boot 代码时，可以使用下面的方法来更新 linux 镜像中的 u-boot 进行测试

a. 首先将编译好的 u-boot 的 deb 包上传到开发板的 linux 系统中

```
test@test:~/orangepi-build$ cd output/debs/u-boot
test@test:~/orangepi_build/output/debs/u-boot$ scp \
linux-u-boot-current-orangepizero3w_x.x.x_arm64.deb root@192.168.1.xxx:/root
```

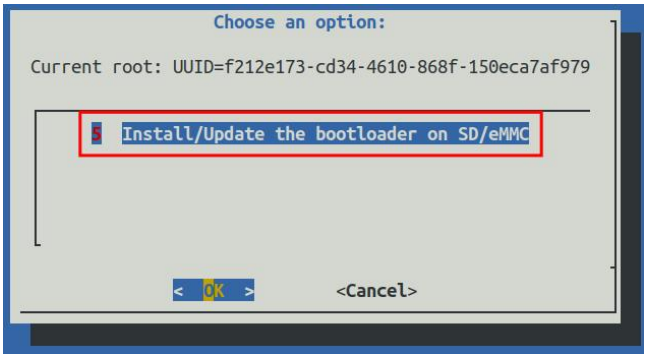
b. 再安装刚才上传的新的 u-boot 的 deb 包

```
orangepi@orangepi:~$ sudo dpkg -i
linux-u-boot-current-orangepizero3w_x.x.x_arm64.deb
```

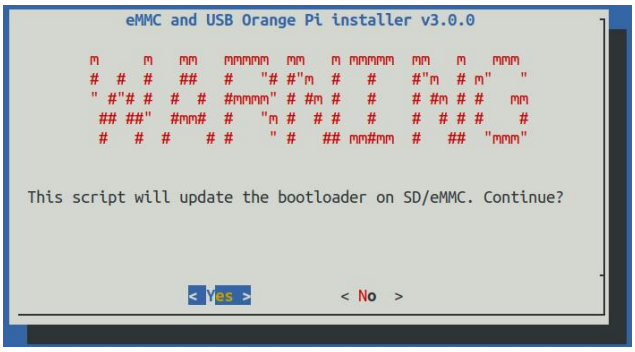
c. 然后运行 nand-sata-install 脚本

```
orangepi@orangepi:~$ sudo nand-sata-install
```

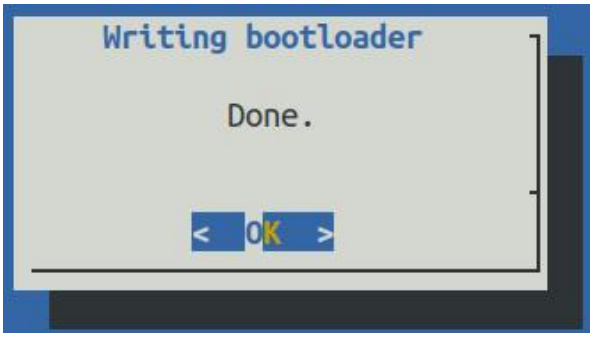
d. 然后选择 **5 Install/Update the bootloader on SD/eMMC**



e. 按下回车键后首先会弹出一个 Warning



f. 再按下回车键就会开始更新 u-boot，更新完后会显示下面的信息



g. 然后就可以重启开发板来测试 u-boot 的修改是否生效了

4. 4. 编译 linux 内核

如果从 github 下载代码很慢，可以在./build.sh 后面加入 GITEE_SERVER=yes 这个参数，这样在编译的时候都会从 gitee 网站去下载。

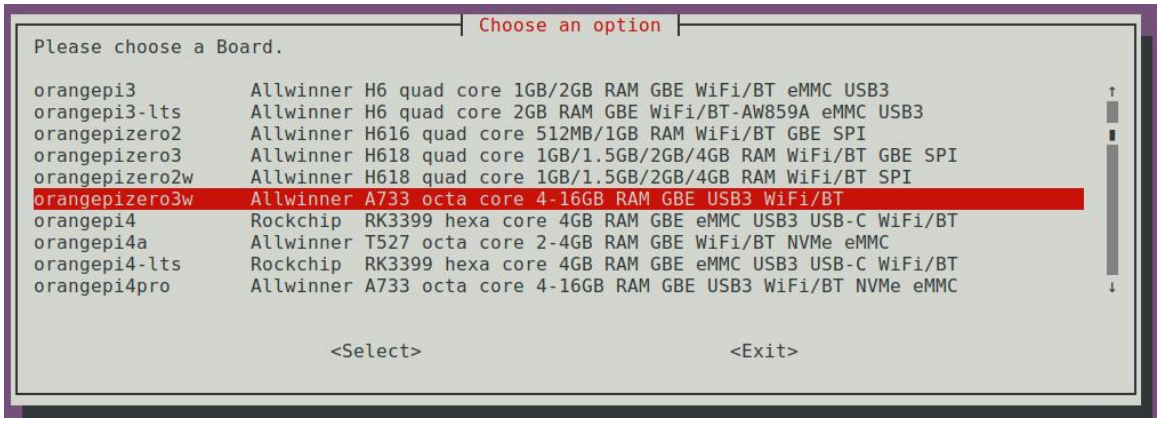
加入 GITEE_SERVER=yes 参数的编译命令如下所示:

test@test:~/orangepi-build$ **sudo ./build.sh GITEE_SERVER=yes**

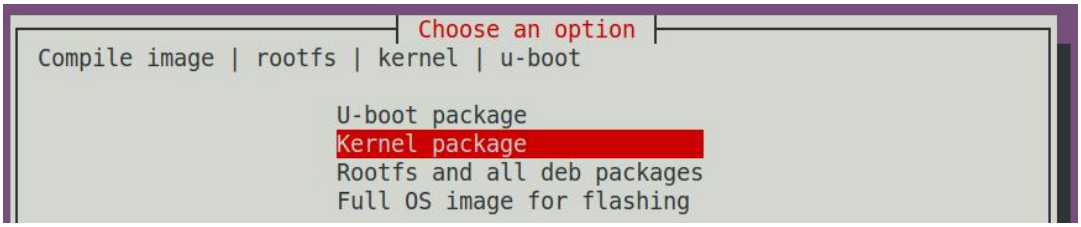
1) 运行 **build.sh** 脚本，记得加 sudo 权限

```
test@test:~/orangepi-build$ sudo ./build.sh
```

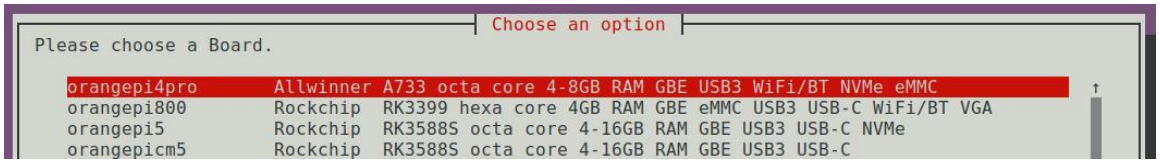
2) 接着选择开发板的型号



3) 选择 **Kernel package**，然后回车



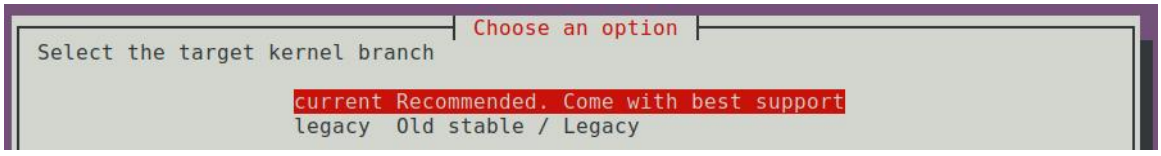
4) 然后会提示是否需要显示内核配置界面，如果不需要修改内核配置，则选择第一个即可，如果需要修改内核配置，则选择第二个



5) 然后选择内核源码的分支类型

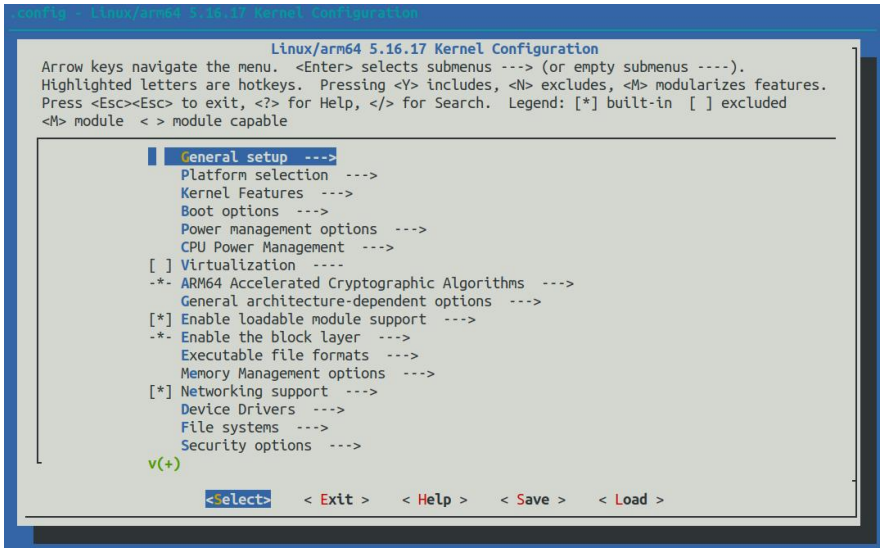
a. legacy 分支会编译 linux5.15 内核源码

b. current 分支会编译 linux6.6 内核源码



6) 如果第 3)步选择了需要显示内核配置菜单（第二个选项），则会弹出通过 **make**

menuconfig 打开的内核配置的界面，此时可以直接修改内核的配置，修改完后再保存退出即可，退出后会开始编译内核源码。



a. 如果不需要修改内核的配置选项，在运行 build.sh 脚本时，传入 **KERNEL_CONFIGURE=no** 就可临时屏蔽弹出内核的配置界面了

```
test@test:~/orangepi-build$ sudo ./build.sh KERNEL_CONFIGURE=no
```

b. 也可以设置 orangepi-build/userpatches/config-default.conf 配置文件中的 **KERNEL_CONFIGURE=no**，这样可以永久禁用这个功能

c. 编译内核的时候如果提示下面的错误，这是由于 Ubuntu PC 的终端界面太小，导致 make menuconfig 的界面无法显示，请把 Ubuntu PC 的终端调到最大，然后重新运行 build.sh 脚本

```
  HOSTCC  scripts/kconfig/mconf.o
  HOSTCC  scripts/kconfig/lxdialog/checklist.o
  HOSTCC  scripts/kconfig/lxdialog/util.o
  HOSTCC  scripts/kconfig/lxdialog/inputbox.o
  HOSTCC  scripts/kconfig/lxdialog/textbox.o
  HOSTCC  scripts/kconfig/lxdialog/yesno.o
  HOSTCC  scripts/kconfig/lxdialog/menubox.o
  HOSTLD  scripts/kconfig/mconf
scripts/kconfig/mconf  Kconfig
Your display is too small to run Menuconfig!
It must be at least 19 lines by 80 columns.
scripts/kconfig/Makefile:28: recipe for target 'menuconfig' failed
make[1]: *** [menuconfig] Error 1
Makefile:560: recipe for target 'menuconfig' failed
make: *** [menuconfig] Error 2
[ error ] ERROR in function compile_kernel [ compilation.sh:376 ]
[ error ] Error kernel menuconfig failed
[ o.k. ] Process terminated
```

7) 编译 current 分支内核源码时提示的部分信息说明如下：

a. linux 内核源码的版本

```
[ o.k. ] Compiling current kernel [ 5.15.147 ]
```

b. 使用的交叉编译工具链的版本

```
[ o.k. ] Compiler version [ aarch64-linux-gnu-gcc 11 ]
```

c. 内核默认使用的配置文件以及它存放的路径如下所示

```
[ o.k. ] Using kernel config file
[ orangepi-build/external/config/kernel/linux-5.15-sun60iw2-current.config ]
```

d. 编译生成的内核相关的 deb 包的路径

```
[ o.k. ] Target directory [ output/debs/ ]
```

e. 编译生成的内核镜像 deb 包的包名

```
[ o.k. ] File name [ linux-image-current-sun60iw2_x.x.x_arm64.deb ]
```

f. 编译使用的时间

```
[ o.k. ] Runtime [ 10 min ]
```

g. 最后会显示重复编译上一次选择的内核的编译命令，使用下面的命令无需通过图形界面选择，可以直接开始编译内核源码

```
[ o.k. ] Repeat Build Options [ sudo ./build.sh  BOARD=orangepizero3w
BRANCH=current BUILD_OPT=kernel KERNEL_CONFIGURE=no ]
```

8) 查看编译生成的内核相关的 deb 包

a. **linux-dtb-current-sun60iw2_x.x.x_arm64.deb** 包含有内核使用的 dtb 文件

b. **linux-headers-current-sun60iw2_x.x.x_arm64.deb** 包含内核头文件

c. **linux-image-current-sun60iw2_x.x.x_arm64.deb** 包含内核镜像和内核模块

```
test@test:~/orangepi-build$ ls output/debs/linux-*
output/debs/linux-dtb-current-sun60iw2_x.x.x_arm64.deb
output/debs/linux-headers-current-sun60iw2_x.x.x_arm64.deb
output/debs/linux-image-current-sun60iw2_x.x.x_arm64.deb
```

9) orangepi-bulid 编译系统编译 linux 内核源码时首先会将 linux 内核源码和 github 服务器的 linux 内核源码进行同步，所以如果想修改 linux 内核的源码，首先需要关闭源码的更新功能（**需要完整编译过一次 linux 内核源码后才能关闭这个功能，否则会提示找不到 linux 内核的源码**），否则所作的修改都会被还原，方法如下：

设置 **userpatches/config-default.conf** 中的 IGNORE_UPDATES 变量为“yes”

```
test@test:~/orangepi-build$ vim userpatches/config-default.conf
IGNORE_UPDATES="yes"
```

10) 如果对内核做了修改，可以使用下面的方法来更新开发板 linux 系统的内核和内

核模块

a. 将编译好的 linux 内核的 deb 包上传到开发板的 linux 系统中

```
test@test:~/orangepi-build$ cd output/debs
test@test:~/orangepi-build/output/debs$ scp \
linux-image-current-sun60iw2_x.x.x_arm64.deb root@192.168.1.xxx:/root
```

b. 再安装刚才上传的新的 linux 内核的 deb 包

```
orangepi@orangepi:~$ sudo dpkg -i linux-image-current-sun60iw2_x.x.x_arm64.deb
```

c. 然后重启开发板，再查看内核相关的修改是否已生效

```
orangepi@orangepi:~$ sudo reboot
```

4.5. 编译 rootfs

如果从 github 下载代码很慢，可以在./build.sh 后面加入 GITEE_SERVER=yes 这个参数，这样在编译的时候都会从 gitee 网站去下载。

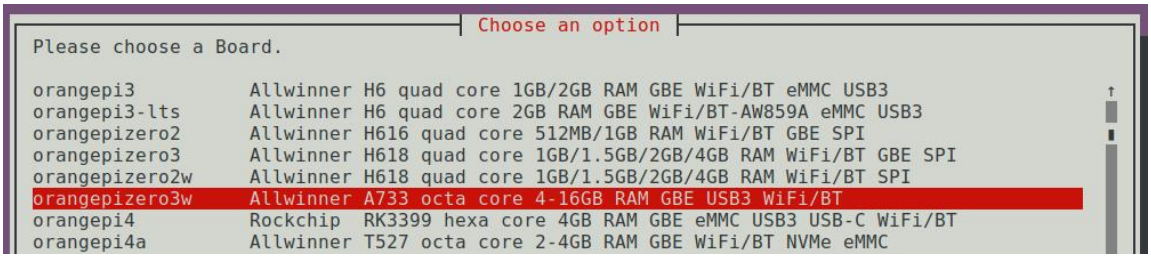
加入 GITEE_SERVER=yes 参数的编译命令如下所示:

```
test@test:~/orangepi-build$ sudo ./build.sh GITEE_SERVER=yes
```

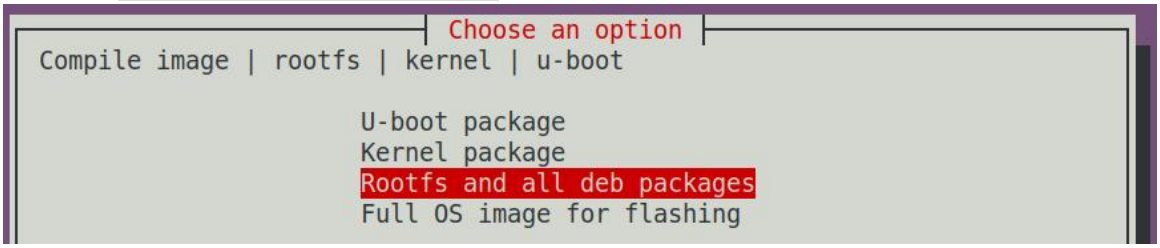
1) 运行 build.sh 脚本，记得加 sudo 权限

```
test@test:~/orangepi-build$ sudo ./build.sh
```

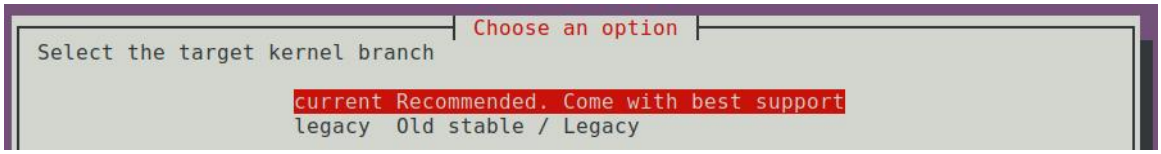
2) 接着选择开发板的型号



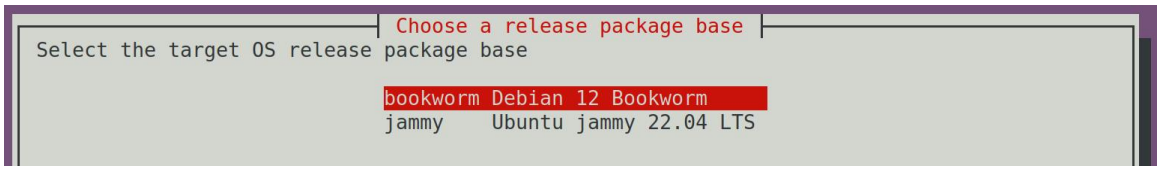
3) 选择 **Rootfs and all deb packages**，然后回车



4) 然后选择内核源码的分支类型，目前内核源码维护的 rootfs 使用同一套

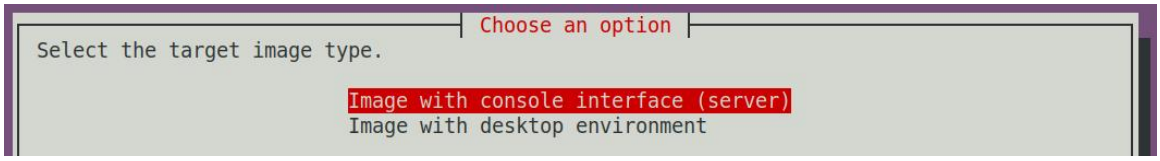


5) 然后选择 rootfs 的类型

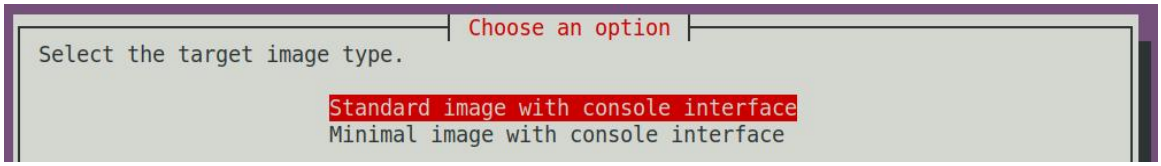


6) 然后选择镜像的类型

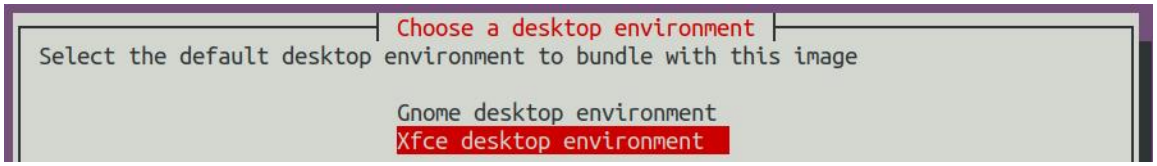
a. **Image with console interface (server)**表示服务器版的镜像，体积比较小
b. **Image with desktop environment** 表示带桌面的镜像，体积比较大

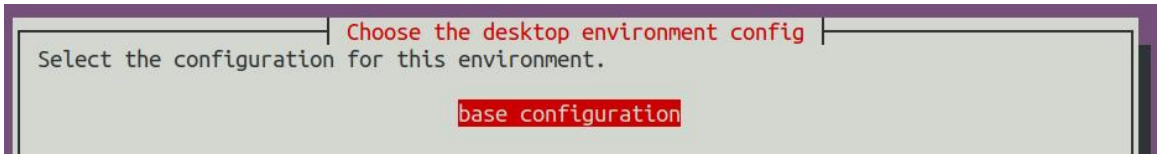


7) 如果是编译服务器版的镜像，还可以选择编译 Standard 版本或者 Minimal 版本，Minimal 版本预装的软件会比 Standard 版本少很多(**没特殊需求请不要选择 Minimal 版本，因为很多东西默认没有预装，部分功能可能用不了**)

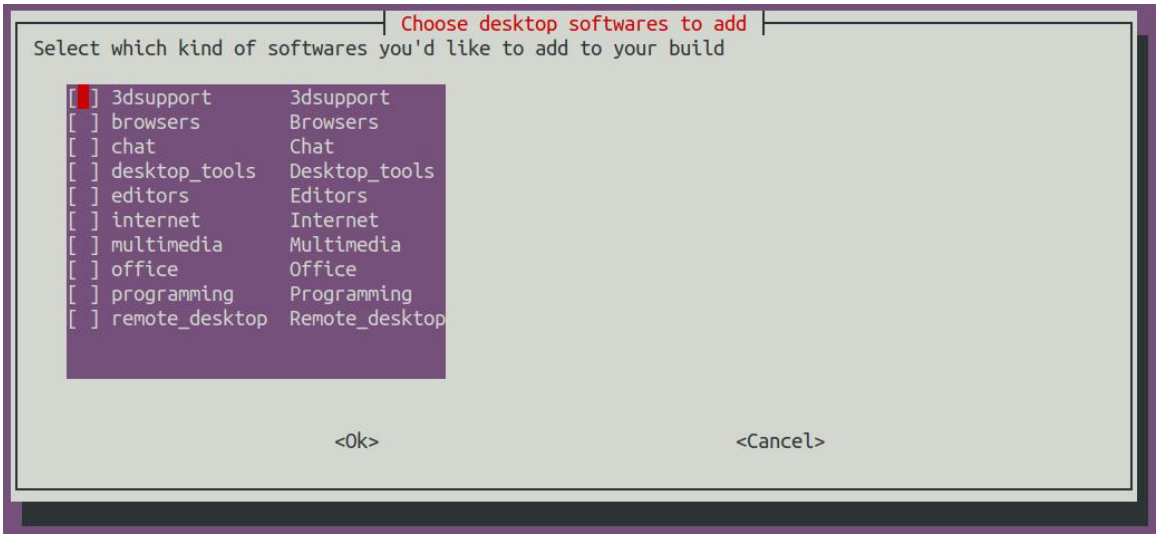


8) 如果是编译桌面版本的镜像还需要选择桌面环境的类型，目前只维护 GNOME，所以请选择 GNOME 类型的桌面





然后可以选择需要安装的额外的软件包。这里请按下回车键直接跳过。



9) 然后就会开始编译 rootfs，编译时提示的部分信息说明如下

a. rootfs 的类型

[o.k.] local not found [Creating new rootfs cache for **jammy**]

b. 编译生成的 rootfs 压缩包的存放路径

[o.k.] Target directory [**orangepi-build/external/cache/rootfs**]

c. 编译生成的 rootfs 压缩包的名字

[o.k.] File name [**jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4**]

10) 查看编译生成的 rootfs 压缩包

a. **jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4** 是 rootfs 的压缩包，名字各字段的含义为

a) **jammy** 表示 rootfs 的 linux 发行版的类型

b) **xfce** 表示 rootfs 为桌面版的类型，如果为 **cli** 则表示服务器版类型

c) **arm64** 表示 rootfs 的架构类型

d) **25250ec7002de9e81a41de169f1f89721** 是由 rootfs 安装的所有软件包的包名生成的 MD5 哈希值，只要没有修改 rootfs 安装的软件包的列表，那么这个值就不会变，编译脚本会通过这个 MD5 哈希值来判断是否需要重新编译 rootfs

b. **jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4.list** 列出了 rootfs 安装的所有软件包的包名

```
test@test:~/orangepi-build$ ls external/cache/rootfs/
jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4
jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4.current
jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4.list
```

11) 如果需要的 rootfs 在 **external/cache/rootfs** 下已经存在，那么再次编译 rootfs 就会直接跳过编译过程，不会重新开始编译，编译镜像的时候也会去 **external/cache/rootfs** 下查找是否已经有缓存可用的 rootfs，如果有就直接使用，这样可以节省大量的下载编译时间

4.6. 编译 linux 镜像

如果从 github 下载代码很慢，可以在./build.sh 后面加入 GITEE_SERVER=yes 这个参数，这样在编译的时候都会从 gitee 网站去下载。

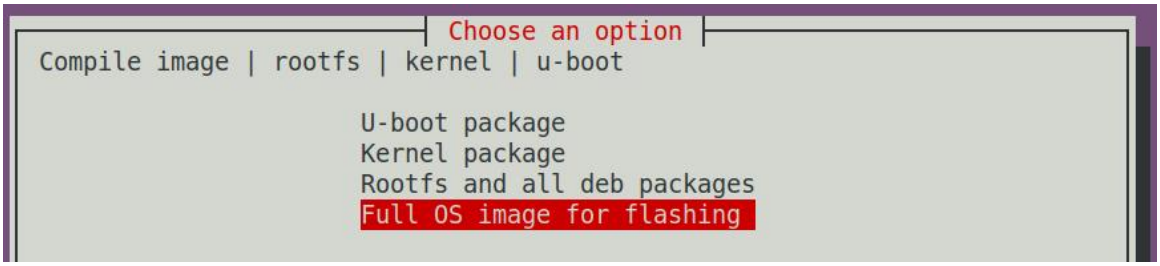
加入 GITEE_SERVER=yes 参数的编译命令如下所示:

```
test@test:~/orangepi-build$ sudo ./build.sh GITEE_SERVER=yes
```

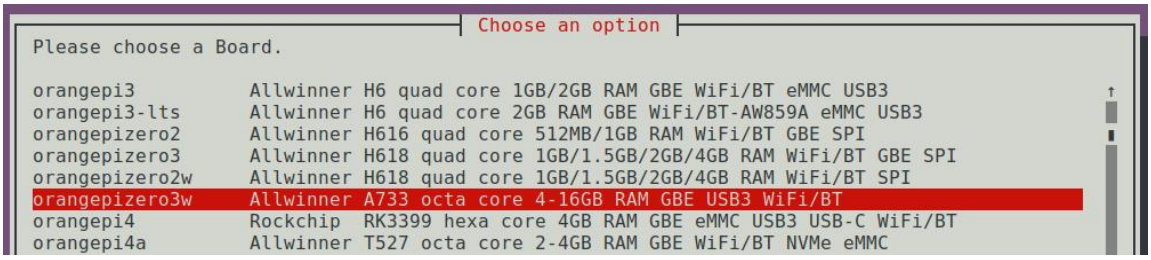
1) 运行 **build.sh** 脚本，记得加 sudo 权限

```
test@test:~/orangepi-build$ sudo ./build.sh
```

2) 选择 **Full OS image for flashing**，然后回车



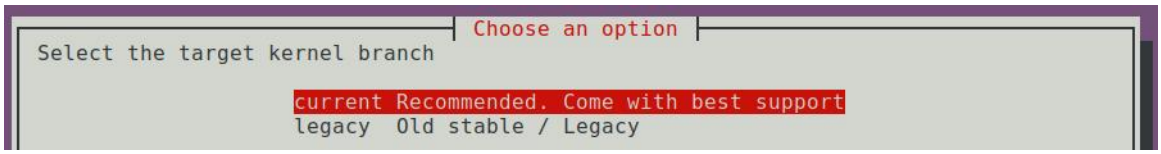
3) 然后选择开发板的型号



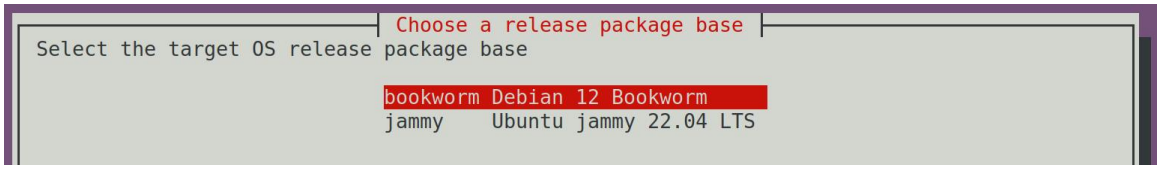
4) 然后选择内核源码的分支类型

a. legacy 分支会编译 linux5.15 内核源码

b. current 分支会编译 linux6.6 内核源码



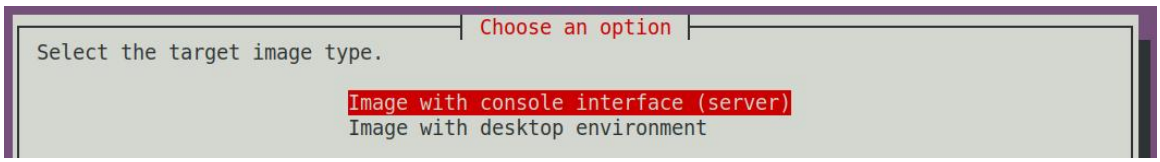
5) 然后选择 rootfs 的类型



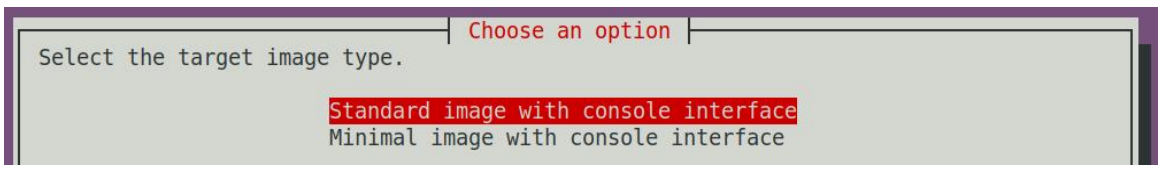
6) 然后选择镜像的类型

a. **Image with console interface (server)**表示服务器版的镜像，体积比较小

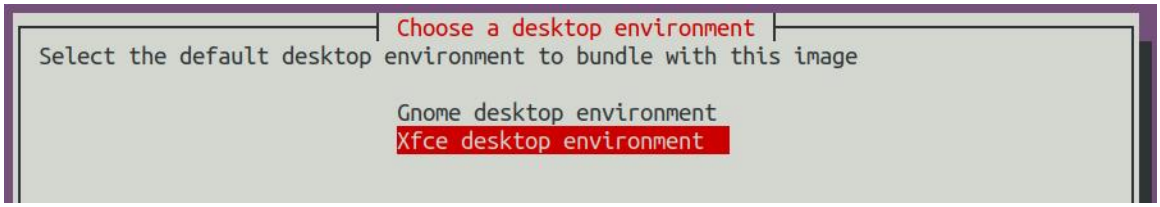
b. **Image with desktop environment** 表示带桌面的镜像，体积比较大

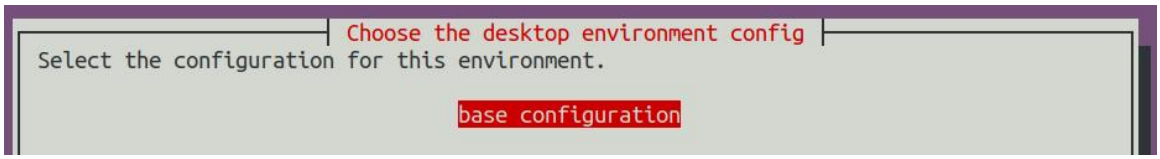


7) 如果是编译服务器版的镜像，还可以选择编译 Standard 版本或者 Minimal 版本，Minimal 版本预装的软件会比 Standard 版本少很多(**没特殊需求请不要选择 Minimal 版本，因为很多东西默认没有预装，部分功能可能用不了**)

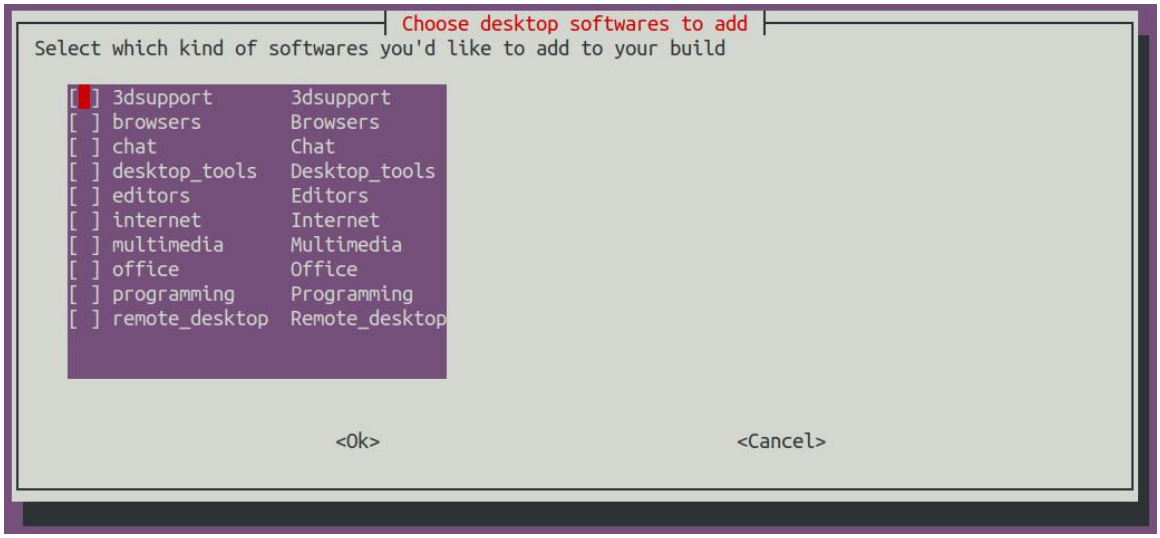


8) 如果是编译桌面版本的镜像还需要选择桌面环境的类型，目前只维护 XFCE，所以请选择 XFCE 类型的桌面





然后可以选择需要安装的额外的软件包。这里请按下回车键直接跳过。



9) 然后就会开始编译 linux 镜像，编译的大致流程如下

a. 初始化 Ubuntu PC 的编译环境，安装编译过程需要的软件包
b. 下载 u-boot 和 linux 内核的源码（如果已经缓存，则只更新代码）
c. 编译 u-boot 源码，生成 u-boot 的 deb 包
d. 编译 linux 源码，生成 linux 相关的 deb 包
e. 制作 linux firmware 的 deb 包
f. 制作 orangepi-config 工具的 deb 包
g. 制作板级支持的 deb 包
h. 如果是编译 desktop 版镜像，还会制作 desktop 相关的 deb 包
i. 检查 rootfs 是否已经缓存，如果没有缓存，则重新制作 rootfs，如果已经缓存，则直接解压使用
j. 安装前面生成的 deb 包到 rootfs 中

k. 对不同的开发板和不同类型镜像做一些特定的设置，如预装额外的软件包，修改系统配置等
l. 然后制作镜像文件，并格式化分区，默认类型为 ext4
m. 再将配置好的 rootfs 拷贝到镜像的分区中
n. 然后更新 initramfs
o. 最后将 u-boot 的 bin 文件通过 dd 命令写入到镜像中

10) 编译完镜像后会提示下面的信息

a. 编译生成的镜像的存放路径

```
[ o.k. ] Done building
[ output/images/Orangepizero3w_x.x.x_debian_jammy_linux5.15.xx_xfce_desktop/
Orangepizero3w_x.x.x_debian_jammy_linux5.15.xx_xfce_desktop.img ]
```

b. 编译使用的时间

```
[ o.k. ] Runtime [ 19 min ]
```

c. 重复编译镜像的命令，使用下面的命令无需通过图形界面选择，可以直接开始编译镜像

```
[ o.k. ] Repeat Build Options [ sudo ./build.sh  BOARD=orangepizero3w
BRANCH=current BUILD_OPT=image RELEASE=jammy BUILD_MINIMAL=no
BUILD_DESKTOP=no KERNEL_CONFIGURE=yes ]
```

5. Android 13 系统使用说明

5.1. 已支持的 Android 版本

Android 版本	内核版本
Android 13	**linux5.15**

5.2. Android 13 功能适配情况

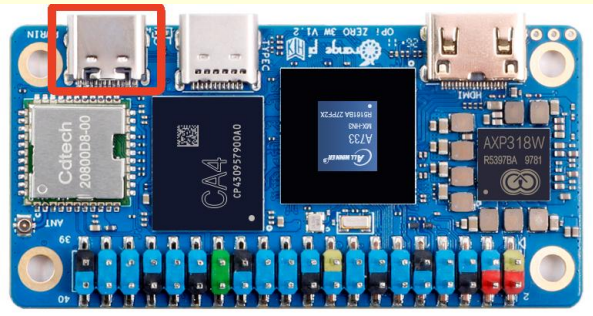
功能	Android 13
HDMI 视频	**OK**
HDMI 音频	**OK**
TypeC USB3.0	**OK**
TypeC USB2.0	**OK**
TypeC DP 显示	**OK**
TF 卡启动	**OK**
eMMC	**OK**
UFS	**OK**
WIFI	**OK**
蓝牙	**OK**
CAM1 接口	**OK**
CAM2 接口	**OK**
MIPI LCD	**OK**
FAN 散热风扇	**OK**
LED 灯	**OK**
40 pin GPIO	**OK**
40 pin I2C	**OK**
40 pin SPI	**OK**
40 pin UART	**OK**
40 pin PWM	**OK**

温度传感器	OK
GPU	OK

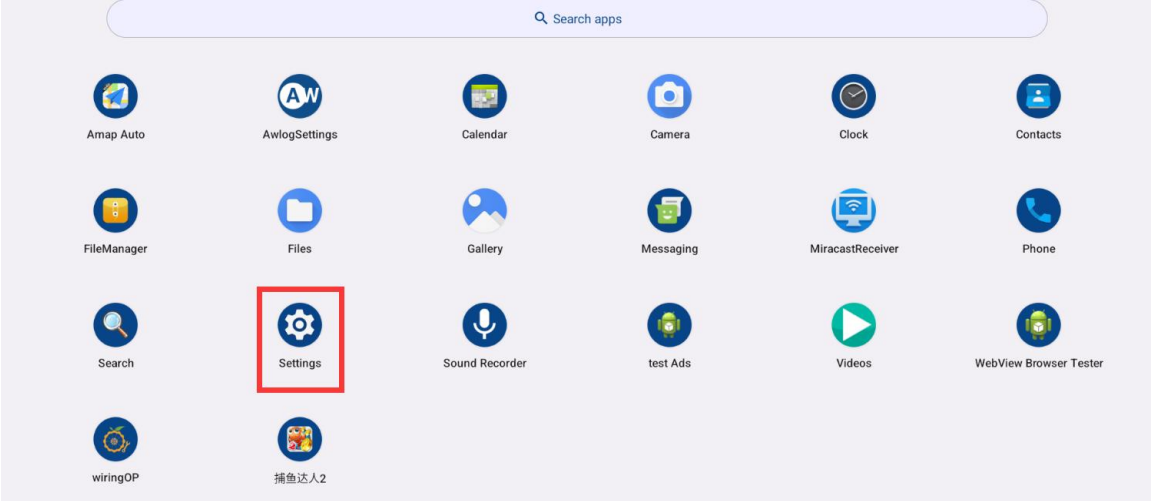
5.3. ADB 的使用方法

5.3.1. USB OTG 模式切换的方法

开发板有 2 个 TypeC 接口，其中下图红框标注的 Type C USB 2.0 接口既可以支持 Host 模式，又可以支持 Device 模式，另外的 TypeC 接口只支持 Host 模式。



1) 首先打开 Settings



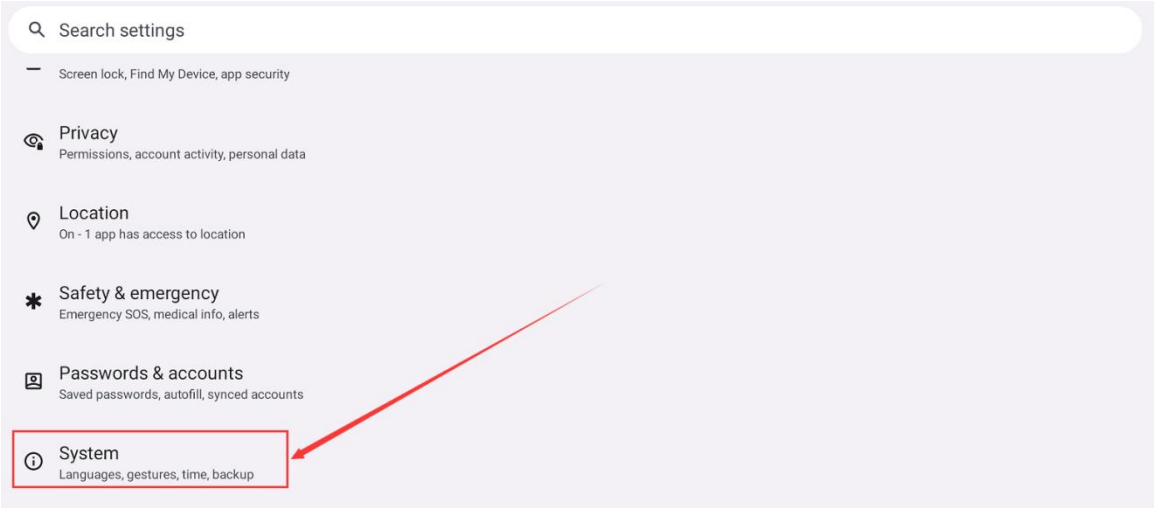
2) 然后找到 **About tablet**



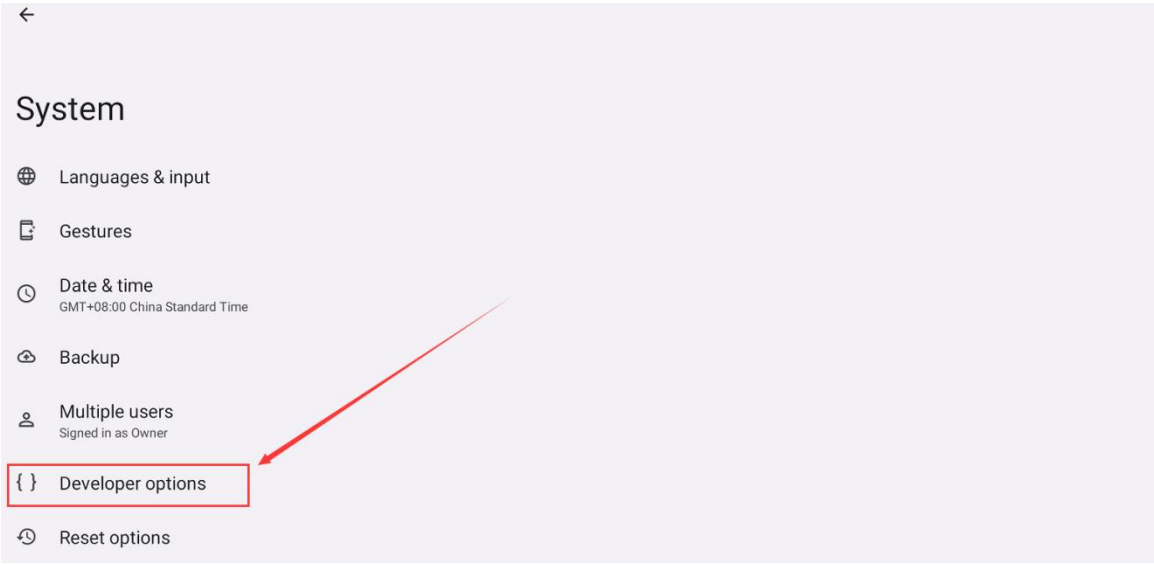
3) 然后用鼠标多次点击 **Build number** 选项，直到出现 **You are now a developer!**的提示



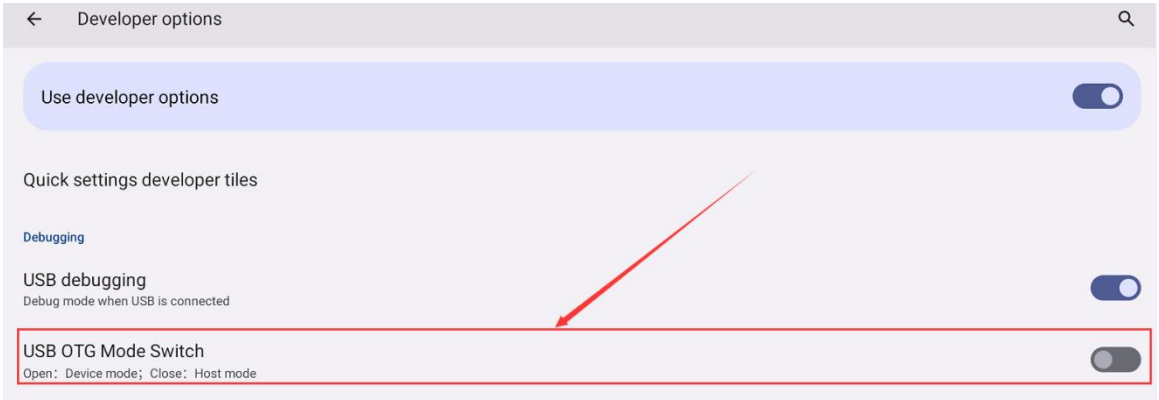
4) 然后返回上一级菜单，选择 **System**



5) 然后选择 **Developer options**



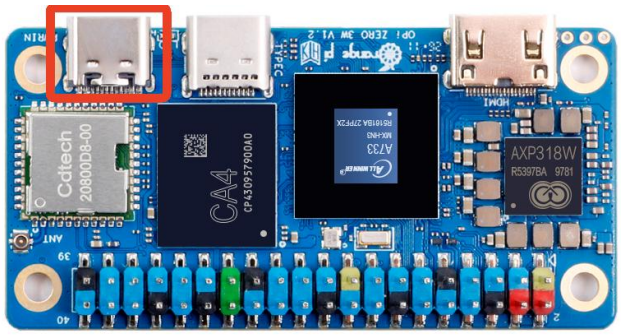
6) 最后找到 **USB OTG Mode Switch** 开关，**打开开关切换为 Device 模式，关闭开关切换为 Host 模式**



5.3.2. 使用数据线连接 adb 调试

1) 准备一根 USB Typc C 接口的数据线， USB 接口一端插入电脑的 USB 接口中，USB Type C 接口一端插入开发板的 TypeC 接口中（位置请见下面右边图片）。在这种情况下是由电脑的 USB 接口给开发板供电，所以请确保电脑的 USB 接口能提供足够的功率驱动开发板。





2) 然后按照 **USB OTG 模式切换的方法**切换到 USB Device 模式。

3) 然后在 Ubuntu PC 上安装 adb 工具

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y adb
```

4) 查看识别到 ADB 设备

```
test@test:~$ adb devices
List of devices attached
4c00146473c28651dd0     device
```

5) 然后在 Ubuntu PC 上通过 adb shell 就可以登录 android 系统

```
test@test:~$ adb shell
a733-demo:/ #
```

5.3.3. 使用网络连接 adb 调试

使用网络 adb 无需 Type-C 数据线来连接电脑和开发板，而是通过网络来通信，所以首先请确保开发板的有线或者无线网络已经连接好了，然后获取开发板的 IP 地址，后面要用到。

1) 确保 Android 系统的 **service.adb.tcp.port** 设置为 5555 端口号

```
console:/ # getprop | grep "adb.tcp"
[service.adb.tcp.port]: [5555]
```

2) 如果 **service.adb.tcp.port** 没有设置，可以在串口中使用下面的命令设置网络 adb 的端口号

```
console:/ # setprop service.adb.tcp.port 5555

console:/ # stop adbd
console:/ # start adbd
```

3) 在 Ubuntu PC 上安装 adb 工具

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y adb
```

4) 然后在 Ubuntu PC 上连接网络 adb

```
test@test:~$ adb connect 192.168.1.xxx:5555     (需要修改为开发板的 IP 地址)
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 192.168.1.xxx:5555

test@test:~$ adb devices
List of devices attached
192.168.1.xxx:5555          device
```

5) 然后在 Ubuntu PC 上通过 adb shell 就可以登录 android 系统

```
test@test:~$ adb shell
a733-demo:/ #
```

5.3.4. HDMI 转 VGA 显示测试

1) 首先需要准备下面的配件

a. HDMI 转 VGA 转换器



b. 一根 VGA 线和一个 Mini HDMI 公转 HDMI 母的转接头



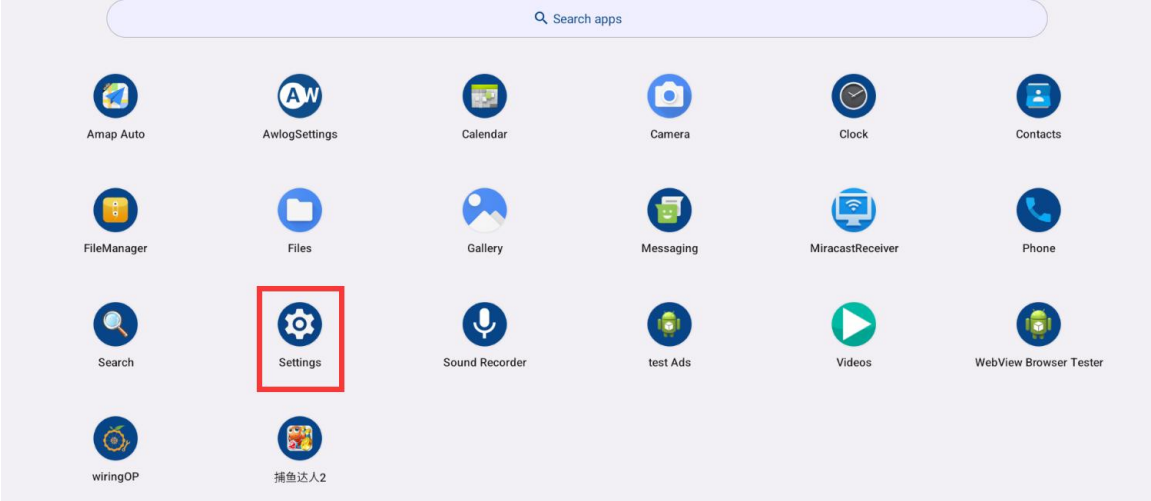


c. 一个支持 VGA 接口的显示器或者电视

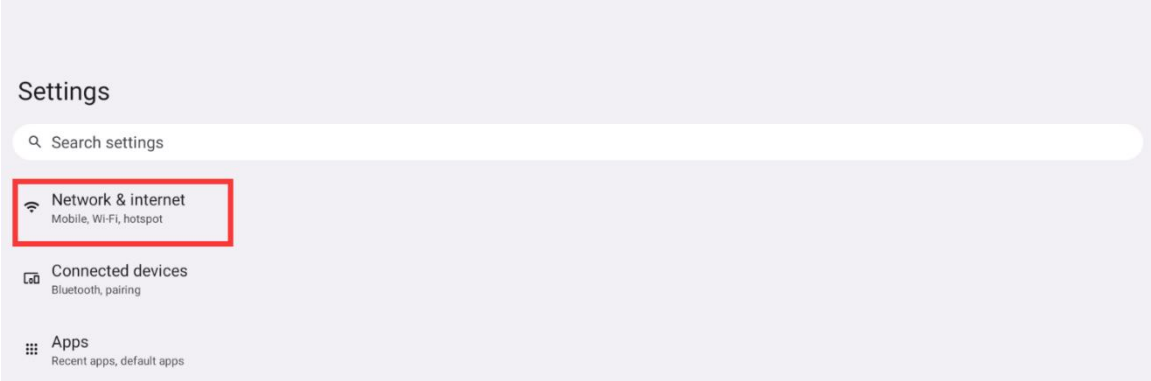
2) 然后把线连接好，HDMI 转 VGA 就能够正常显示了

5.4. WI-FI 的连接方法

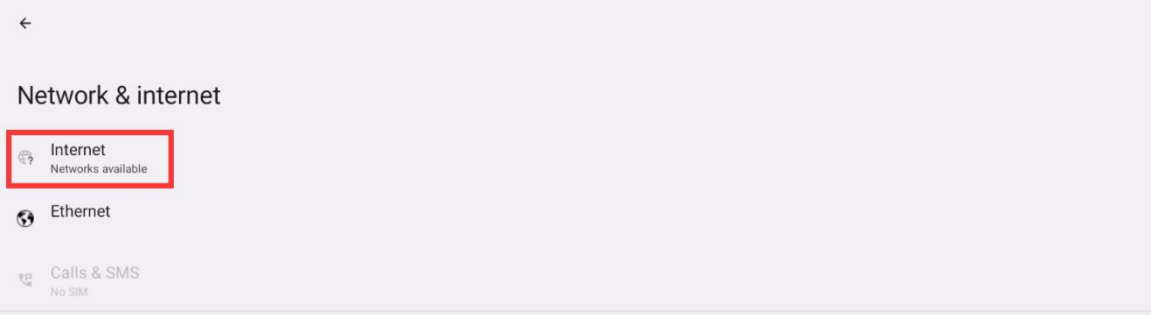
1) 首先选择 **Settings**



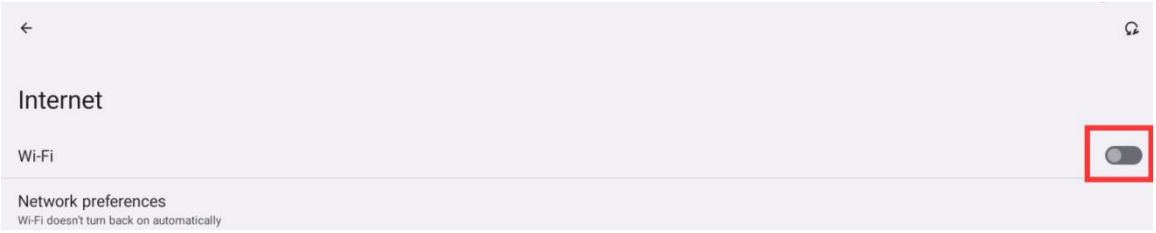
2) 然后选择 **Network & Internet**



3) 然后选择 **Internet**



4) 然后打开 WI-FI



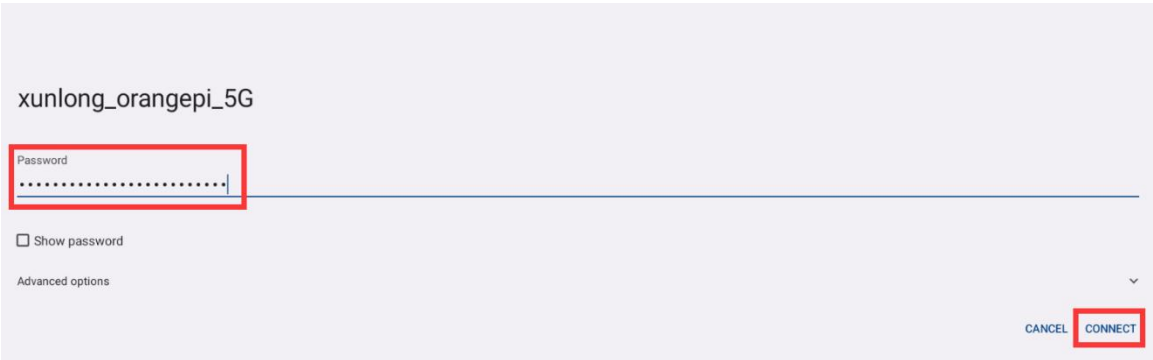
5) 打开 WI-FI 后在 **Available networks** 下面就可以看到搜索到的信号



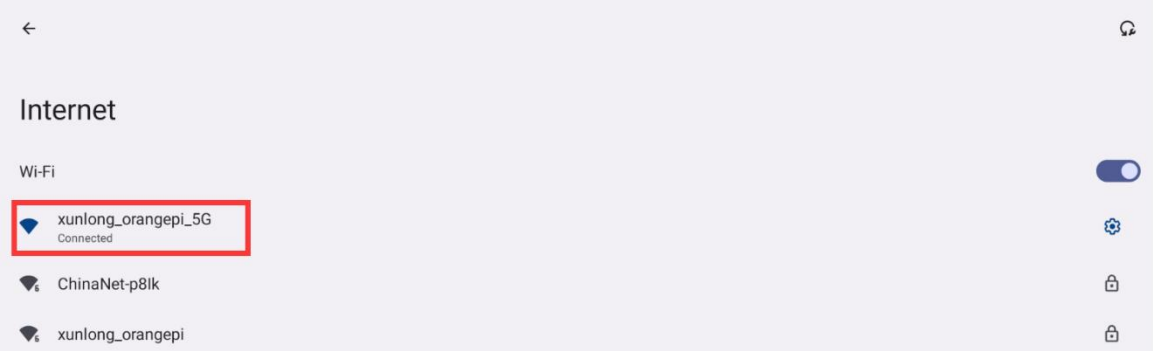
6) 选择想连接的 WI-FI 后会弹出下图所示的密码输入界面



7) 然后使用键盘输入 WI-FI 对应的密码，再使用鼠标点击虚拟键盘中的回车按钮就会开始连接 WI-FI 了

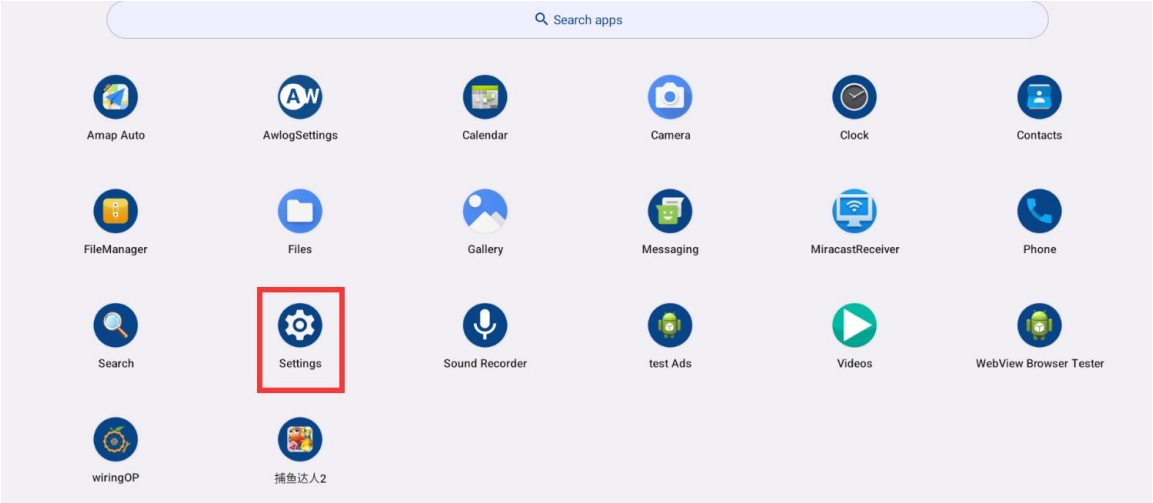


8) WI-FI 连接成功后的显示如下图所示

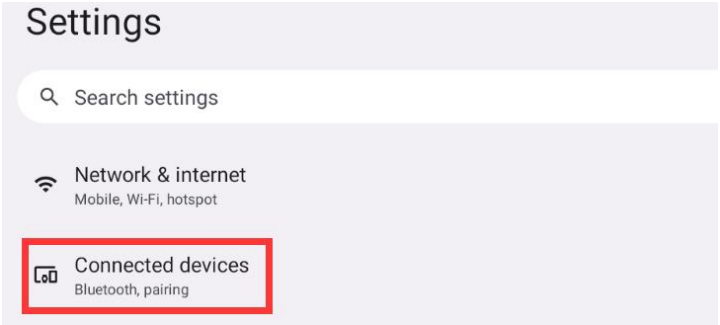


5.5. 蓝牙的连接方法

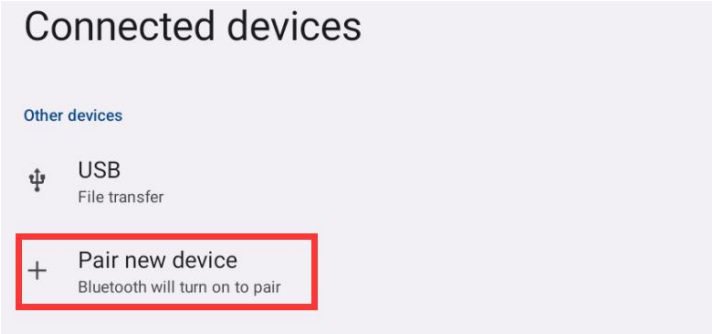
1) 首先选择 **Settings**



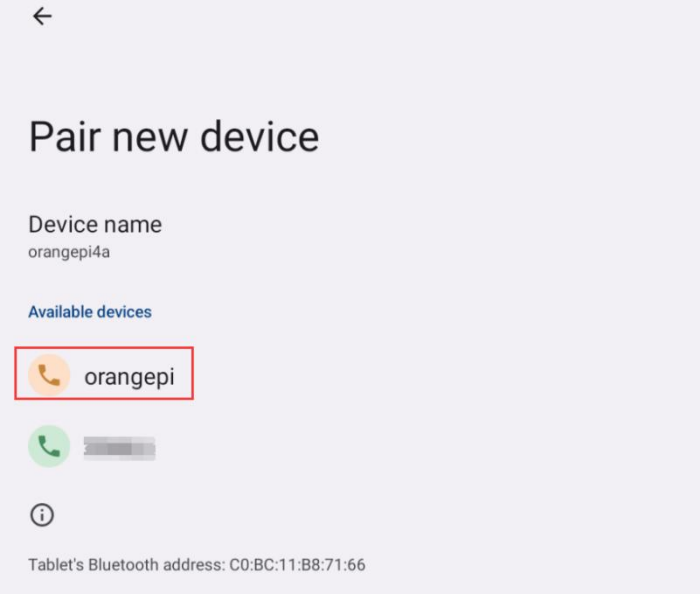
2) 然后选择 **Connected devices**



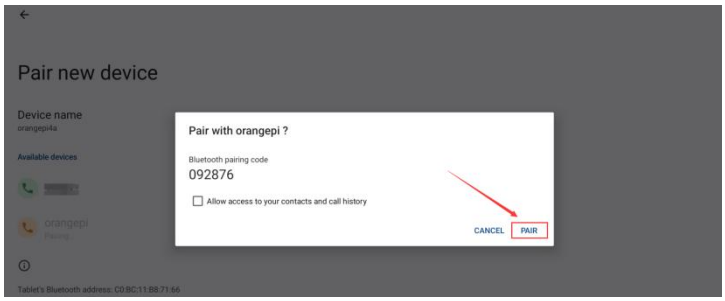
3) 然后选择 **Pair new device** 开始扫描周围的蓝牙设备



4) 搜索到的蓝牙设备会在 **Available devices** 下面显示出来

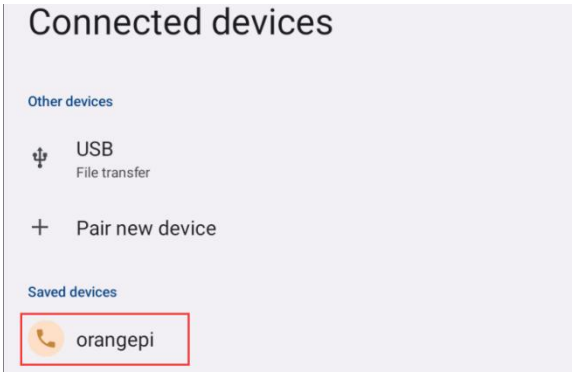


5) 然后点击想要连接的蓝牙设备就可以开始配对了，当弹出下面的界面时，请使用鼠标选择 **Pair** 选项

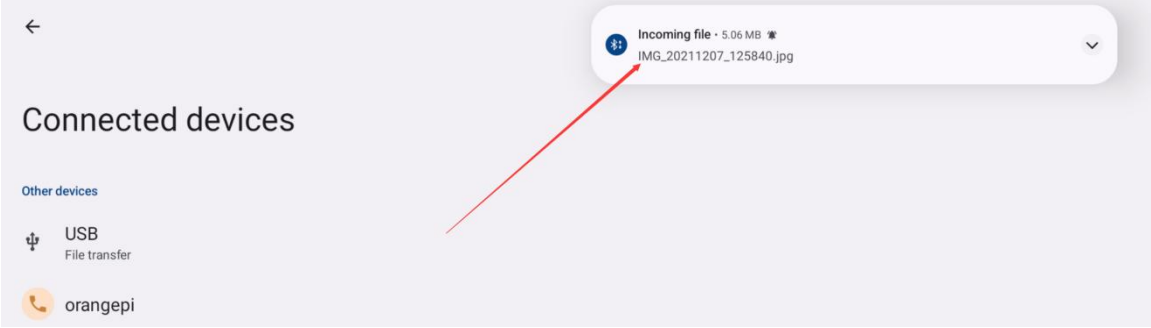


6) 这里测试的是开发板和安卓手机蓝牙的配置过程，此时在手机上会弹出确认界面，在手机上也点击配对按钮后就会开始配对过程

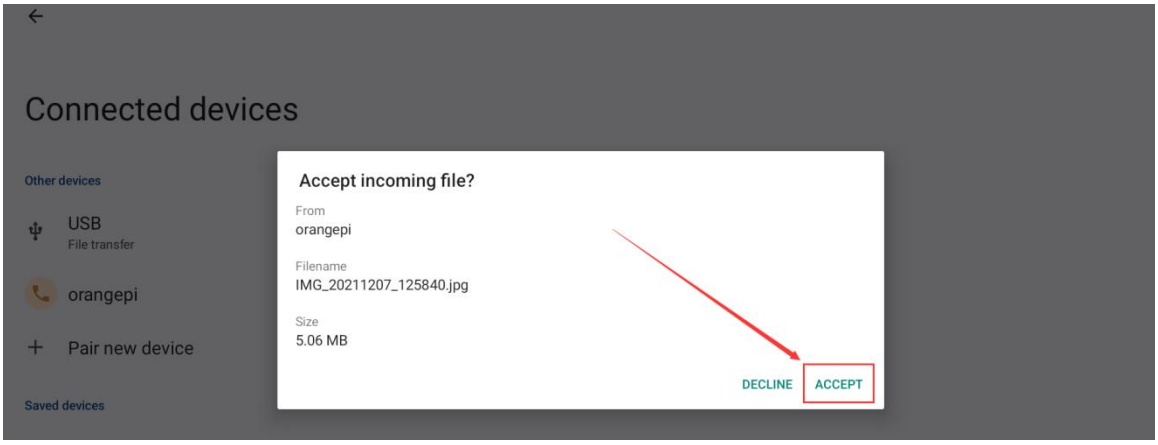
7) 配对完成后，再打开 **Paired devices** 下面就可以看到已配对的蓝牙设备



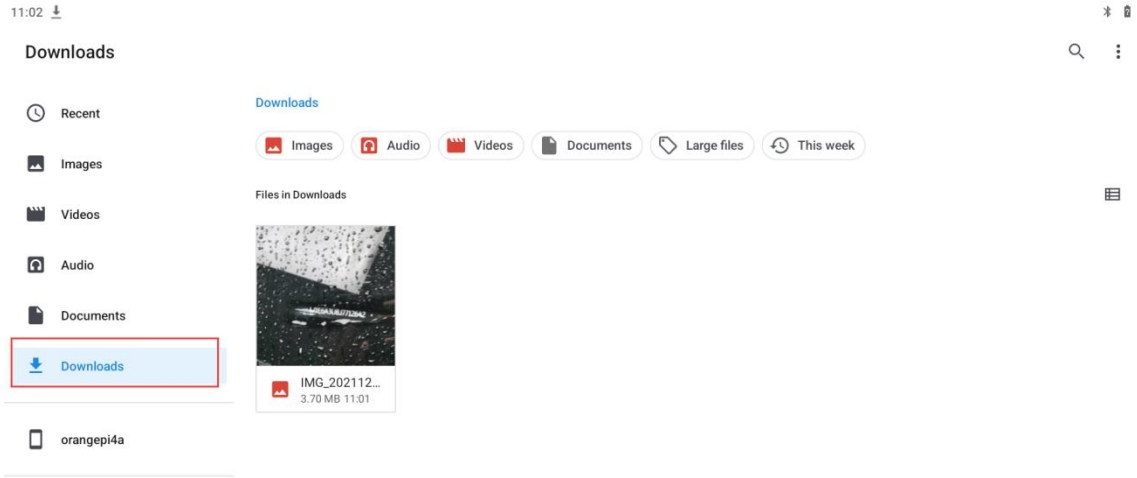
8) 此时可以使用手机蓝牙给开发板发送一张图片，发送后，在开发板的安卓系统中可以看到下面的提示，然后点击 **Incoming file**



9) 然后在弹出的窗口中点击 **Accept** 就可以开始接收手机发过来的图片了

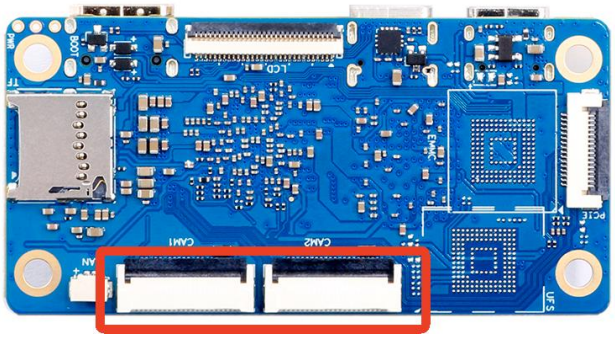


10) 开发板 Android 系统蓝牙接收到的图片可以打开文件管理器的 **Download** 目录查看

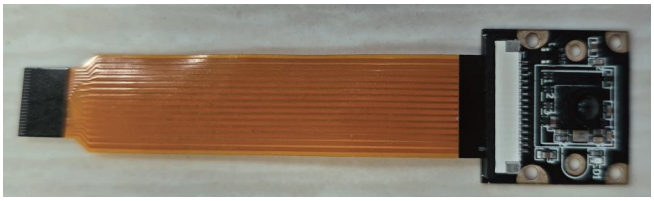


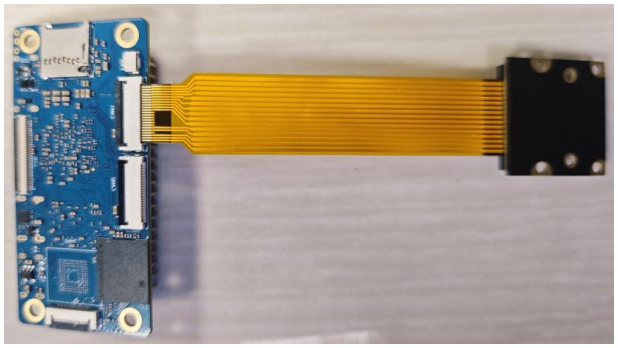
5.6. MIPI 摄像头的使用方法

1) 开发板上总共有 2 个摄像头接口，目前只支持接 IMX219 摄像头，CAM1 和 CAM2 的位置如下所示：

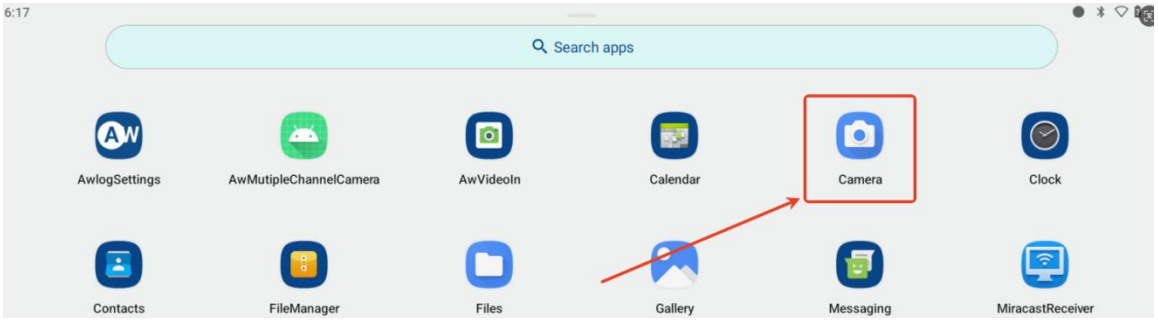


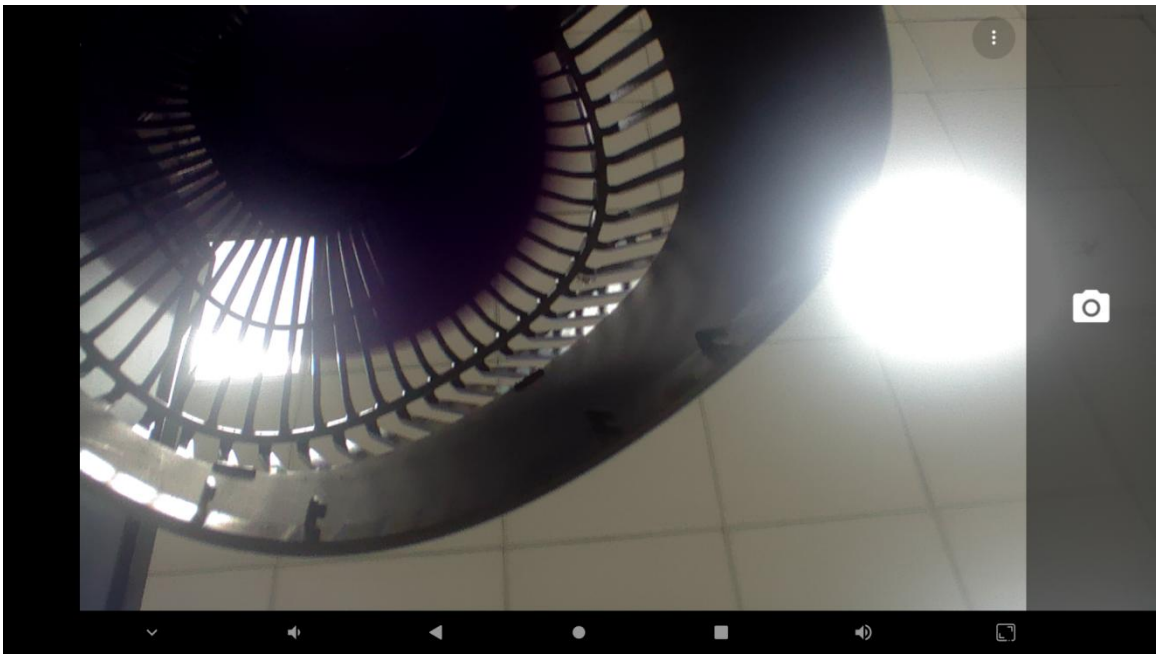
2) 摄像头接口规格是 22PIN 0.5mm 间距，需要通过 22PIN 转 15PIN 的排线将 IMX219 摄像头连接到在开发板上，CAM1 和 CAM2 接法都是一样的





3) 连接好摄像头后，在桌面中点击相机图标就可以预览摄像头了





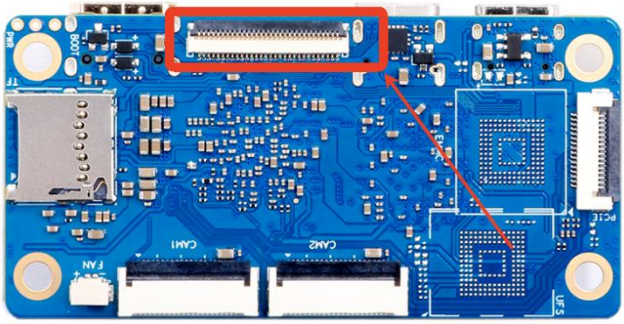
5.7. 10.1 寸 MIPI 屏幕的使用方法

请确保使用的 Android 镜像为下面的版本的镜像:

OrangePiZero3W_A733_Android13_lcd_vX.X.img

1) 首先需要组装好屏幕，请参考 10.1 寸 MIPI 屏幕的组装方法

2) 开发板上 mipi lcd 屏幕的接口的位置如下图所示：



3) 将组装好的屏幕接到开发板的 LCD 接口，注意拔掉 HDMI 接口，给板子接通 Type-C 电源，并上电，系统启动后，就可以看到屏幕显示如下图所示（**默认为竖屏**）



5.8. USB 摄像头使用方法

1) 首先在开发板的 USB 接口中插入 USB（UVC 协议）摄像头

2) USB 摄像头如果识别正常，在/dev 下会生成相应的 video 设备节点

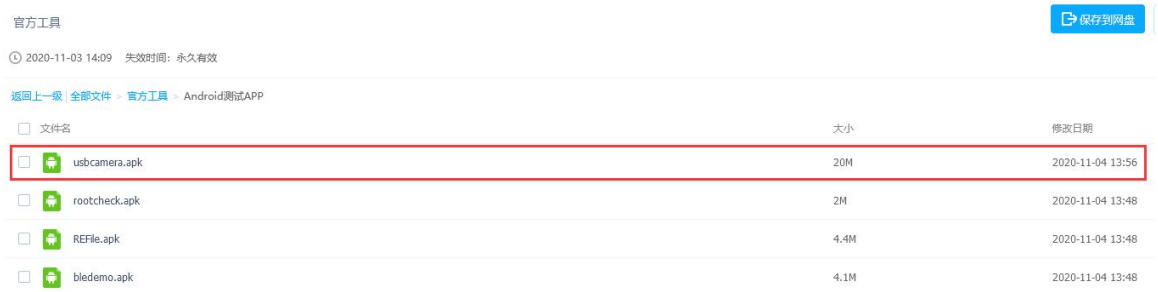
```
console:/ # ls /dev/video*
```

```
/dev/video0
```

3) 然后确保 Ubuntu PC 和开发板的 adb 连接正常，adb 的使用方法请参考 **ADB 的使用方法**一小节的说明

4) 在开发板资料下载页面的**官方工具**中下载 USB 摄像头测试 APP

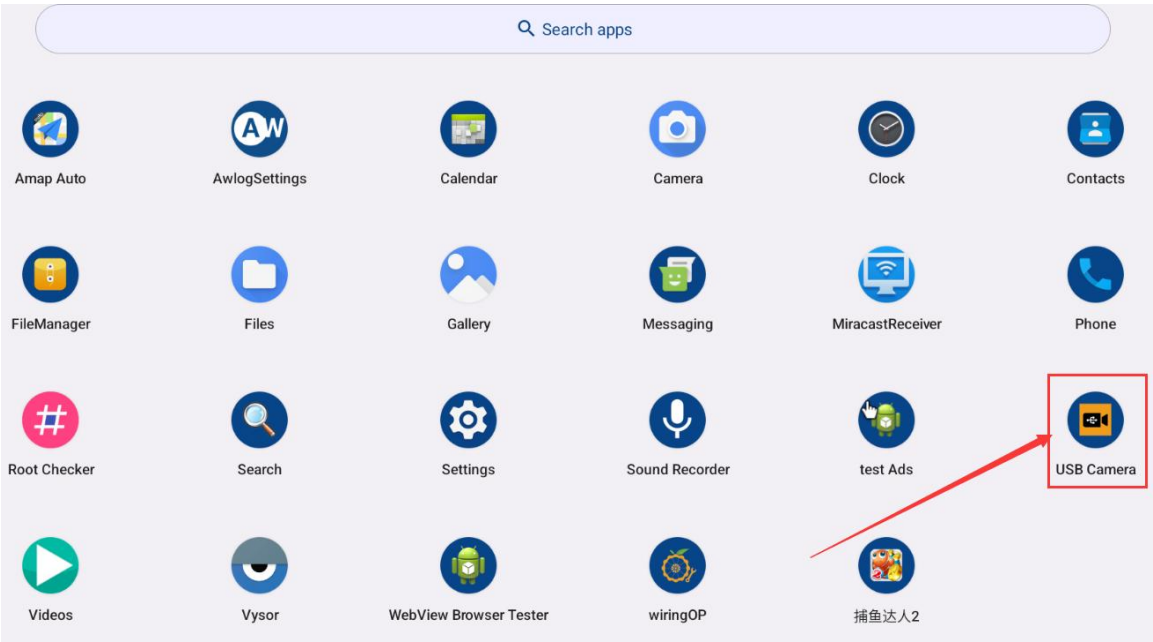




5) 然后使用 adb 命令安装 USB 摄像头测试 APP 到 Android 系统中，当然也可以使用 U 盘拷贝的方式进行安装

```
test@test:~$ adb install usbcamera.apk
```

6) 安装完后在 Android 的 APP 界面可以看到 USB 摄像头的启动图标



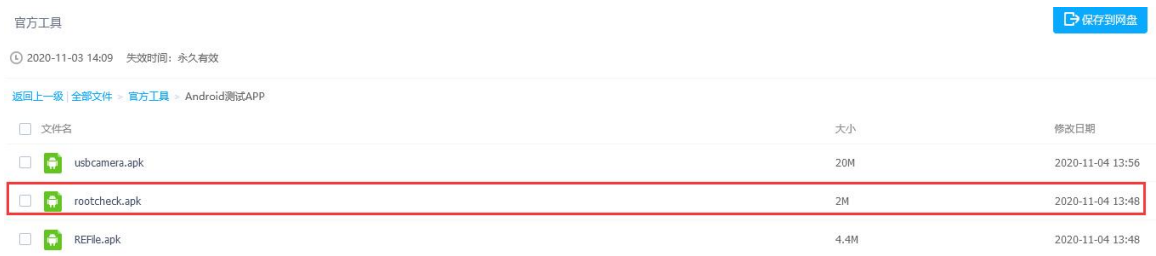
7) 然后双击打开 USB 摄像头 APP 就可以看到 USB 摄像头的输出视频了

5.9. Android 系统 ROOT 说明

Orange Pi 发布的 Android 系统已经 ROOT，可以使用下面的方法来测试。

1) 在开发板资料下载页面的**官方工具**中下载 **rootcheck.apk**



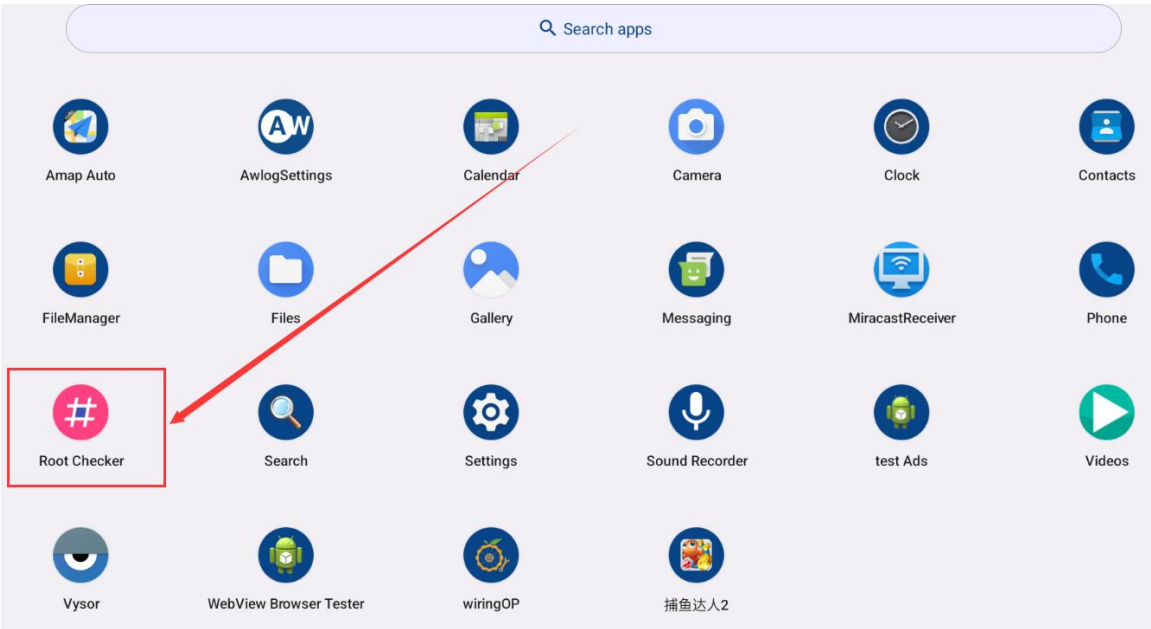


2) 然后确保 Ubuntu PC 和开发板的 adb 连接正常，adb 的使用方法请参考 **ADB 的使用方法**一小节的说明

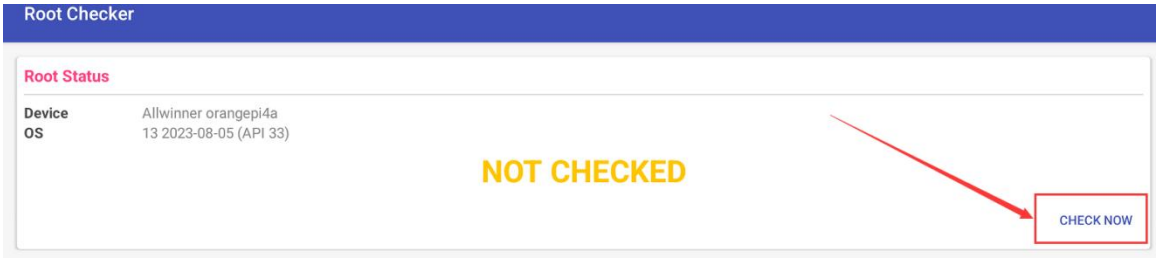
3) 然后使用 adb 命令安装 rootcheck.apk 到 Android 系统中，当然也可以使用 U 盘拷贝的方式进行安装

```
test@test:~$ adb install rootcheck.apk
```

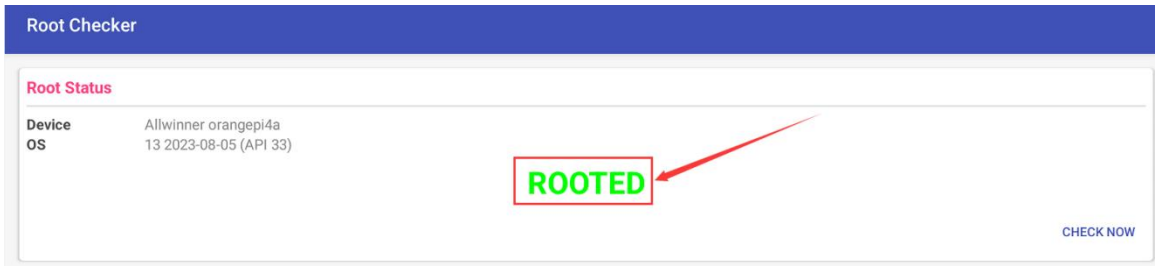
4) 安装完后在 Android 的的 APP 界面可以看到 ROOT 测试工具的启动图标



5) 第一次打开 **ROOT 测试工具**后的显示界面如下图所示



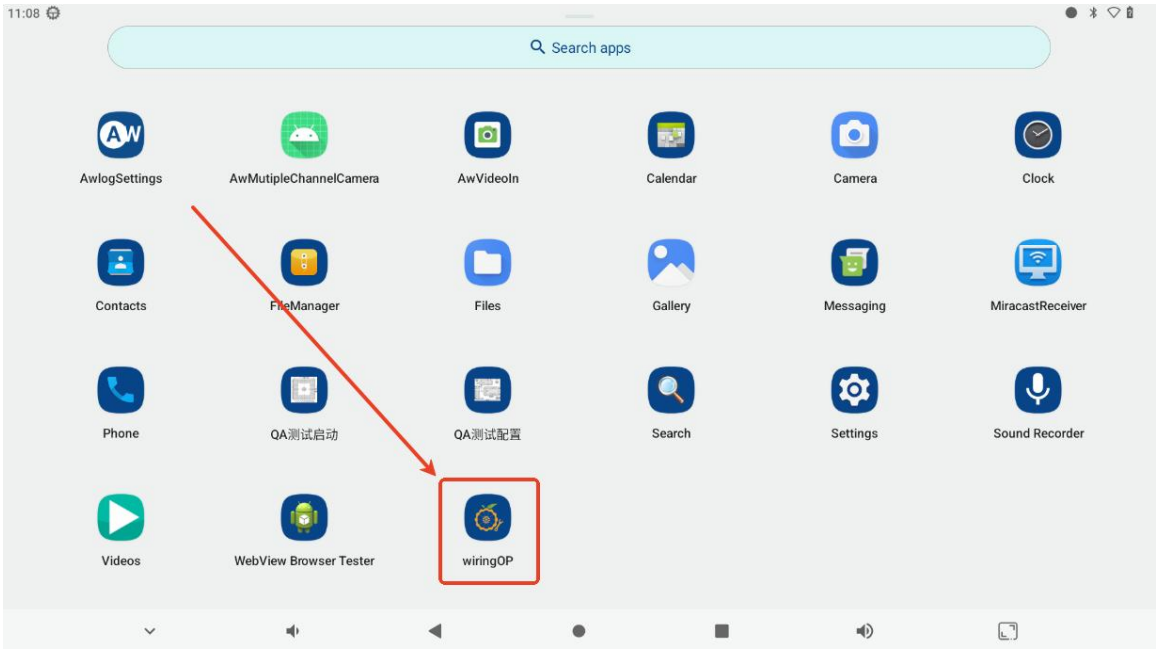
6) 然后就可以点击 **CHECK NOW** 开始 Android 系统的 ROOT 状态的检查，检查完后的显示如下所示，可以看到 Android 系统已取得 ROOT 权限



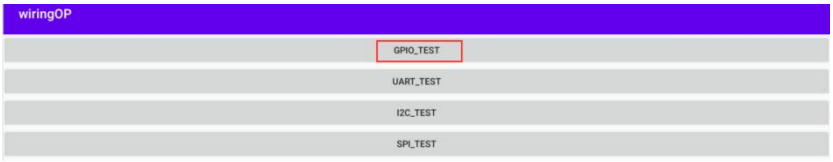
5.10. 40 pin 接口 GPIO、UART、SPI 测试

5.10.1. 40 pin 的 GPIO 口测试方法

1) 首先在桌面中打开 wiringOP APP



2) 然后点击 **GPIO_TEST** 按钮打开 GPIO 测试界面



3) GPIO 测试界面如下图所示，左边的两排 **CheckBox** 按钮跟 40 pin 引脚是一一对应的关系。当勾选 **CheckBox** 按钮时，对应的 GPIO 引脚会被设置为 **OUT** 模式，引脚电平设置为高电平；当取消勾选时，GPIO 引脚电平设置为低电平；当点击右边的 **GPIO READALL** 按钮时，可以获取到 wPi 号、GPIO 模式、引脚电平等信息。当点击 **BLINK ALL GPIO** 按钮时，所有引脚会不断地切换高低电平



4) 然后点击 **GPIO READALL** 按钮，输出信息如下图所示：

1

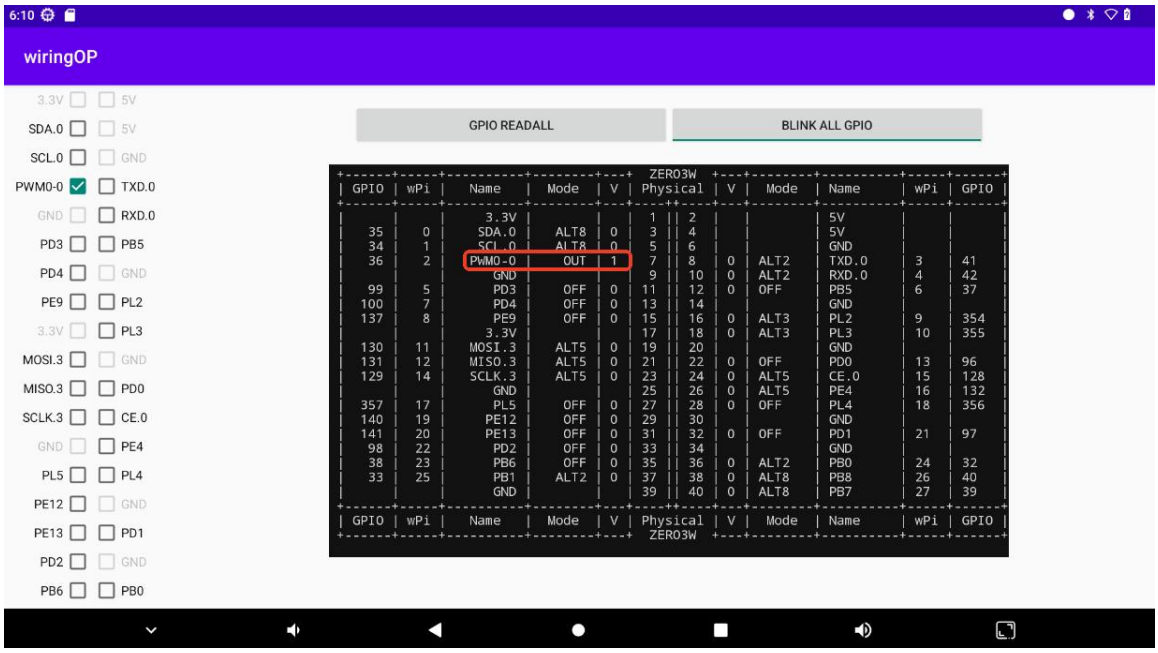


5) 开发板 40 pin 中总共有 28 个 GPIO 口可以使用，下面以 7 号引脚——对应 GPIO 为 PB4 ——对应 wPi 序号为 2——为例演示如何设置 GPIO 口的高低电平。首先点击 7 号引脚对应的 **CheckBox** 按钮，当按钮为选中状态时，7 号引脚会设置为高电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 **3.3v**，说明设置高电平成功



6) 然后点击 **GPIO READALL** 按钮，可以看到当前的 7 号引脚模式为 **OUT**，引脚

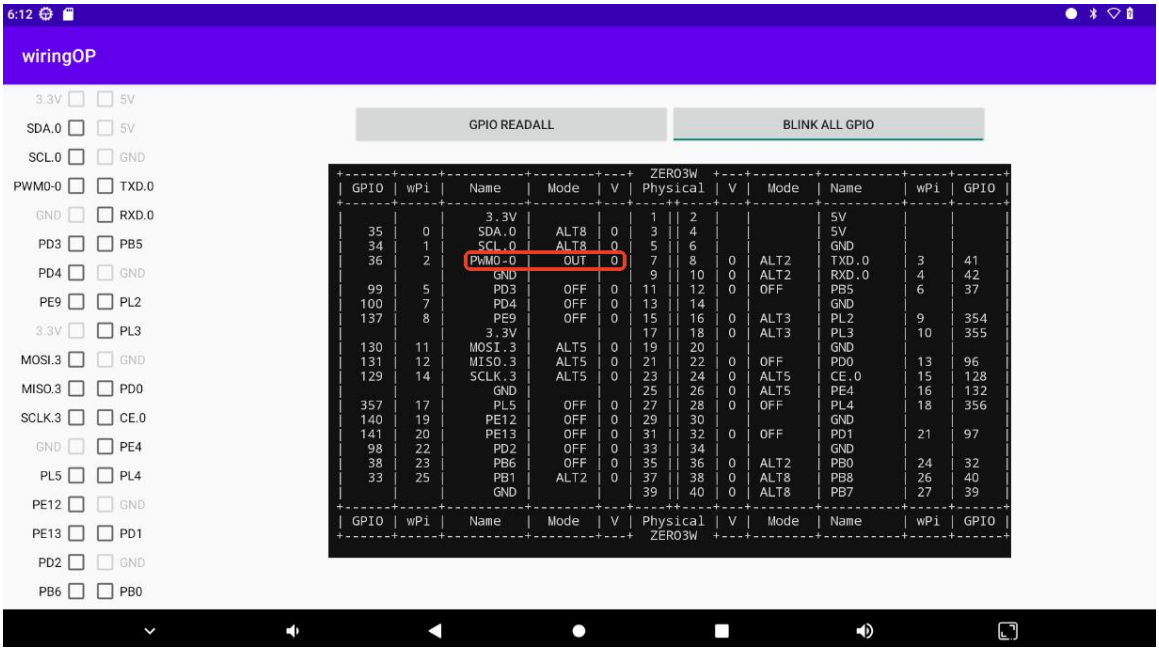
电平为高电平



7) 再次点击下图的 **CheckBox** 按钮取消勾选状态，7 号引脚会设置为低电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 **0v**，说明设置低电平成功



8) 然后点击 **GPIO READALL** 按钮，可以看到当前的 7 号引脚模式为 OUT，引脚电平为低电平

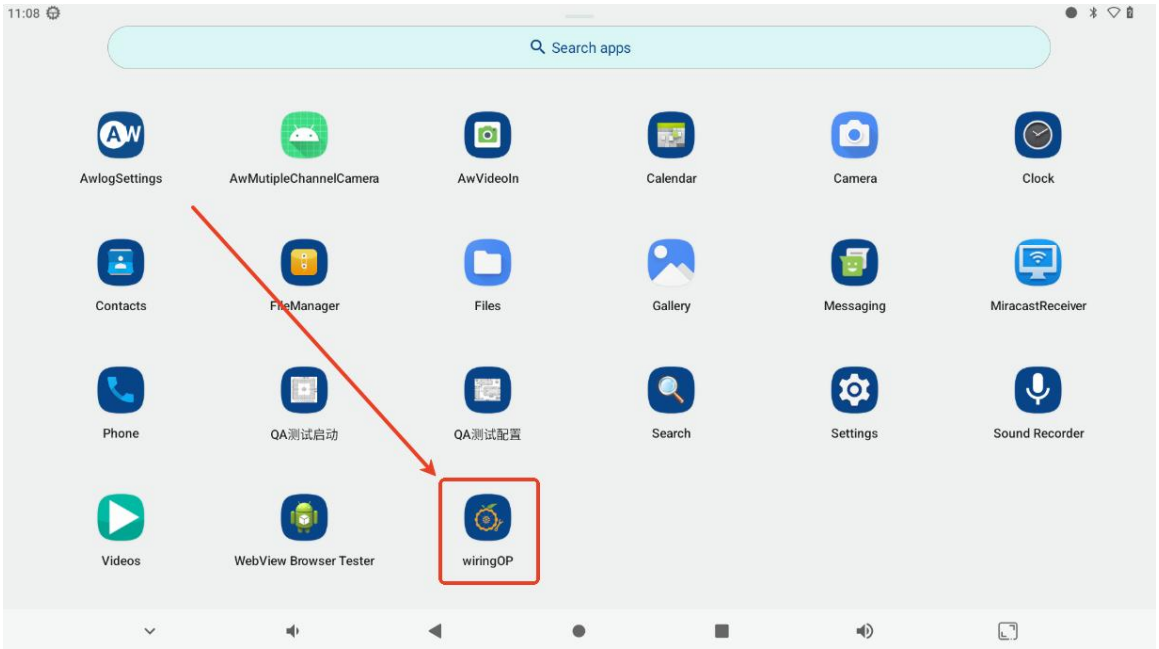


5.10.2. 40 pin 的 UART 测试方法

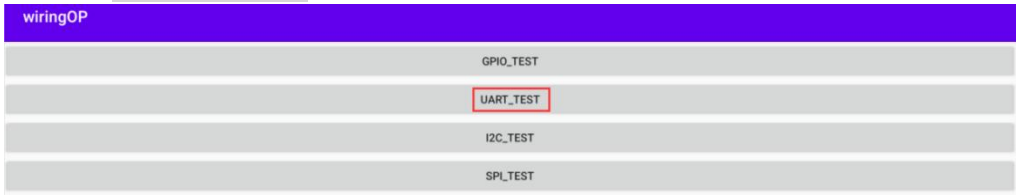
1) Android 中默认打开了 **UART2** 和 **UART7**，对应的设备节点为**/dev/ttyAS2** 和 **/dev/ttyAS7**

```
a733-demo:/ $ ls /dev/ttyAS*
/dev/ttyAS0    /dev/ttyAS1    /dev/ttyAS2    /dev/ttyAS7
```

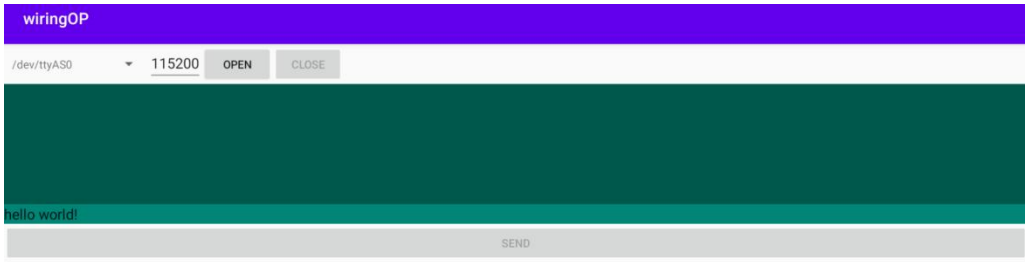
2) 首先在桌面中打开 wiringOP APP



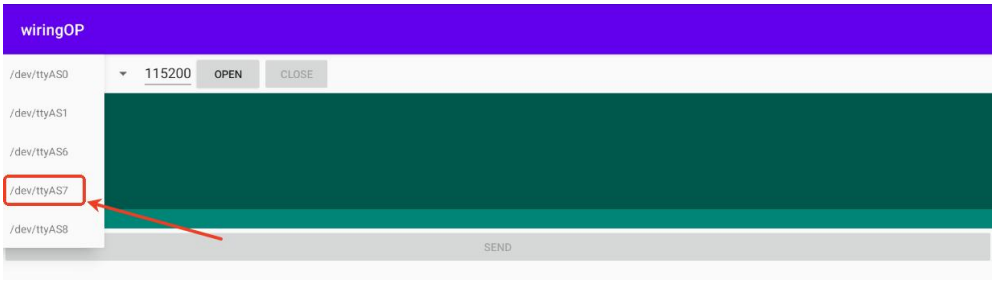
3) 然后点击 **UART_TEST** 按钮打开 UART 测试界面



4) wiringOP 的串口测试界面如下图所示



5) 以测试 UART7 为例，在选择框中选择**/dev/ttyAS7** 节点



6) 再在编辑框中输入想要设置的波特率，然后点击 **OPEN** 按钮打开**/dev/ttyAS7** 节点，打开成功后，**OPEN** 按钮变为不可选中状态，**CLOSE** 按钮和 **SEND** 按钮变为可选中状态

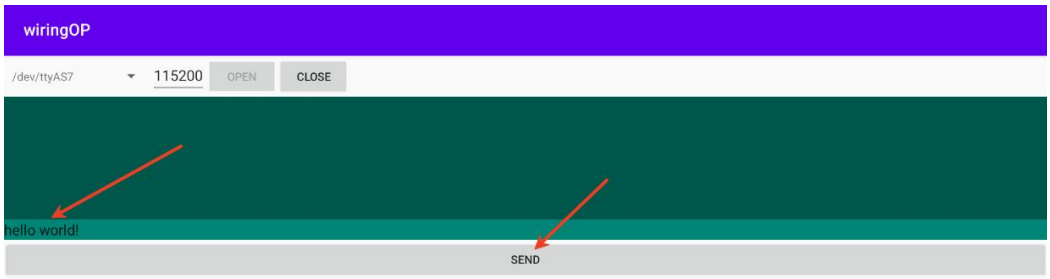


7) 然后使用杜邦线短接 uart7 的 rx 和 tx 引脚

	uart2	uart7

tx 引脚	对应 40 pin 的 11 号引脚	对应 40 pin 的 16 号引脚
rx 引脚	对应 40 pin 的 13 号引脚	对应 40 pin 的 18 号引脚

8) 然后可以在下面的发送编辑框中输入一段字符，点击 **SEND** 按钮开始发送

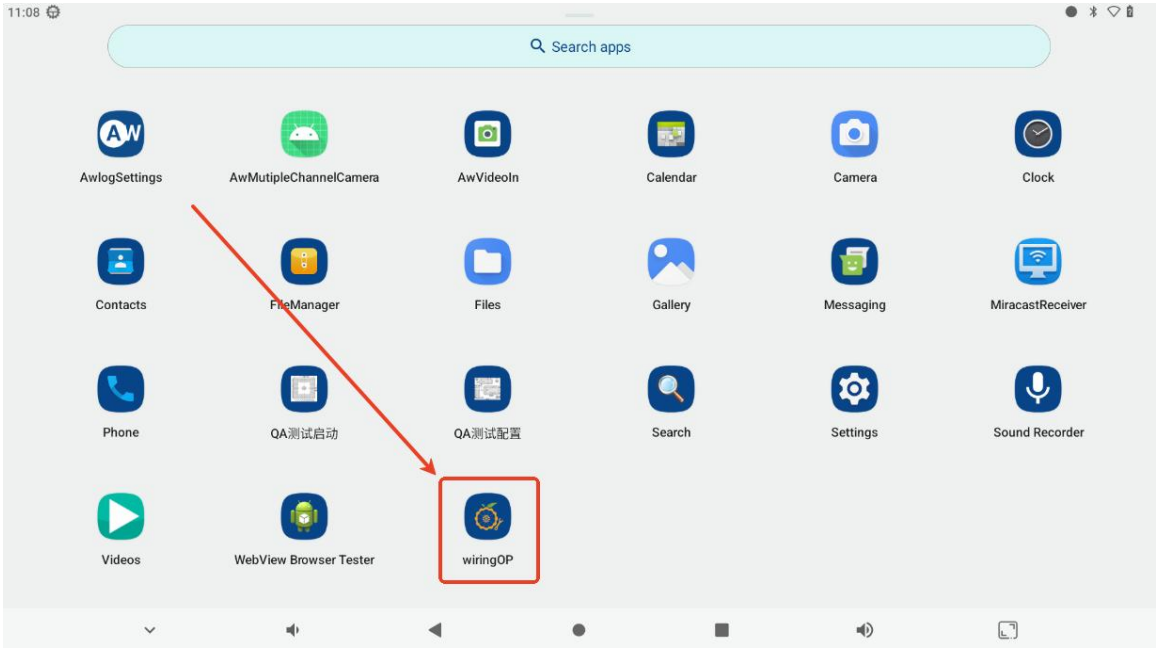


9) 如果一切正常，接收框内会显示已接收到的字符串



5.10.3. 40 pin 的 SPI 测试方法

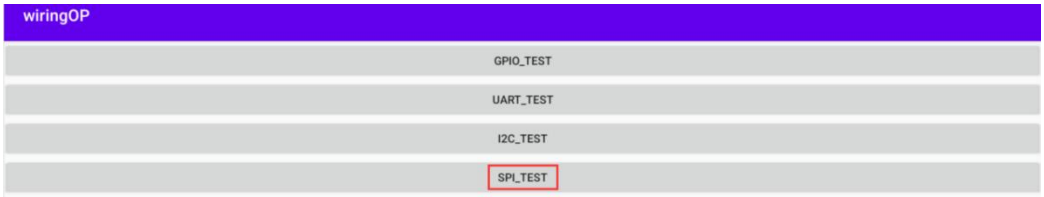
1) 40 pin 中可以用的 SPI 为 SPI3，对应的设备节点为**/dev/spidev3.0**



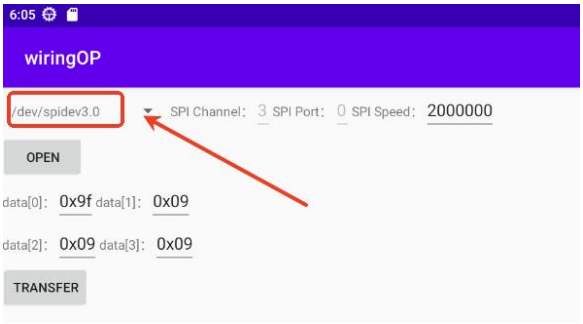
2) 这里演示下通过 **w25q64** 模块来测试 SP3 接口，首先在 SPI3 接口接入 w25q64 模块

3) 然后在桌面中打开 wiringOP APP

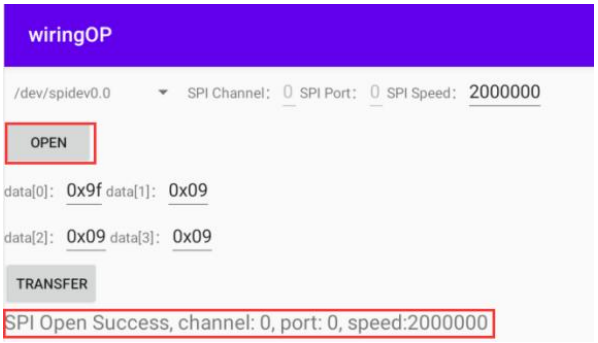
4) 然后点击 **SPI_TEST** 按钮打开 SPI 的测试界面



5) 然后在左上角选择 spi 的设备节点，请选择**/dev/spidev3.0**



6) 然后点击 **OPEN** 按钮初始化 SPI



7) 然后填充需要发送的字节，比如读取 SPIFlash 的 ID 信息，在 data[0]中填入地址 0x9f，然后点击 **TRANSFER** 按钮



8) 最后 APP 会显示读取到的 SPI Flash 的 ID 信息



9) w25q64 模块的 MANUFACTURER ID 为 EFh，Device ID 为 4017h，跟上面读取到的值是对应的（h 代表是 16 进制）

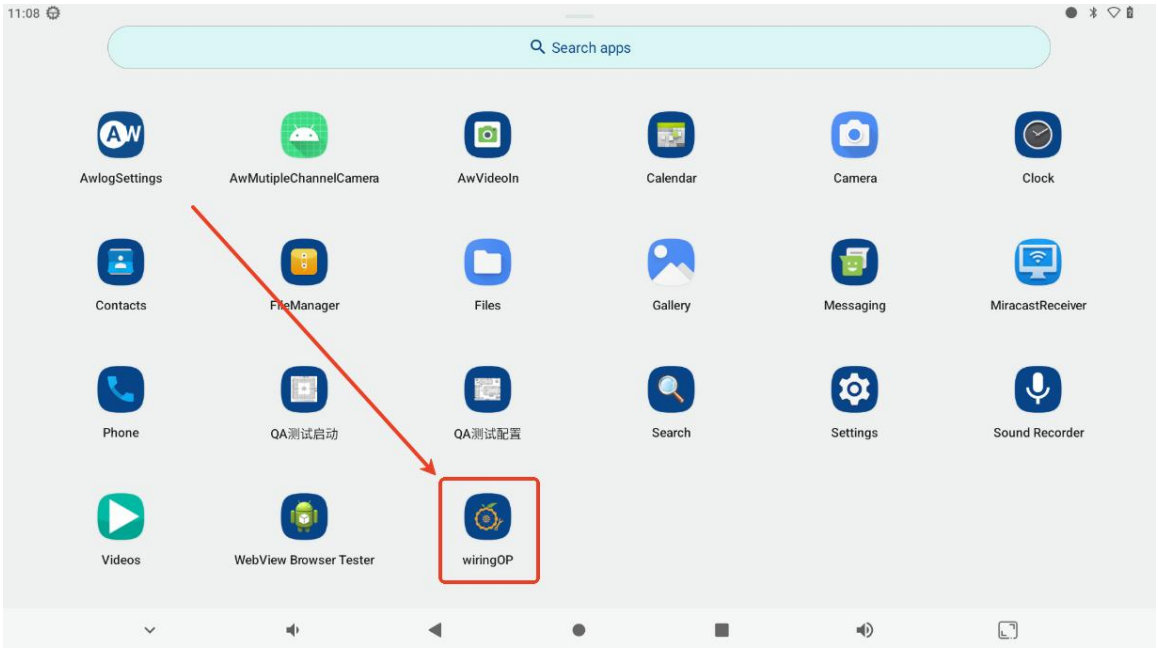
MANUFACTURER ID	**(MF7 - MF0)**	
Winbond Serial Flash	EFh	
Device ID	**(ID7 - ID0)**	**(ID15 - ID0)**
Instruction	**ABh, 90h, 92h, 94h**	**9Fh**
W25Q64FV (SPI)	16h	4017h
W25Q64FV (QPI)	16h	6017h

5. 10. 4. 40 pin 的 I2C 测试方法

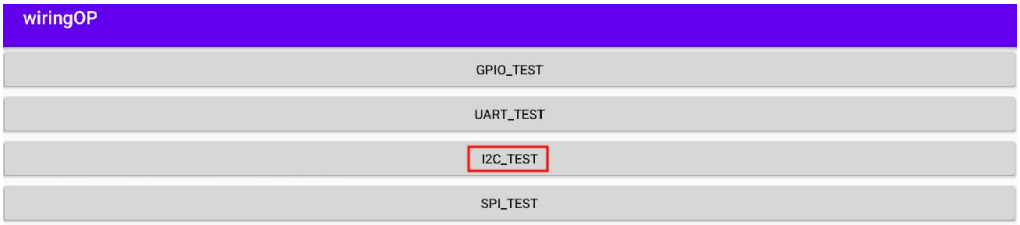
1) Android 中默认打开了 40 pin 中的 i2c0 和 i2c1，对应的设备节点分别为**/dev/i2c-0** 和**/dev/i2c-1**

```
console:/ # ls /dev/i2c-*
/dev/i2c-0    /dev/i2c-1
```

2) 首先在桌面中打开 wiringOP APP



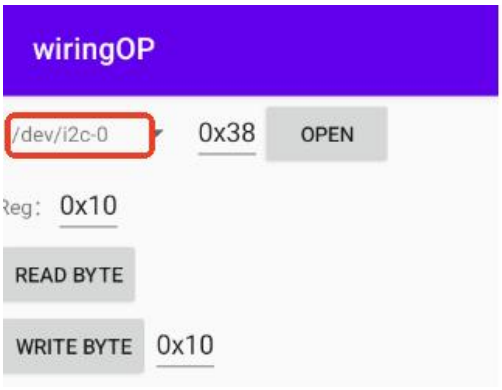
3) 然后点击 **I2C_TEST** 按钮打开 i2c 的测试界面



4) wiringOP 的 i2c 测试界面如下图所示



5) 以测试 i2c0 为例，选择框中默认就是**/dev/i2c-0 节点**



6) 然后在 40 pin 的 i2c0 引脚上接一个 i2c 设备，这里以 ds1307 rtc 模块为例



RTC 模块的引脚	开发板 40 pin 对应的引脚
5V	**2 号引脚**
GND	**6 号引脚**
SDA	**3 号引脚**
SCL	**5 号引脚**

7) ds1307 rtc 模块的 i2c 地址为 0x68，接好线后，我们可以在串口命令行中使用 **i2cdetect -y -r 0** 命令查看下是否能扫描到 ds1307 rtc 模块的 i2c 地址。如下图所示，如果能看到 0x68 这个地址，说明 ds1307 rtc 模块接线正确。

```
console:/ # i2cdetect -y 0
```

```
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

8) 然后在 wiringOP 中设置 i2c 的地址为 0x68，再点击 **OPEN** 按钮打开 i2c0

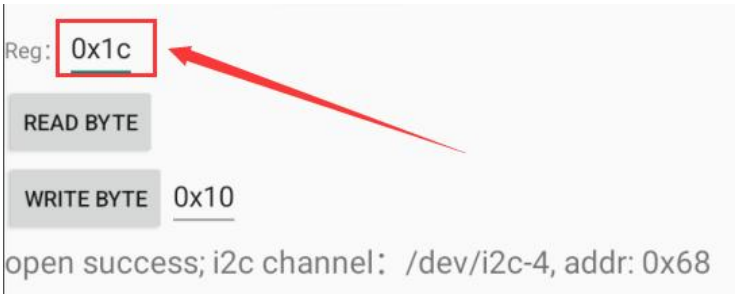


9) 点击 **OPEN** 按钮打开 i2c0 后的显示如下所示：



10) 然后我们测试下往 rtc 模块的寄存器中写入一个值，比如往 0x1c 地址写入 0x55

a. 我们首先设置需要写入的寄存器的地址为 0x1c



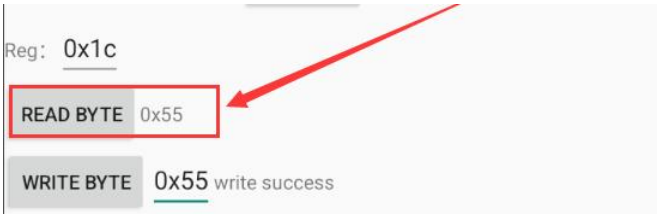
b. 然后设置需要写入的值为 0x55



c. 然后点击 **WRITE BYTE** 按钮执行写入的动作



11) 然后点击 **READ BYTE** 按钮读取下 0x1c 寄存器的值，如果显示为 0x55，就说明 i2c 读写测试通过

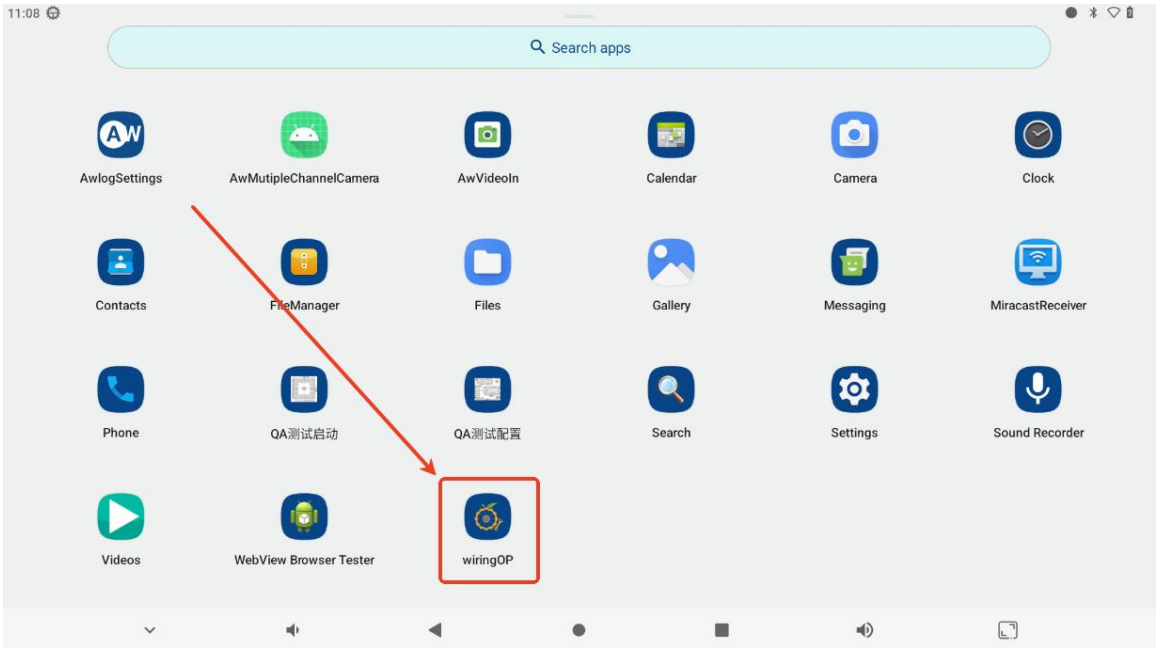


5.10.5. 40 pin 的 PWM 测试方法

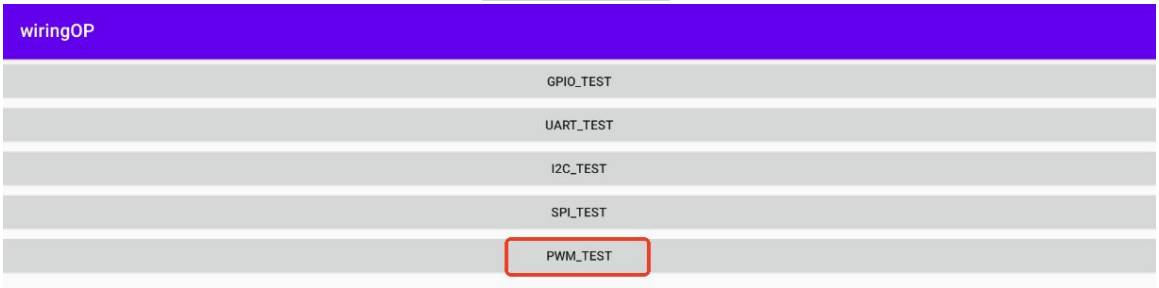
1) Android 默认开启了 **PWM0_0**、**PWM0_1** 和 **PWM0_2**，对应的引脚在 40pin 的所在位置如下图所示

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PB3	35	3	4		5V		
		TWI0-SCK	PB2	34	5	6		GND		
		PWM0-0	PB4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PB1	33	13	14		GND		
			PE9	137	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PB6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PB8	TWI1-SDA	
			GND		39	40	39	PB7	TWI1-SCL	PWM0-9

2) 首先点击 wiringOP 图标打开 wiringOP APP



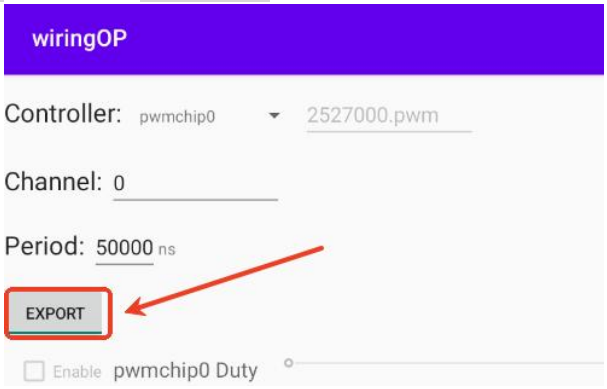
3) 然后在 wiringOP 的主界面点击 **PWM_TEST** 按钮进入 PWM 的测试界面



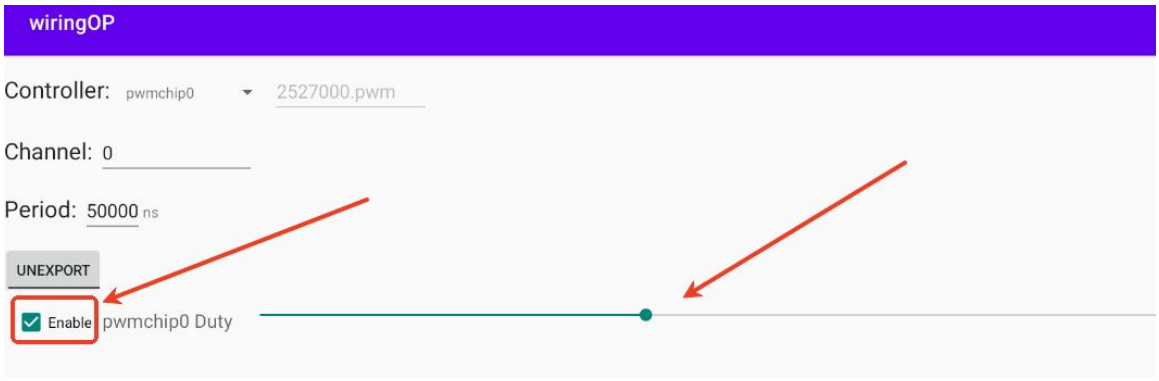
4) **PWM0_0、PWM0_1** 和 **PWM0_2** 的基地址是 **2527000**，以测试 **PWM0_0** 为例，**PWM0_0** 对应的基地址是 **2527000**，这里下拉框选择 pwmchip0 时右边显示的就是 **2527000.pwm**， 如果显示的基地址不对，请点击下拉选项选择其它的 pwmchip，直到右边显示 **2527000** 为止



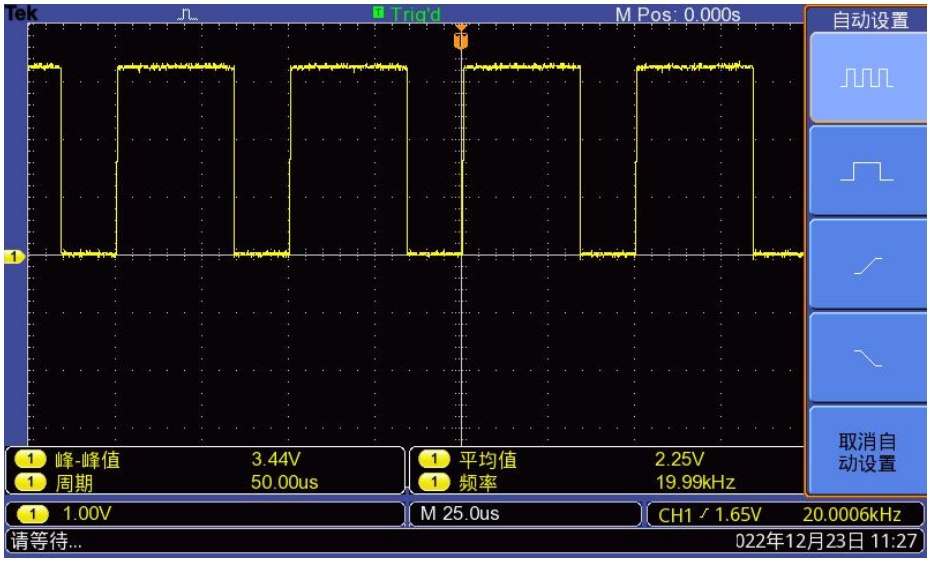
5) 由于 **PWM0_0** 对应的是 PWM0 的第 0 个通道，所以 Channel 要设置为 0，并确认 PWM 的周期，默认的配置是 **50000ns**，转换为 PWM 频率是 **20KHz**，可自行修改，点击 **EXPORT** 按钮导出 **PWM0_0**



6) 然后拖动下面的拖动条，就可以改变 PWM 的占空比，然后勾选 Enable 就可以输出 PWM 波形了



7) 然后使用示波器测量开发板 40pin 中的第 7 号引脚就可以看到下面的波形了



6. Android 13 源码的编译方法

6.1. 下载 Android 13 的源码

1) 首先从百度或者谷歌网盘下载 Android 13 源码的分卷压缩包

2) Android 13 源码的分卷压缩包下载完后，请先检查下 MD5 校验和是否正确，如果不正确，请重新下载源码。检查 MD5 校验和的方法如下所示：

```
test@test:~$ md5sum -c md5sum
a733_android13.tar.gz00: OK
a733_android13.tar.gz01: OK
.....
```

3) 然执行下面的命令解压出安卓源码

```
test@test:~$ cat a733_android13.tar.gz0* | tar -xvzf -
```

6.2. 编译 Android 13 的源码

Android13 的编译是在安装有 Ubuntu 22.04 的 x86_64 电脑上进行的，其它版本的 Ubuntu 系统包依赖可能会有一些区别，Ubuntu 22.04 amd64 版本的镜像下载地址如下所示：
https://repo.huaweicloud.com/ubuntu-releases/22.04/ubuntu-22.04.2-desktop-amd64.iso

编译 Android13 源码的 x86_64 电脑硬件配置建议内存为 16GB 或 16GB 以上，硬盘空间建议预留 200GB 或以上，CPU 核心数越多越好。

1) 首先安装编译 Android13 源码需要的软件包

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y git gnupg flex bison gperf build-essential \
zip curl zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 \
lib32ncurses5-dev x11proto-core-dev libx11-dev lib32z1-dev ccache \
libgl1-mesa-dev libxml2-utils xsltproc unzip u-boot-tools python-is-python3 \
```

```
libssl-dev libncurses5 clang gawk
```

2) 然后编译 **longan** 文件夹中的代码，里面主要包含 u-boot 和 linux 内核

a. 首先运行 **./build.sh config** 设置编译选项

```
test@test:~$ cd a733_android13/longan
test@test:~/a733_android13/longan$ ./build.sh config
========ACTION List: mk_config ;========
options :
All available platform:
    0. android
    1. linux
Choice [android]: 0
All available ic:
    0. a523
    1. a527
    2. a733
    3. t527
    4. t736
Choice [a733]: 2
All available board:
    0. QA
    1. demo_aiot
    2. demo_car
    3. evb1
    4. fpga
    5. perf1
    6. pro2
    7. pro3
Choice [demo_aiot]: 1
All available flash:
    0. default
    1. nor
Choice [default]: 0
Setup BSP files
INFO: Prepare toolchain ...
```

b. 然后运行**./build.sh** 脚本就可以开始编译了

```
test@test:~/a733_android13/longan$ ./build.sh
```

c. 编译完成后会看到下面的输出

```
sun60iw2p1 compile all(Kernel+modules+boot.img) successful



INFO: build dts ...
INFO: Prepare toolchain ...
Setup BSP files
'/home/test/a733_android13/longan/out/a733/kernel/staging/sunxi.dtb' ->
'/home/test/a733_android13/longan/out/a733/demo/android/sunxi.dtb'
INFO: build rootfs ...
INFO: skip make rootfs for android
INFO: ----------------------------------------
INFO: build Tina OK.
INFO: ----------------------------------------
```

3) 然后使用下面的命令编译安卓源码并生成最终的安卓镜像

```
test@test:~$ cd a733_android13
test@test:~/a733_android13$ source build/envsetup.sh
test@test:~/a733_android13$ lunch a733_demo_aiot_arm64-userdebug
test@test:~/a733_android13$ make -j8
test@test:~/a733_android13$ pack
```

4) 编译生成的安卓镜像存放路径为：

```
longan/out/a733_android13_demo_aiot_uart0.img
```

7. 附录

7.1. 用户手册更新历史

版本	日期	更新说明
v1.0	2026-03-26	初始版本

7.2. 镜像更新历史

日期	更新说明
2026-03-26	OrangePi_Zero3W_A733_Android13_v1.0.tar.gz OrangePi_Zero3W_A733_Android13_lcd_v1.0.tar.gz Orangepizero3w_1.0.0_ubuntu_jammy_server_linux5.15.147.7z Orangepizero3w_1.0.0_debian_bullseye_server_linux5.15.147.7z Orangepizero3w_1.0.0_debian_bookworm_server_linux5.15.147.7z Orangepizero3w_1.0.0_ubuntu_jammy_desktop_xfce_linux5.15.147.7z Orangepizero3w_1.0.0_debian_bullseye_desktop_xfce_linux5.15.147.7z Orangepizero3w_1.0.0_debian_bookworm_desktop_xfce_linux5.15.147.7z Orangepizero3w_1.0.0_ubuntu_jammy_server_linux6.6.98.7z Orangepizero3w_1.0.0_debian_bullseye_server_linux6.6.98.7z Orangepizero3w_1.0.0_debian_bookworm_server_linux6.6.98.7z Orangepizero3w_1.0.0_ubuntu_jammy_desktop_xfce_linux6.6.98.7z Orangepizero3w_1.0.0_debian_bullseye_desktop_xfce_linux6.6.98.7z Orangepizero3w_1.0.0_debian_bookworm_desktop_xfce_linux6.6.98.7z * 初始版本